Fpt University

TRƯỜNG ĐẠI HỌC FPT

FOOTBALL FIELD BOOKING SYSTEM

Software Design Specification

– Can Tho, July 2026 –

Record of Changes

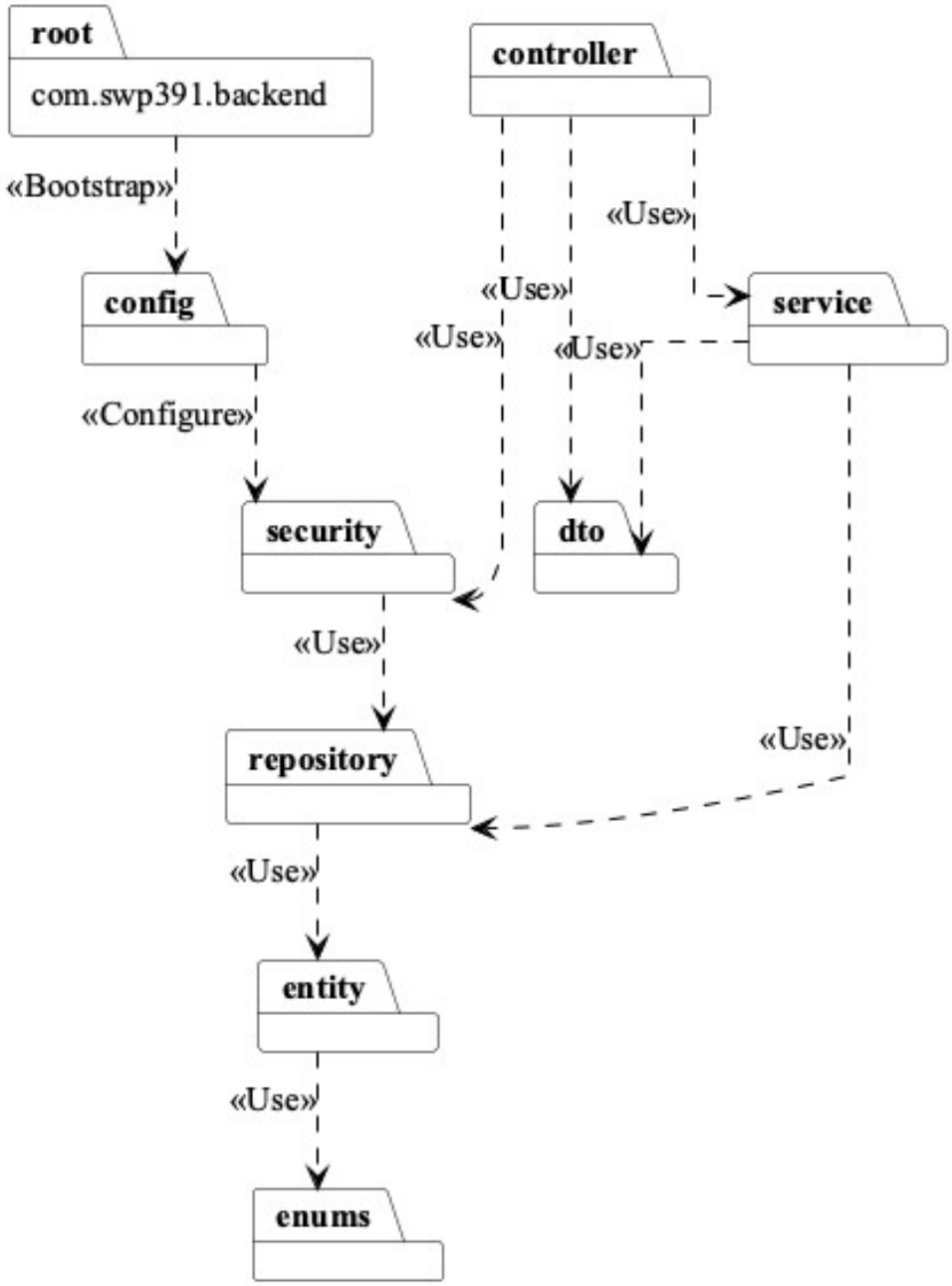
Date	A* M, D	In charge	Change Description
24/07/26	A	BonVT	Added Account & Customer Status requirements and design specifications.
24/07/26	A	NgocPA	Added Booking Lifecycle requirements and design specifications.
24/07/26	A	BaoNG	Added Field, Slot, Service & Issue Management requirements and design specifications.
24/07/26	A	AnNP	Added Payment Gateway, Refund & Invoice requirements and design specifications.
24/07/26	A	AnPTT	Added Promotion, Membership, Notification & Report requirements and design specifications.

*A - Added M - Modified D - Deleted

Contents

I. Overview

1. Code Packages



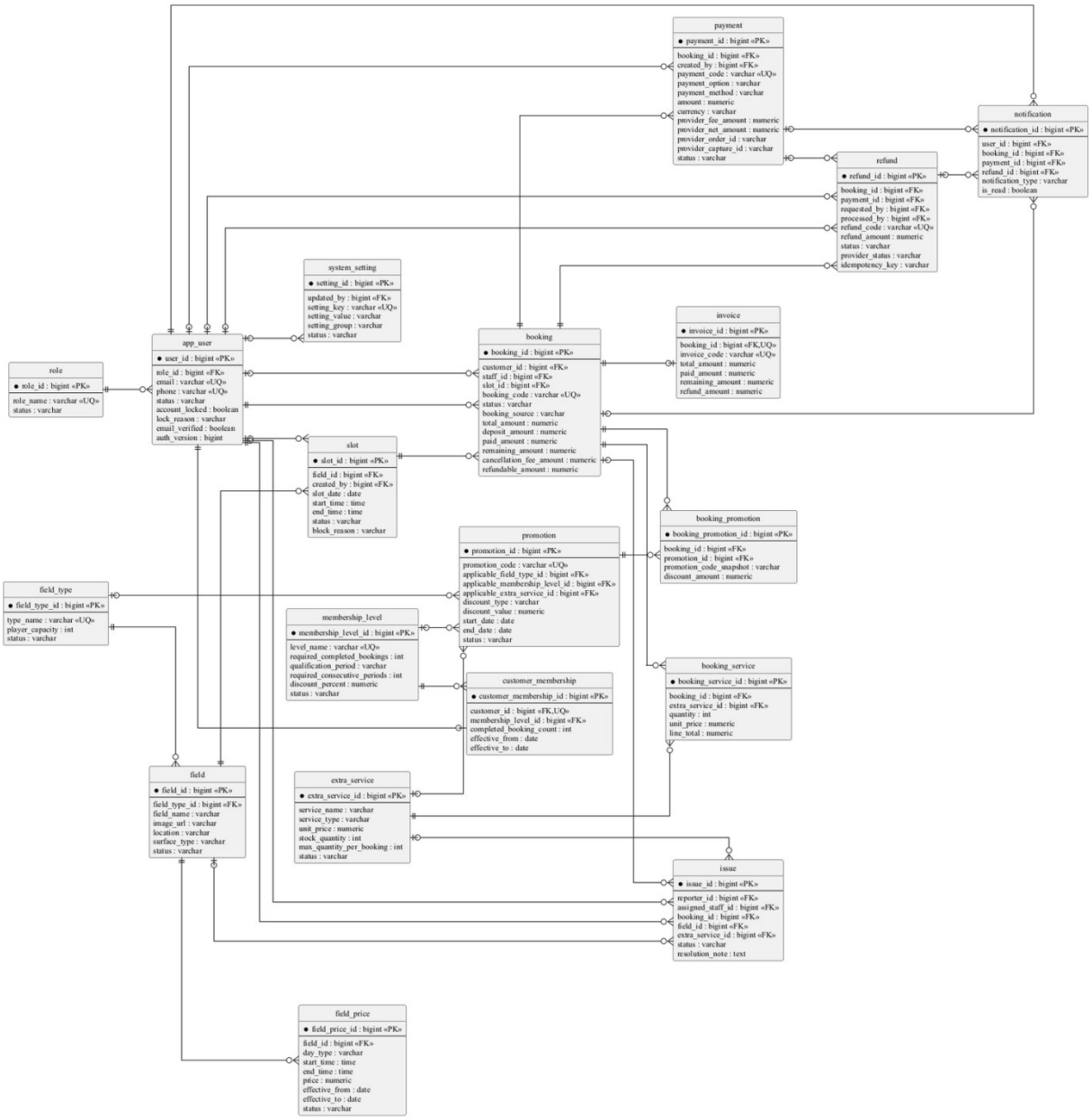
Package descriptions

No	Package	Description
01	*com.swp391.backend*	*1 Java file(s): BackendApplication. Count the Spring Boot entry point separately from feature packages.*
02	*com.swp391.backend.config*	*4 Java file(s): CorsConfig, DataSeeder, PostgresDdlSafetyEnvironmentPostProcessor, SecurityConfig. Authorization rules for many Admin/Staff endpoints live in SecurityConfig, not inside every service method.*

03	*com.swp391.backend.controller*	9 Java file(s): AccountController, ApiExceptionHandler, AvailabilityAssistantController, BookingController, FieldOperationController, ImageUploadController, PaymentController, PromotionReportController, and SystemController.
04	*com.swp391.backend.dto*	*1 Java file(s): ApiRequests. Request records are nested in one ApiRequests class rather than separate DTO files.*
05	*com.swp391.backend.entity*	*20 Java file(s): AppUser, AuditEntity, Booking, BookingPromotion, BookingServiceItem, CustomerMembership, ExtraService, FieldPrice, FieldType, FootballField, Invoice, Issue, MembershipLevel, Notification, Payment, Promotion, Refund, Role, Slot, SystemSetting. There are 19 table-backed entities plus the mapped superclass AuditEntity; do not count AuditEntity as a table.*
06	*com.swp391.backend.enums*	*13 Java file(s): AccountStatus, BookingSource, BookingStatus, CommonStatus, DiscountType, IssueStatus, NotificationType, PaymentMethod, PaymentOption, PaymentStatus, RefundStatus, ServiceType, SlotStatus. Status/value sets are Java enums persisted as strings; they are not lookup tables.*
07	*com.swp391.backend.repository*	*19 Java file(s): AppUserRepository, BookingPromotionRepository, BookingRepository, BookingServiceItemRepository, CustomerMembershipRepository, ExtraServiceRepository, FieldPriceRepository, FieldTypeRepository, FootballFieldRepository, InvoiceRepository, IssueRepository, MembershipLevelRepository, NotificationRepository, PaymentRepository, PromotionRepository, RefundRepository, RoleRepository, SlotRepository, SystemSettingRepository. Only BookingServiceItemRepository contains explicit JPQL; most queries are Spring Data derived or inherited CRUD.*
08	*com.swp391.backend.security*	*4 Java file(s): CustomUserDetailsService, JwtAuthenticationFilter, JwtService, SecurityUser. JWT identity and role authorization are cross-cutting implementation components, not persistence entities.*
09	*com.swp391.backend.service*	16 Java file(s): AccountService, ApiException, AvailabilityAssistantService, BookingMaintenanceScheduler, BookingWorkflowService, DomainSupportService, FieldOperationService, ImageStorageService, PayPalCheckoutService, PaymentWorkflowService, PromotionReportService, SlotGenerationScheduler, SlotGenerationService, SystemQueryService, VerificationEmailDelivery, and VerificationEmailService.

2. Database Design

a. Database Schema



b. Table Description

No	Table	Description
01	*role*	*Primary key: role_id. Use role, not roles or user_role.*
02	*app_user*	Primary key: user_id. Stores Customer, Staff, and Admin identities. status is the single persisted source of truth for active/inactive/locked access; lock_reason explains an

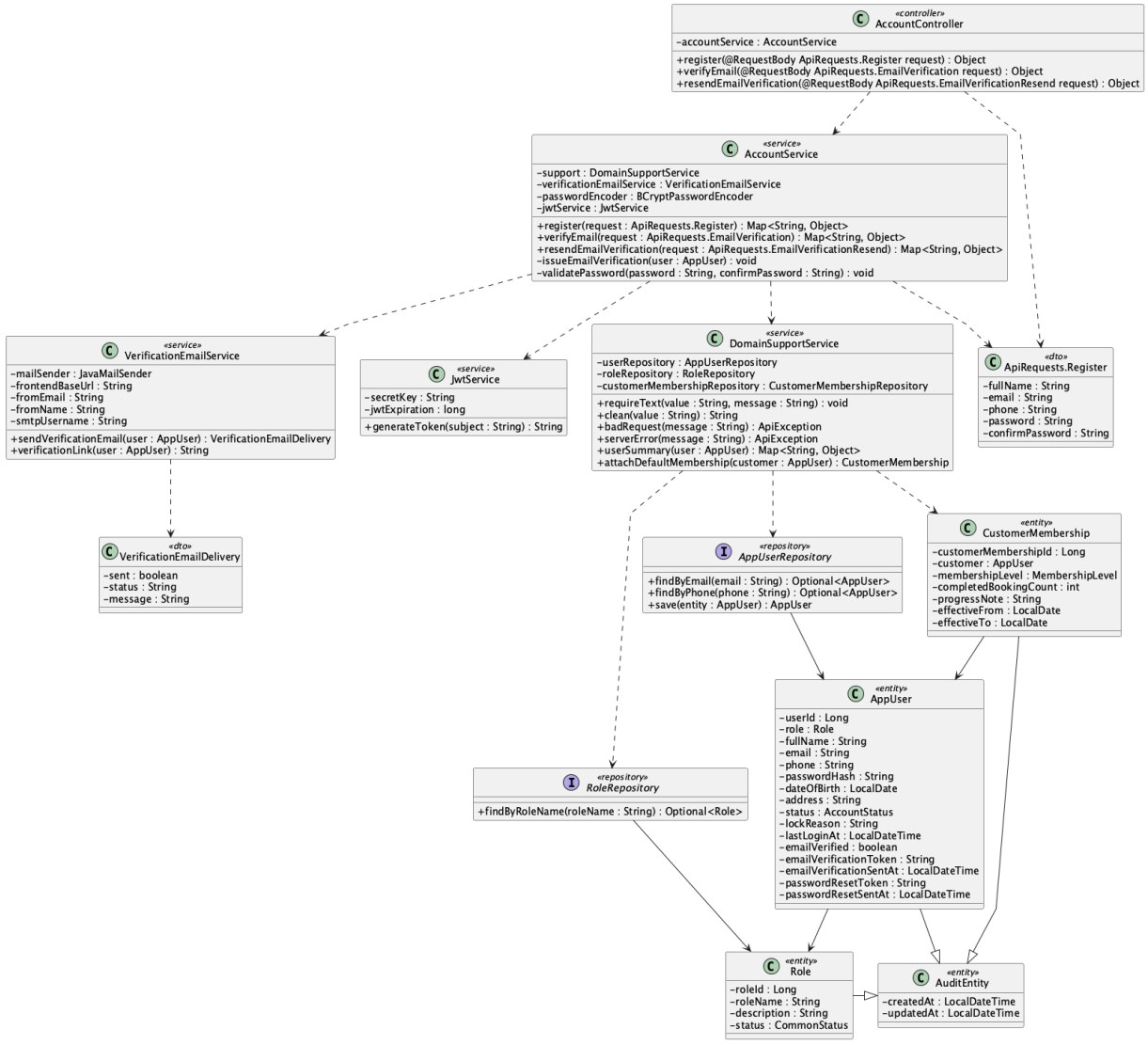
		administrative Customer lock and auth_version revokes JWTs.
03	*membership_level*	*Primary key: membership_level_id. Stores lifetime/monthly/weekly completed-booking rules with consecutive-week requirements added by V10.*
04	*customer_membership*	*Primary key: customer_membership_id. One row per customer is enforced by later integrity migration.*
05	*field_type*	Primary key: field_type_id. Normalizes reusable field-type name and player capacity so multiple fields do not duplicate the same catalogue metadata.
06	*field*	*Primary key: field_id. Physical table is field although the entity is FootballField.*
07	*field_price*	Primary key: field_price_id. Keeps effective weekday/weekend and time-band prices separate from field identity so pricing can change over time without rewriting a field.
08	*slot*	*Primary key: slot_id. Unavailable periods are represented by slot.status='blocked'; no unavailable_slot table exists.*
09	*booking*	*Primary key: booking_id. Lifecycle, financial snapshots, and timestamps live together; there is no booking_status_history table.*
10	*extra_service*	*Primary key: extra_service_id. Use extra_service, not service.*
11	*booking_service*	*Primary key: booking_service_id. Physical table is booking_service although the entity/repository use BookingServiceItem naming.*
12	*issue*	*Primary key: issue_id. Resolution fields are on issue; no issue_resolution table exists.*
13	*promotion*	*Primary key: promotion_id. Applicability dimensions are nullable columns on promotion, not join tables.*
14	*booking_promotion*	*Primary key: booking_promotion_id. Stores applied code/discount snapshots; ordinary product scope is one code per booking.*
15	*payment*	*Primary key: payment_id. Provider order/capture/status/currency/idempotency fields are on payment; V11 also records the exact provider fee and provider net amount returned by PayPal.*
16	*invoice*	*Primary key: invoice_id. booking_id is unique, so the relationship is one invoice per booking.*
17	*refund*	*Primary key: refund_id. Provider status, idempotency key, and gateway message are added by V9; V11 reconciles completed legacy refunds with booking, invoice, and payment status.*

18	*notification*	*Primary key: notification_id. Supports optional booking_id/payment_id/refund_id, but current shared notifyUser only supplies booking_id.*
19	*system_setting*	Primary key: setting_id. Stores administrator-editable deposit, refund, payment-timeout, reminder, and automatic slot-generation policies as key/value configuration; it is not redundant field data.

II. Code Designs

1. Register account

a. Class Diagram



b. Class Specifications

AccountController

No	Method	Description
01	register(request : ApiRequests.Register) : Object	Accepts a registration request and delegates the use case to AccountService.
02	verifyEmail(request : ApiRequests.EmailVerification) : Object	Validates the email-verification link submitted by the client.
03	resendEmailVerification(request : ApiRequests.EmailVerificationResend) : Object	Requests a new verification token and delivery attempt.

AccountService

No	Method	Description
01	register(request : ApiRequests.Register) : Map<String, Object>	Validates unique identity and password policy, creates the Customer, attaches default membership, sends verification, and returns JWT/user data.
02	verifyEmail(request : ApiRequests.EmailVerification) : Map<String, Object>	Validates a one-hour token and marks the account email as verified.
03	resendEmailVerification(request : ApiRequests.EmailVerificationResend) : Map<String, Object>	Reissues a verification token for an unverified account.
04	issueEmailVerification(user : AppUser) : void	Stores a new 16-character token and issue timestamp.
05	validatePassword(password : String, confirmPassword : String) : void	Enforces length, character classes, no whitespace, and matching confirmation.

DomainSupportService

No	Method	Description
01	requireText(value : String, message : String) : void	Rejects a missing required string.
02	clean(value : String) : String	Trims optional input before validation and persistence.
03	attachDefaultMembership(user : AppUser) : void	Creates the initial membership record for the new Customer.
04	userSummary(user : AppUser) : Map<String, Object>	Builds the authenticated account response without exposing the password hash.

VerificationEmailService

No	Method	Description
01	sendVerificationEmail(user : AppUser) :	Sends the verification link and reports delivery status without rolling back account creation.

No	Method	Description
	VerificationEmailDelivery	
02	verificationLink(user : AppUser) : String	Builds the frontend verification URL from user ID and token.

JwtService

No	Method	Description
01	generateToken(user : SecurityUser) : String	Creates a signed JWT containing role and auth-version claims.

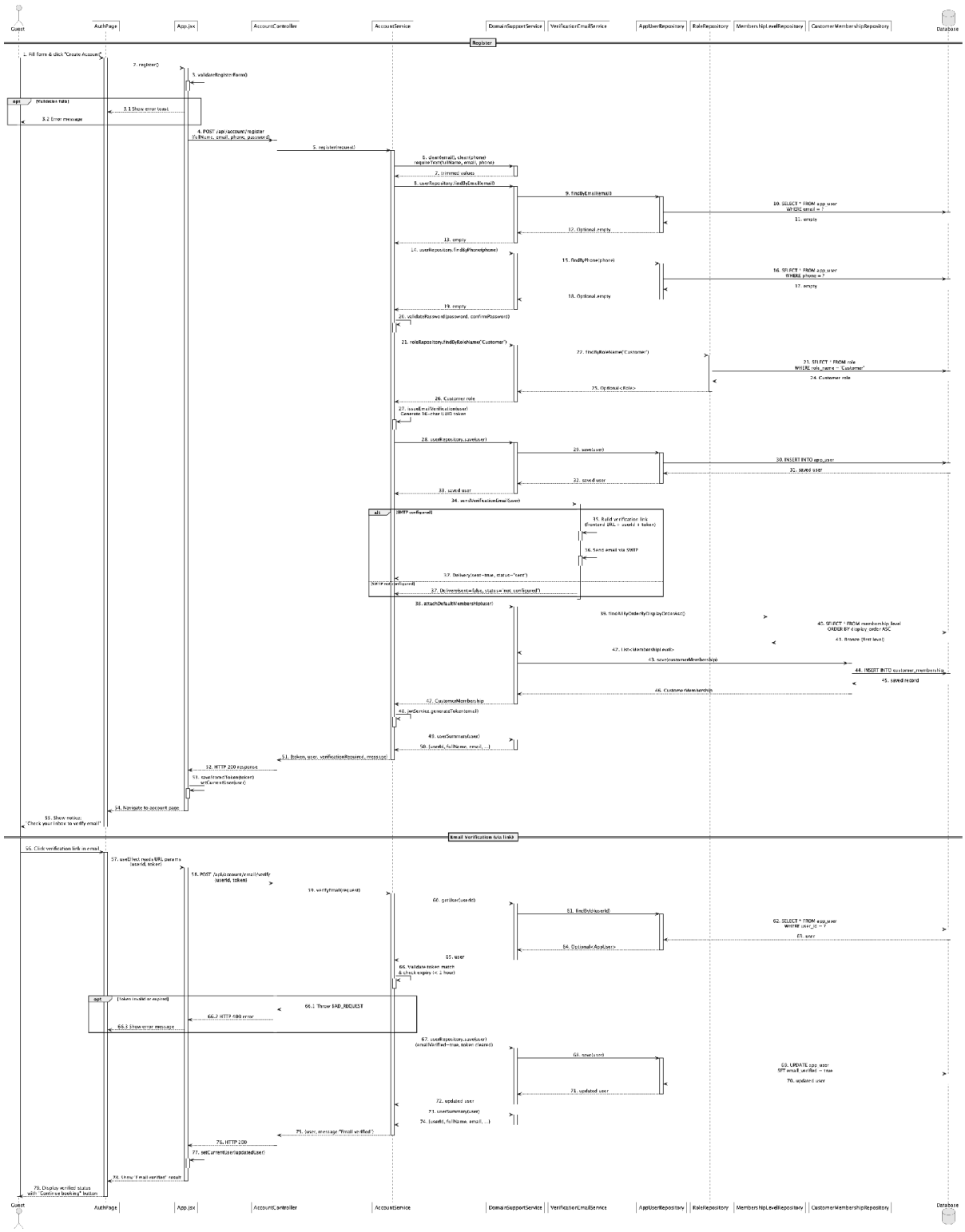
AppUserRepository

No	Method	Description
01	findByEmail(email : String) : Optional<AppUser>	Loads an account and role by unique email.
02	findByPhone(phone : String) : Optional<AppUser>	Loads an account and role by unique phone.
03	save(user : AppUser) : AppUser	Persists the new account.

RoleRepository

No	Method	Description
01	findByRoleName(roleName : String) : Optional<Role>	Loads the seeded Customer role.

c. Sequence Diagram



d. Database Queries

== Register account ==

1. 1. Check email uniqueness

```
SELECT u.*, r.* FROM app_user u
LEFT JOIN role r ON r.role_id = u.role_id
WHERE u.email = ?;
```

2. 2. Check phone uniqueness

```
SELECT u.*, r.* FROM app_user u
LEFT JOIN role r ON r.role_id = u.role_id
WHERE u.phone = ?;
```

3. 3. Load Customer role

```
SELECT * FROM role WHERE role_name = 'Customer';
```

4. 4. Insert Customer account

```
INSERT INTO app_user
(role_id, full_name, email, phone, password_hash, status,
 auth_version, email_verified, email_verification_token, email_verification_sent_at, created_at, updated_at)
VALUES (?, ?, ?, ?, ?, 'active', 0, false, ?, ?, ?, ?);
```

5. 5. Attach default membership

```
SELECT * FROM membership_level ORDER BY display_order ASC;
INSERT INTO customer_membership
(customer_id, membership_level_id, completed_booking_count, progress_note, effective_from, created_at, updated_at)
VALUES (?, ?, 0, 'Default membership', ?, ?, ?);
```

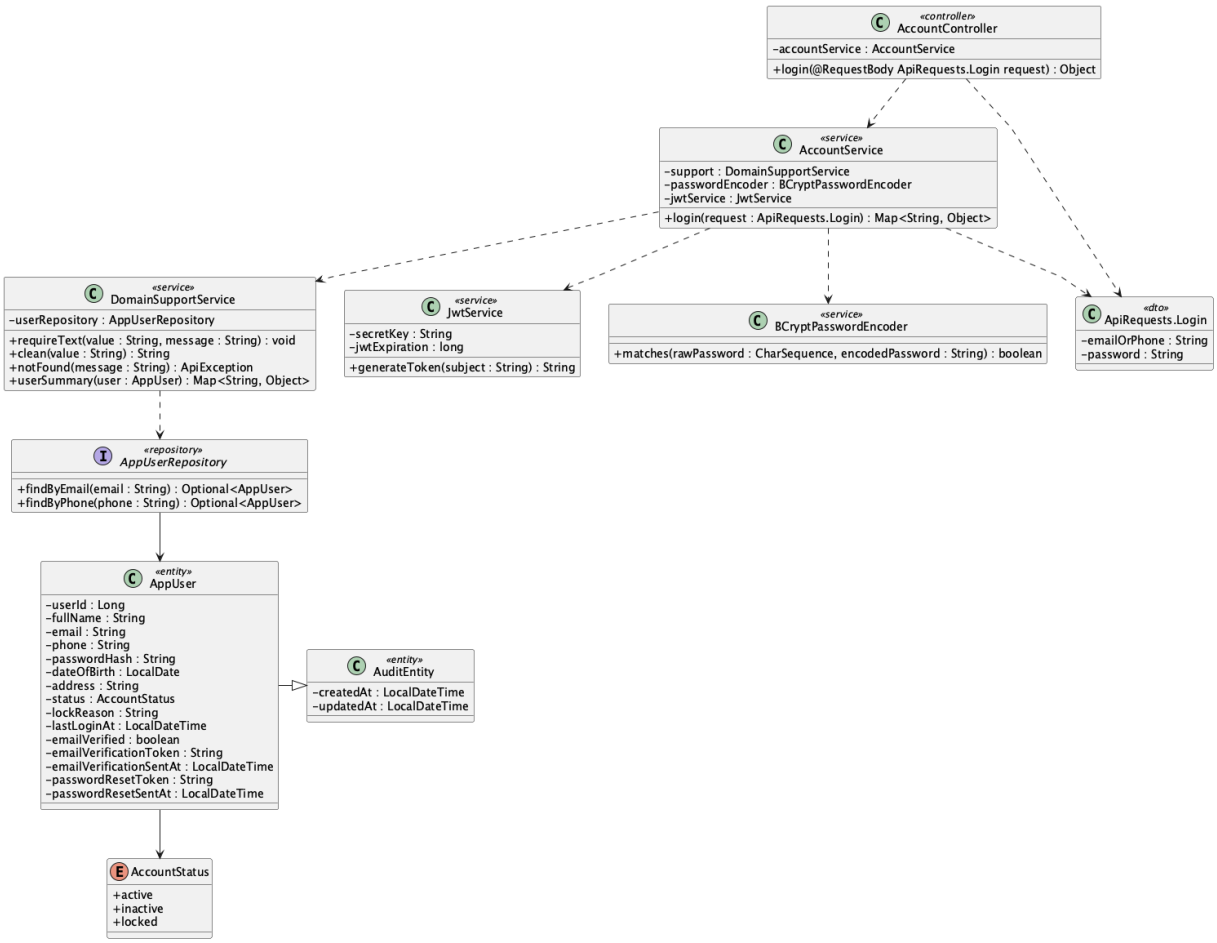
6. 6. Verify email

```
SELECT * FROM app_user WHERE user_id = ?;
UPDATE app_user SET email_verified = true, email_verification_token = NULL,
```

email_verification_sent_at = NULL, updated_at = ? WHERE user_id = ?;

2. Login

a. Class Diagram



b. Class Specifications

AccountController

No	Method	Description
01	login(request : ApiRequests.Login) : Object	Accepts email-or-phone credentials and returns the service response.

AccountService

No	Method	Description
01	login(request : ApiRequests.Login) : Map<String, Object>	Loads the account, checks BCrypt credentials and active status, records login time, and returns JWT/user data.

BCryptPasswordEncoder

No	Method	Description
01	matches(rawPassword : CharSequence, encodedPassword : String) : boolean	Compares the submitted password with the stored BCrypt hash.

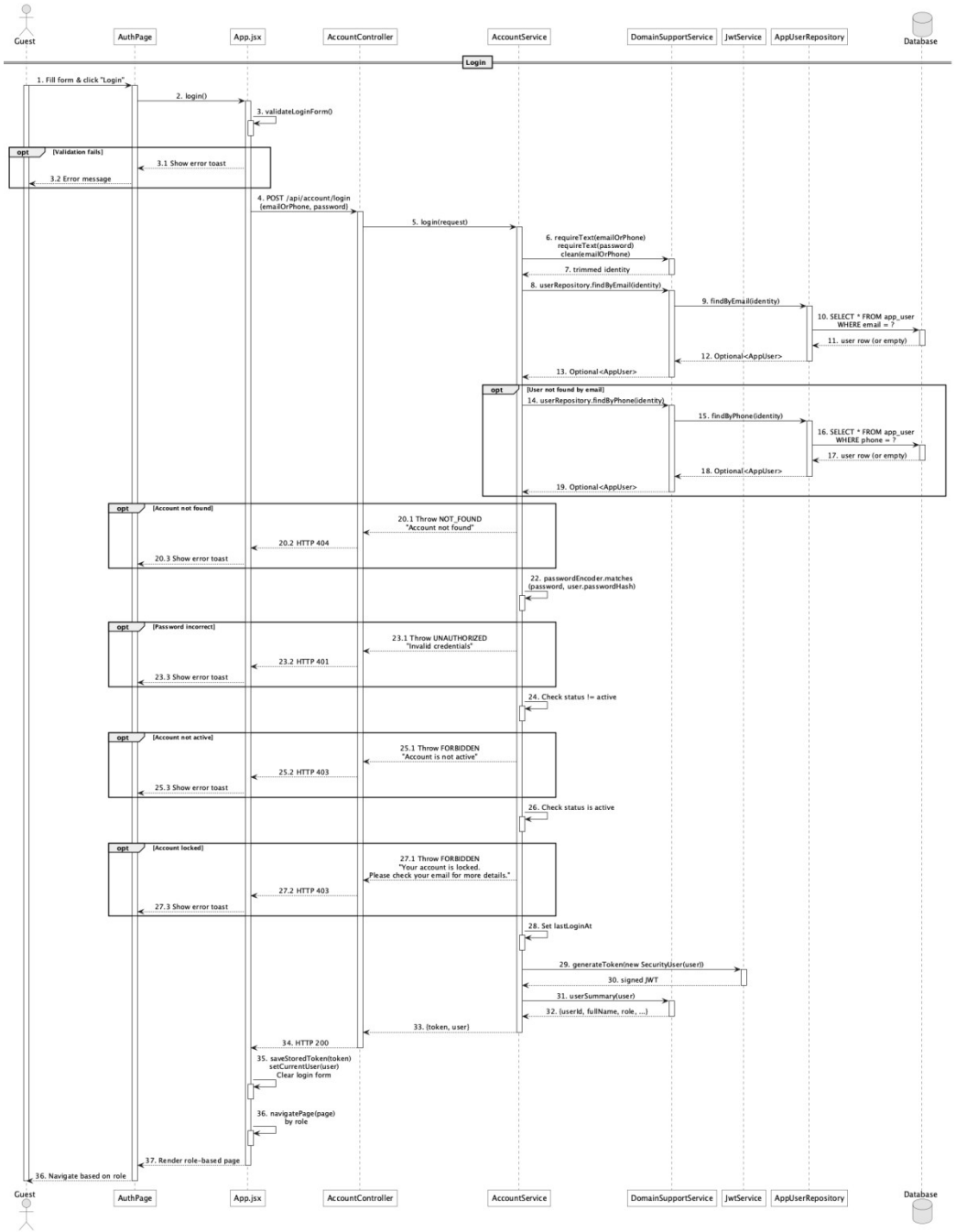
JwtService

No	Method	Description
01	generateToken(user : SecurityUser) : String	Issues a JWT with username, role, and current auth version.

AppUserRepository

No	Method	Description
01	findByEmail(email : String) : Optional<AppUser>	Finds the account by email with role loaded.
02	findByPhone(phone : String) : Optional<AppUser>	Finds the account by phone when email lookup fails.

c. Sequence Diagram



d. Database Queries

== Login ==

1. 1. Find account by email

SELECT u.*, r.* FROM app_user u LEFT JOIN role r ON r.role_id = u.role_id

WHERE u.email = ?;

2. 2. Find account by phone

SELECT u.*, r.* FROM app_user u LEFT JOIN role r ON r.role_id = u.role_id

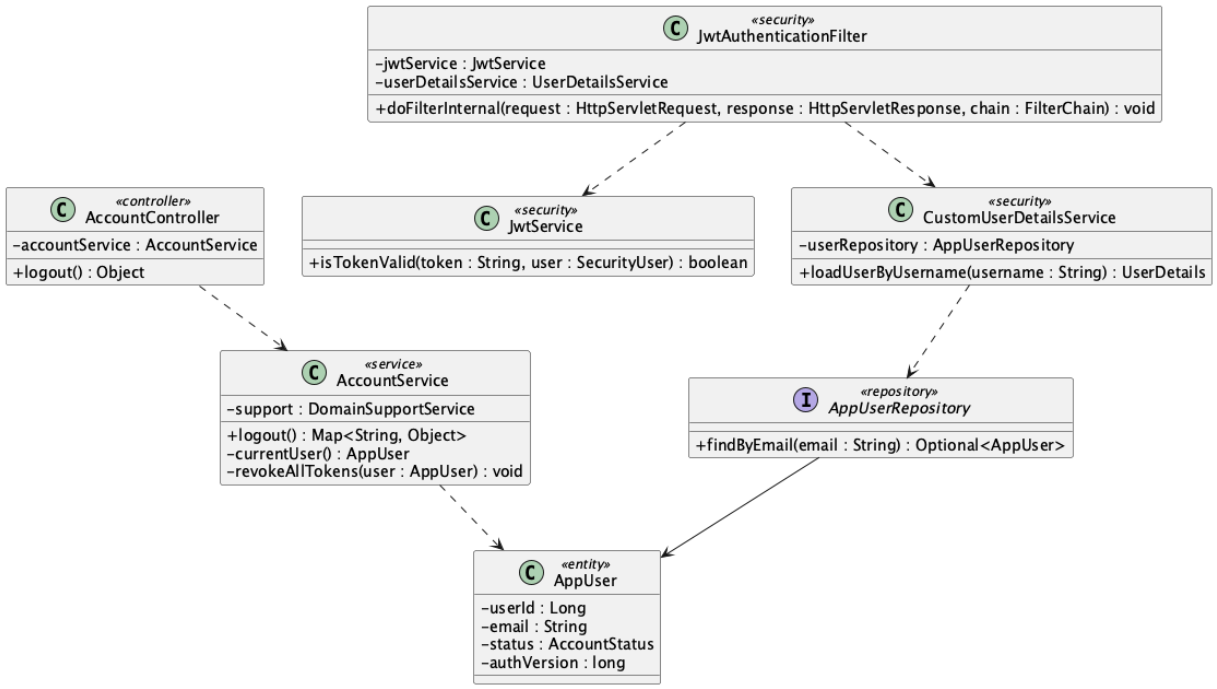
WHERE u.phone = ?;

3. 3. Record successful login

UPDATE app_user SET last_login_at = ?, updated_at = ? WHERE user_id = ?;

3. Logout

a. Class Diagram



b. Class Specifications

AccountController

No	Method	Description
01	logout() : Object	Requires authentication and delegates server-side token revocation.

AccountService

No	Method	Description
01	logout() : Map<String, Object>	Resolves the current account, increments auth version, and returns sign-out confirmation.

No	Method	Description
02	revokeAllTokens(user : AppUser) : void	Invalidates all JWTs issued with the previous auth-version value.

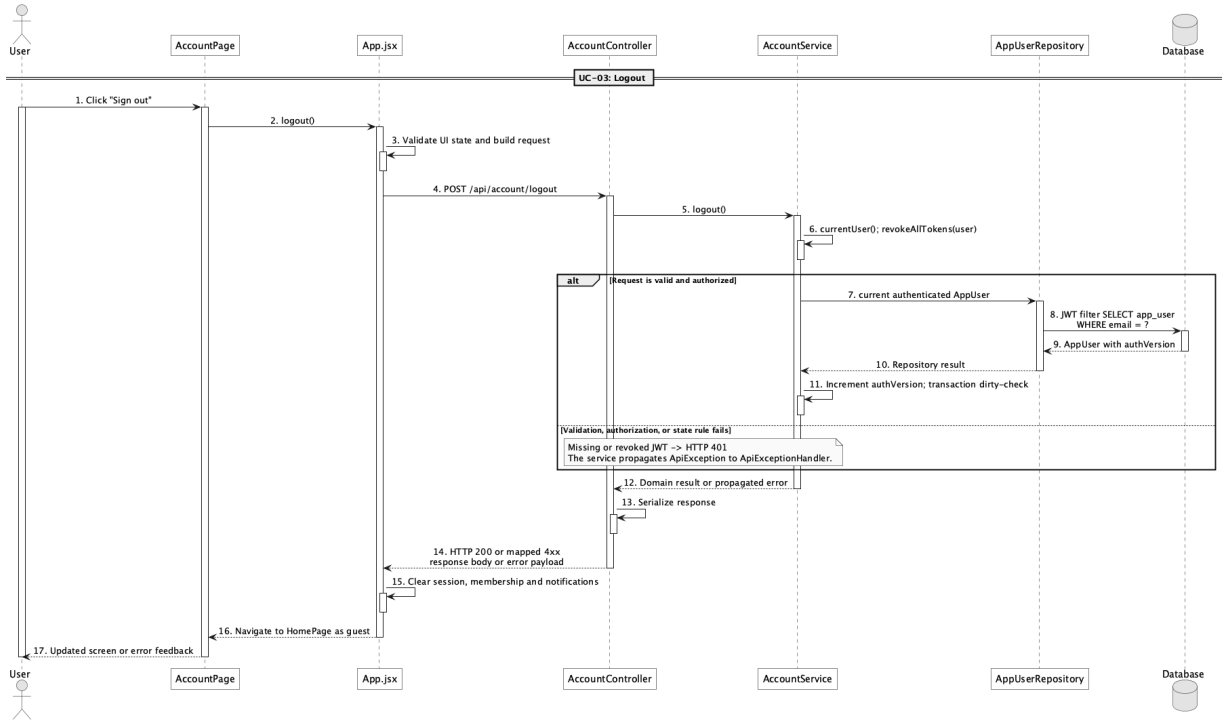
JwtAuthenticationFilter

No	Method	Description
01	doFilterInternal(request, response, chain) : void	Rejects a later request when the token auth-version no longer matches the database account.

JwtService

No	Method	Description
01	isTokenValid(token : String, user : SecurityUser) : boolean	Checks subject, expiration, and auth-version claim.

c. Sequence Diagram



d. Database Queries

== Logout ==

1. 1. Load authenticated account

SELECT u.*, r.* FROM app_user u LEFT JOIN role r ON r.role_id = u.role_id

WHERE u.email = ?;

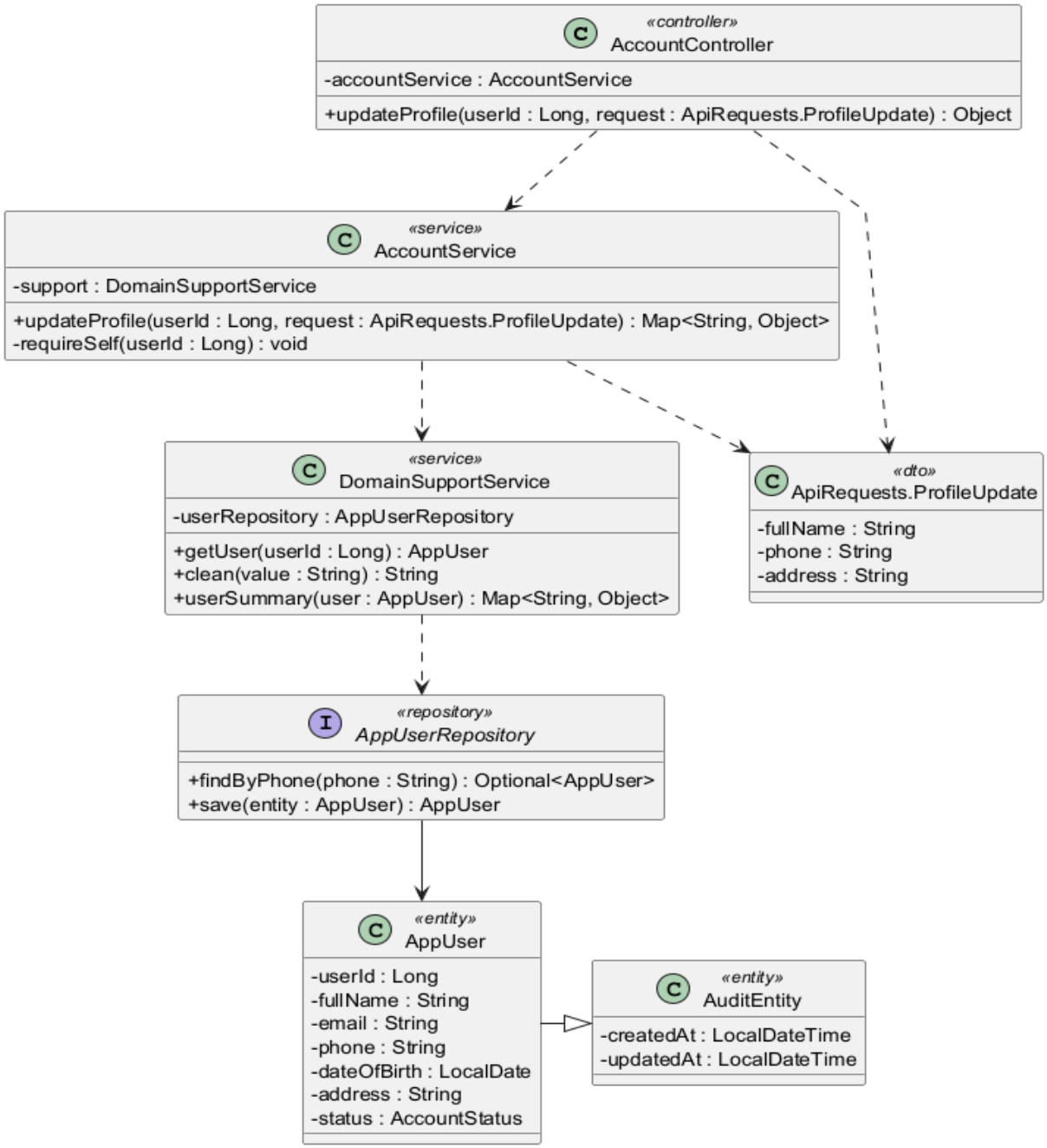
2. 2. Revoke issued JWTs

UPDATE app_user SET auth_version = auth_version + 1, updated_at = ?

WHERE user_id = ?;

4. Manage personal profile

a. Class Diagram



b. Class Specifications

AccountController

No	Method	Description
01	updateProfile(userId : Long, request : ApiRequests.ProfileUpdate) : Object	Receives allowed profile changes for the authenticated user's own account.

AccountService

No	Method	Description
01	updateProfile(userId : Long, request : ApiRequests.ProfileUpdate) : Map<String, Object>	Checks self-or-Admin access, validates phone uniqueness, and updates full name, phone, and address.
02	requireSelf(userId : Long) : void	Prevents a non-Admin user from updating another account.

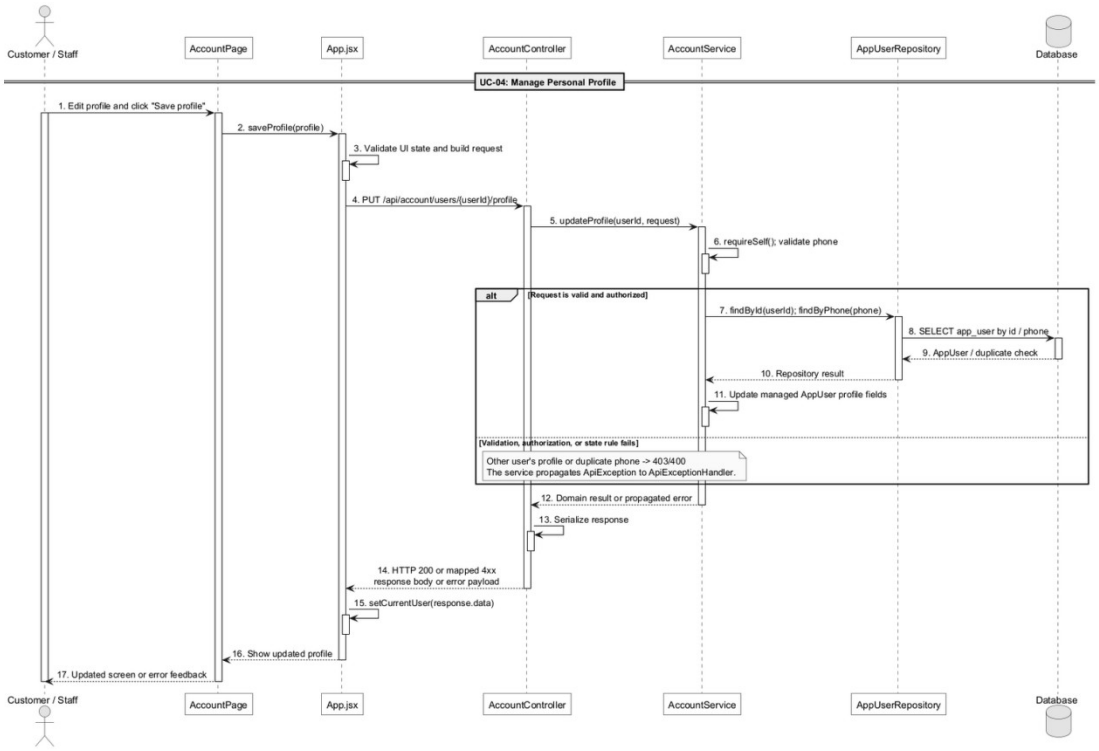
DomainSupportService

No	Method	Description
01	getUser(userId : Long) : AppUser	Loads the target account or raises not-found.
02	clean(value : String) : String	Normalizes optional profile input.
03	userSummary(user : AppUser) : Map<String, Object>	Returns the updated safe profile representation.

AppUserRepository

No	Method	Description
01	findByPhone(phone : String) : Optional<AppUser>	Checks that a replacement phone is not owned by another account.

c. Sequence Diagram



d. Database Queries

== Manage personal profile ==

1. 1. Load profile

SELECT u.*, r.* FROM app_user u LEFT JOIN role r ON r.role_id = u.role_id

WHERE u.user_id = ?;

2. 2. Check replacement phone

SELECT * FROM app_user WHERE phone = ? AND user_id <> ?;

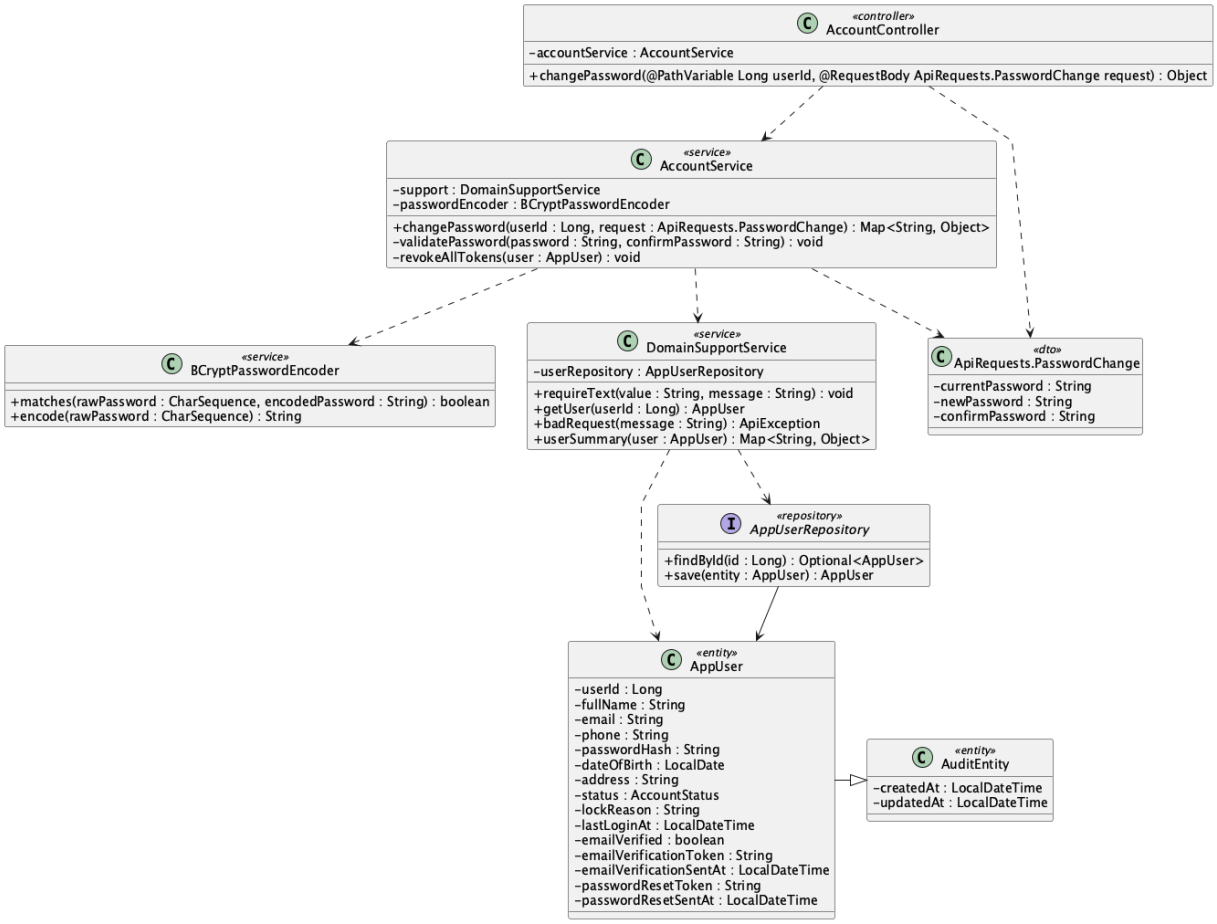
3. 3. Update allowed profile fields

UPDATE app_user SET full_name = ?, phone = ?, address = ?, updated_at = ?

WHERE user_id = ?;

5. Change password

a. Class Diagram



b. Class Specifications

AccountController

No	Method	Description
01	changePassword(userId : Long, request : ApiRequests.PasswordChange) : Object	Accepts current and new password data for the authenticated account.

AccountService

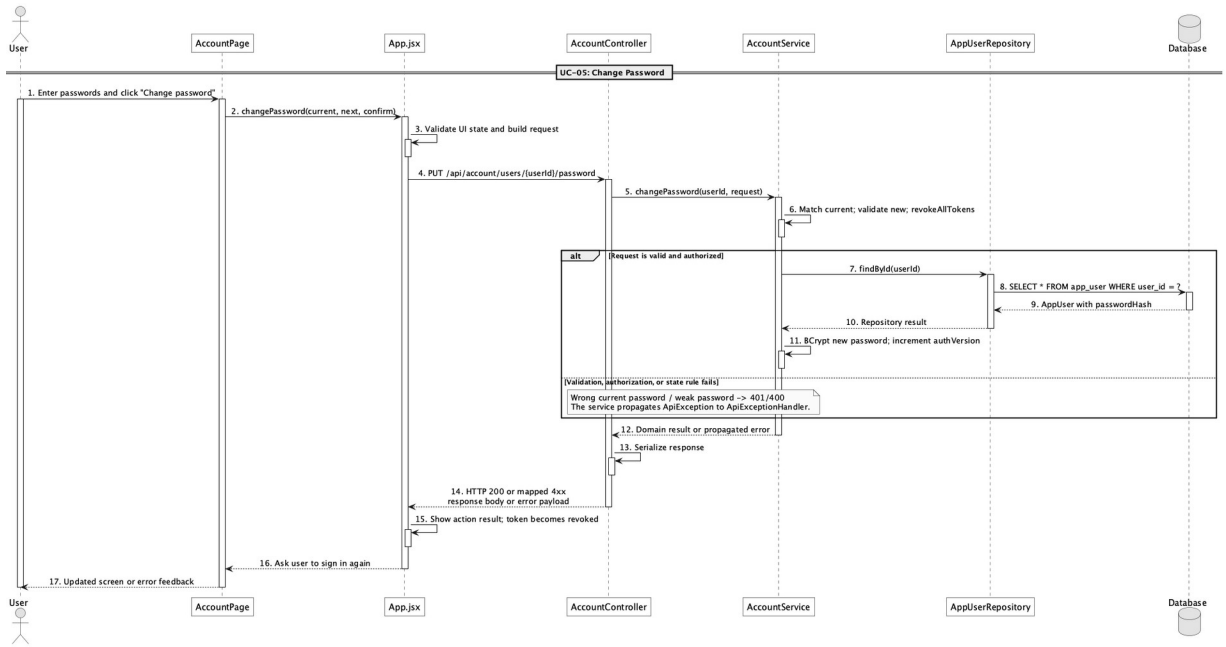
No	Method	Description
01	changePassword(userId : Long, request : ApiRequests.PasswordChange) : Map<String, Object>	Checks current password, validates a different new password, stores its hash, and revokes tokens.
02	validatePassword(password : String, confirmPassword : String) : void	Enforces the shared password policy and confirmation.

No	Method	Description
03	revokeAllTokens(user : AppUser) : void	Increments auth version after the password changes.

BCryptPasswordEncoder

No	Method	Description
01	matches(rawPassword : CharSequence, encodedPassword : String) : boolean	Checks current/unchanged password conditions.
02	encode(rawPassword : CharSequence) : String	Produces the replacement BCrypt hash.

c. Sequence Diagram



d. Database Queries

== Change password ==

1. 1. Load account for credential check

SELECT * FROM app_user WHERE user_id = ?;

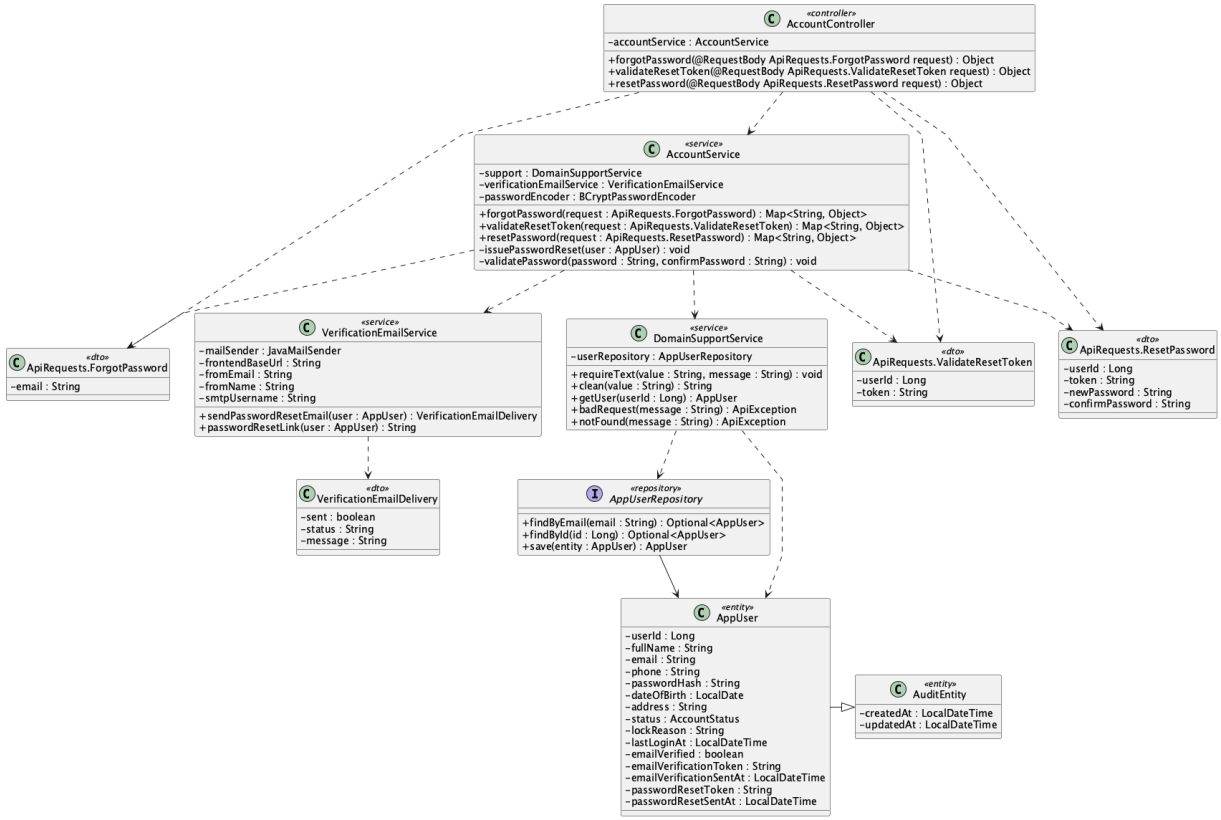
2. 2. Store password and revoke tokens

UPDATE app_user SET password_hash = ?, auth_version = auth_version + 1, updated_at = ?

WHERE user_id = ?;

6. Forgot/reset password

a. Class Diagram



b. Class Specifications

AccountController

No	Method	Description
01	forgotPassword(request : ApiRequests.ForgotPassword) : Object	Starts the neutral, anti-enumeration reset-request flow.
02	validateResetToken(request : ApiRequests.ValidateResetToken) : Object	Validates user ID, token, and one-hour age.
03	resetPassword(request : ApiRequests.ResetPassword) : Object	Applies a valid new password and completes the reset.

AccountService

No	Method	Description
01	forgotPassword(request : ApiRequests.ForgotPassword) : Map<String, Object>	Issues and emails a token only for an active matching account while returning a neutral message in every case.
02	validateResetToken(request :	Rejects a mismatched or expired reset token.

No	Method	Description
	ApiRequests.ValidateResetToken) : Map<String, Object>	
03	resetPassword(request : ApiRequests.ResetPassword) : Map<String, Object>	Validates the token/password, clears reset state, and revokes prior JWTs.
04	issuePasswordReset(user : AppUser) : void	Stores a new 16-character token and issue time.

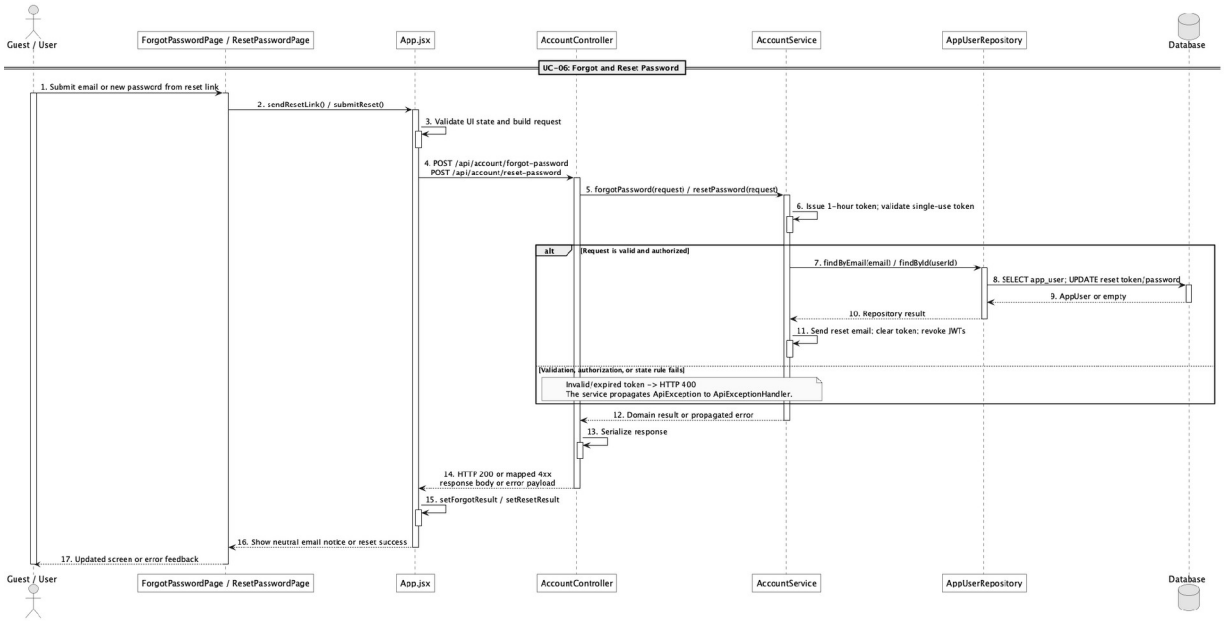
VerificationEmailService

No	Method	Description
01	sendPasswordResetEmail(user : AppUser) : VerificationEmailDelivery	Sends the reset link to the registered email.
02	passwordResetLink(user : AppUser) : String	Builds the frontend reset URL from user ID and token.

AppUserRepository

No	Method	Description
01	findByEmail(email : String) : Optional<AppUser>	Finds an eligible reset account without changing the public response.
02	save(user : AppUser) : AppUser	Persists issued reset data.

c. Sequence Diagram



d. Database Queries

== Forgot/reset password ==

1. 1. Find reset account

```
SELECT u.*, r.* FROM app_user u LEFT JOIN role r ON r.role_id = u.role_id
WHERE u.email = ?;
```

2. 2. Issue reset token

```
UPDATE app_user SET password_reset_token = ?, password_reset_sent_at = ?, updated_at = ?
WHERE user_id = ? AND status = 'active';
```

3. 3. Validate reset link

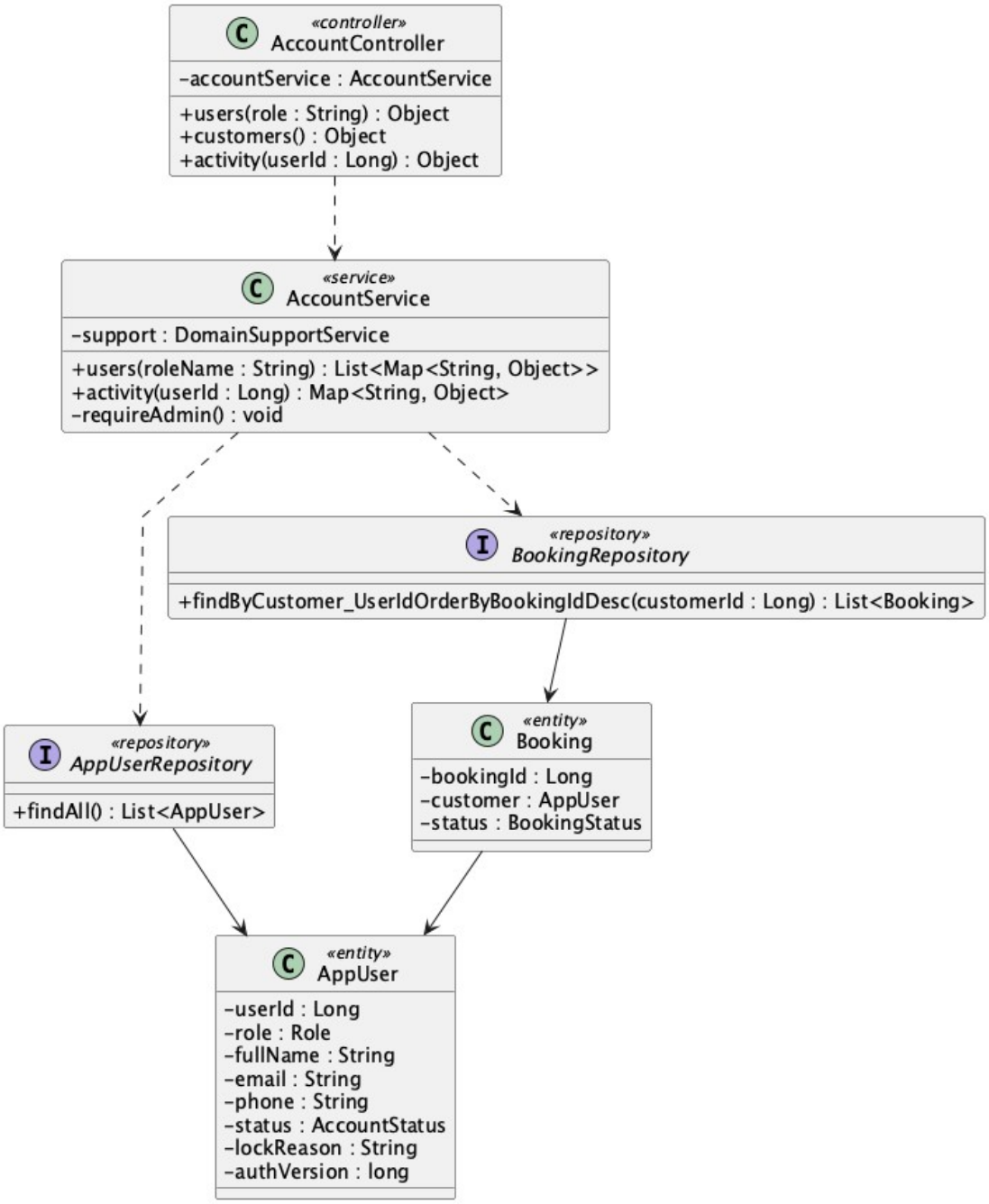
```
SELECT * FROM app_user WHERE user_id = ? AND password_reset_token = ?
AND password_reset_sent_at > ?;
```

4. 4. Complete password reset

```
UPDATE app_user SET password_hash = ?, password_reset_token = NULL,
password_reset_sent_at = NULL, auth_version = auth_version + 1, updated_at = ?
WHERE user_id = ?;
```

7. Manage customer accounts

a. Class Diagram



b. Class Specifications

AccountController

No	Method	Description
01	users(role : String) : Object	Returns an Admin-only account list optionally filtered by role.
02	activity(userId : Long) : Object	Returns selected customer activity details.

AccountService

No	Method	Description
01	users(roleName : String) : List<Map<String, Object>>	Checks Admin role, loads accounts, and applies the requested role filter.
02	activity(userId : Long) : Map<String, Object>	Builds the customer's recent operational history.

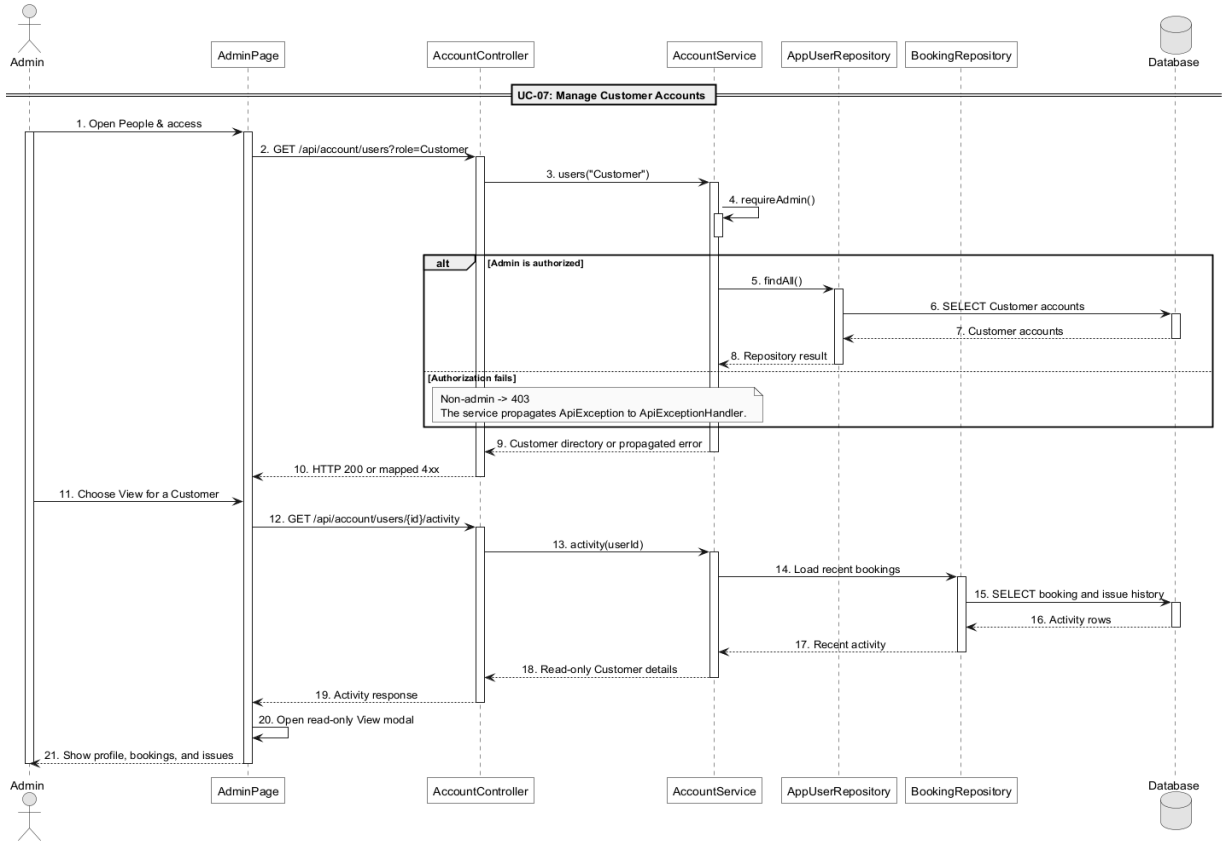
AppUserRepository

No	Method	Description
01	findAll() : List<AppUser>	Loads accounts for the Admin list.

BookingRepository

No	Method	Description
01	findByCustomer_UserIdOrderByBookingIdDesc(customerId : Long) : List<Booking>	Loads the customer's bookings for activity details.

c. Sequence Diagram



d. Database Queries

== Manage customer accounts ==

List Customer accounts

SELECT u.*, r.* FROM app_user u JOIN role r ON r.role_id = u.role_id

WHERE r.role_name = 'Customer' ORDER BY u.user_id DESC;

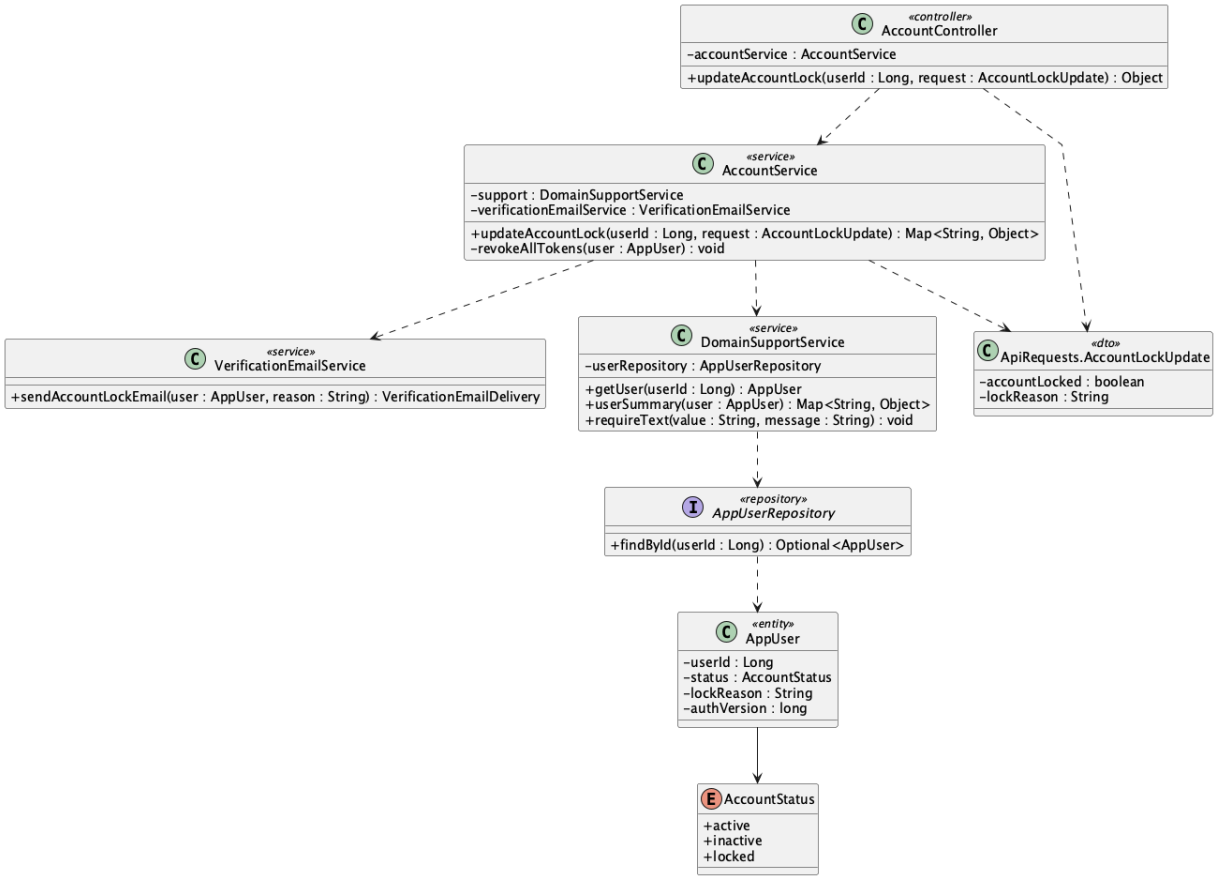
Read Customer activity

SELECT * FROM booking WHERE customer_id = ? ORDER BY booking_id DESC;

SELECT * FROM issue WHERE reporter_id = ? ORDER BY issue_id DESC;

e. Account lock/unlock design

e.1 Class Diagram



e.2 Class Specifications

AccountController

No	Method	Description
01	updateAccountLock(userId : Long, request : ApiRequests.AccountLockUpdate) : Object	Receives the Admin account lock/unlock decision for a Customer.

AccountService

No	Method	Description
01	updateAccountLock(userId : Long, request : ApiRequests.AccountLockUpdate) : Map<String, Object>	Checks Admin and Customer role, requires a reason when locking, stores locked/active state, revokes tokens, and reports email delivery.
02	requireAdmin() : void	Rejects non-Admin account-management requests.
03	revokeAllTokens(user : AppUser) : void	Invalidates every active Customer token immediately.

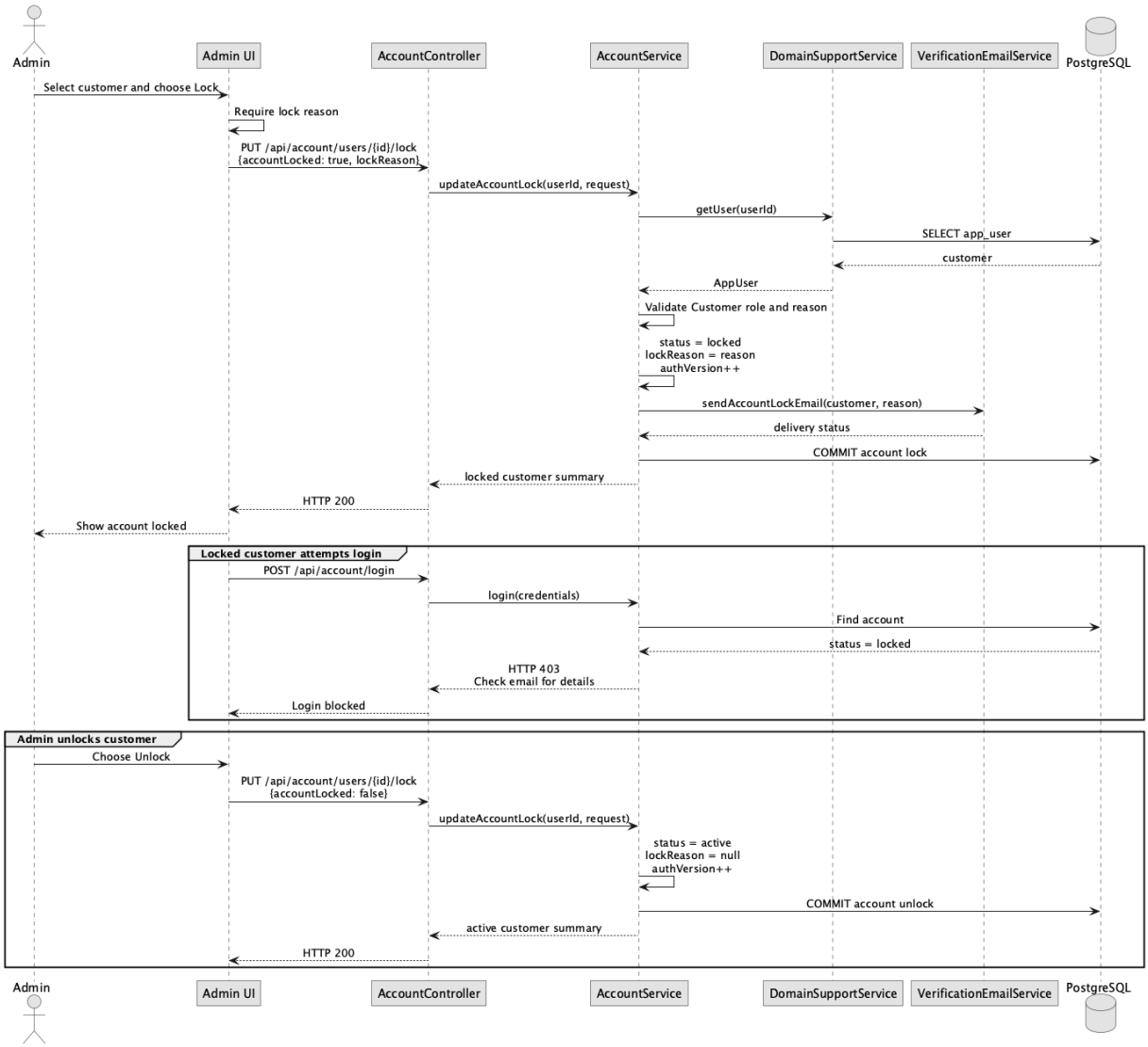
VerificationEmailService

No	Method	Description
01	sendAccountLockEmail(user : AppUser, reason : String) : VerificationEmailDelivery	Emails the Customer the account-lock reason and returns delivery status.
02	accountLockPlainText(user : AppUser, reason : String) : String	Builds the plain-text account-lock email.

DomainSupportService

No	Method	Description
01	getUser(userId : Long) : AppUser	Loads the selected target account.
02	clean(value : String) : String	Normalizes the lock reason.
03	requireText(value : String, message : String) : void	Requires a reason when the account is locked.
04	userSummary(user : AppUser) : Map<String, Object>	Returns the updated summary including accountLocked, lockReason, status, and authVersion.
05	badRequest(message : String) : ApiException	Builds a validation error for unsupported target accounts.

e.3 Sequence Diagram



e.4 Database Queries

== Lock/unlock customer account ==

1. Load and verify Customer account

SELECT u.*, r.* FROM app_user u JOIN role r ON r.role_id = u.role_id

WHERE u.user_id = ? AND r.role_name = 'Customer';

2. Lock customer account

UPDATE app_user SET status = 'locked', lock_reason = ?,

auth_version = auth_version + 1, updated_at = ? WHERE user_id = ?;

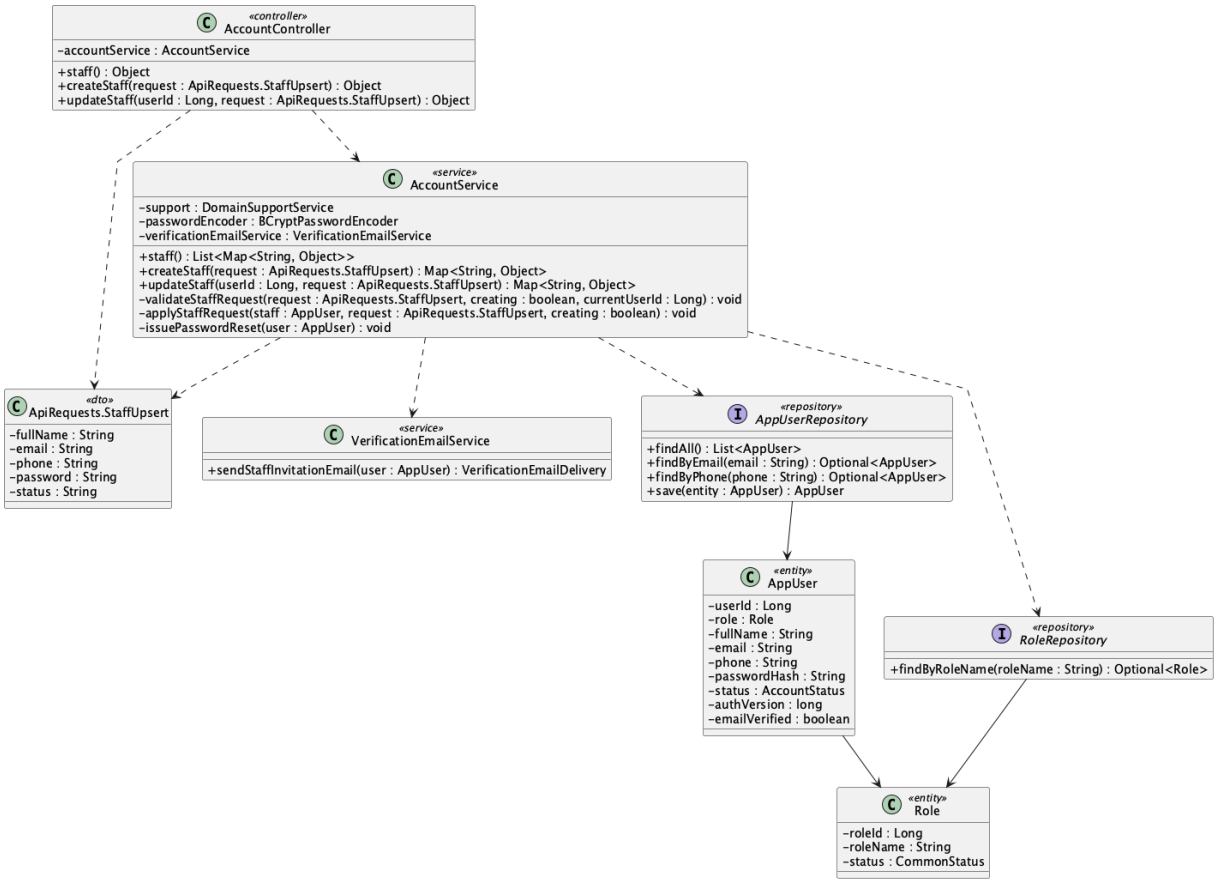
3. Unlock customer account

UPDATE app_user SET status = 'active', lock_reason = NULL,

auth_version = auth_version + 1, updated_at = ? WHERE user_id = ?;

8. Manage staff accounts

a. Class Diagram



b. Class Specifications

AccountController

No	Method	Description
01	staff() : Object	Returns the Admin-only Staff list.
02	createStaff(request : ApiRequests.StaffUpsert) : Object	Creates a Staff account from Admin input.
03	updateStaff(userId : Long, request : ApiRequests.StaffUpsert) : Object	Updates the selected Staff account.

AccountService

No	Method	Description
01	staff() : List<Map<String, Object>>	Loads and filters Staff accounts for Admin.
02	createStaff(request : ApiRequests.StaffUpsert) : Map<String, Object>	Creates the Staff identity with an unknown internal credential, issues a setup token, and reports invitation delivery.
03	updateStaff(userId : Long, request : ApiRequests.StaffUpsert) : Map<String, Object>	Updates Staff identity/status and rejects any Admin-submitted password change.
04	validateStaffRequest(request, creating, currentUserId) : void	Enforces required identity fields, formats, uniqueness, and role boundaries.
05	issuePasswordReset(user : AppUser) : void	Creates the one-hour token used by the Staff invitation link.

AppUserRepository

No	Method	Description
01	findAll() : List<AppUser>	Loads accounts for the Staff filter.
02	findByEmail(email : String) : Optional<AppUser>	Checks Staff email uniqueness.
03	save(user : AppUser) : AppUser	Persists a newly provisioned Staff account.

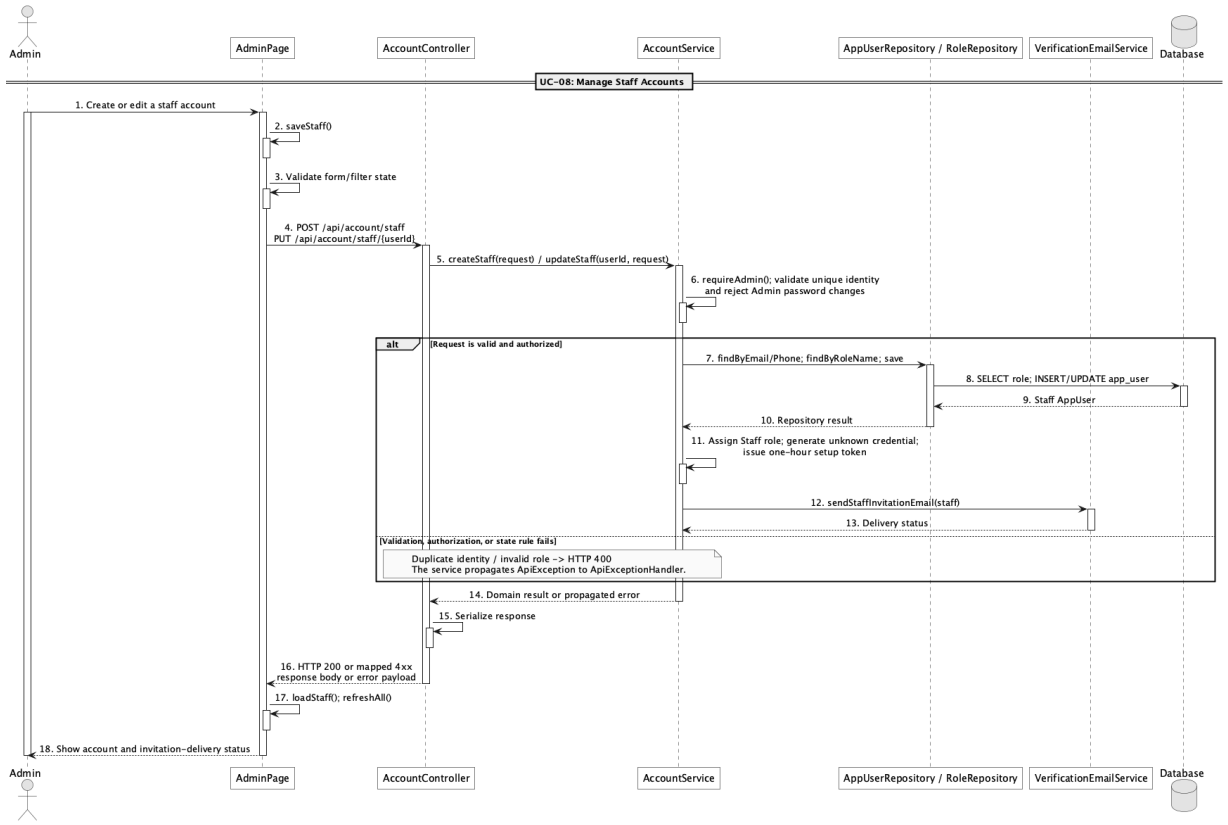
RoleRepository

No	Method	Description
01	findByRoleName(roleName : String) : Optional<Role>	Loads the seeded Staff role.

VerificationEmailService

No	Method	Description
01	sendStaffInvitationEmail(user : AppUser) : VerificationEmailDelivery	Sends the Staff-owned password setup link and returns the delivery result.

c. Sequence Diagram



d. Database Queries

== Manage staff accounts ==

1. 1. List Staff accounts

SELECT u.*, r.* FROM app_user u JOIN role r ON r.role_id = u.role_id

WHERE r.role_name = 'Staff' ORDER BY u.user_id DESC;

2. 2. Check Staff identity uniqueness

SELECT * FROM app_user WHERE (email = ? OR phone = ?) AND user_id <> ?;

3. 3. Create Staff account and setup token

INSERT INTO app_user

(role_id, full_name, email, phone, password_hash, status, auth_version, email_verified, password_reset_token, password_reset_sent_at, created_at, updated_at)

VALUES (?, ?, ?, ?, ?, ?, 0, true, ?, ?, ?, ?);

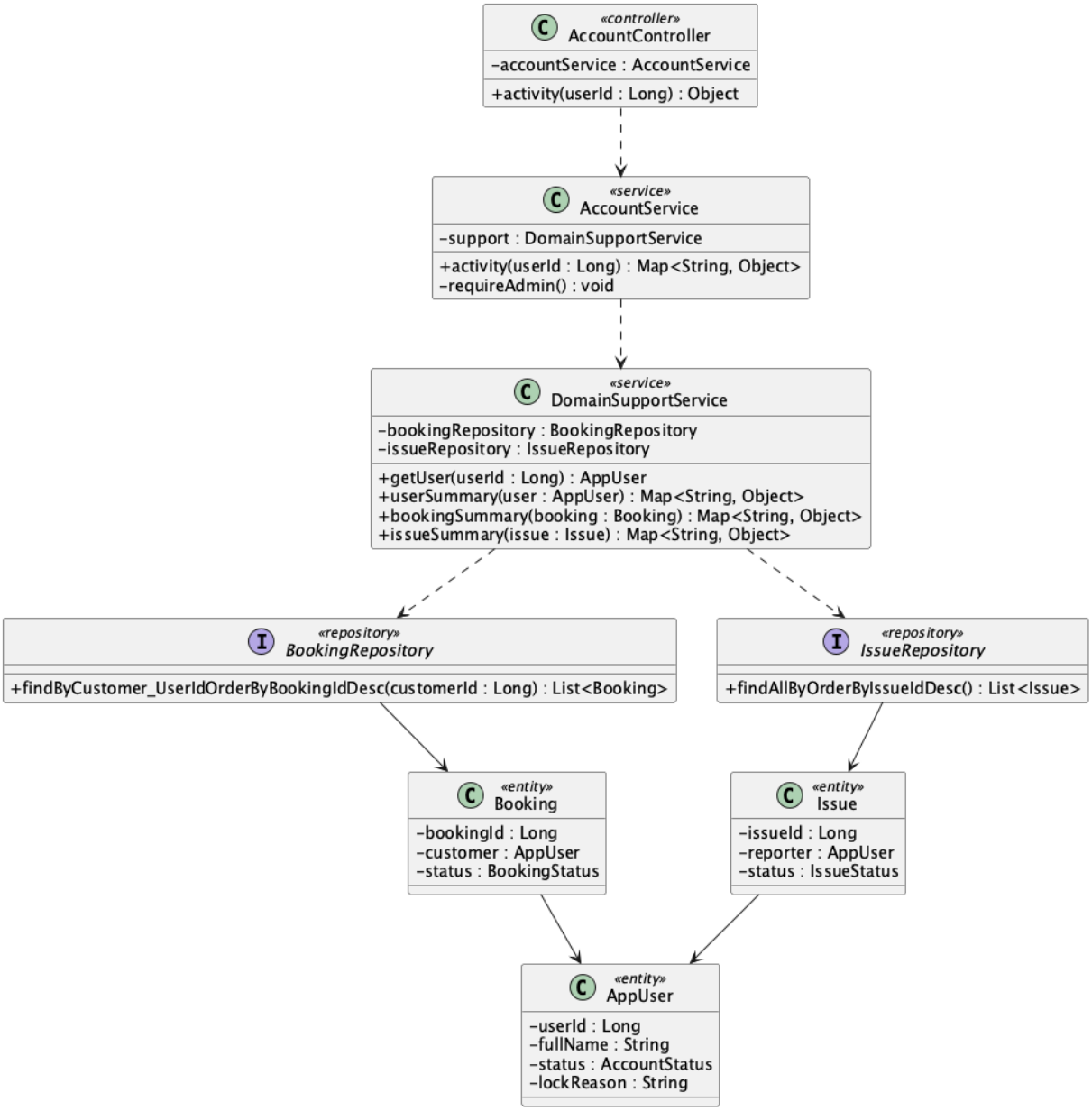
4. 4. Update Staff identity or status

UPDATE app_user SET full_name = ?, email = ?, phone = ?, status = ?, auth_version = ?, updated_at = ?

WHERE user_id = ?;

9. View customer activity status

a. Class Diagram



b. Class Specifications

AccountController

No	Method	Description
01	activity(userId : Long) : Object	Returns the selected customer's operational activity

No	Method	Description
		response.

AccountService

No	Method	Description
01	activity(userId : Long) : Map<String, Object>	Checks Admin access and aggregates Customer details, recent bookings, completed count, and recent issues.
02	requireAdmin() : void	Rejects Customer activity requests from non-Admin accounts.

BookingRepository

No	Method	Description
01	findByCustomer_UserIdOrderByBookingIdDesc(customerId : Long) : List<Booking>	Loads recent bookings and supplies the completed-booking count source.

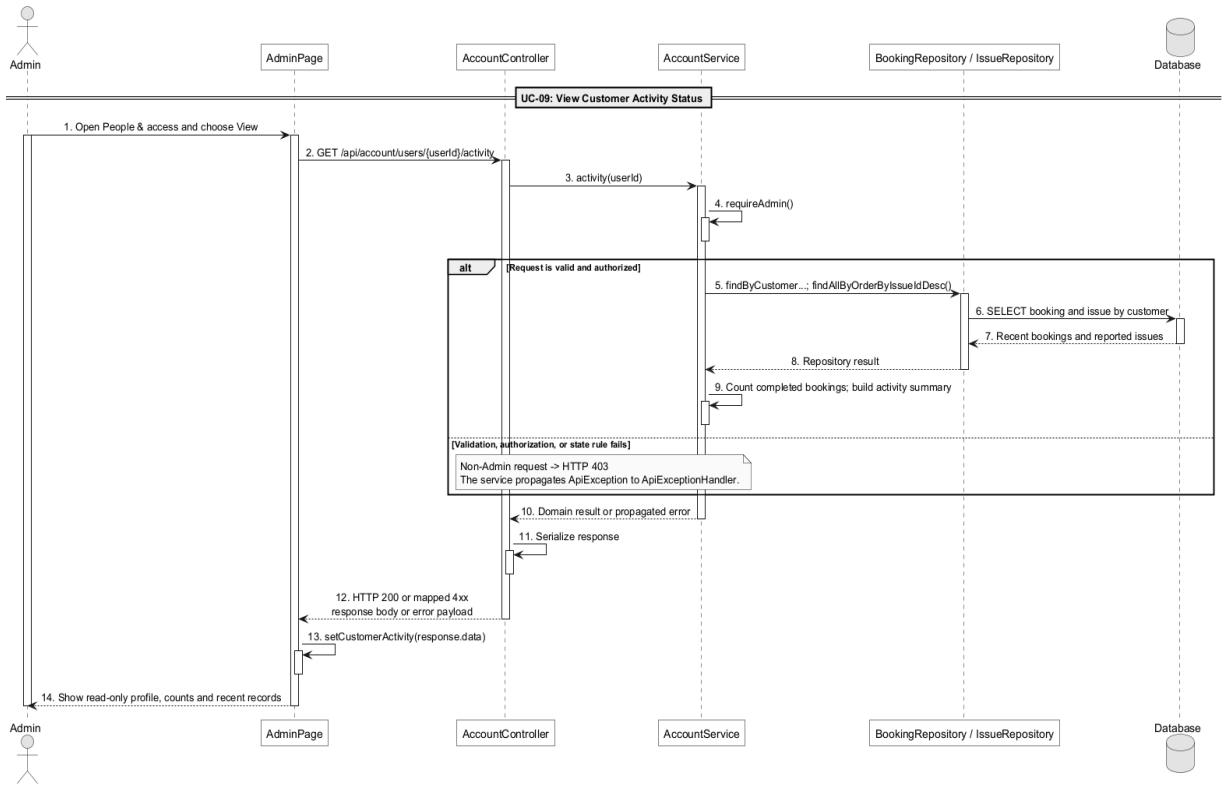
IssueRepository

No	Method	Description
01	findAllByOrderByIssueIdDesc() : List<Issue>	Loads issues before filtering to the selected reporter and limiting the response.

DomainSupportService

No	Method	Description
01	bookingSummary(booking : Booking) : Map<String, Object>	Builds each safe booking row.
02	issueSummary(issue : Issue) : Map<String, Object>	Builds each safe issue row.
03	userSummary(user : AppUser) : Map<String, Object>	Builds customer and membership summary data.

c. Sequence Diagram



d. Database Queries

== View customer activity status ==

1. 1. Load Customer summary

SELECT u.*, r.* FROM app_user u JOIN role r ON r.role_id = u.role_id

WHERE u.user_id = ?;

2. 2. Load recent bookings

SELECT * FROM booking WHERE customer_id = ? ORDER BY booking_id DESC LIMIT 20;

3. 3. Count completed bookings

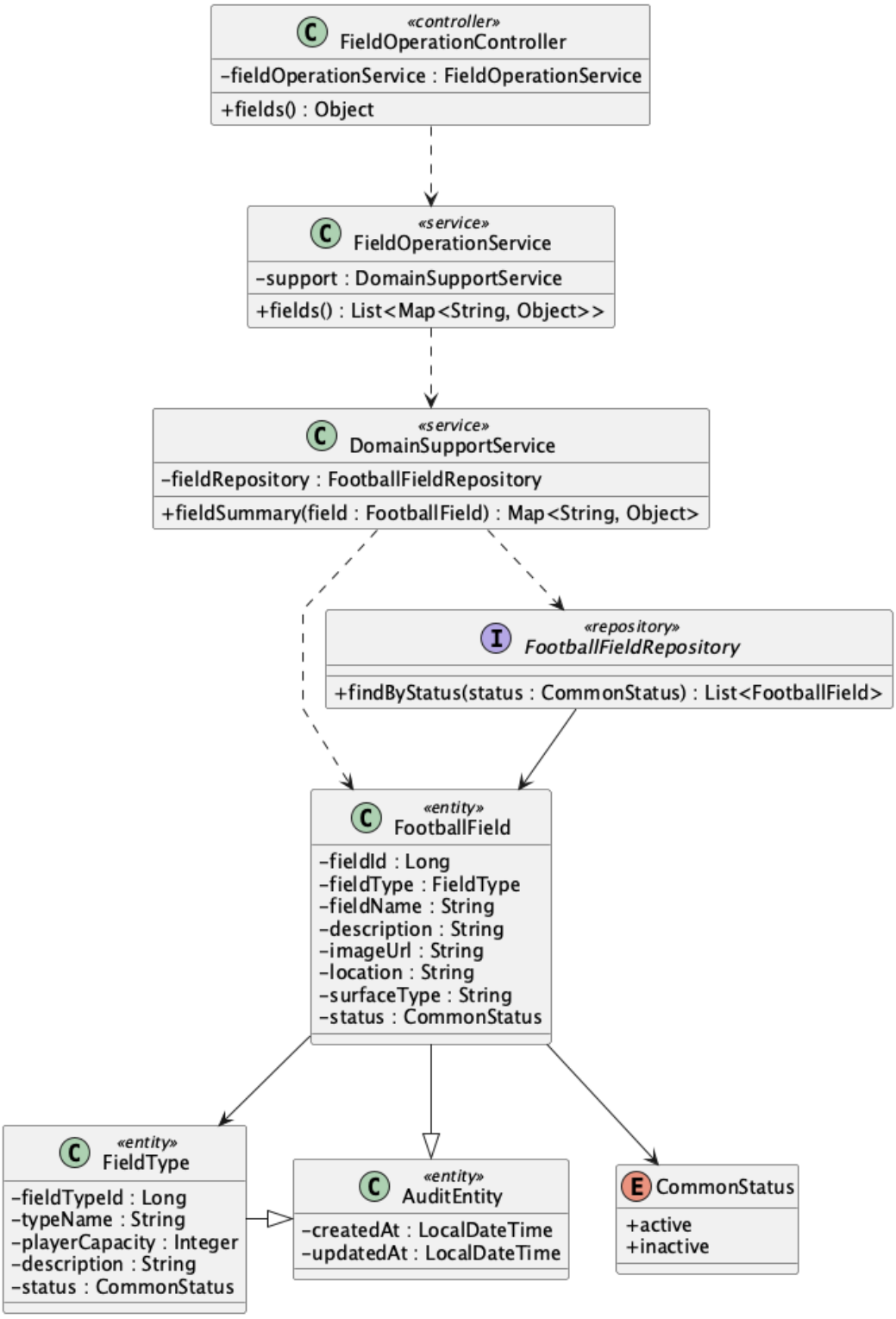
SELECT COUNT(*) FROM booking WHERE customer_id = ? AND status = 'completed';

4. 4. Load recent reported issues

SELECT * FROM issue WHERE reporter_id = ? ORDER BY issue_id DESC LIMIT 20;

10. View field list

a. Class Diagram



b. Class Specifications

FieldOperationController

No	Method	Description
01	public Object fields()	Exposes GET /api/fields for the public catalogue.

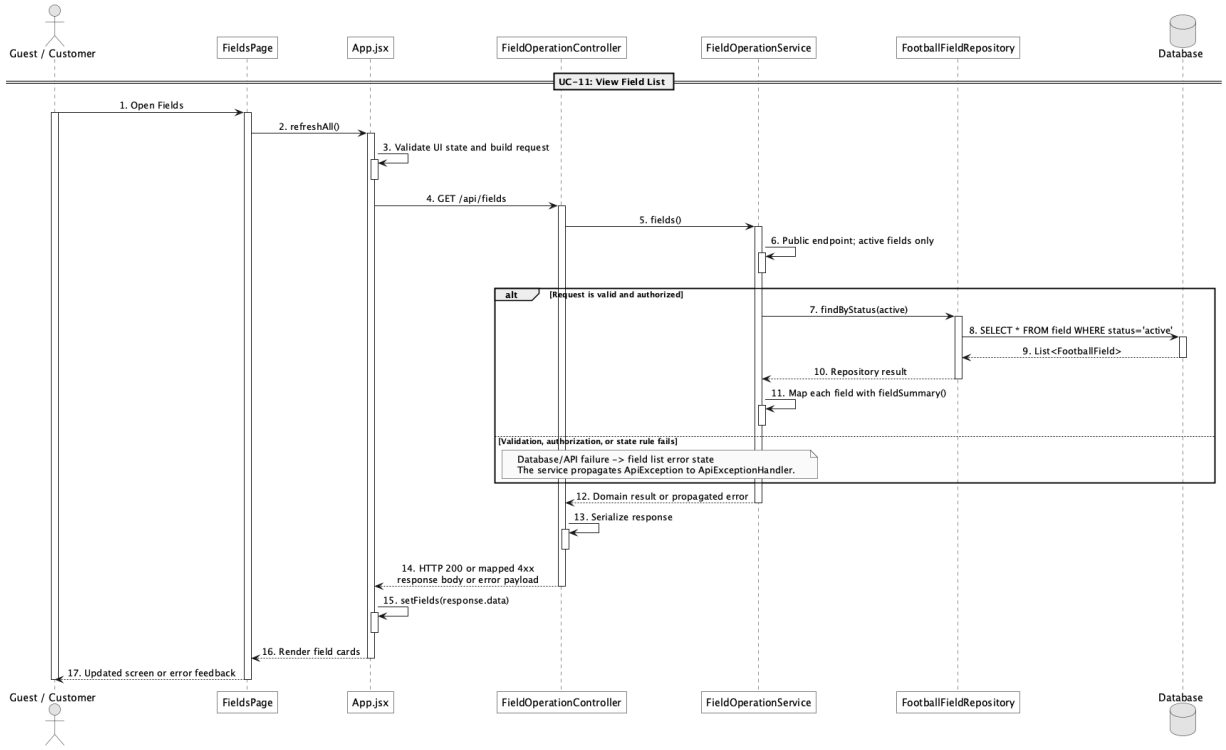
FieldOperationService

No	Method	Description
01	public List<Map<String, Object>> fields()	Loads active fields and maps them to catalogue summaries.

FootballFieldRepository

No	Method	Description
01	List<FootballField> findByStatus(CommonStatus status)	Spring Data lookup used to exclude inactive fields.

c. Sequence Diagram



d. Database Queries

== View field list ==

1. Active field catalogue

SELECT f.field_id, f.field_name, ft.type_name, ft.player_capacity,

f.location, f.surface_type, f.image_url, f.status

FROM field f

JOIN field_type ft ON ft.field_type_id = f.field_type_id

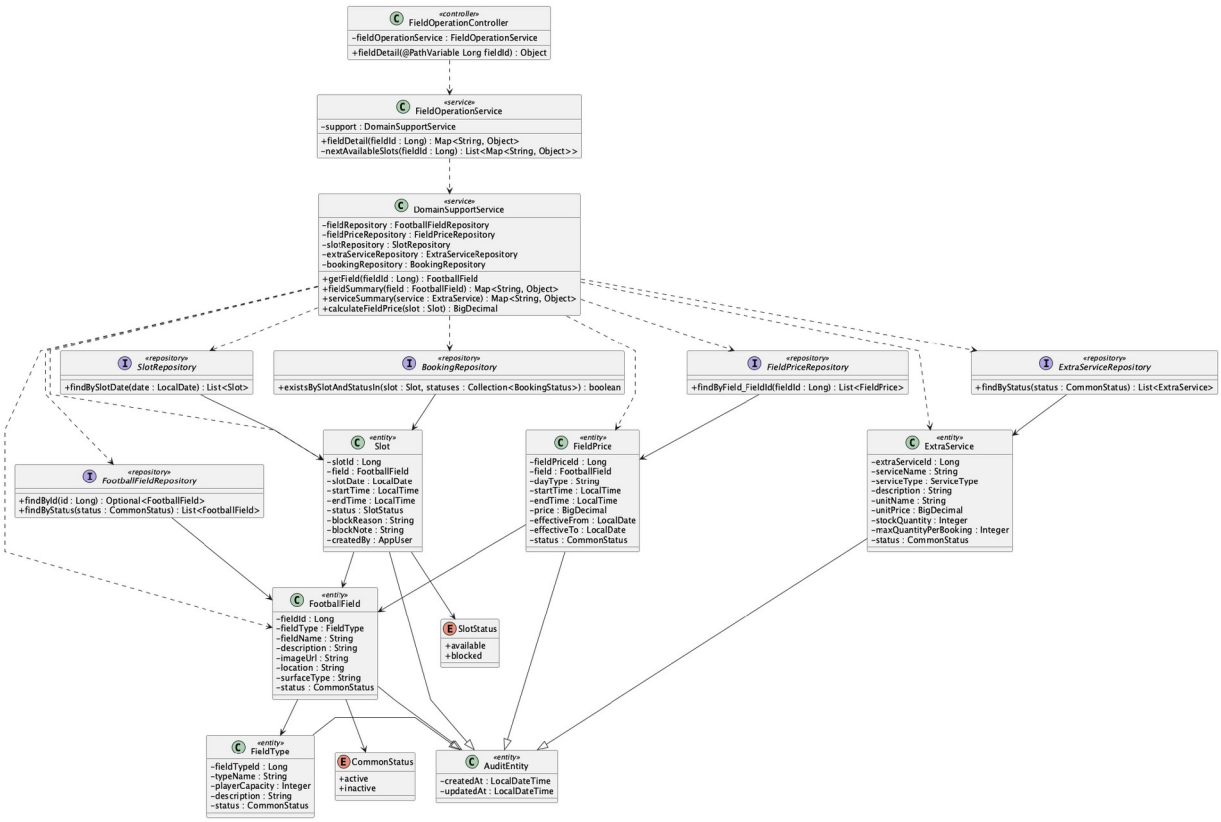
WHERE f.status = 'active'

ORDER BY f.field_id;

Implementation note: The physical table name is field; FootballField is the Java entity name.

11. View field detail

a. Class Diagram



b. Class Specifications

FieldOperationController

No	Method	Description
01	public Object fieldDetail(Long fieldId)	Exposes GET /api/fields/{fieldId}.

FieldOperationService

No	Method	Description
01	public Map<String, Object> fieldDetail(Long fieldId)	Combines field identity, active price bands, services, and next available slots.

DomainSupportService

No	Method	Description
01	FootballField getField(Long fieldId)	Loads the field or rejects an unknown identifier.
02	Map<String, Object> fieldSummary(FootballField field)	Maps the field and field-type attributes returned to the client.
03	Map<String, Object> serviceSummary(ExtraService service)	Maps one active add-on service for the detail response.
04	BigDecimal calculateFieldPrice(Slot slot)	Resolves the effective USD price band for an available slot.

FootballFieldRepository

No	Method	Description
01	Optional<FootballField> findById(Long fieldId)	Loads the selected field.

FieldPriceRepository

No	Method	Description
01	List<FieldPrice> findByField_FieldId(Long fieldId)	Loads price bands for the selected field.

SlotRepository

No	Method	Description
01	List<Slot> findBySlotDate(LocalDate slotDate)	Loads candidate slots for the next-eight-day availability summary.

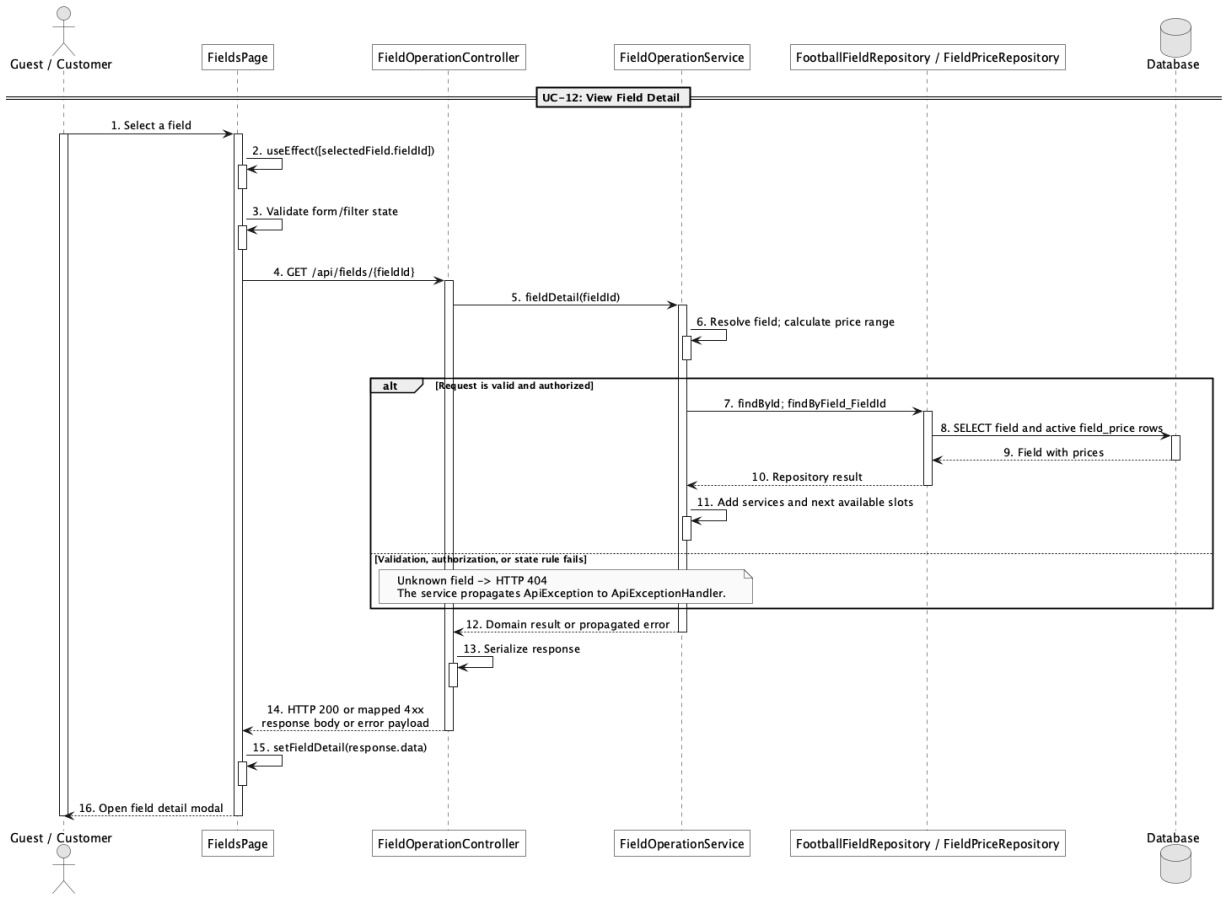
ExtraServiceRepository

No	Method	Description
01	List<ExtraService> findByStatus(CommonStatus status)	Loads the active add-on catalogue shown with the field.

BookingRepository

No	Method	Description
01	boolean existsBySlotAndStatusIn(Slot slot, Collection<BookingStatus> statuses)	Excludes slots already reserved by an active booking.

c. Sequence Diagram



d. Database Queries

== View field detail ==

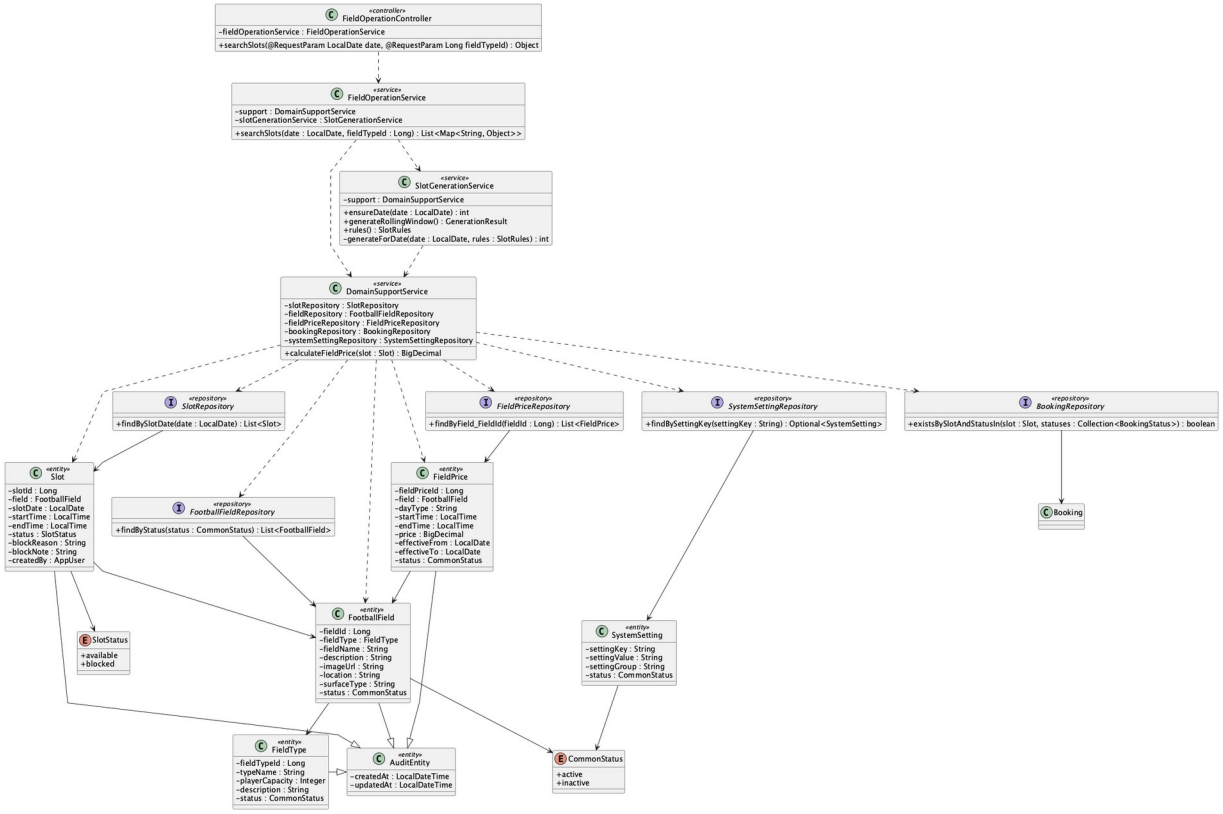
1. Field detail with active price bands

```
SELECT f.field_id, f.field_name, ft.type_name, f.description, f.image_url,
    f.location, f.surface_type, fp.day_type, fp.start_time, fp.end_time, fp.price
FROM field f
JOIN field_type ft ON ft.field_type_id = f.field_type_id
LEFT JOIN field_price fp ON fp.field_id = f.field_id AND fp.status = 'active'
WHERE f.field_id = :fieldId
ORDER BY fp.day_type, fp.start_time;
```

Implementation note: Reviews are outside the approved scope: there is no review entity, repository, table, or workflow. The response returns an empty reviews list and an explicit availability message.

12. Search available fields by date/time

a. Class Diagram



b. Class Specifications

FieldOperationController

No	Method	Description
01	public Object searchSlots(LocalDate date, Long fieldTypeId)	Exposes GET /api/slots/search with optional date and fieldTypeId filters.

FieldOperationService

No	Method	Description
01	searchSlots(date : LocalDate, fieldTypeId : Long) : List<Map<String, Object>>	Ensures the requested day is materialized, then returns active-field slots with booking state, availability, and calculated USD price.

SlotGenerationService

No	Method	Description
01	ensureDate(date : LocalDate) : int	Materializes missing non-elapsed slots for every active field from the persisted opening, closing, and duration rules.
02	rebuildRollingWindow() : int	Rebuilds only unbooked available auto-generated rows and preserves bookings, Block exceptions, and manual audit rows.
03	validateRuleChange(key : String, value : String) : void	Validates opening/closing order, 30-minute duration steps, divisible operating windows, and the 1–90 day horizon.

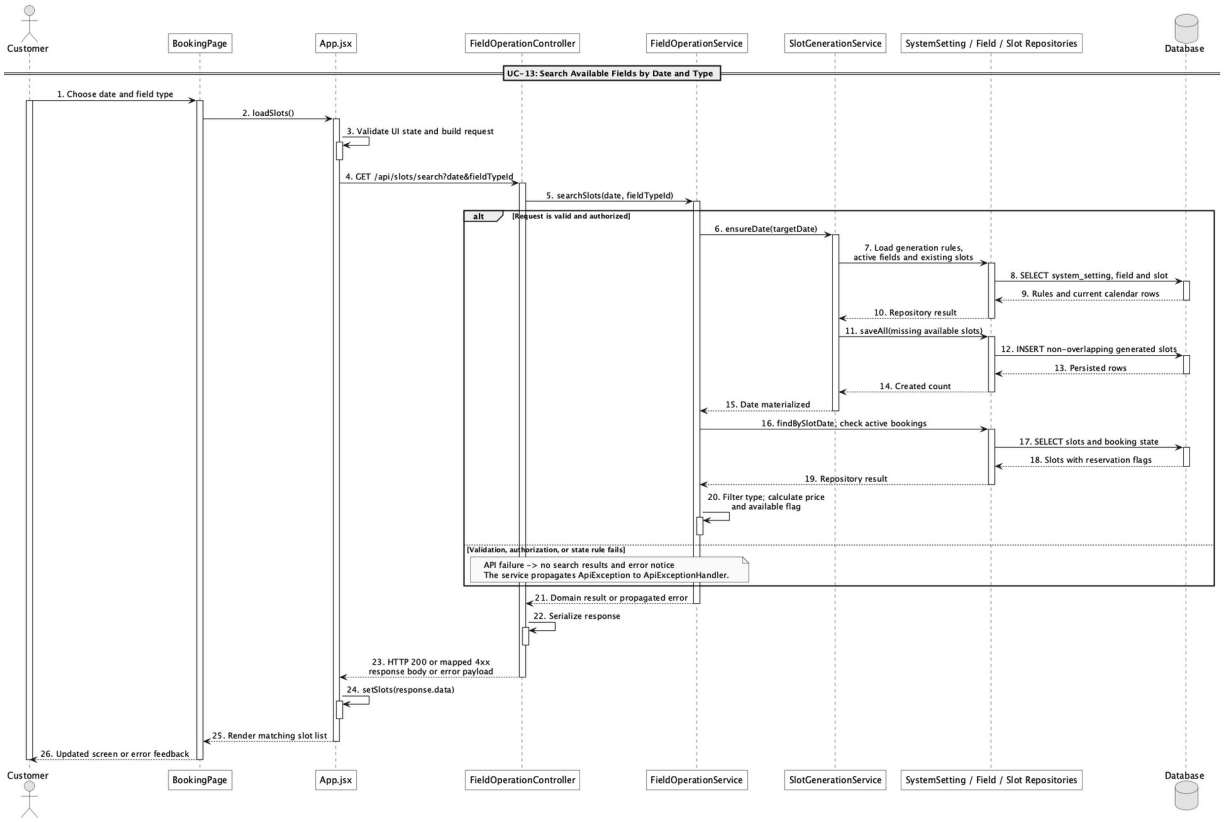
SlotRepository

No	Method	Description
01	findBySlotDate(date : LocalDate) : List<Slot>	Loads the materialized slots for the requested date.
02	findByField_FieldIdAndSlotDate(fieldId : Long, date : LocalDate) : List<Slot>	Supports overlap-safe materialization for one field and date.

BookingRepository

No	Method	Description
01	boolean existsBySlotAndStatusIn(Slot slot, Collection<BookingStatus> statuses)	Marks slots reserved by pending, confirmed, or checked-in bookings.

c. Sequence Diagram



d. Database Queries

== Search available fields by date/time ==

1. Searchable slot availability

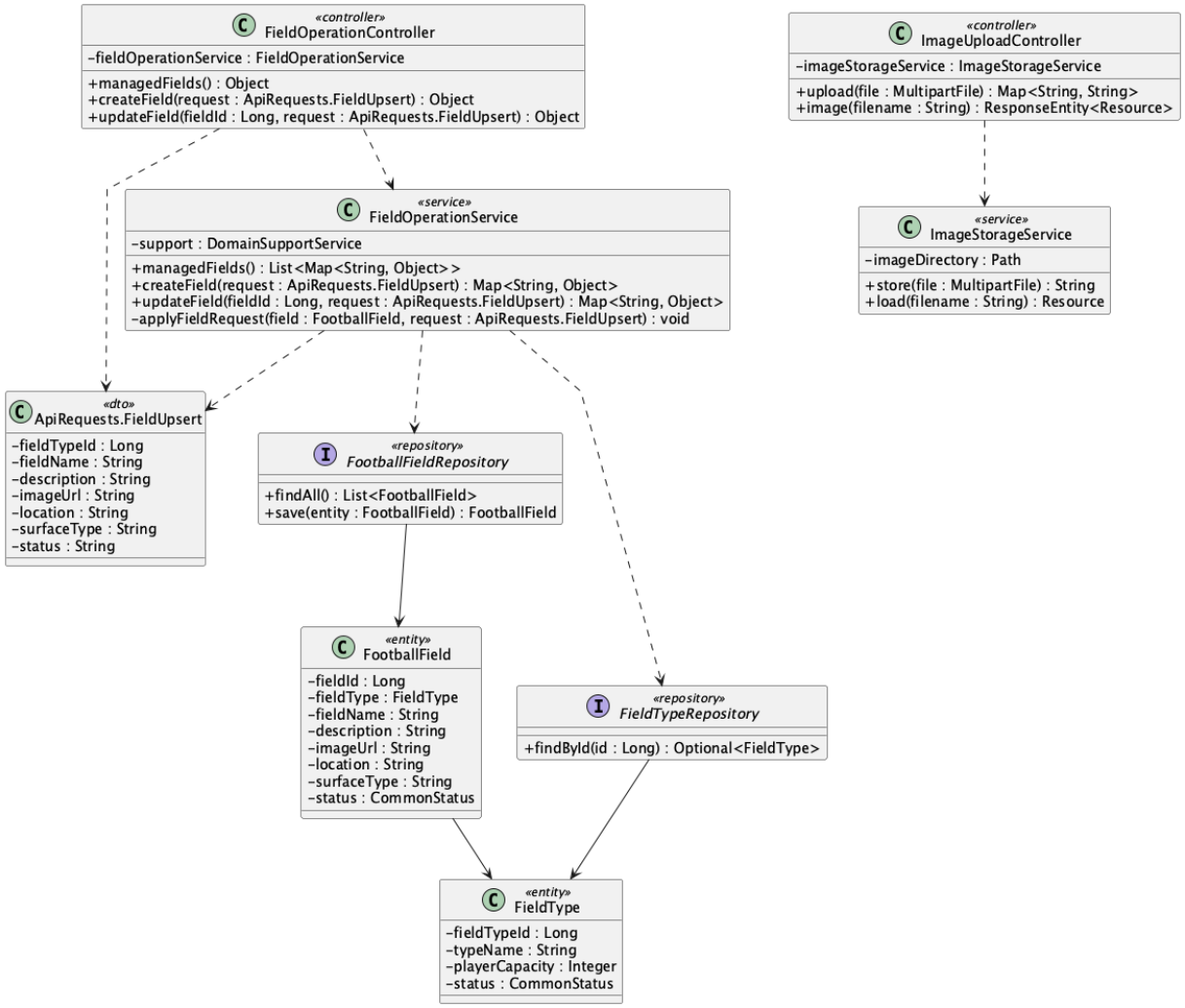
```
SELECT s.slot_id, s.slot_date, s.start_time, s.end_time, f.field_id, f.field_name,
      ft.type_name, CASE WHEN EXISTS (
        SELECT 1 FROM booking b
        WHERE b.slot_id = s.slot_id
          AND b.status IN ('pending','confirmed','checked_in')
      ) THEN 'booked' ELSE s.status END AS effective_status
FROM slot s
JOIN field f ON f.field_id = s.field_id AND f.status = 'active'
JOIN field_type ft ON ft.field_type_id = f.field_type_id
WHERE s.slot_date = :slotDate
  AND (:fieldTypeId IS NULL OR ft.field_type_id = :fieldTypeId)
```

ORDER BY f.field_name, s.start_time;

Implementation note: Missing availability is first materialized from the Admin opening, closing, duration, and horizon rules. The API then accepts a date and field type; time is represented by returned start_time/end_time values.

13. Manage football fields

a. Class Diagram



b. Class Specifications

FieldOperationController

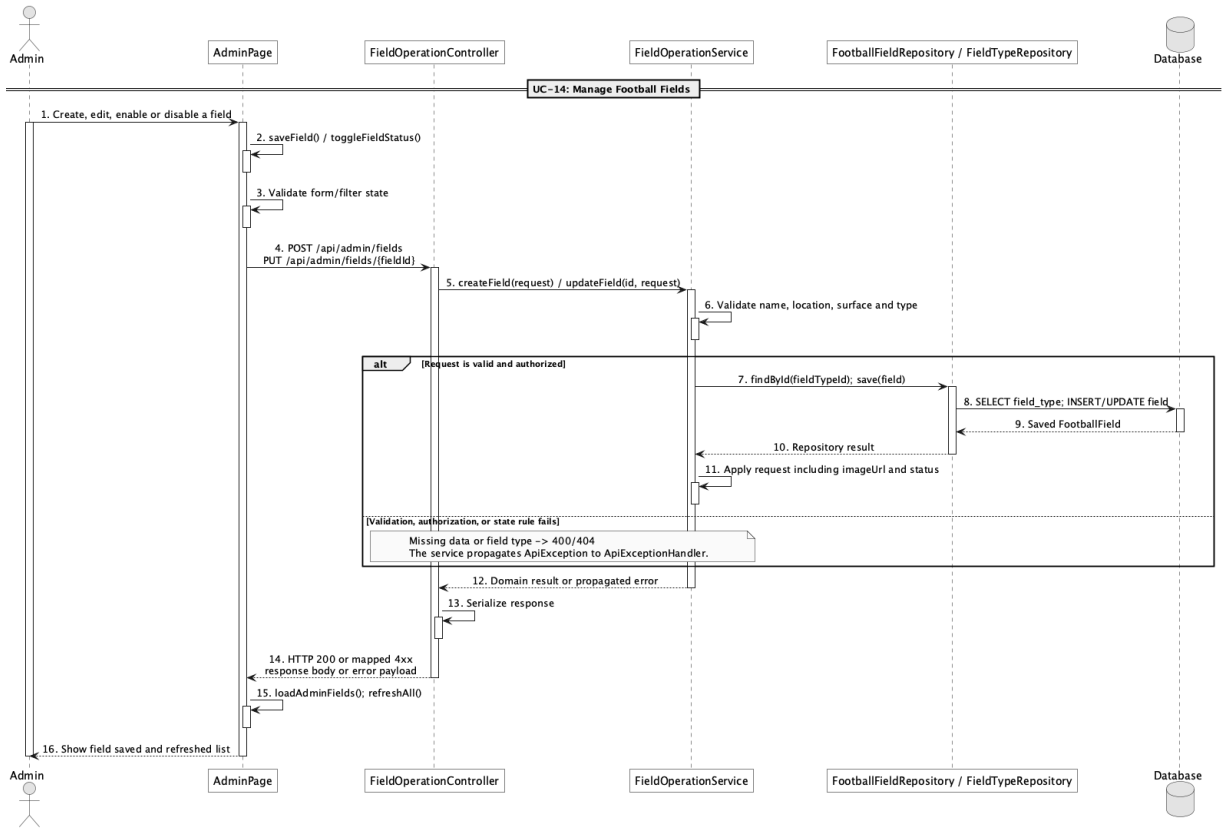
No	Method	Description
01	public Object managedFields()	Lists all fields for administration.
02	public Object createField(ApiRequests.FieldUpsert request)	Creates a field through POST /api/admin/fields.
03	public Object updateField(Long fieldId, ApiRequests.FieldUpsert	Updates field data and active/inactive state.

No	Method	Description
	request)	

FieldOperationService

No	Method	Description
01	public List<Map<String, Object>> managedFields()	Returns the complete field catalogue including inactive rows.
02	public Map<String, Object> createField(ApiRequests.FieldUpsert request)	Validates and persists a new FootballField.
03	public Map<String, Object> updateField(Long fieldId, ApiRequests.FieldUpsert request)	Applies an administrative field update.

c. Sequence Diagram



d. Database Queries

== Manage football fields ==

1. Create or update a field

INSERT INTO field (field_type_id, field_name, description, image_url, location, surface_type, status, created_at, updated_at)

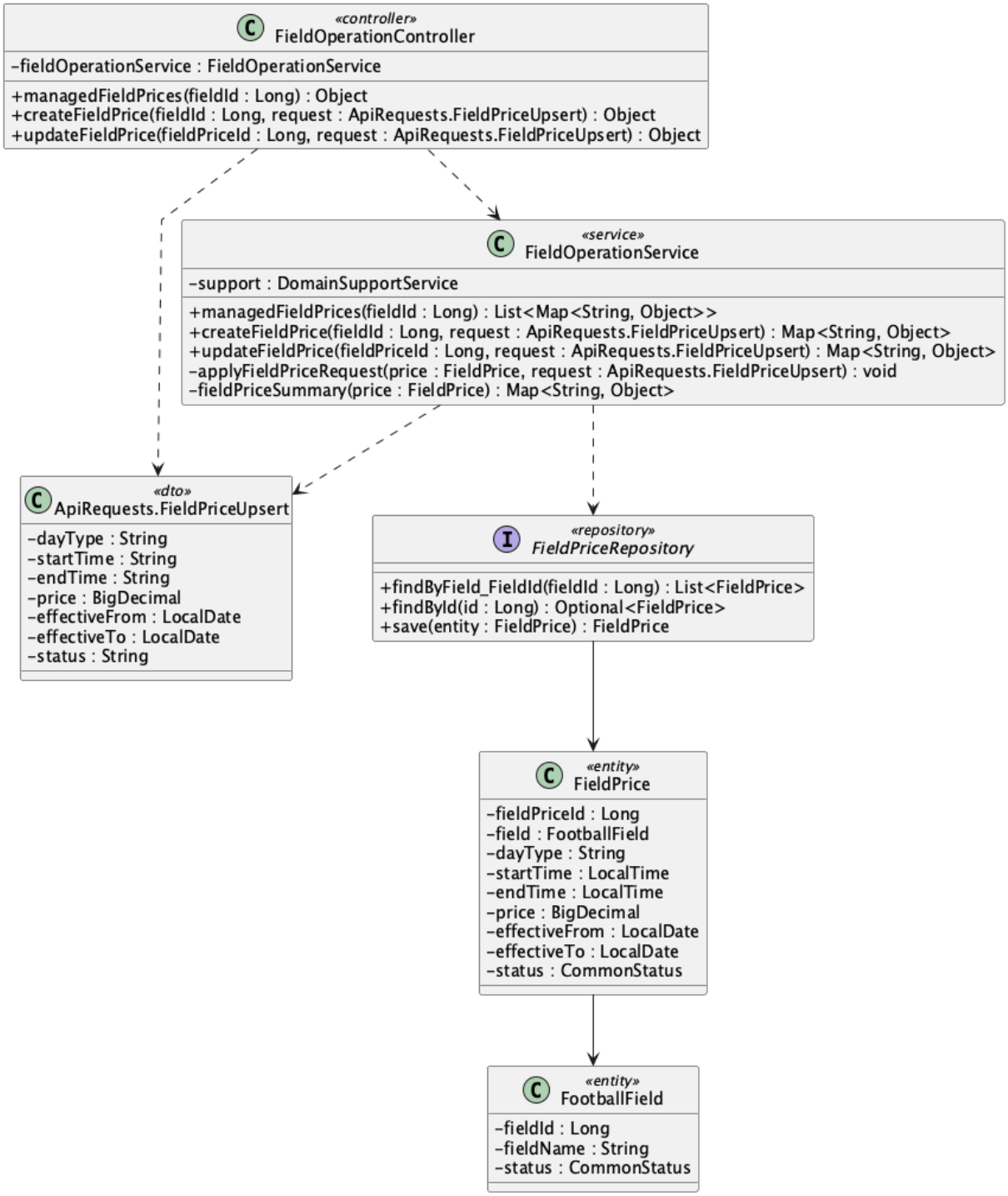
```
VALUES (:fieldTypeId, :fieldName, :description, :imageUrl, :location, :surfaceType, :status,
CURRENT_TIMESTAMP, CURRENT_TIMESTAMP);


UPDATE field
SET field_type_id = :fieldTypeId, field_name = :fieldName, description = :description,
    image_url = :imageUrl, location = :location, surface_type = :surfaceType,
    status = :status, updated_at = CURRENT_TIMESTAMP
WHERE field_id = :fieldId;
```

Implementation note: Role protection is centralized in SecurityConfig through /api/admin/**.

14. Manage field pricing by time range

a. Class Diagram



b. Class Specifications

FieldOperationController

No	Method	Description
01	public Object managedFieldPrices(Long fieldId)	Lists all pricing rules for one field.

No	Method	Description
02	public Object createFieldPrice(Long fieldId, ApiRequests.FieldPriceUpsert request)	Creates a day/time price band.
03	public Object updateFieldPrice(Long fieldPriceId, ApiRequests.FieldPriceUpsert request)	Updates an existing price band.

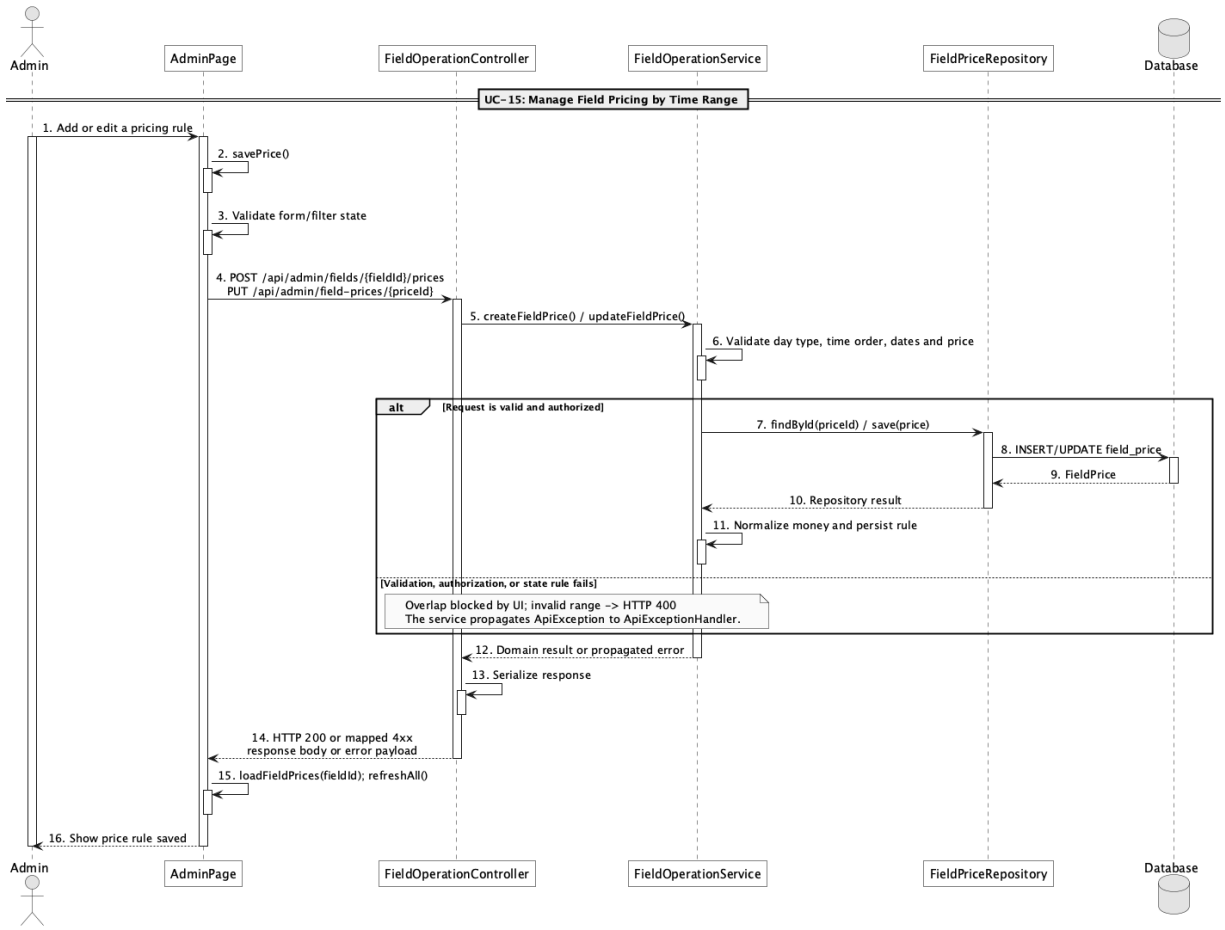
FieldOperationService

No	Method	Description
01	public List<Map<String, Object>> managedFieldPrices(Long fieldId)	Sorts pricing rules by day type and start time.
02	public Map<String, Object> createFieldPrice(Long fieldId, ApiRequests.FieldPriceUpsert request)	Validates day type, time range, amount, effective dates, and status.
03	public Map<String, Object> updateFieldPrice(Long fieldPriceId, ApiRequests.FieldPriceUpsert request)	Reuses the same pricing validation for edits.

FieldPriceRepository

No	Method	Description
01	List<FieldPrice> findByField_FieldId(Long fieldId)	Reads pricing rules for pricing and field-detail flows.

c. Sequence Diagram



d. Database Queries

== Manage field pricing by time range ==

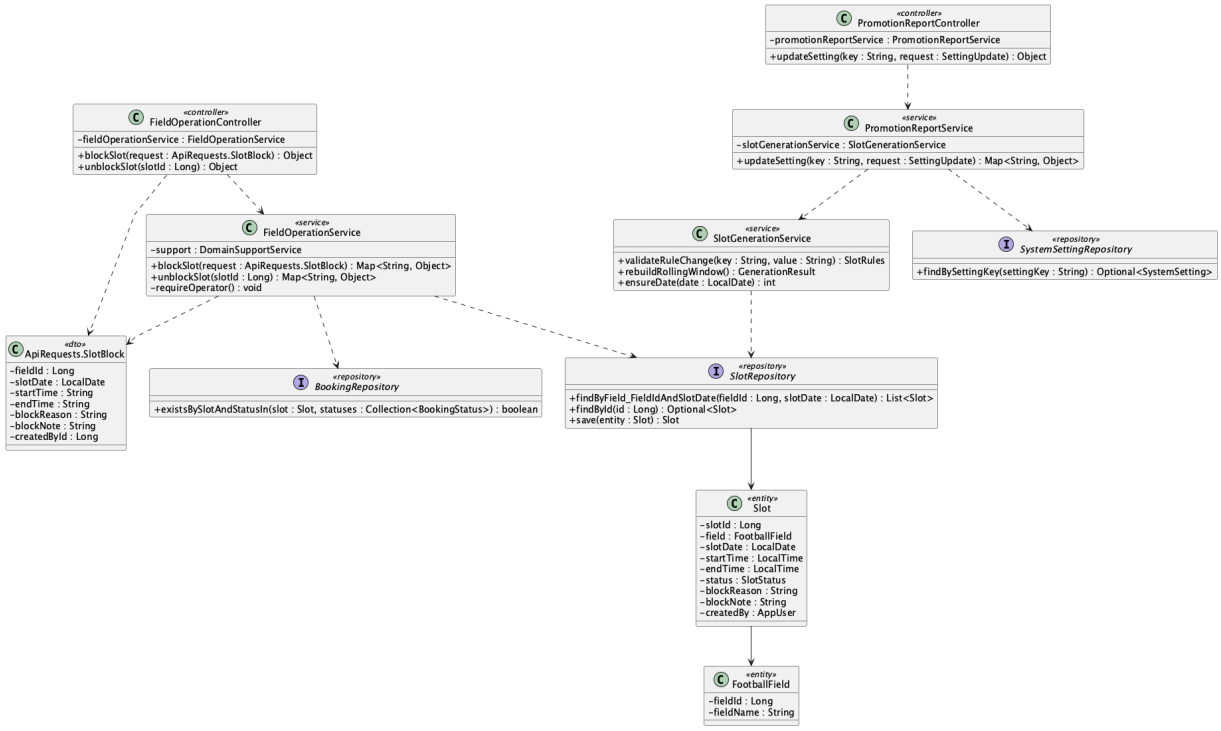
1. Field pricing rules

```
SELECT field_price_id, field_id, day_type, start_time, end_time, price,
    effective_from, effective_to, status
FROM field_price
WHERE field_id = :fieldId
ORDER BY day_type, start_time;
```

Implementation note: Allowed day_type values in service validation are all, weekday, and weekend.

15. Manage unavailable slots

a. Class Diagram



b. Class Specifications

FieldOperationController

No	Method	Description
01	public Object blockSlot(ApiRequests.SlotBlock request)	Creates or changes a time slot to blocked.
02	public Object unblockSlot(Long slotId)	Restores a blocked slot to available.

FieldOperationService

No	Method	Description
01	blockSlot(request : ApiRequests.SlotBlock) : Map<String, Object>	Blocks every overlapping generated slot, rejects active-booking conflicts, or records a future manual exception outside the current horizon.
02	unblockSlot(slotId : Long) : Map<String, Object>	Returns a blocked exception to available without recreating the operating calendar manually.

PromotionReportService

No	Method	Description
01	updateSetting(key : String, value : String) : Map<String, Object>	Lets Admin persist one automatic-slot rule after validating the combined rule set, then safely rebuilds the rolling window.

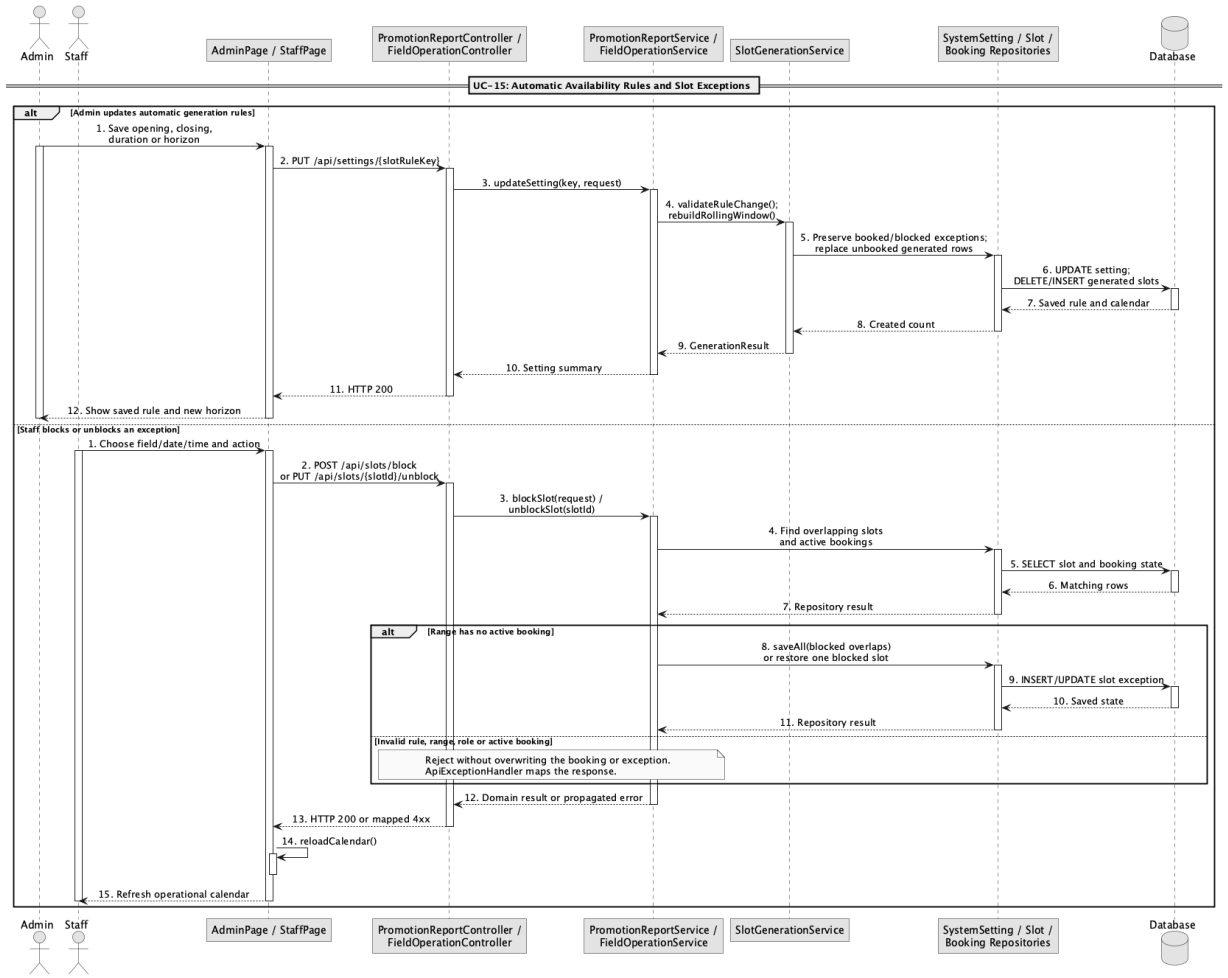
SlotGenerationService

No	Method	Description
01	ensureDate(date : LocalDate) : int	Materializes missing non-elapsed slots for every active field from the persisted opening, closing, and duration rules.
02	rebuildRollingWindow() : int	Rebuilds only unbooked available auto-generated rows and preserves bookings, Block exceptions, and manual audit rows.
03	validateRuleChange(key : String, value : String) : void	Validates opening/closing order, 30-minute duration steps, divisible operating windows, and the 1–90 day horizon.

SlotRepository

No	Method	Description
01	findByField_FieldIdAndSlotDate(fieldId : Long, date : LocalDate) : List<Slot>	Loads every same-day row so Block can cover all overlapping generated periods.
02	findBySlotDateBetweenOrderBySlotDateAscStartTimeAsc(from : LocalDate, to : LocalDate) : List<Slot>	Loads the rolling window for a safe Admin-rule rebuild.

c. Sequence Diagram



d. Database Queries

== Manage unavailable slots ==

1. Block and unblock a slot

UPDATE slot

SET status = 'blocked', block_reason = :reason, block_note = :note,

created_by = :operatorId, updated_at = CURRENT_TIMESTAMP

WHERE slot_id = :slotId;

UPDATE slot

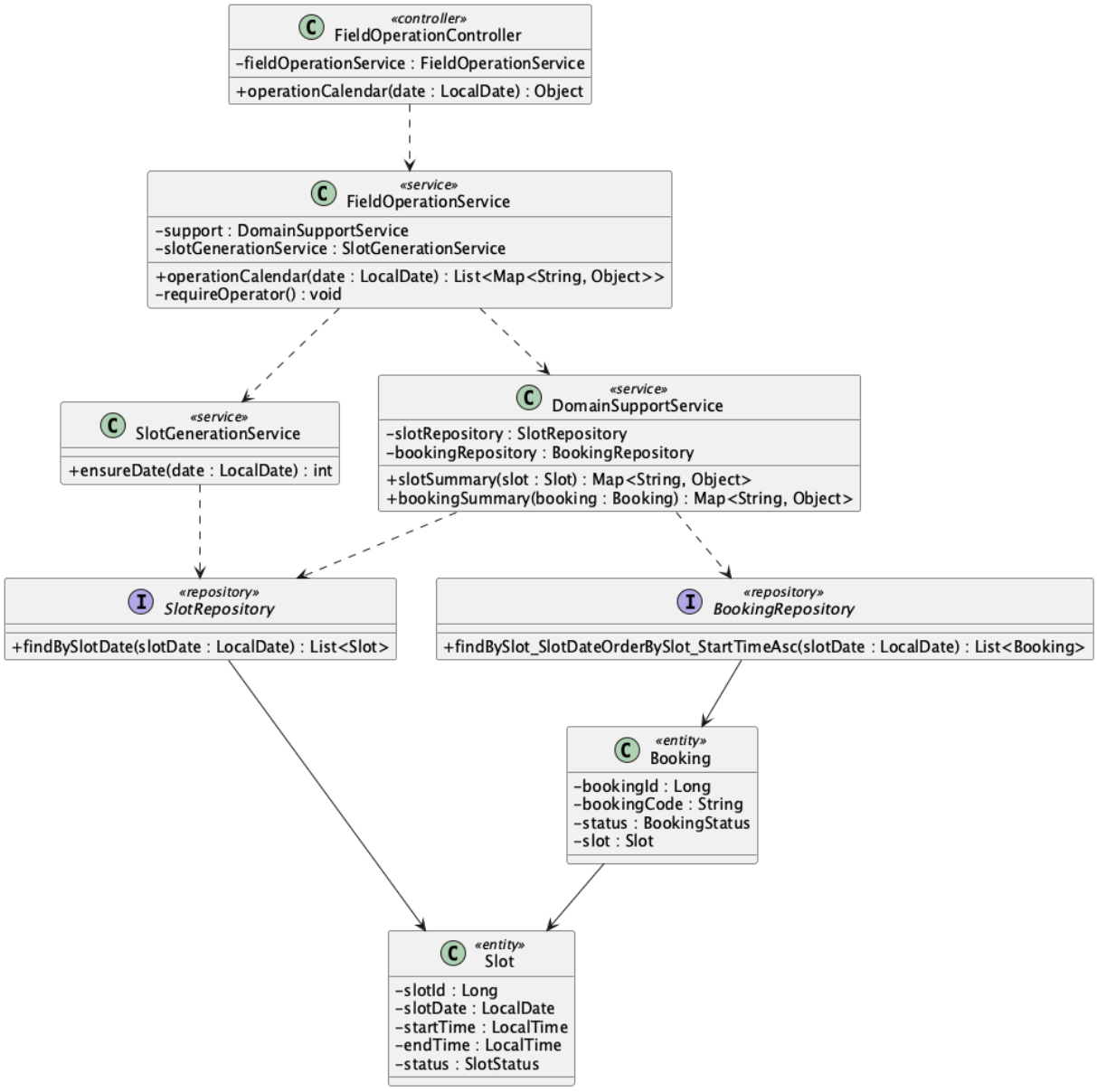
SET status = 'available', block_reason = NULL, block_note = NULL, updated_at = CURRENT_TIMESTAMP

WHERE slot_id = :slotId AND status = 'blocked';

Implementation note: Block covers every overlapping generated slot and is rejected when any overlap has an active booking. A future exception may also be recorded before that date enters the rolling horizon.

16. View field operation calendar

a. Class Diagram



b. Class Specifications

FieldOperationController

No	Method	Description
01	public Object operationCalendar(LocalDate date)	Exposes the staff/admin daily operations view.

FieldOperationService

No	Method	Description
01	operationCalendar(date : LocalDate) : List<Map<String, Object>>	Materializes the requested date, then merges slots and bookings into available, blocked, booked, and operational states.

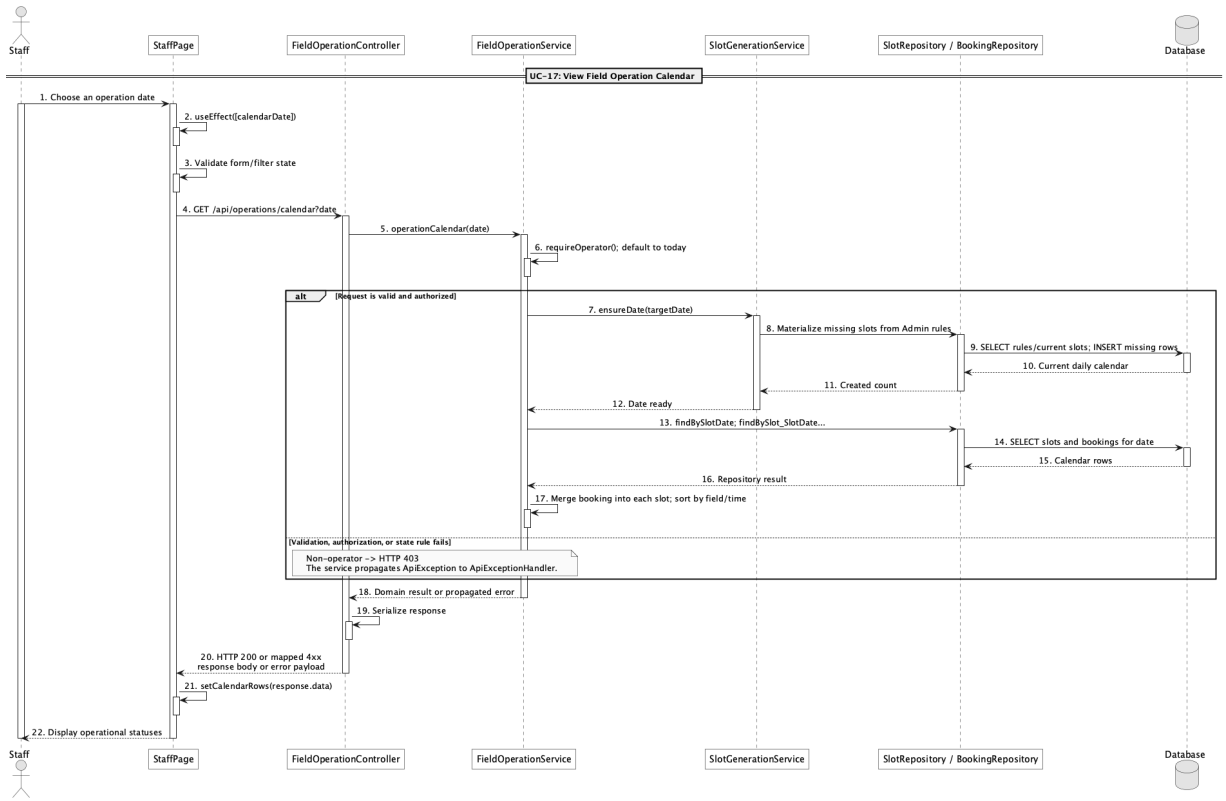
SlotGenerationService

No	Method	Description
01	ensureDate(date : LocalDate) : int	Materializes missing non-elapsed slots for every active field from the persisted opening, closing, and duration rules.

BookingRepository

No	Method	Description
01	List<Booking> findBySlot_SlotDateOrderBySlot_StartTimeAsc(LocalDate slotDate)	Loads bookings for the selected operating date.

c. Sequence Diagram



d. Database Queries

== View field operation calendar ==

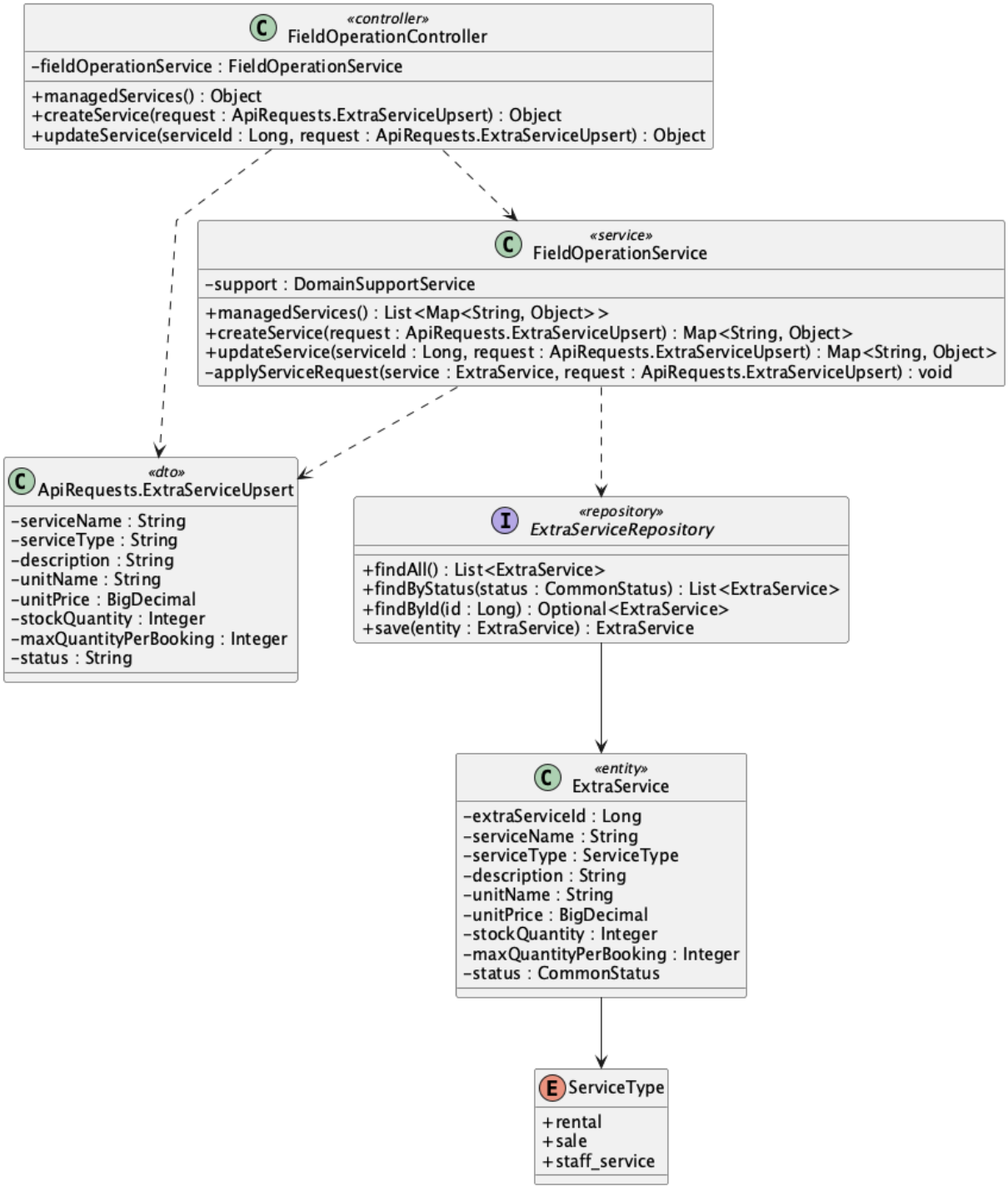
1. Daily field operations

```
SELECT s.slot_id, f.field_name, s.start_time, s.end_time, s.status AS slot_status,
    b.booking_id, b.booking_code, b.status AS booking_status, b.customer_id, b.staff_id
FROM slot s
JOIN field f ON f.field_id = s.field_id
LEFT JOIN booking b ON b.slot_id = s.slot_id
WHERE s.slot_date = :operationDate
ORDER BY f.field_name, s.start_time;
```

Implementation note: The requested date is materialized before display. This remains a date-focused operations calendar, not a month-grid scheduling engine.

17. Manage extra services

a. Class Diagram



b. Class Specifications

FieldOperationController

No	Method	Description
01	public Object managedServices()	Lists active and inactive services for Admin.
02	public Object	Creates an extra service.

No	Method	Description
	createService(ApiRequests.ExtraServiceUpsert request)	
03	public Object updateService(Long serviceId, ApiRequests.ExtraServiceUpsert request)	Updates service pricing, inventory, limits, and status.

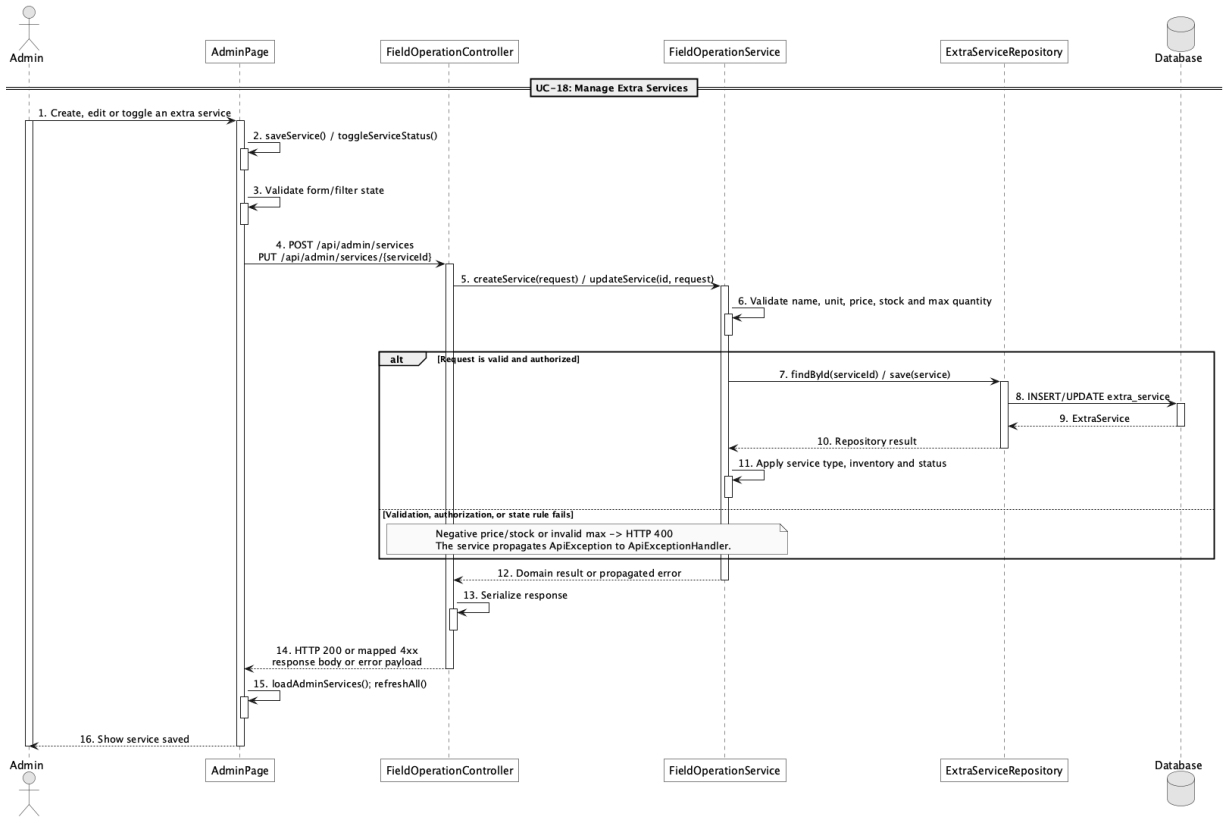
FieldOperationService

No	Method	Description
01	public List<Map<String, Object>> managedServices()	Returns the complete service inventory.
02	public Map<String, Object> createService(ApiRequests.ExtraServiceUpsert request)	Validates and persists a new ExtraService.
03	public Map<String, Object> updateService(Long serviceId, ApiRequests.ExtraServiceUpsert request)	Applies validated changes to an ExtraService.

ExtraServiceRepository

No	Method	Description
01	List<ExtraService> findByStatus(CommonStatus status)	Supplies active services to public and booking flows.

c. Sequence Diagram



d. Database Queries

== Manage extra services ==

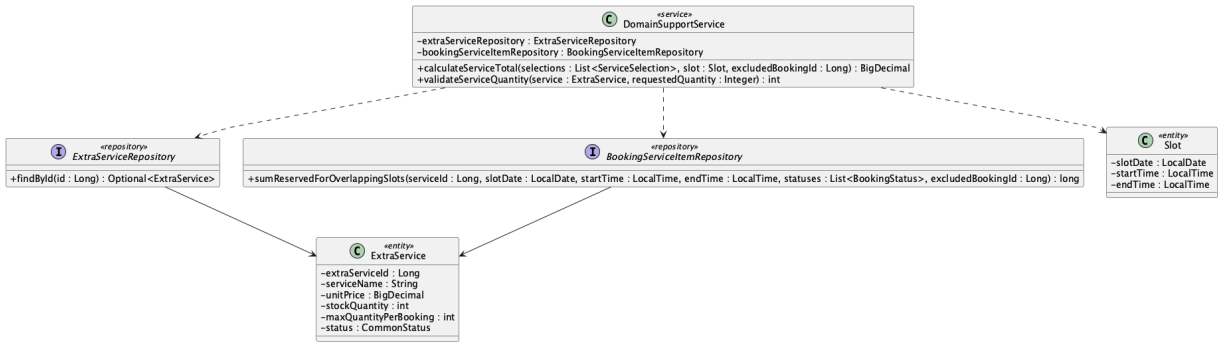
1. Managed service inventory

```
SELECT extra_service_id, service_name, service_type, description, unit_name,
    unit_price, stock_quantity, max_quantity_per_booking, status
FROM extra_service
ORDER BY extra_service_id;
```

Implementation note: Service types map to rental, sale, and staff_service.

18. Check extra service availability

a. Class Diagram



b. Class Specifications

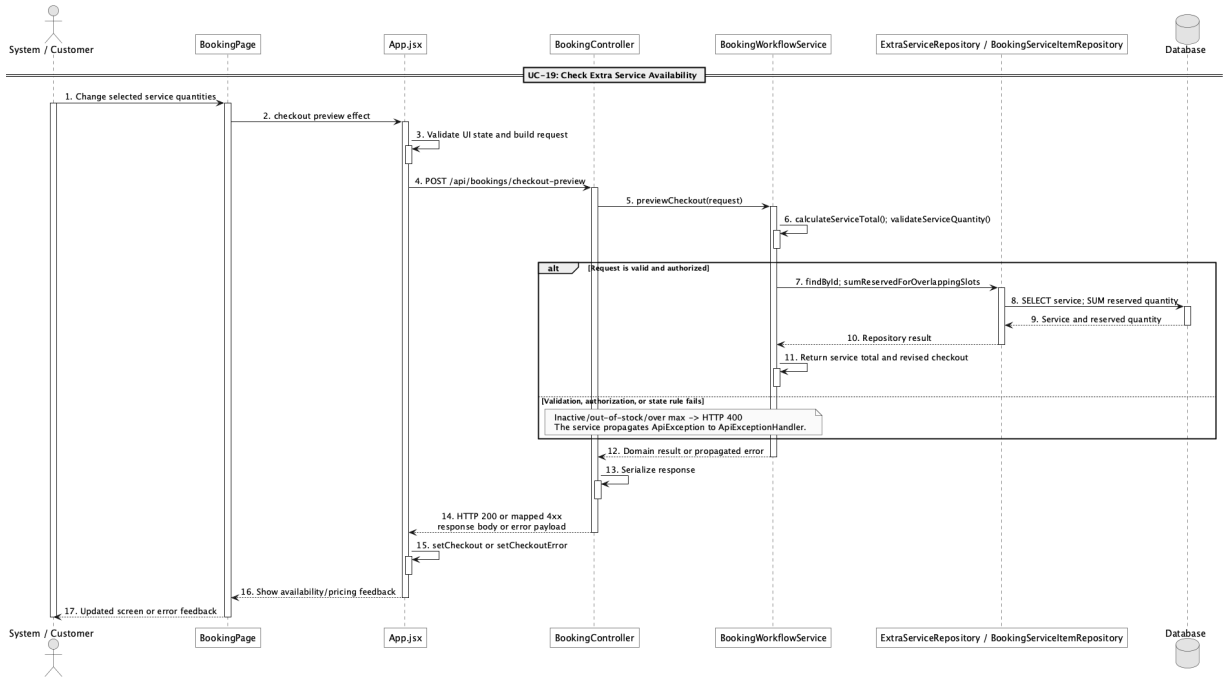
DomainSupportService

No	Method	Description
01	BigDecimal calculateServiceTotal(List<ApiRequests.ServiceSelection> selections, Slot slot, Long excludedBookingId)	Checks active status, per-booking maximum, stock reserved in overlapping active bookings, and computes total.
02	int validateServiceQuantity(ExtraService service, Integer requestedQuantity)	Enforces active status, maximum quantity, and base stock constraints.

BookingServiceItemRepository

No	Method	Description
01	long sumReservedForOverlappingSlots(Long serviceId, LocalDate slotDate, LocalTime startTime, LocalTime endTime, List<BookingStatus> statuses, Long excludedBookingId)	JPQL aggregate for inventory consumed by overlapping bookings.

c. Sequence Diagram



d. Database Queries

== Check extra service availability ==

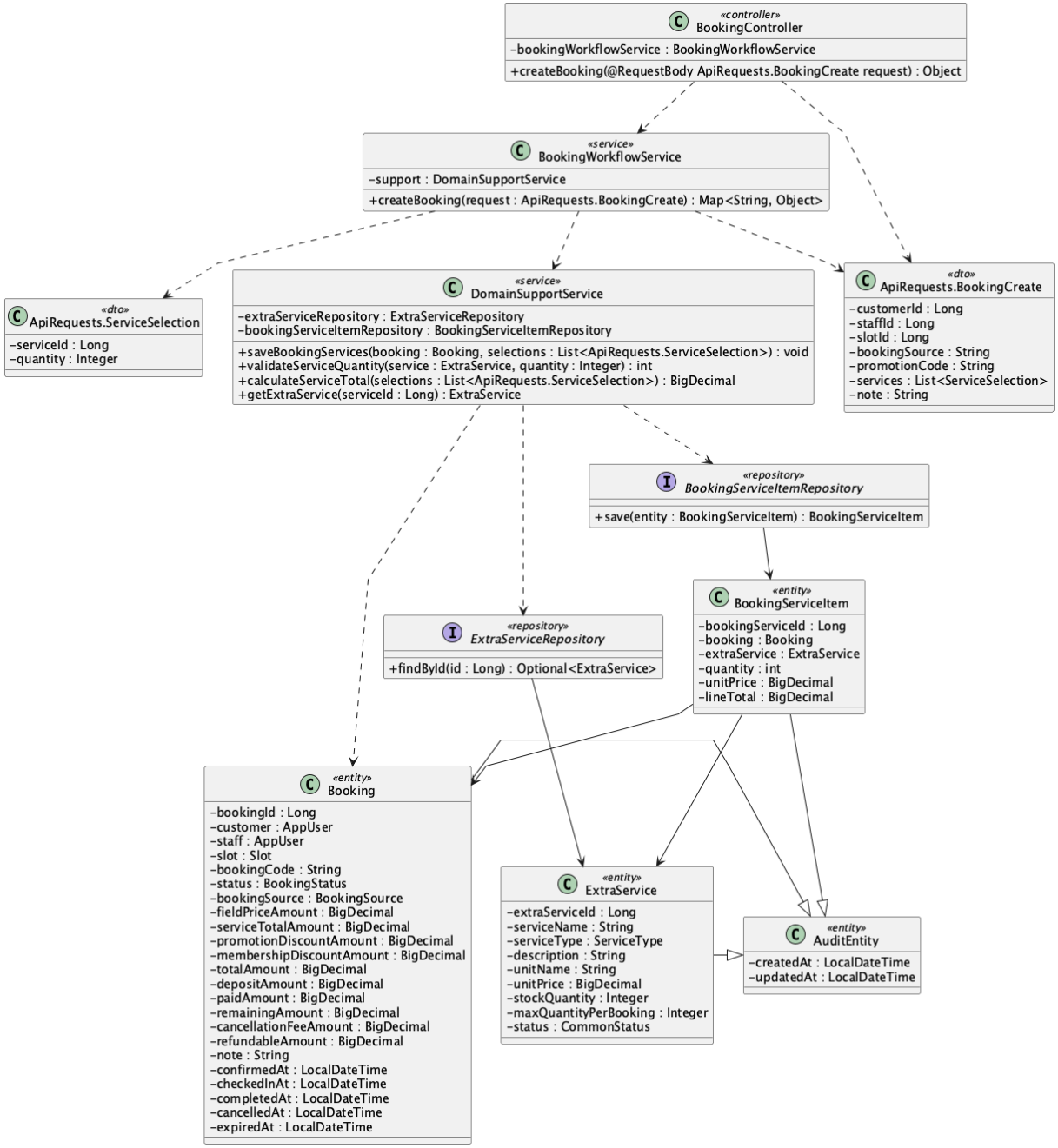
1. Reserved service quantity for overlapping slots

```
SELECT COALESCE(SUM(bs.quantity), 0) AS reserved_quantity
FROM booking_service bs
JOIN booking b ON b.booking_id = bs.booking_id
JOIN slot s ON s.slot_id = b.slot_id
WHERE bs.extra_service_id = :serviceId
  AND b.status IN ('pending','confirmed','checked_in')
  AND s.slot_date = :slotDate
  AND s.start_time < :endTime
  AND s.end_time > :startTime
  AND (:excludedBookingId IS NULL OR b.booking_id <> :excludedBookingId);
```

Implementation note: The repository implementation is JPQL; this SQL is the equivalent physical-table query.

19. Add extra services to booking

a. Class Diagram



b. Class Specifications

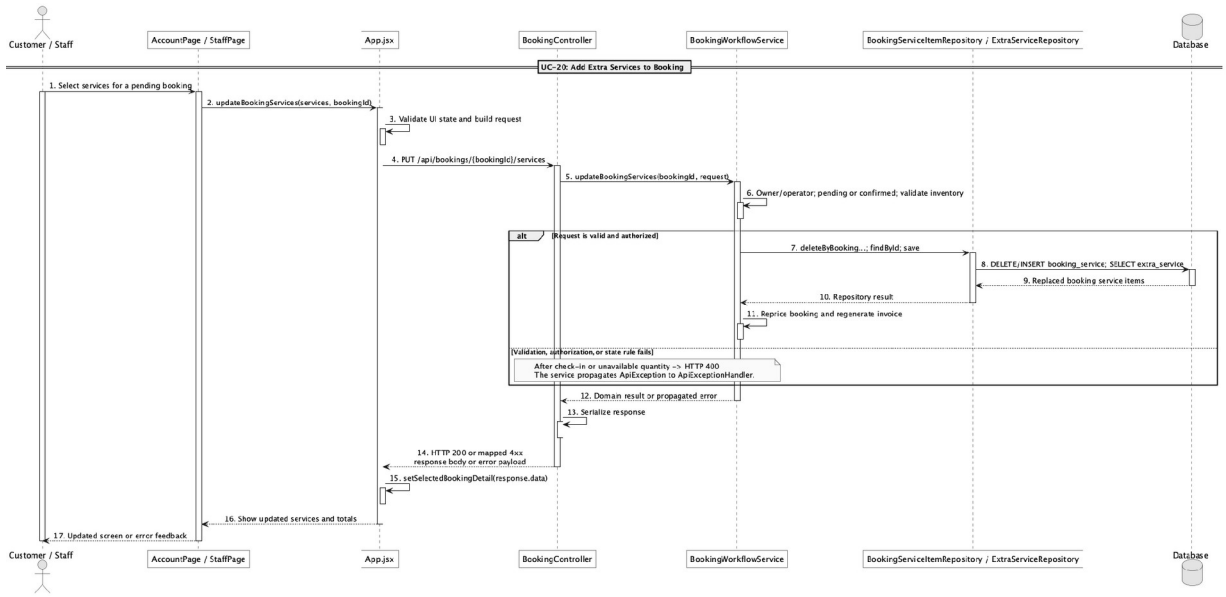
BookingWorkflowService

No	Method	Description
01	public Map<String, Object> createBooking(ApiRequests.BookingCreate request)	Accepts service selections as part of online or walk-in booking creation.

DomainSupportService

No	Method	Description
01	BigDecimal calculateServiceTotal(List<ApiRequests.ServiceSelection> selections, Slot slot, Long excludedBookingId)	Prices and validates selected services.
02	void saveBookingServices(Booking booking, List<ApiRequests.ServiceSelection> selections)	Persists unit-price and line-total snapshots in booking_service.

c. Sequence Diagram



d. Database Queries

== Add extra services to booking ==

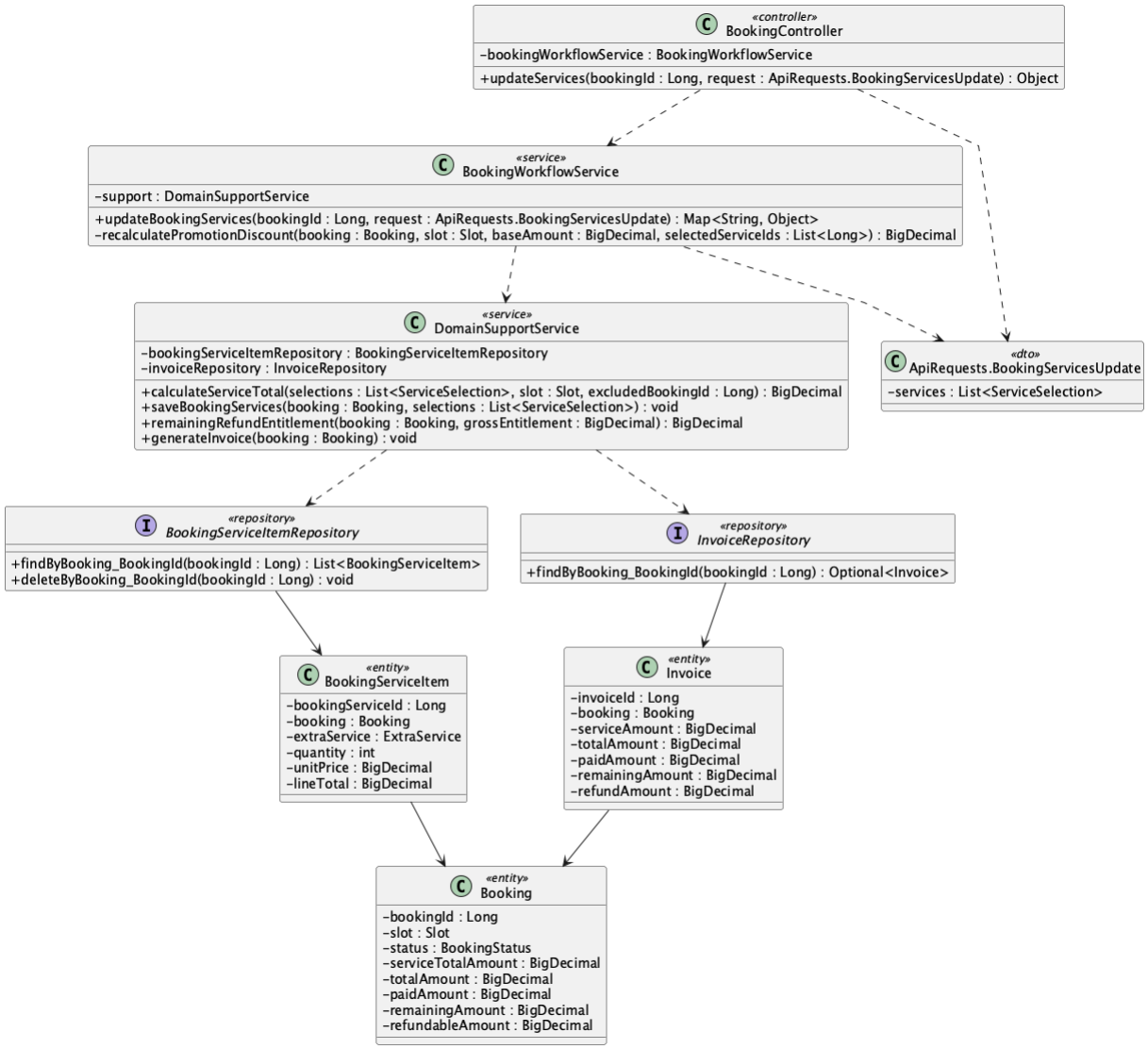
1. Persist booking service snapshots

```
INSERT INTO booking_service
  (booking_id, extra_service_id, quantity, unit_price, line_total, created_at, updated_at)
SELECT :bookingId, es.extra_service_id, :quantity, es.unit_price,
      es.unit_price * :quantity, CURRENT_TIMESTAMP, CURRENT_TIMESTAMP
FROM extra_service es
WHERE es.extra_service_id = :serviceId AND es.status = 'active';
```

Implementation note: booking_service stores snapshot prices so later catalogue price changes do not rewrite the booking total.

20. Update extra services before check-in

a. Class Diagram



b. Class Specifications

BookingController

No	Method	Description
01	public Object updateServices(Long bookingId, ApiRequests.BookingServicesUpdate request)	Exposes PUT /api/bookings/{bookingId}/services.

BookingWorkflowService

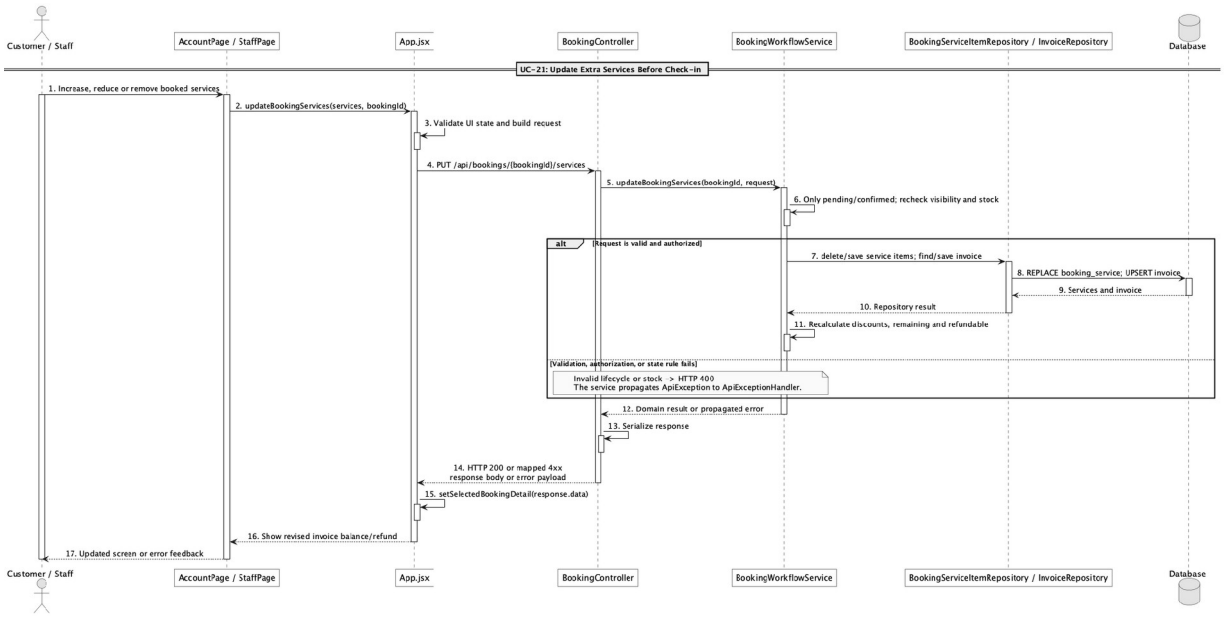
No	Method	Description
01	public Map<String, Object> updateBookingServices(Long	Allows owner/operator changes only while pending or confirmed, then reprices discounts, balance/refund

No	Method	Description
	bookingId, ApiRequests.BookingServicesUpdate request)	delta, and invoice.

BookingServiceItemRepository

No	Method	Description
01	List<BookingServiceItem> findByBooking_BookingId(Long bookingId)	Reads current add-ons for repricing.
02	void deleteByBooking_BookingId(Long bookingId)	Replaces the old service set before saving the revised selections.

c. Sequence Diagram



d. Database Queries

== Update extra services before check-in ==

1. Replace booking services and update totals

DELETE FROM booking_service WHERE booking_id = :bookingId;

UPDATE booking

SET service_total_amount = :serviceTotal, promotion_discount_amount = :promotionDiscount,

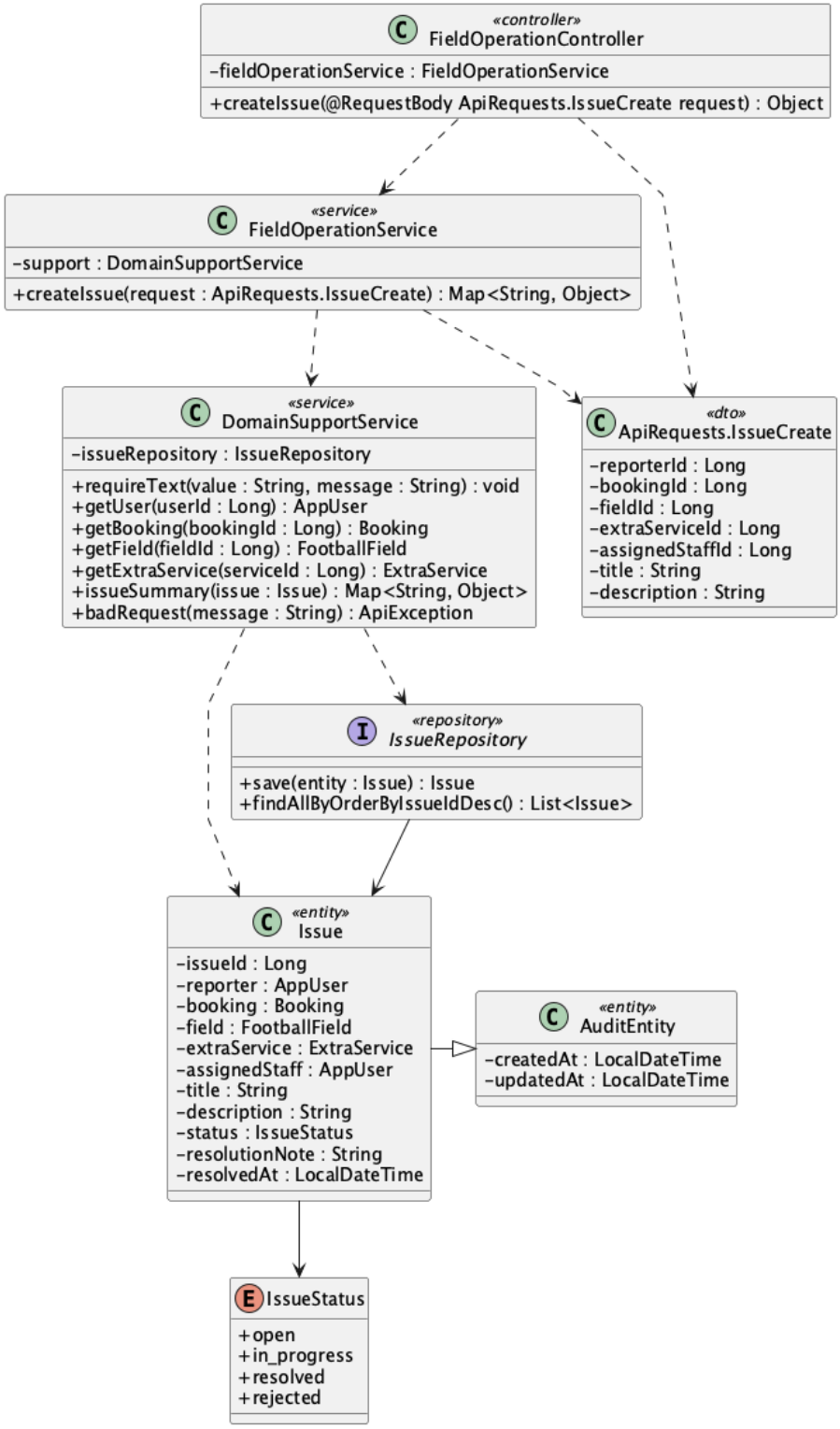
membership_discount_amount = :membershipDiscount, total_amount = :totalAmount,

```
    deposit_amount = :depositAmount, remaining_amount = :remainingAmount,
    refundable_amount = :refundableAmount, updated_at = CURRENT_TIMESTAMP
WHERE booking_id = :bookingId AND status IN ('pending','confirmed');
```

Implementation note: Customer ownership and Staff/Admin access are enforced before the change; edits after check-in are blocked.

21. Report field/service issue

a. Class Diagram



b. Class Specifications

FieldOperationController

No	Method	Description
01	public Object createIssue(ApiRequests.IssueCreate request)	Exposes POST /api/issues.

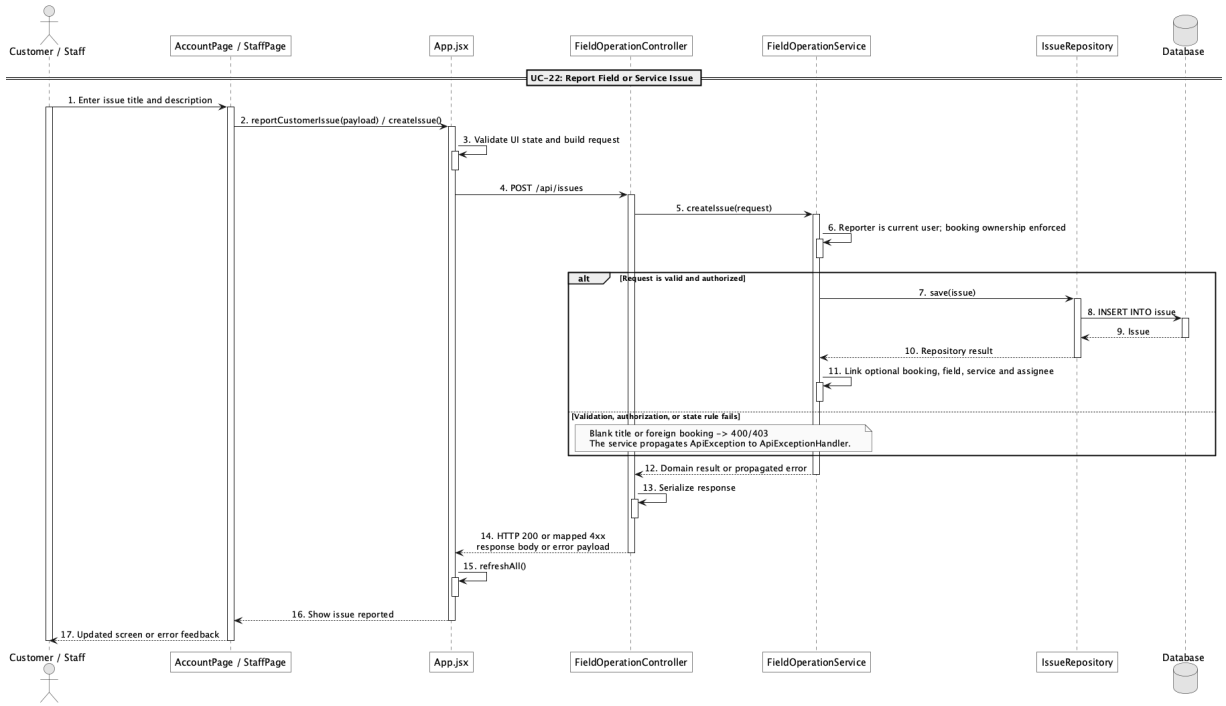
FieldOperationService

No	Method	Description
01	public Map<String, Object> createIssue(ApiRequests.IssueCreate request)	Creates an issue tied optionally to a booking, field, service, and assigned staff member.

IssueRepository

No	Method	Description
01	List<Issue> findAllByOrderByIssueIdDesc()	Supplies the staff issue queue in newest-first order.

c. Sequence Diagram



d. Database Queries

== Report field/service issue ==

1. Create issue report

INSERT INTO issue

(reporter_id, booking_id, field_id, extra_service_id, assigned_staff_id,

title, description, status, created_at, updated_at)

VALUES

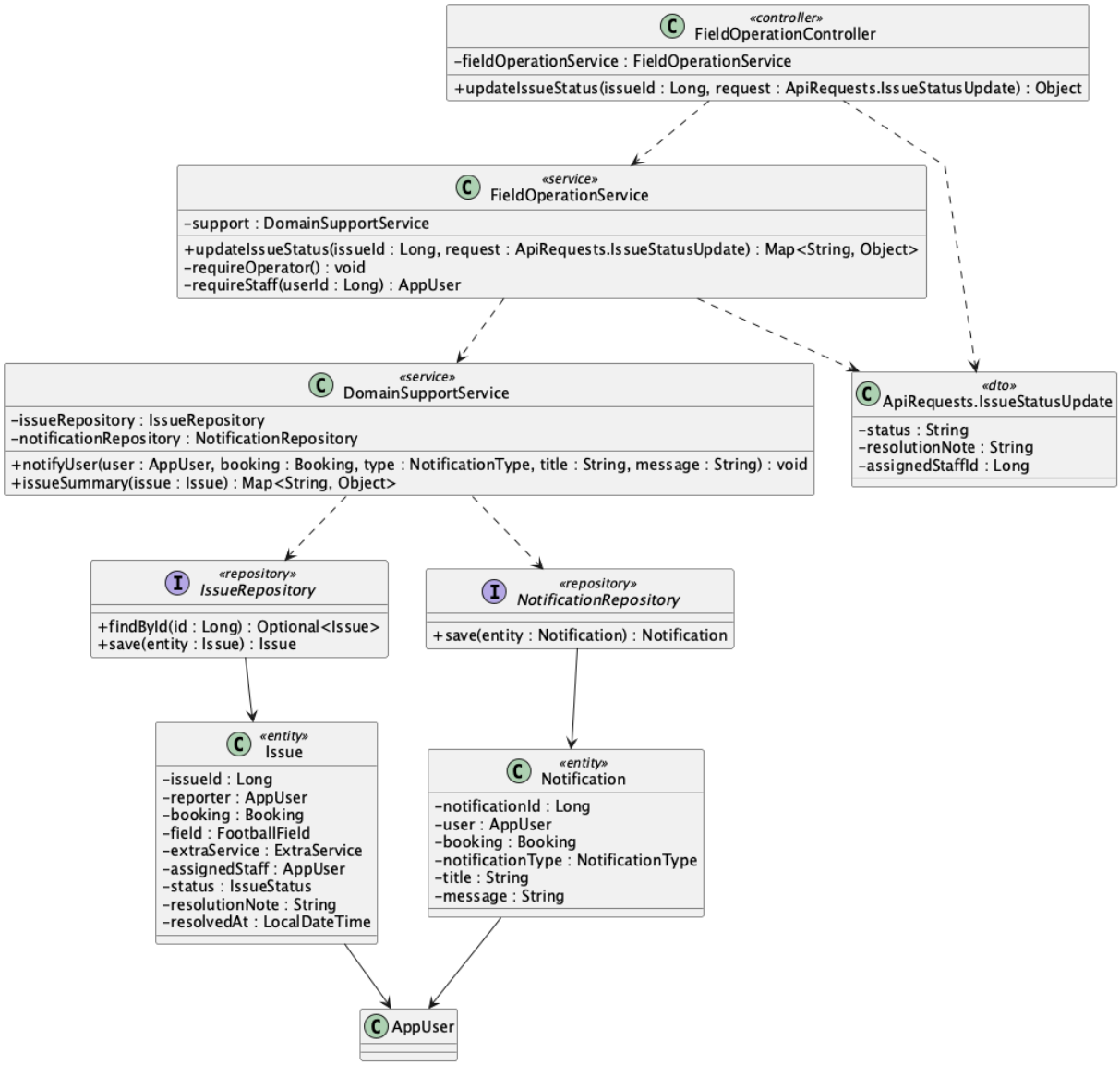
(:reporterId, :bookingId, :fieldId, :extraServiceId, :assignedStaffId,

:title, :description, 'open', CURRENT_TIMESTAMP, CURRENT_TIMESTAMP);

Implementation note: A customer may report only under their own account and, when booking-linked, only for their own booking.

22. Resolve field/service issue

a. Class Diagram



b. Class Specifications

FieldOperationController

No	Method	Description
01	public Object updateIssueStatus(Long issueId, ApiRequests.IssueStatusUpdate request)	Exposes the Staff/Admin issue status endpoint.

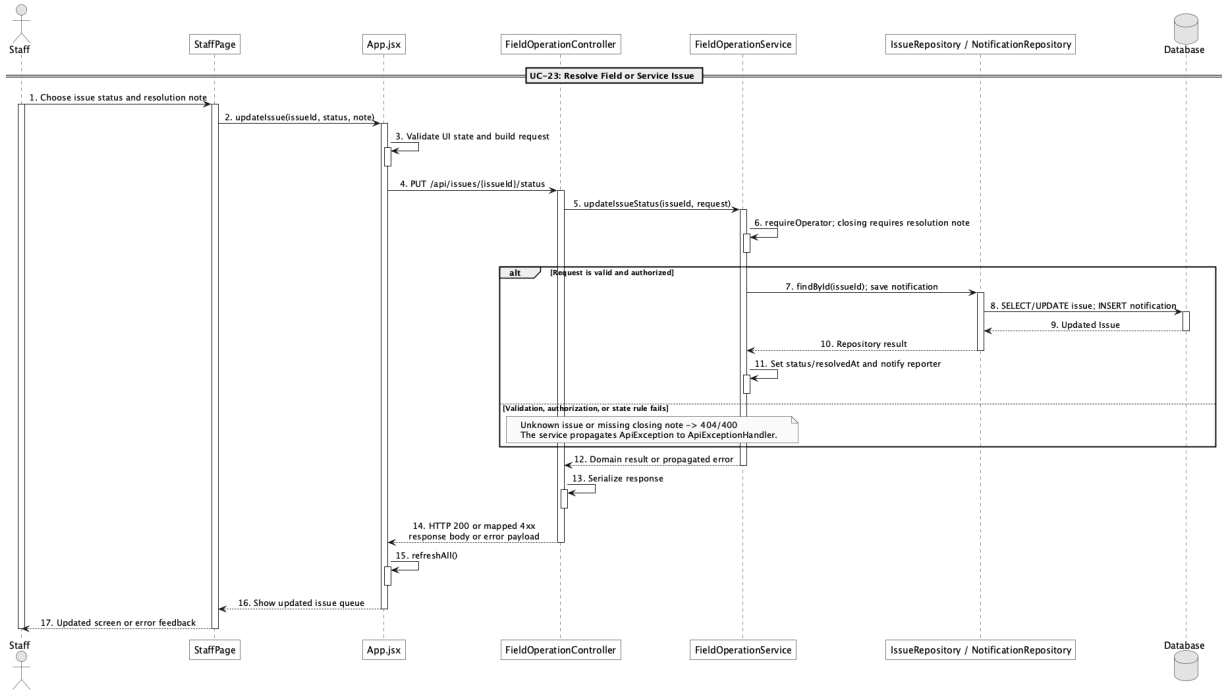
FieldOperationService

No	Method	Description
01	public Map<String, Object> updateIssueStatus(Long issueId, ApiRequests.IssueStatusUpdate request)	Assigns staff, validates closure notes, records resolution time, and notifies the reporter.

DomainSupportService

No	Method	Description
01	void notifyUser(AppUser user, Booking booking, NotificationType type, String title, String message)	Persists the issue status notification.

c. Sequence Diagram



d. Database Queries

== Resolve field/service issue ==

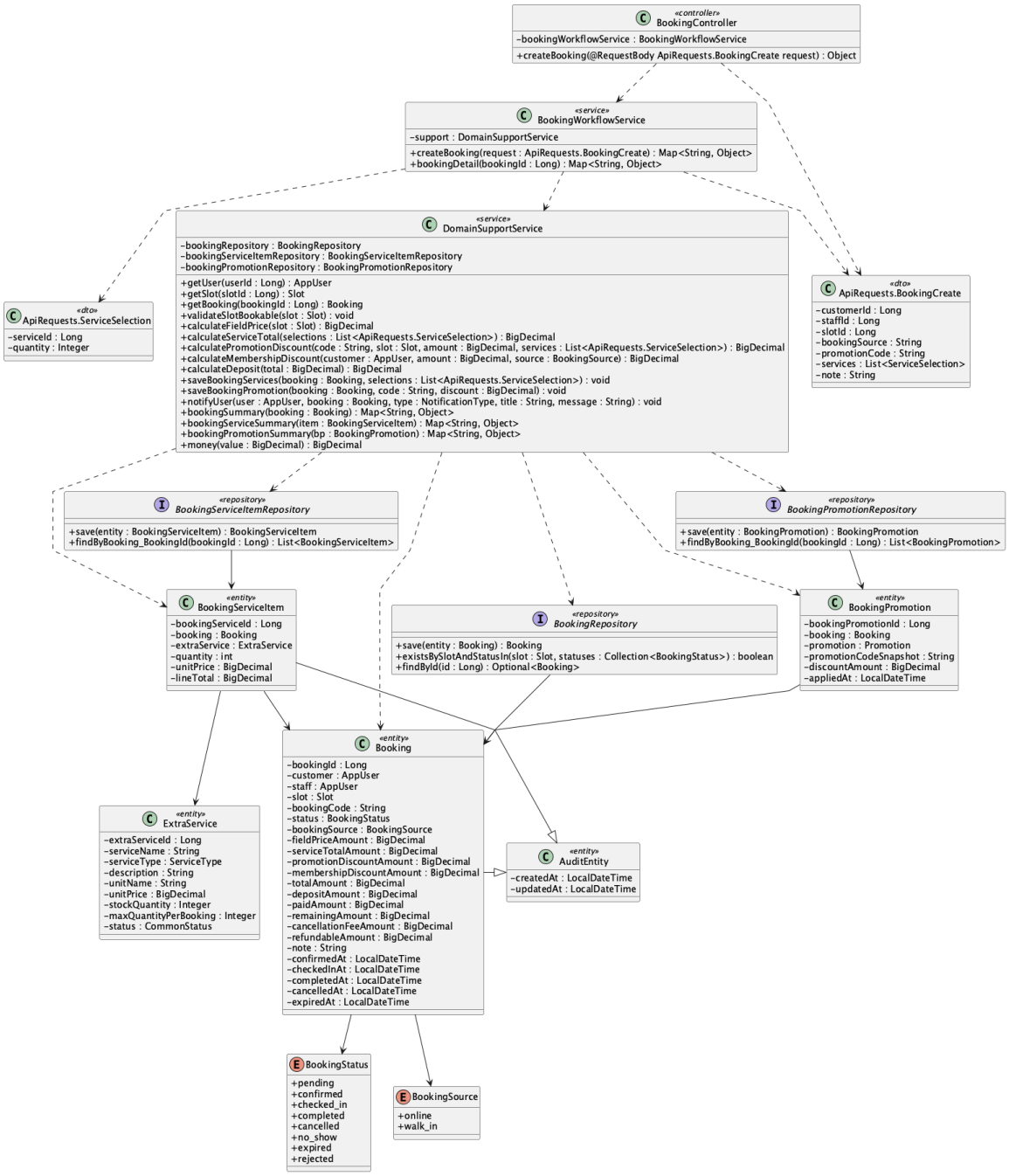
1. Resolve or reject an issue

```
UPDATE issue
SET assigned_staff_id = :assignedStaffId, status = :status,
   resolution_note = :resolutionNote,
   resolved_at = CASE WHEN :status IN ('resolved','rejected') THEN CURRENT_TIMESTAMP ELSE NULL
END,
   updated_at = CURRENT_TIMESTAMP
WHERE issue_id = :issueId;
```

Implementation note: Closing statuses resolved and rejected require a nonblank resolution note.

23. Create online booking

a. Class Diagram



b. Class Specifications

BookingController

No	Method	Description
01	public Object createBooking(ApiRequests.BookingCreate request)	Exposes authenticated POST /api/bookings.

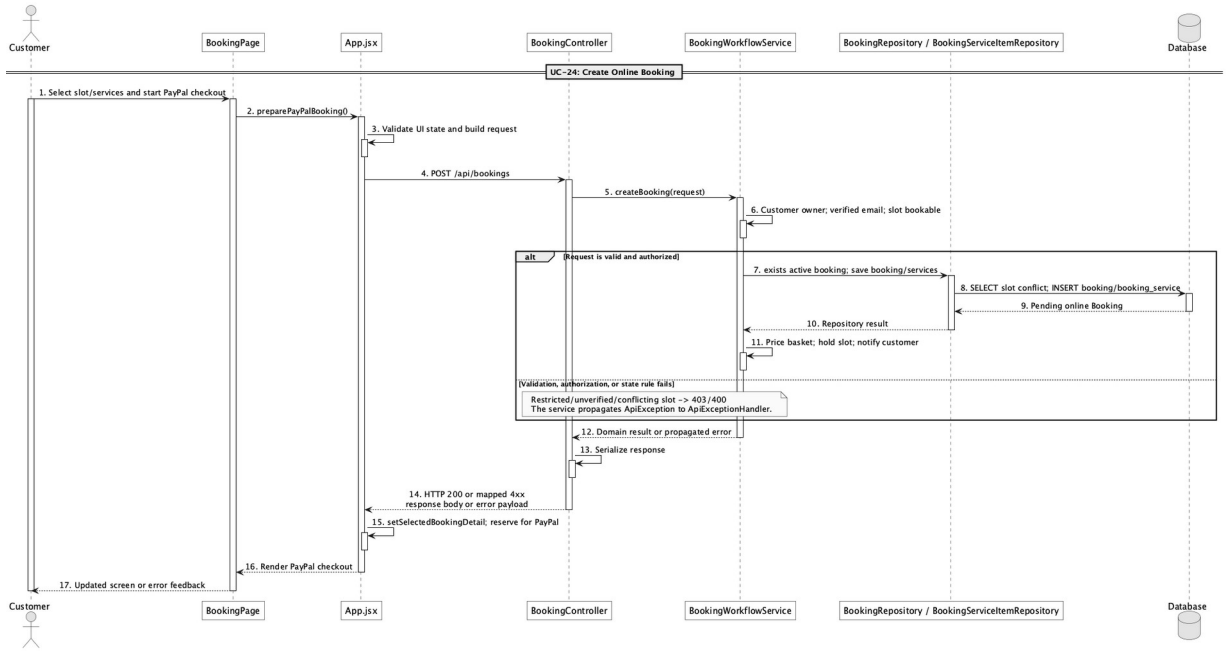
BookingWorkflowService

No	Method	Description
01	public Map<String, Object> createBooking(ApiRequests.BookingCreate request)	Enforces that a Customer creates only their own online booking, then prices and persists it as pending.

DomainSupportService

No	Method	Description
01	void validateSlotBookable(Slot slot)	Rejects blocked, past, or actively reserved slots.

c. Sequence Diagram



d. Database Queries

== Create online booking ==

1. Create pending online booking

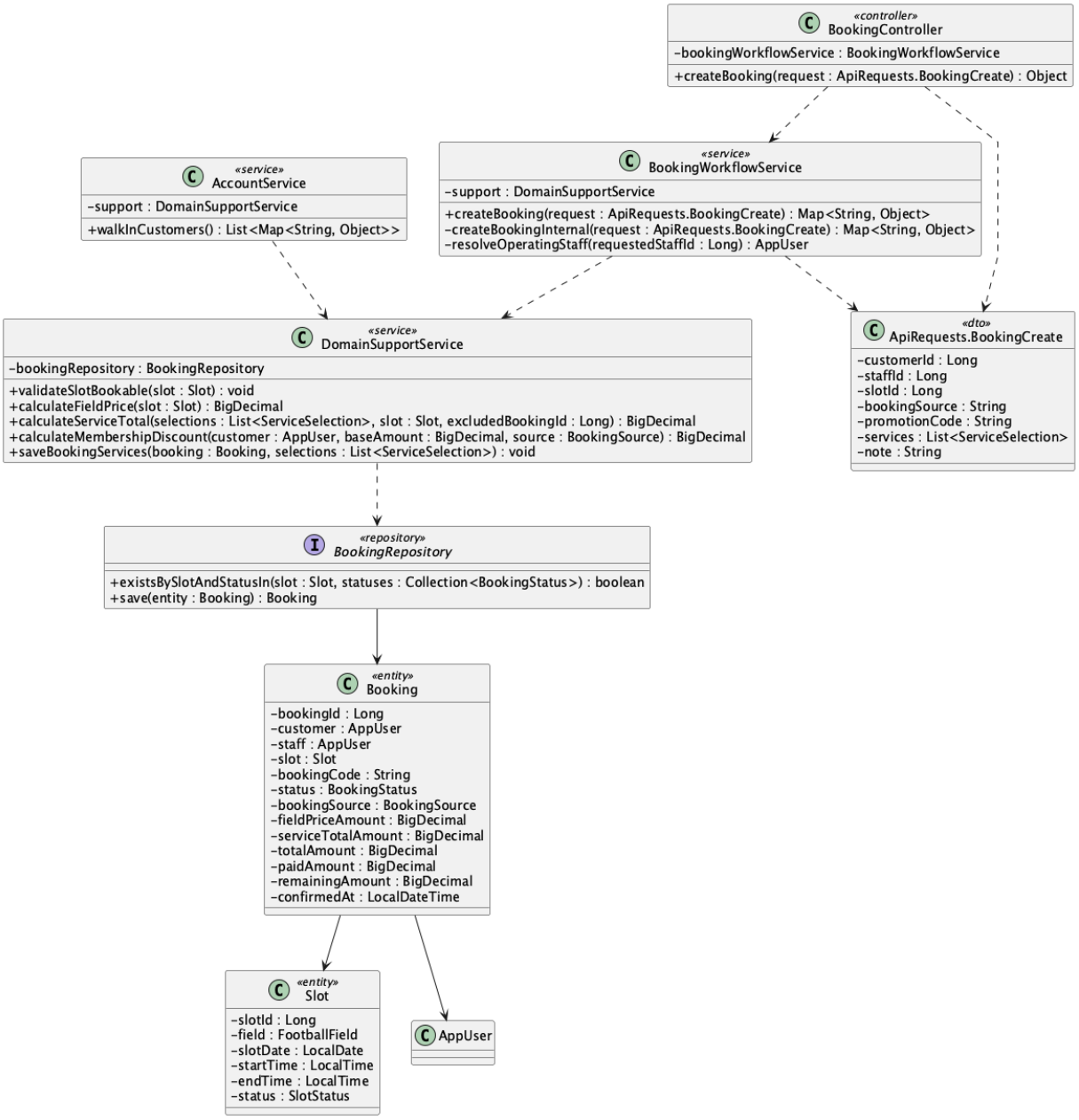
```
INSERT INTO booking
  (customer_id, slot_id, booking_code, status, booking_source, field_price_amount,
   service_total_amount, promotion_discount_amount, membership_discount_amount,
   total_amount, deposit_amount, paid_amount, remaining_amount, created_at, updated_at)
VALUES
  (:customerId, :slotId, :bookingCode, 'pending', 'online', :fieldPrice,
   :serviceTotal, :promotionDiscount, :membershipDiscount, :totalAmount,
```

:depositAmount, 0, :totalAmount, CURRENT_TIMESTAMP, CURRENT_TIMESTAMP);

Implementation note: Online booking is rejected for an unverified email or locked customer account.

24. Create walk-in booking

a. Class Diagram



b. Class Specifications

BookingWorkflowService

No	Method	Description
01	public Map<String, Object> createBooking(ApiRequests.BookingCreate request)	Allows Staff/Admin to create only walk-in bookings for an existing customer.

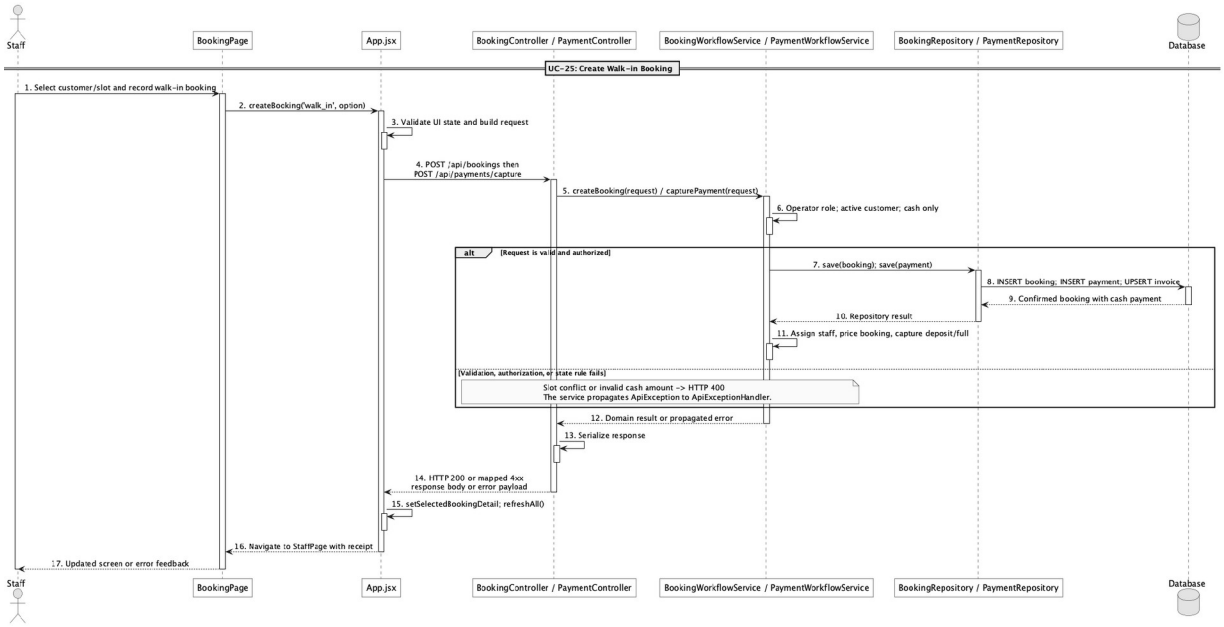
AccountService

No	Method	Description
01	public java.util.List<Map<String, Object>> walkInCustomers()	Provides the counter-safe customer directory used to select an existing customer.

DomainSupportService

No	Method	Description
01	BigDecimal calculateMembershipDiscount(AppUser customer, BigDecimal baseAmount, BookingSource source)	Returns zero membership discount for walk-in source.

c. Sequence Diagram



d. Database Queries

== Create walk-in booking ==

1. Create confirmed walk-in booking

INSERT INTO booking

(customer_id, staff_id, slot_id, booking_code, status, booking_source,

field_price_amount, service_total_amount, total_amount, deposit_amount,

paid_amount, remaining_amount, confirmed_at, created_at, updated_at)

VALUES

(:customerId, :staffId, :slotId, :bookingCode, 'confirmed', 'walk_in',

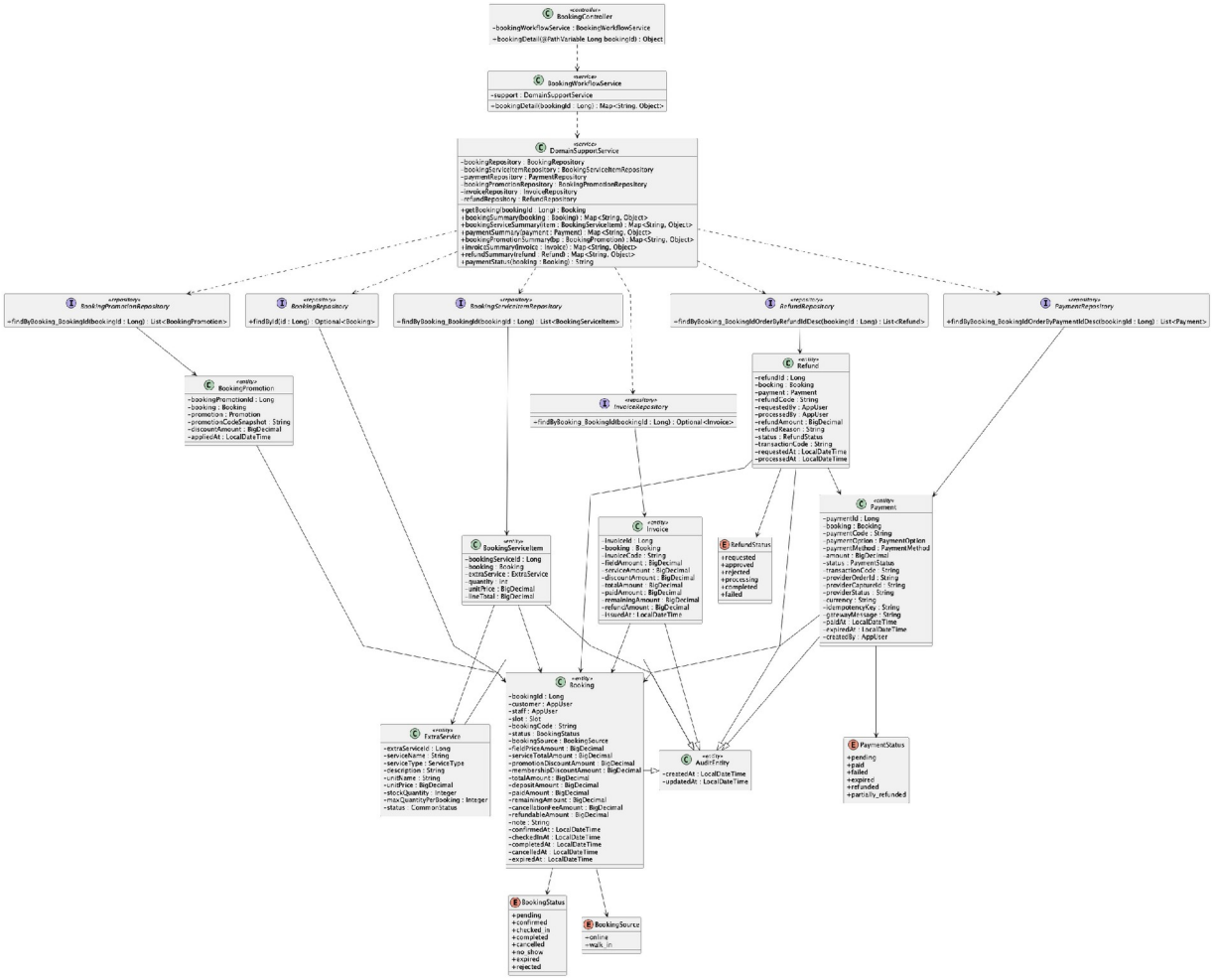
:fieldPrice, :serviceTotal, :totalAmount, :depositAmount,

0, :totalAmount, CURRENT_TIMESTAMP, CURRENT_TIMESTAMP, CURRENT_TIMESTAMP);

Implementation note: The counter booking starts confirmed; cash capture is a separate payment action.

25. View booking detail

a. Class Diagram



b. Class Specifications

BookingController

No	Method	Description
01	public Object bookingDetail(Long bookingId)	Exposes GET /api/bookings/{bookingId}.

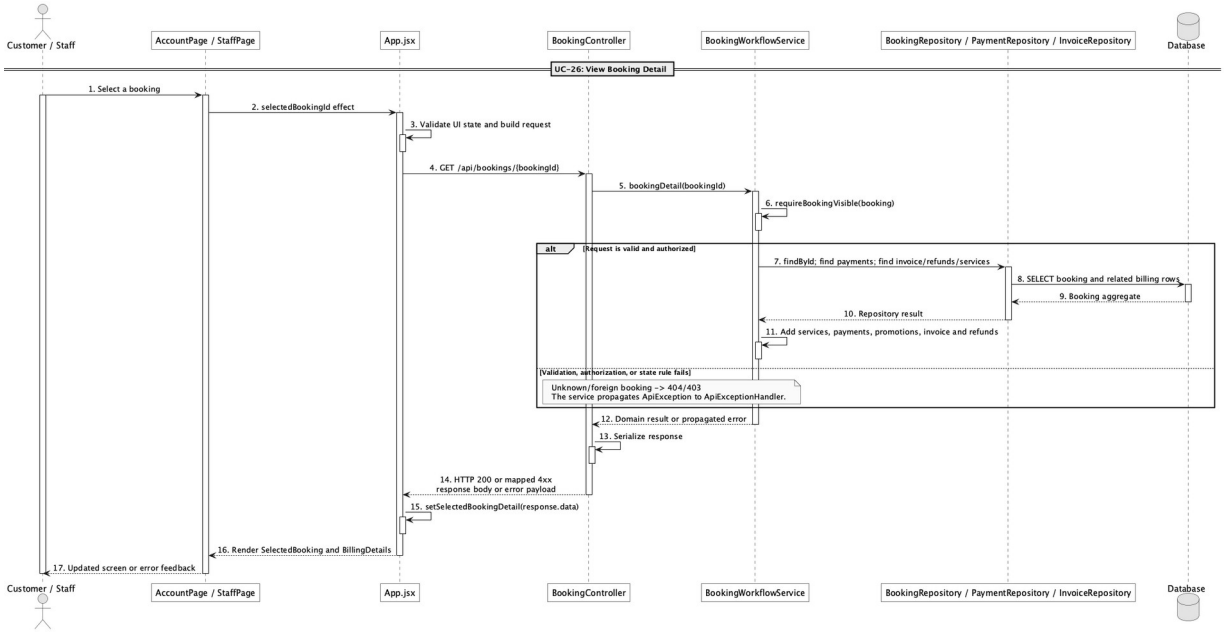
BookingWorkflowService

No	Method	Description
01	public Map<String, Object> bookingDetail(Long bookingId)	Returns the booking plus services, payments, promotions, invoice, refunds, and derived payment status.

InvoiceRepository

No	Method	Description
01	Optional<Invoice> findByBooking_BookingId(Long bookingId)	Loads the one-to-one booking invoice.

c. Sequence Diagram



d. Database Queries

== View booking detail ==

1. Booking detail core

```
SELECT b.*, s.slot_date, s.start_time, s.end_time, f.field_name,
    u.full_name AS customer_name, i.invoice_code, i.total_amount AS invoice_total,
    i.paid_amount AS invoice_paid, i.remaining_amount AS invoice_remaining
FROM booking b
JOIN app_user u ON u.user_id = b.customer_id
```

JOIN slot s ON s.slot_id = b.slot_id

JOIN field f ON f.field_id = s.field_id

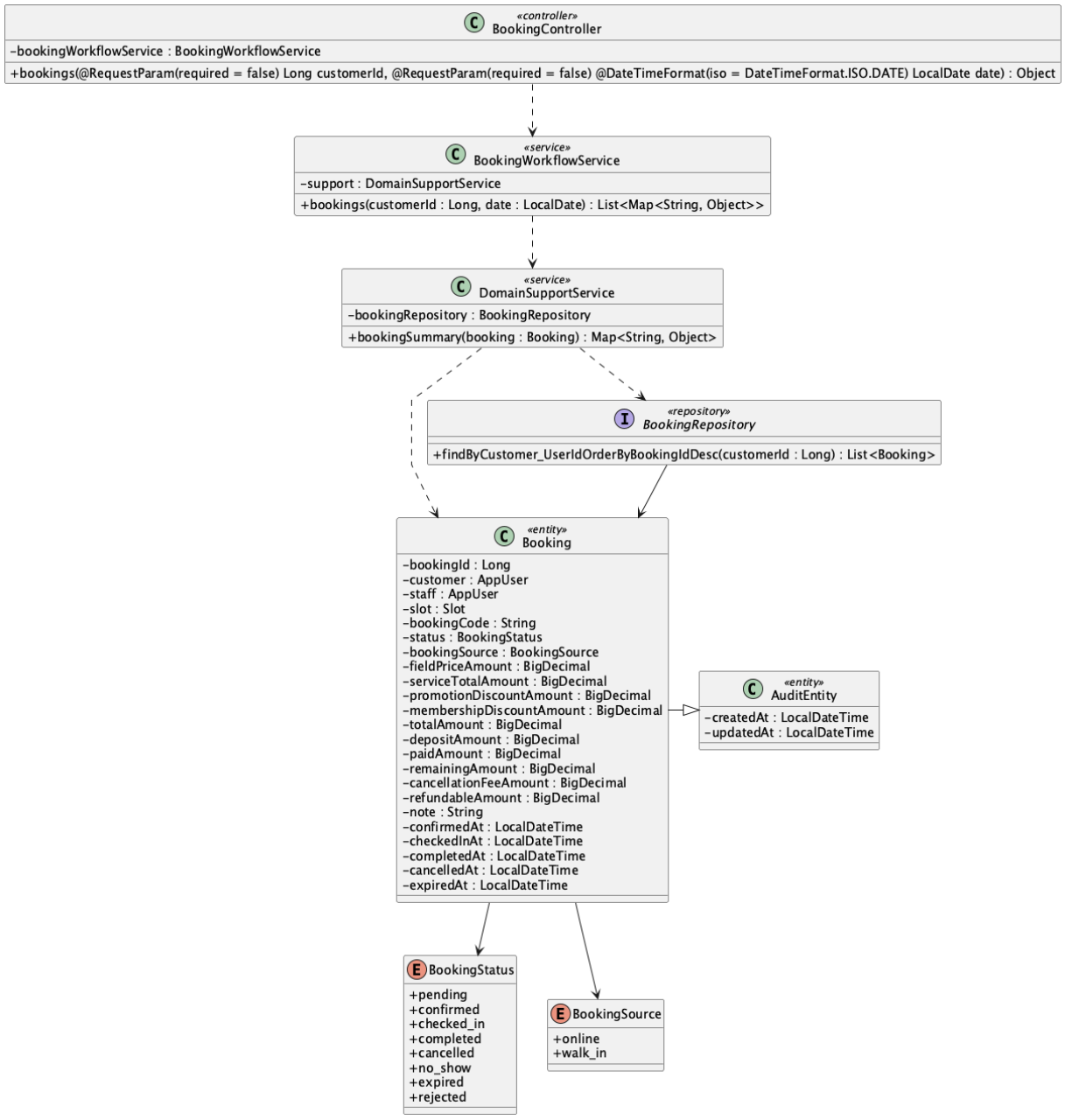
LEFT JOIN invoice i ON i.booking_id = b.booking_id

WHERE b.booking_id = :bookingId;

Implementation note: Visibility is limited to the owning customer or an operator when an authenticated principal is present.

26. View my bookings

a. Class Diagram



b. Class Specifications

BookingController

No	Method	Description
01	public Object bookings(Long customerId, LocalDate date)	Exposes the filtered booking list endpoint.

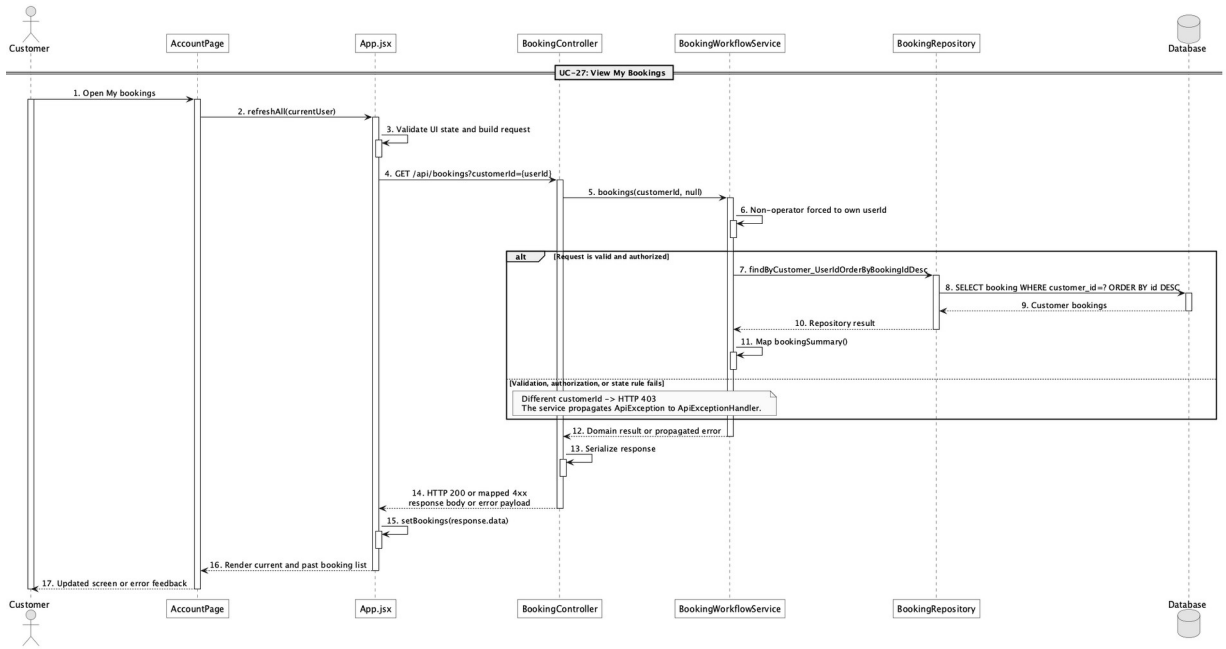
BookingWorkflowService

No	Method	Description
01	public List<Map<String, Object>> bookings(Long customerId, LocalDate date)	Forces non-operators to their own customer id and returns summaries.

BookingRepository

No	Method	Description
01	List<Booking> findByCustomer_UserIdOrderByBookingIdDesc(Long customerId)	Loads a customer's bookings newest first.

c. Sequence Diagram



d. Database Queries

== View my bookings ==

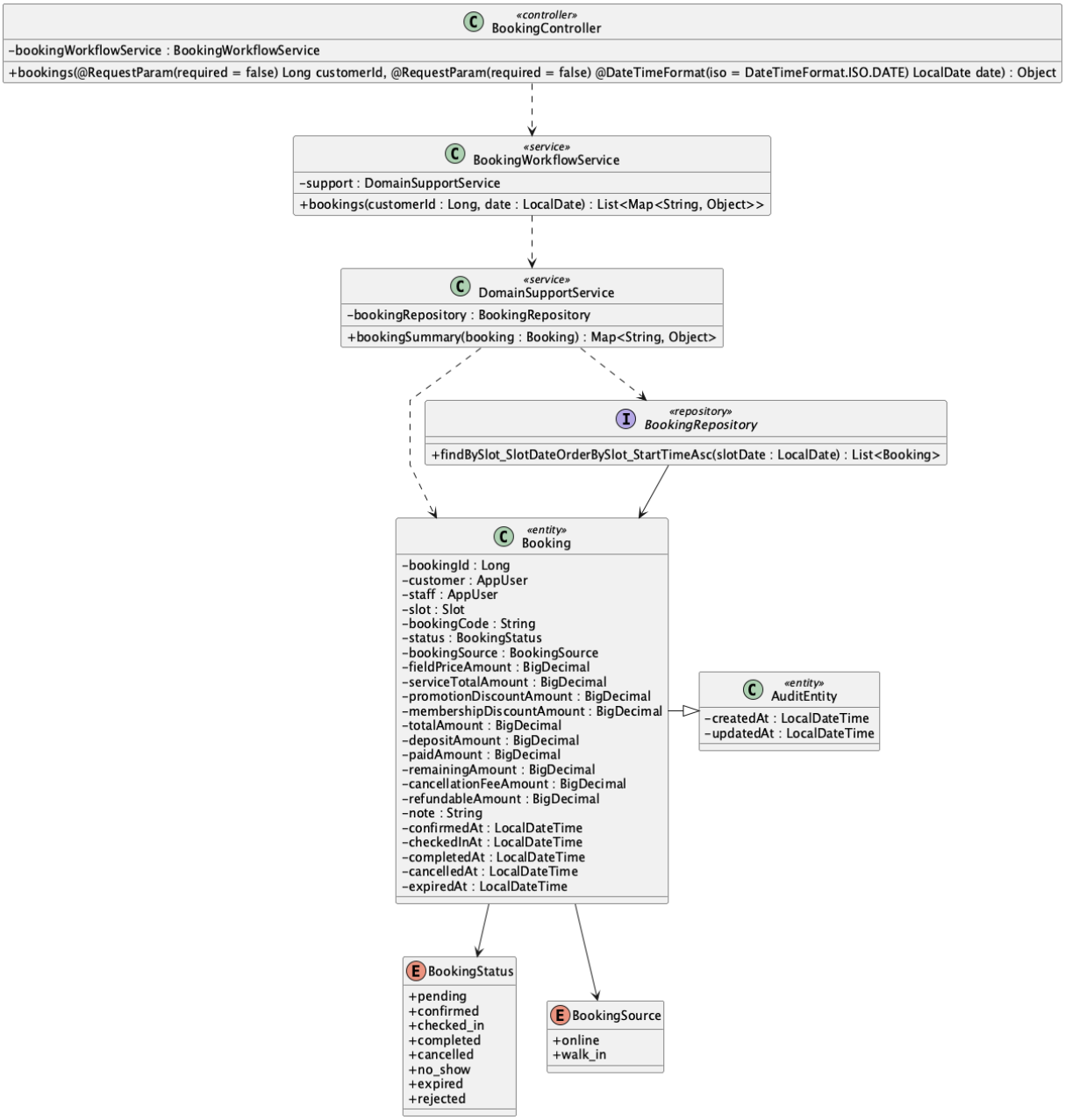
1. Customer booking history

SELECT b.booking_id, b.booking_code, b.status, b.booking_source, b.total_amount,

```
    b.paid_amount, b.remaining_amount, s.slot_date, s.start_time, s.end_time, f.field_name
FROM booking b
JOIN slot s ON s.slot_id = b.slot_id
JOIN field f ON f.field_id = s.field_id
WHERE b.customer_id = :currentCustomerId
ORDER BY b.booking_id DESC;
```

27. View booking calendar

a. Class Diagram



b. Class Specifications

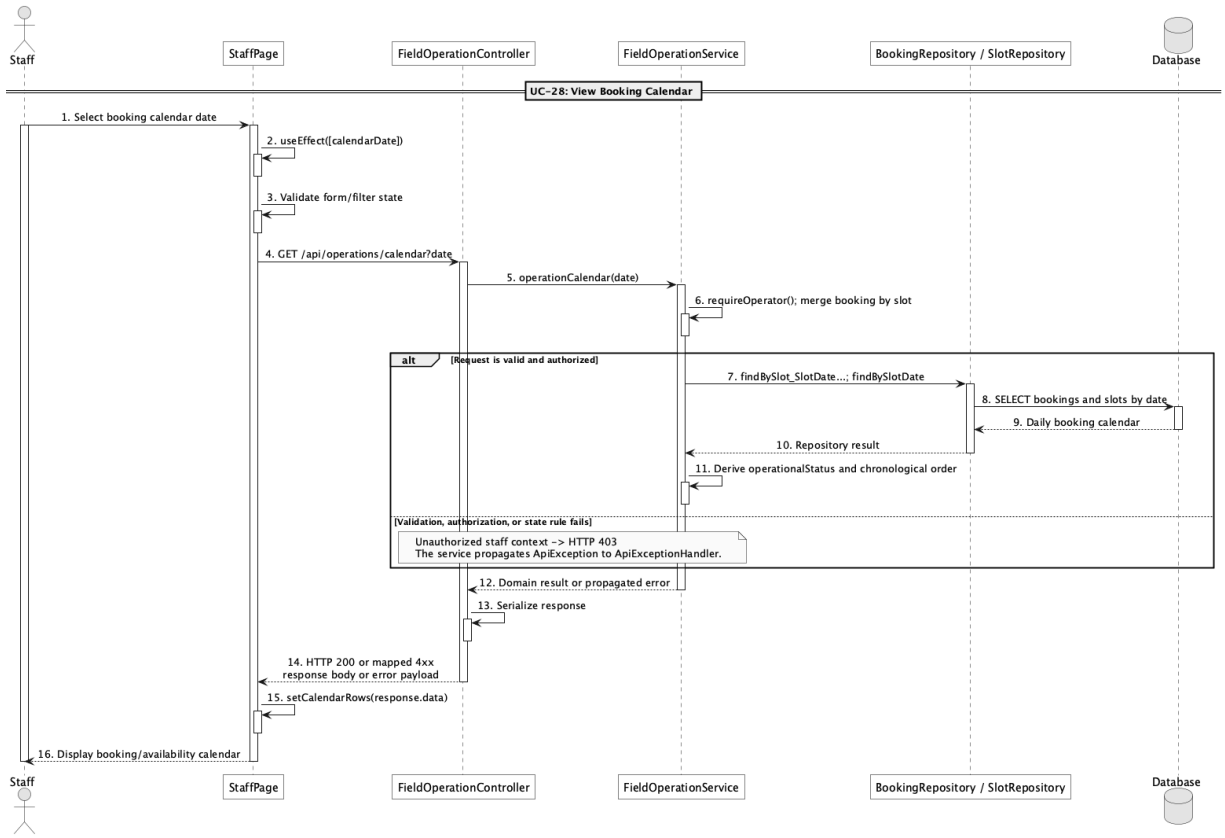
BookingWorkflowService

No	Method	Description
01	public List<Map<String, Object>> bookings(Long customerId, LocalDate date)	Returns booking summaries filtered by date for operators.

BookingRepository

No	Method	Description
01	List<Booking> findBySlot_SlotDateOrderBySlot_StartTimeAsc(LocalDate slotDate)	Orders bookings chronologically for a date.

c. Sequence Diagram



d. Database Queries

== View booking calendar ==

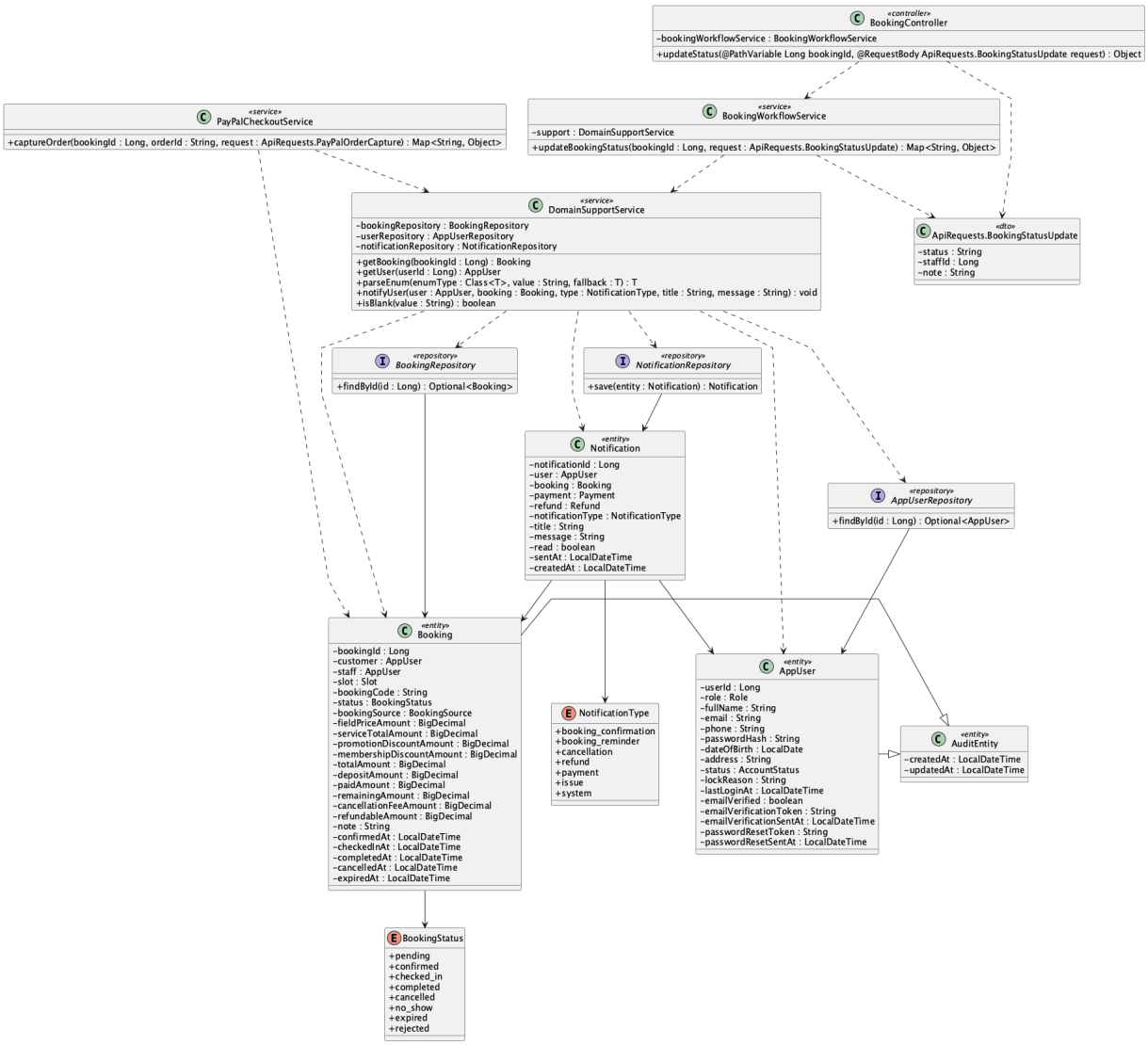
1. Bookings by calendar date

```
SELECT b.booking_id, b.booking_code, b.status, b.customer_id, b.staff_id,
    f.field_name, s.slot_date, s.start_time, s.end_time
FROM booking b
JOIN slot s ON s.slot_id = b.slot_id
JOIN field f ON f.field_id = s.field_id
WHERE s.slot_date = :calendarDate
ORDER BY s.start_time;
```

Implementation note: The implemented view is daily rather than a recurring/monthly calendar model.

28. Confirm booking

a. Class Diagram



b. Class Specifications

BookingController

No	Method	Description
01	public Object updateStatus(Long bookingId, ApiRequests.BookingStatusUpdate request)	Routes lifecycle status updates.

BookingWorkflowService

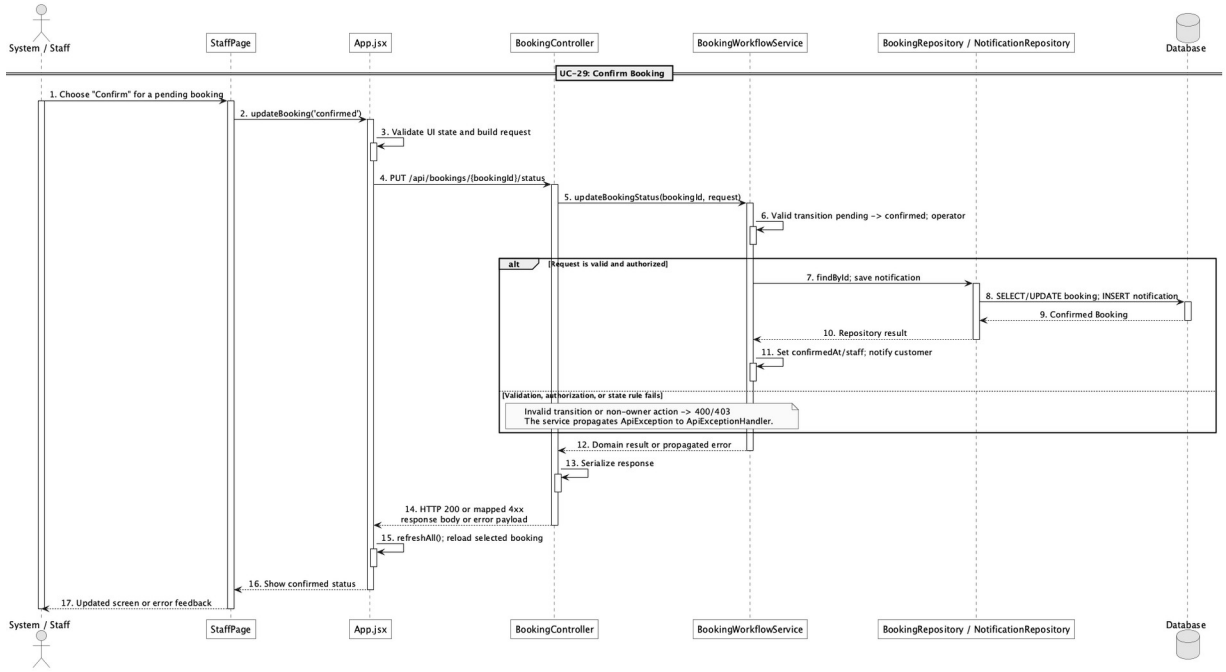
No	Method	Description
01	public Map<String, Object>	Validates pending-to-confirmed transition,

No	Method	Description
	updateBookingStatus(Long bookingId, ApiRequests.BookingStatusUpdate request)	timestamps it, assigns operator, and notifies the customer.

PayPalCheckoutService

No	Method	Description
01	public Map<String, Object> captureOrder(Long bookingId, String orderId, ApiRequests.PayPalOrderCapture request)	Also confirms a pending online booking when captured funds reach the deposit amount.

c. Sequence Diagram



d. Database Queries

== Confirm booking ==

1. Confirm pending booking

UPDATE booking

SET status = 'confirmed', confirmed_at = CURRENT_TIMESTAMP, staff_id = :staffId,

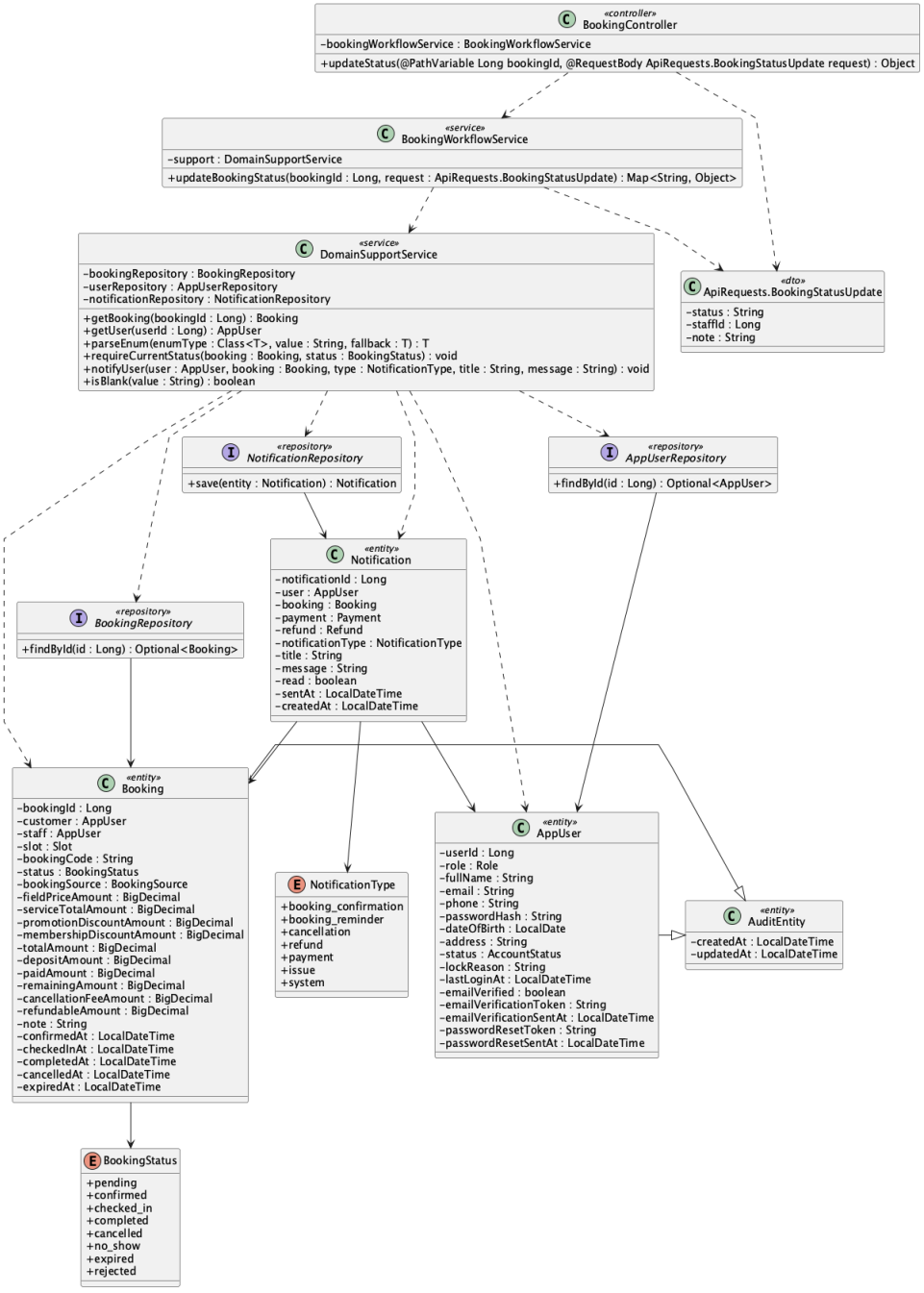
  updated_at = CURRENT_TIMESTAMP

WHERE booking_id = :bookingId AND status = 'pending';

Implementation note: Confirmation can result from an operator lifecycle action or successful qualifying PayPal capture.

29. Reject booking request

a. Class Diagram



b. Class Specifications

BookingController

No	Method	Description
01	public Object updateStatus(Long bookingId, ApiRequests.BookingStatusUpdate request)	Accepts the rejected target status.

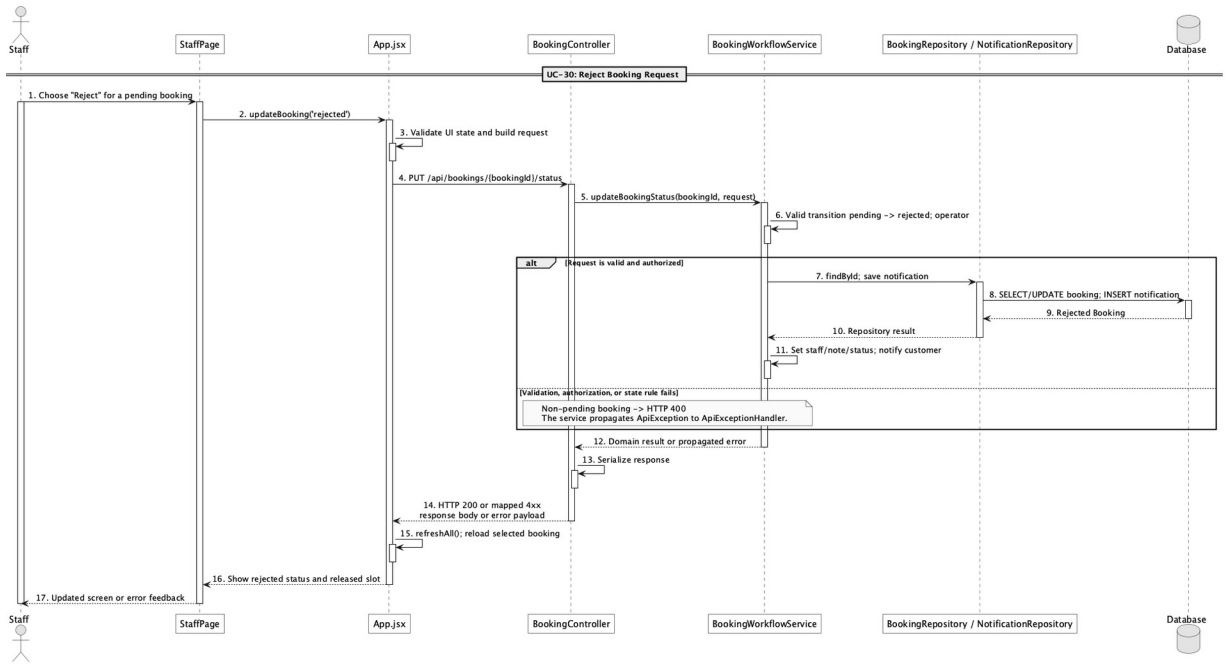
BookingWorkflowService

No	Method	Description
01	public Map<String, Object> updateBookingStatus(Long bookingId, ApiRequests.BookingStatusUpdate request)	Allows pending-to-rejected and creates a customer notification.

DomainSupportService

No	Method	Description
01	void notifyUser(AppUser user, Booking booking, NotificationType type, String title, String message)	Stores the rejection notification.

c. Sequence Diagram



d. Database Queries

== Reject booking request ==

1. Reject pending booking

UPDATE booking

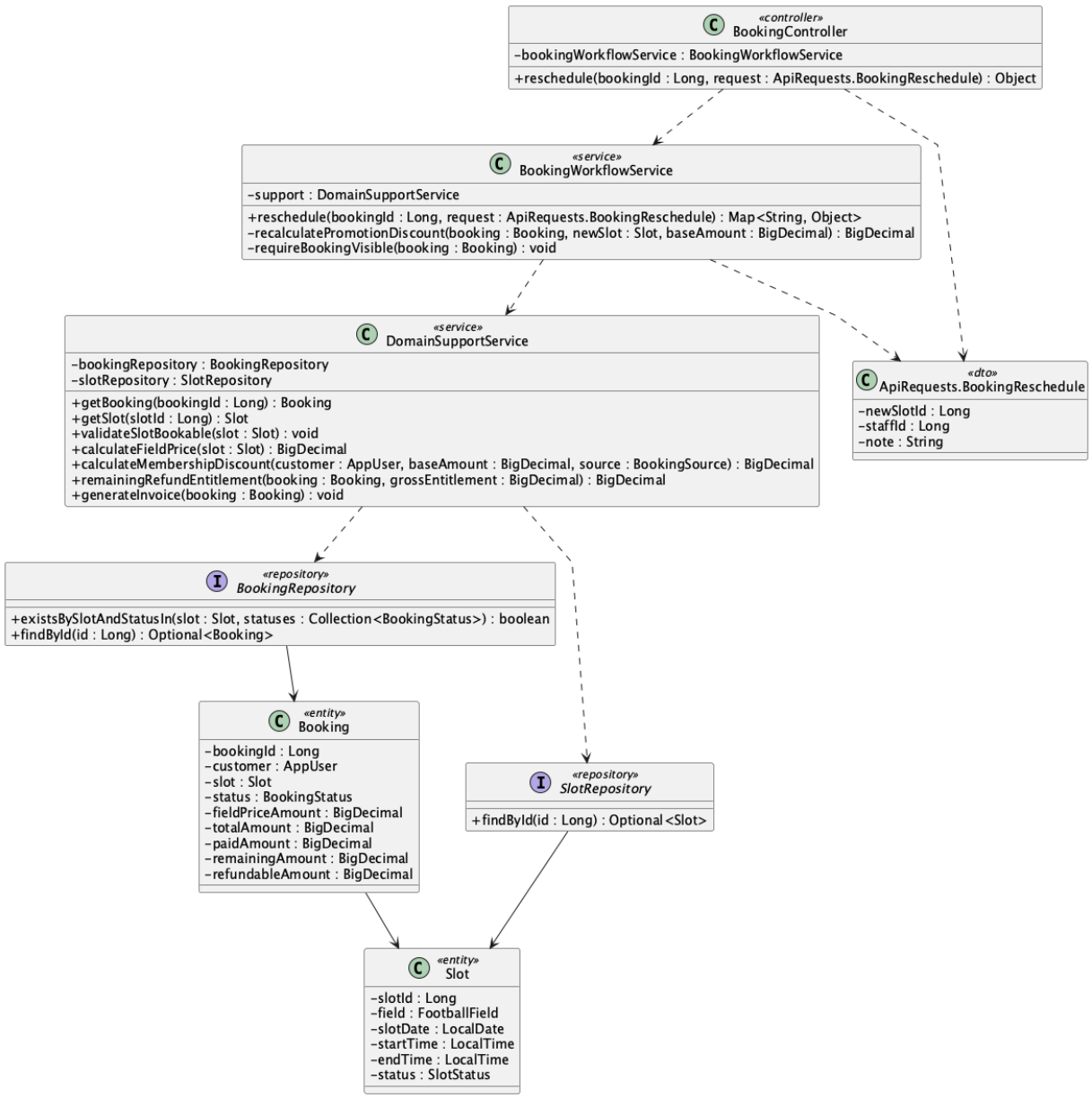
SET status = 'rejected', staff_id = :staffId, note = :note, updated_at = CURRENT_TIMESTAMP

WHERE booking_id = :bookingId AND status = 'pending';

Implementation note: There is no rejected_at column; the audit updated_at and optional note are the persisted timing/context.

30. Reschedule booking

a. Class Diagram



b. Class Specifications

BookingController

No	Method	Description
01	public Object reschedule(Long bookingId, ApiRequests.BookingReschedule request)	Exposes PUT /api/bookings/{bookingId}/reschedule.

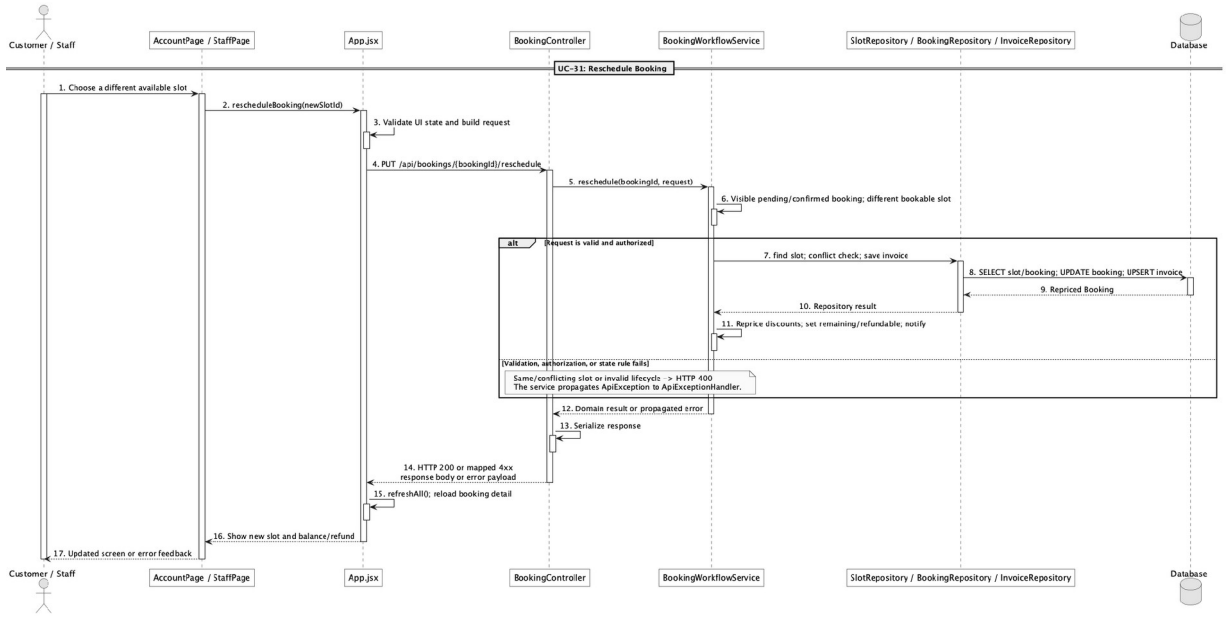
BookingWorkflowService

No	Method	Description
01	public Map<String, Object> reschedule(Long bookingId, ApiRequests.BookingReschedule request)	Moves pending/confirmed booking to a free slot while preserving payment history and repricing totals.

DomainSupportService

No	Method	Description
01	void validateSlotBookable(Slot slot)	Prevents rescheduling into a blocked, past, or actively booked slot.

c. Sequence Diagram



d. Database Queries

== Reschedule booking ==

1. Persist rescheduled slot and price delta

UPDATE booking

SET slot_id = :newSlotId, field_price_amount = :fieldPrice,

promotion_discount_amount = :promotionDiscount, membership_discount_amount = :membershipDiscount,

total_amount = :newTotal, deposit_amount = :depositAmount,

remaining_amount = GREATEST(:newTotal - paid_amount, 0),

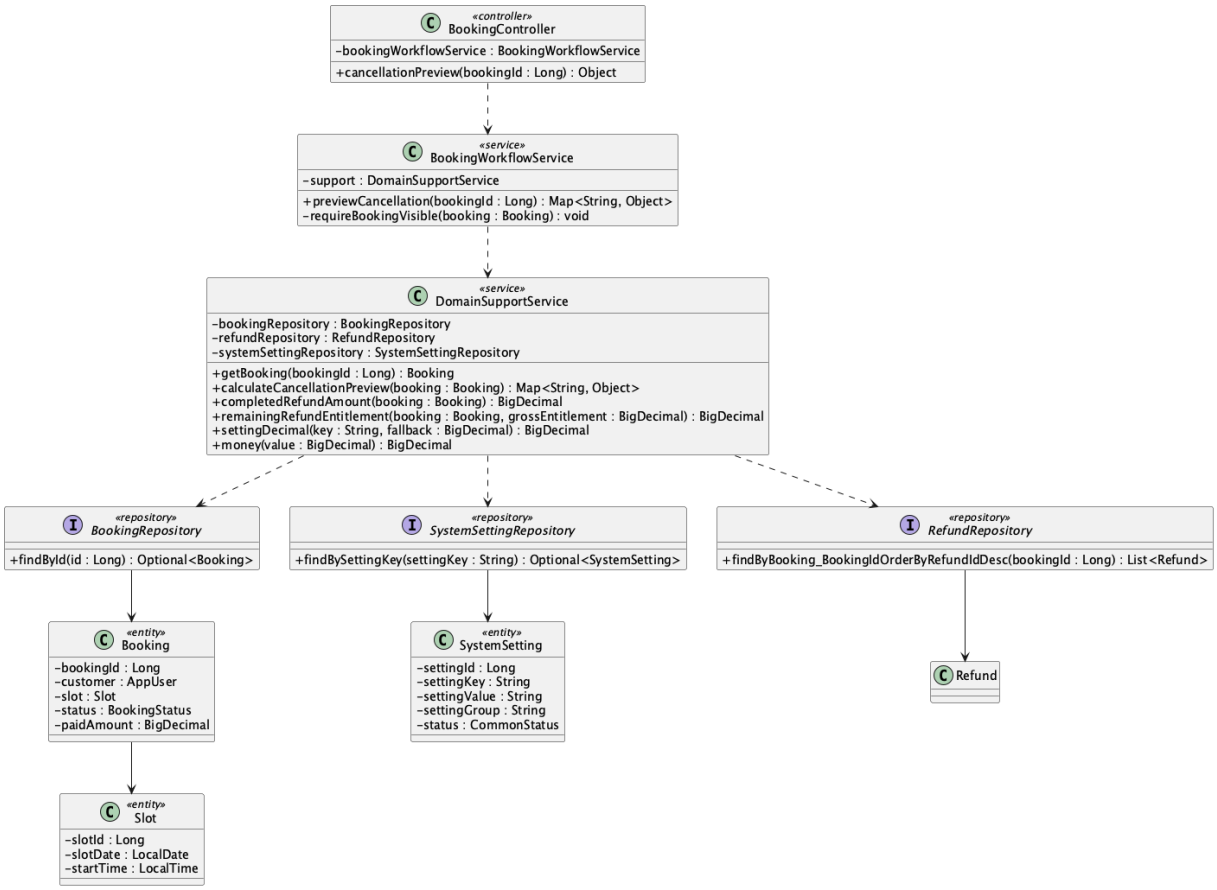
refundable_amount = GREATEST(paid_amount - :newTotal, 0), updated_at = CURRENT_TIMESTAMP

WHERE booking_id = :bookingId AND status IN ('pending','confirmed');

Implementation note: No payment rows are deleted or rewritten during reschedule.

31. Preview cancellation fee/refund

a. Class Diagram



b. Class Specifications

BookingController

No	Method	Description
01	public Object cancellationPreview(Long	Exposes GET /api/bookings/{bookingId}/cancellation-preview.

No	Method	Description
	bookingId)	

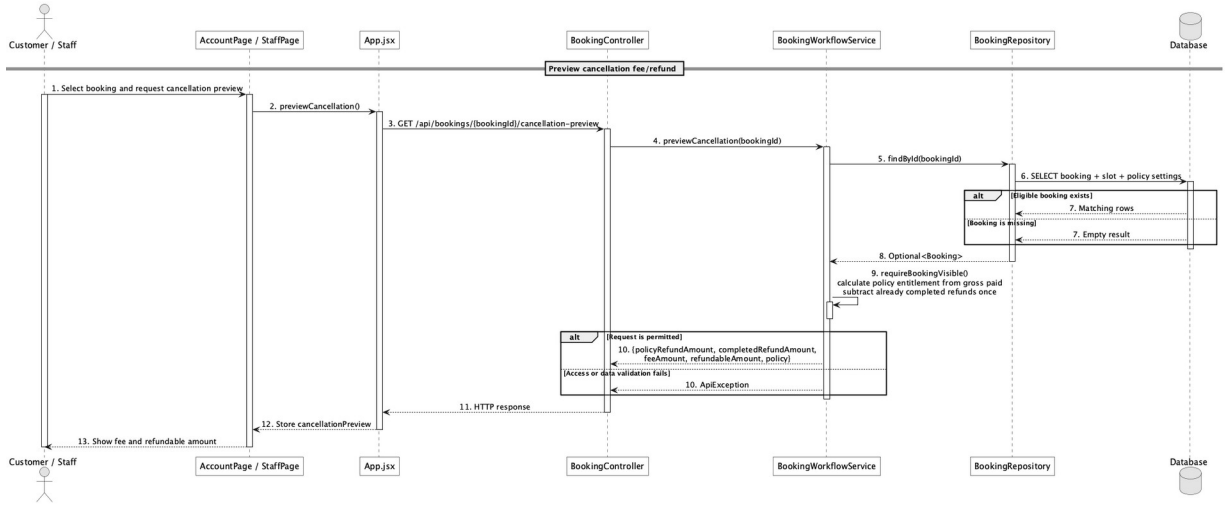
BookingWorkflowService

No	Method	Description
01	public Map<String, Object> previewCancellation(Long bookingId)	Checks visibility and delegates policy calculation without mutating the booking.

DomainSupportService

No	Method	Description
01	Map<String, Object> calculateCancellationPreview(Booking booking)	Calculates fee/refundable amount from paid amount, start time, and refund policy settings.

c. Sequence Diagram



d. Database Queries

== Preview cancellation fee/refund ==

1. Cancellation inputs and policy

SELECT b.booking_id, b.booking_code, b.status, b.paid_amount,

s.slot_date, s.start_time,

MAX(CASE WHEN ss.setting_key = 'refund.before_24h_percent' THEN ss.setting_value END) AS before_24h_percent,

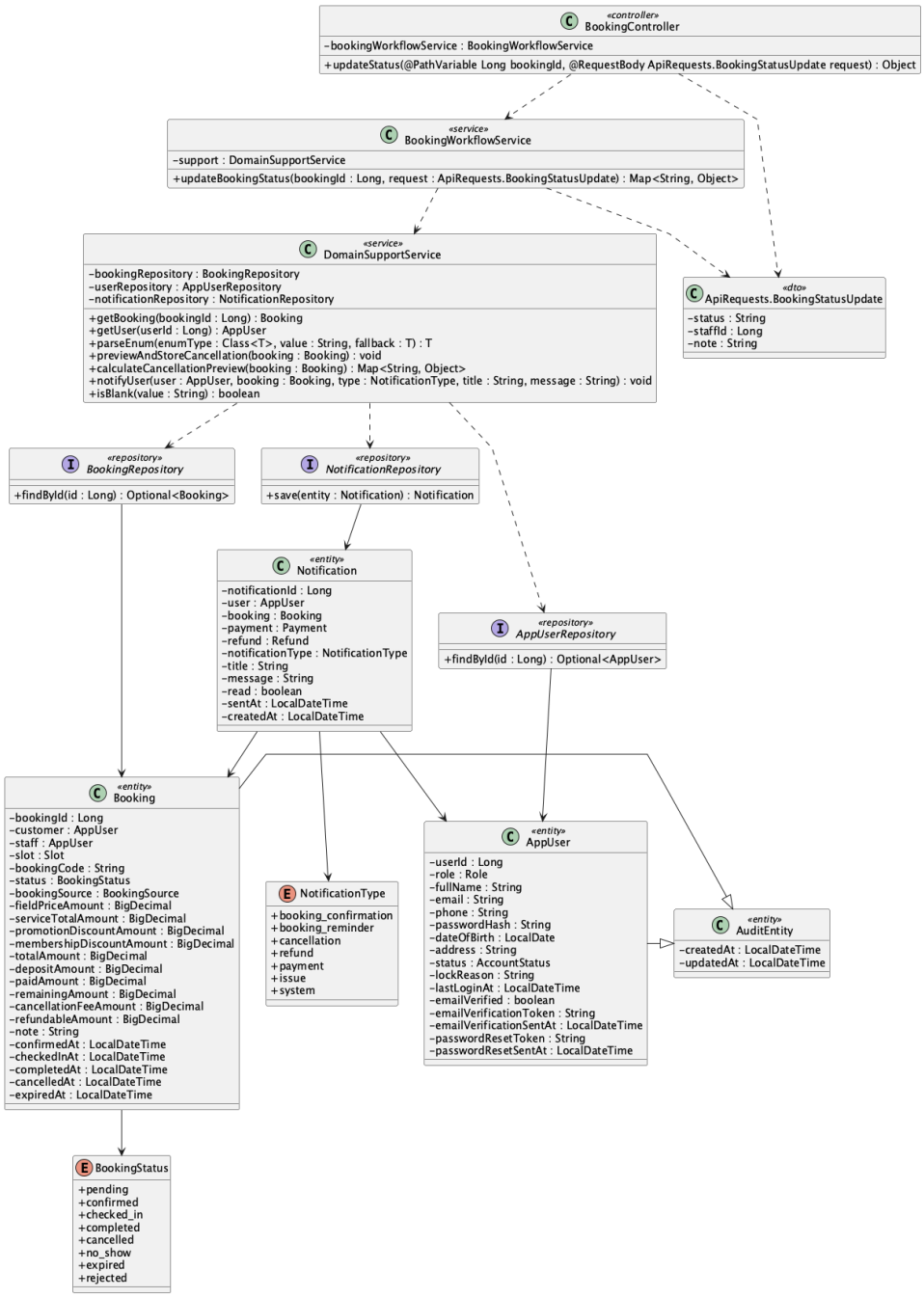
MAX(CASE WHEN ss.setting_key = 'refund.same_day_percent' THEN ss.setting_value END) AS same_day_percent

```
FROM booking b
JOIN slot s ON s.slot_id = b.slot_id
CROSS JOIN system_setting ss
WHERE b.booking_id = :bookingId
GROUP BY b.booking_id, b.booking_code, b.status, b.paid_amount, s.slot_date, s.start_time;
```

Implementation note: Preview is valid only for pending or confirmed bookings.

32. Cancel booking

a. Class Diagram



b. Class Specifications

BookingWorkflowService

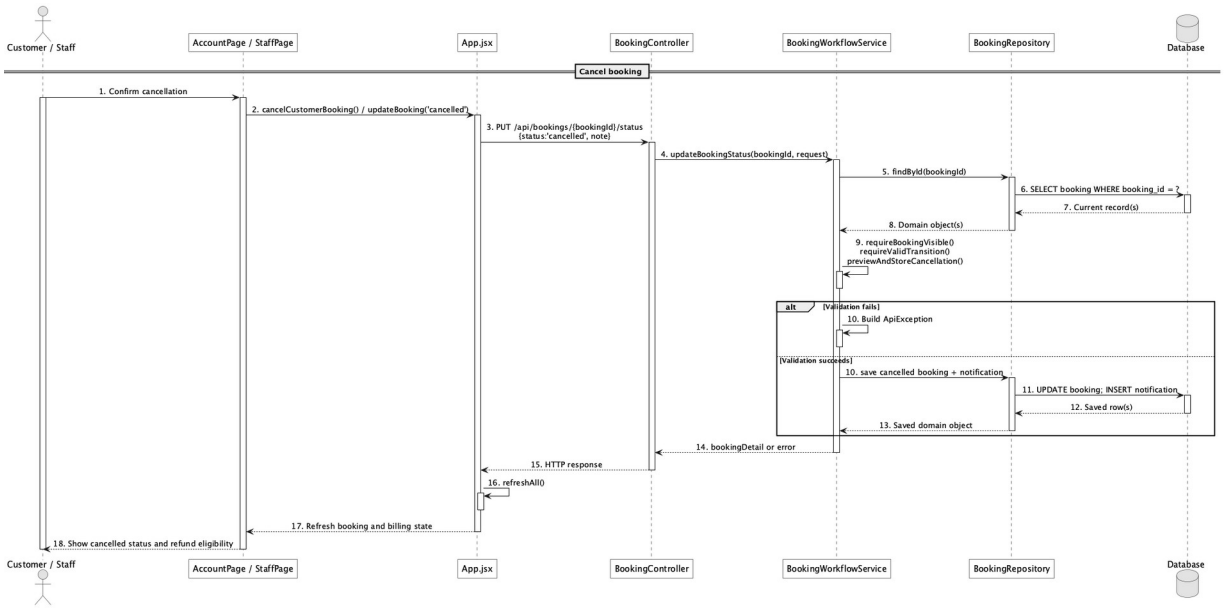
No	Method	Description
01	public Map<String, Object> updateBookingStatus(Long bookingId, ApiRequests.BookingStatusUpdate	Allows owner cancellation or operator cancellation from eligible states.

No	Method	Description
	request)	

DomainSupportService

No	Method	Description
01	void previewAndStoreCancellation(Booking booking)	Stores cancellation fee and refundable amount using the configured policy.
02	void notifyUser(AppUser user, Booking booking, NotificationType type, String title, String message)	Creates the cancellation notification.

c. Sequence Diagram



d. Database Queries

== Cancel booking ==

1. Cancel and store refund calculation

UPDATE booking

SET status = 'cancelled', cancelled_at = CURRENT_TIMESTAMP,

cancellation_fee_amount = :feeAmount, refundable_amount = :refundableAmount,

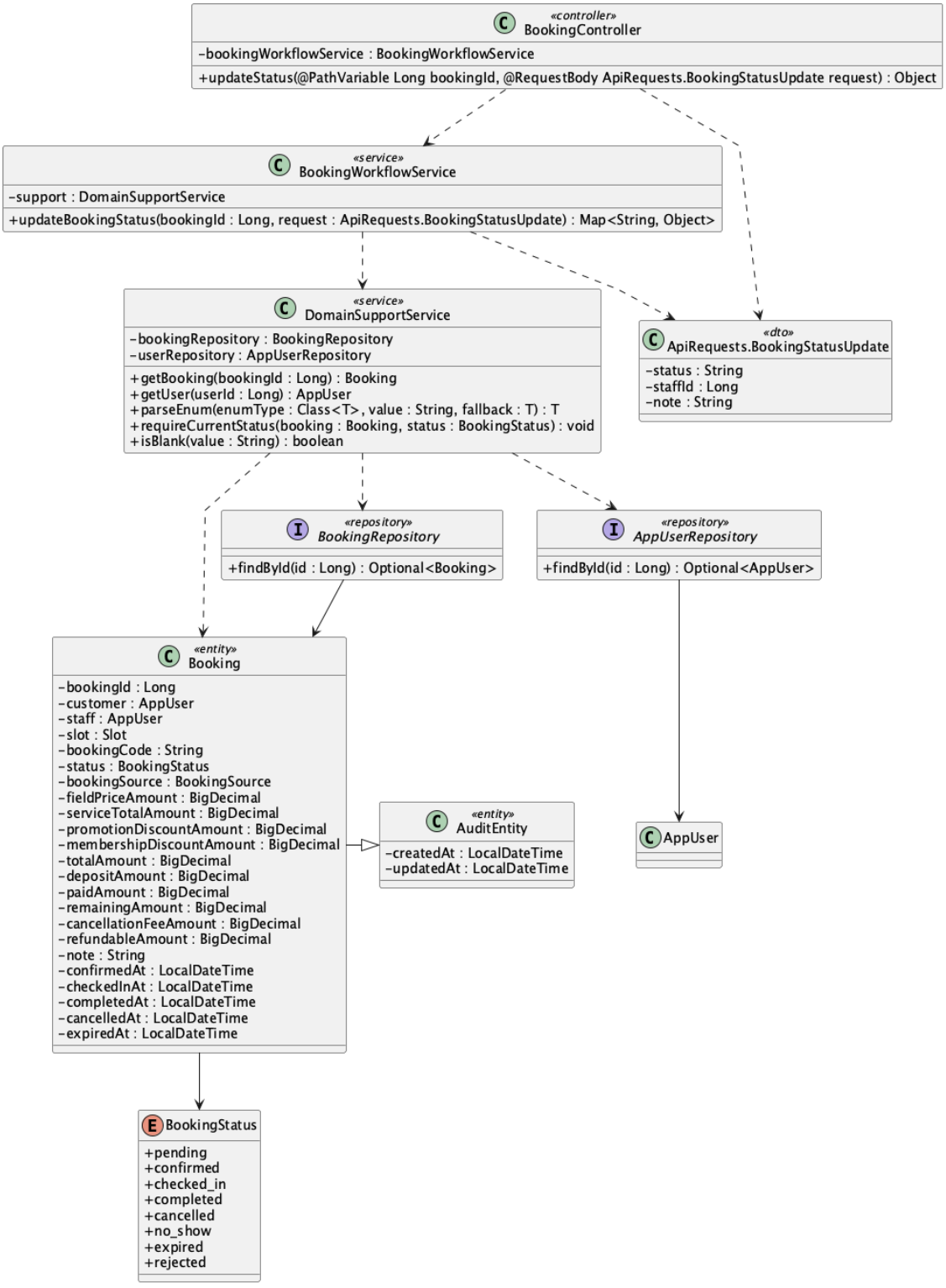
note = COALESCE(:note, note), updated_at = CURRENT_TIMESTAMP

WHERE booking_id = :bookingId AND status IN ('pending','confirmed');

Implementation note: Cancellation computes eligibility but does not itself complete a refund transaction.

33. Check-in booking

a. Class Diagram



b. Class Specifications

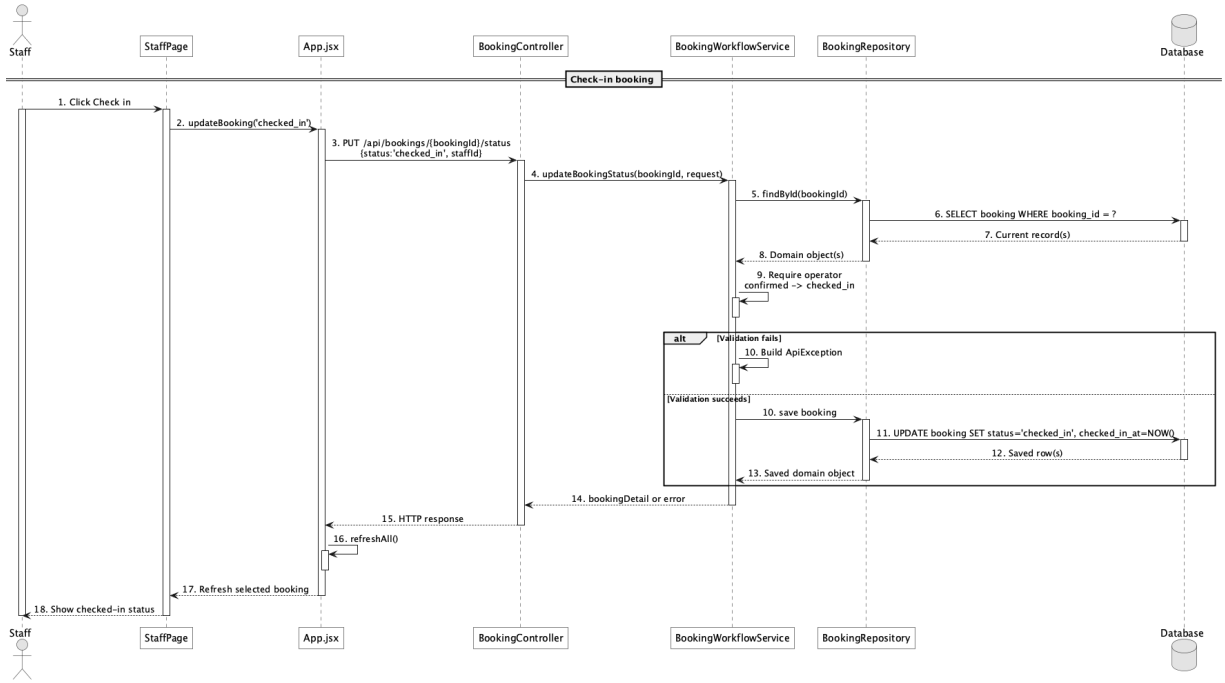
BookingController

No	Method	Description
01	public Object updateStatus(Long bookingId, ApiRequests.BookingStatusUpdate request)	Receives checked_in status from the staff workspace.

BookingWorkflowService

No	Method	Description
01	public Map<String, Object> updateBookingStatus(Long bookingId, ApiRequests.BookingStatusUpdate request)	Allows only confirmed-to-checked_in and records checked_in_at.

c. Sequence Diagram



d. Database Queries

== Check-in booking ==

1. Check in confirmed booking

UPDATE booking

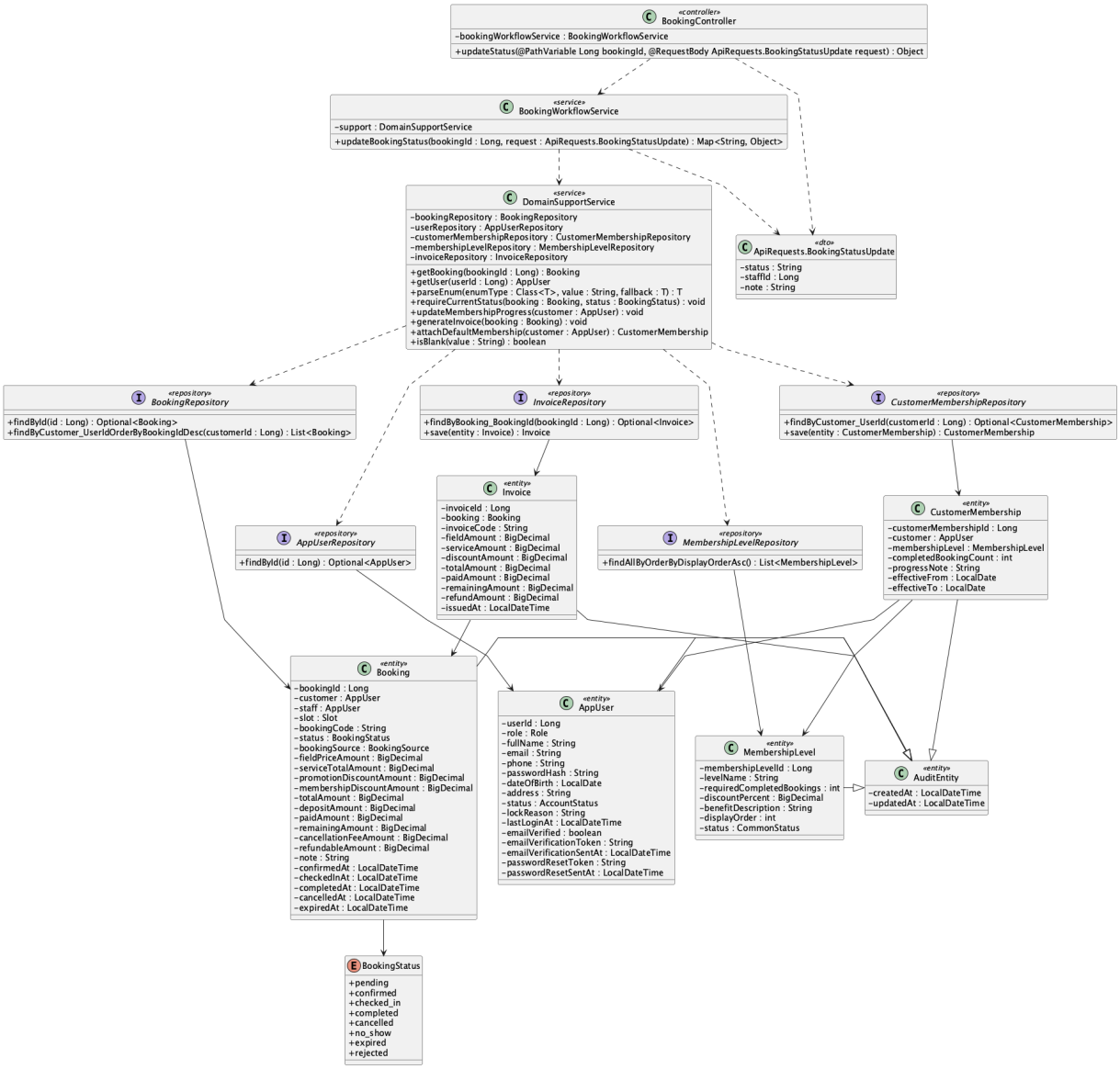
SET status = 'checked_in', checked_in_at = CURRENT_TIMESTAMP, staff_id = :staffId,

updated_at = CURRENT_TIMESTAMP

WHERE booking_id = :bookingId AND status = 'confirmed';

34. Complete booking

a. Class Diagram



b. Class Specifications

BookingWorkflowService

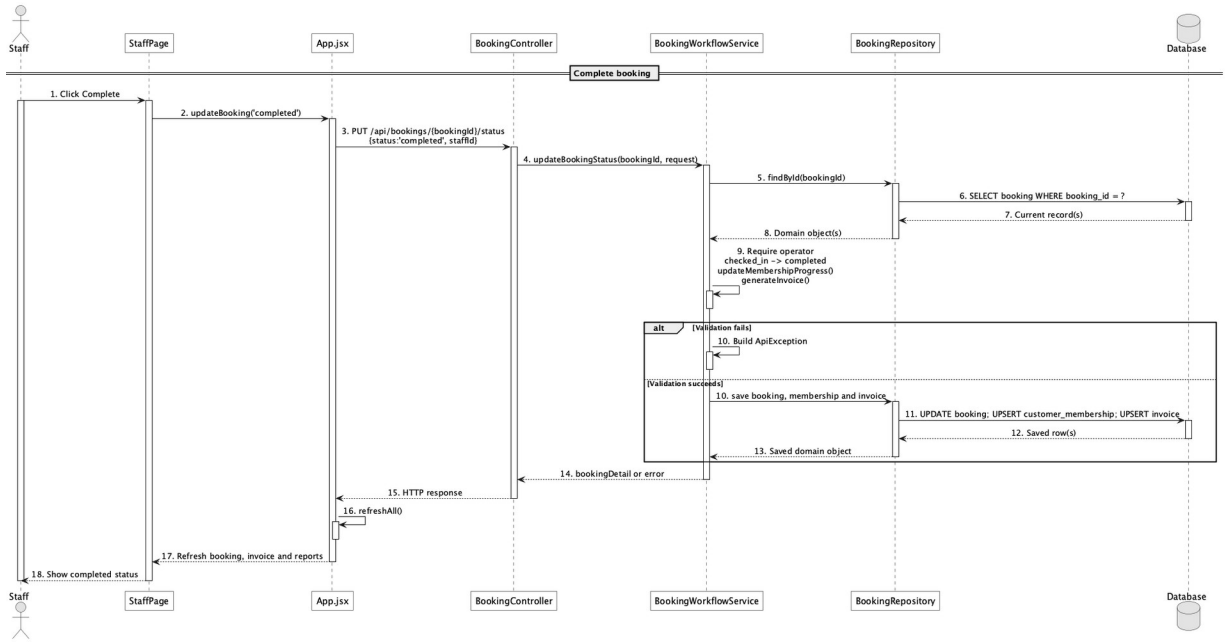
No	Method	Description
01	public Map<String, Object> updateBookingStatus(Long bookingId, ApiRequests.BookingStatusUpdate	Allows checked_in-to-completed, timestamps completion, updates membership, and generates invoice.

No	Method	Description
	request)	

DomainSupportService

No	Method	Description
01	void updateMembershipProgress(AppUser customer)	Refreshes the lifetime audit count and resolves the highest active level whose configured period rule is met.
02	void generateInvoice(Booking booking)	Creates or refreshes the booking invoice.

c. Sequence Diagram



d. Database Queries

== Complete booking ==

1. Complete booking and count membership progress

UPDATE booking

SET status = 'completed', completed_at = CURRENT_TIMESTAMP, updated_at = CURRENT_TIMESTAMP

WHERE booking_id = :bookingId AND status = 'checked_in';

SELECT COUNT(*) AS completed_booking_count

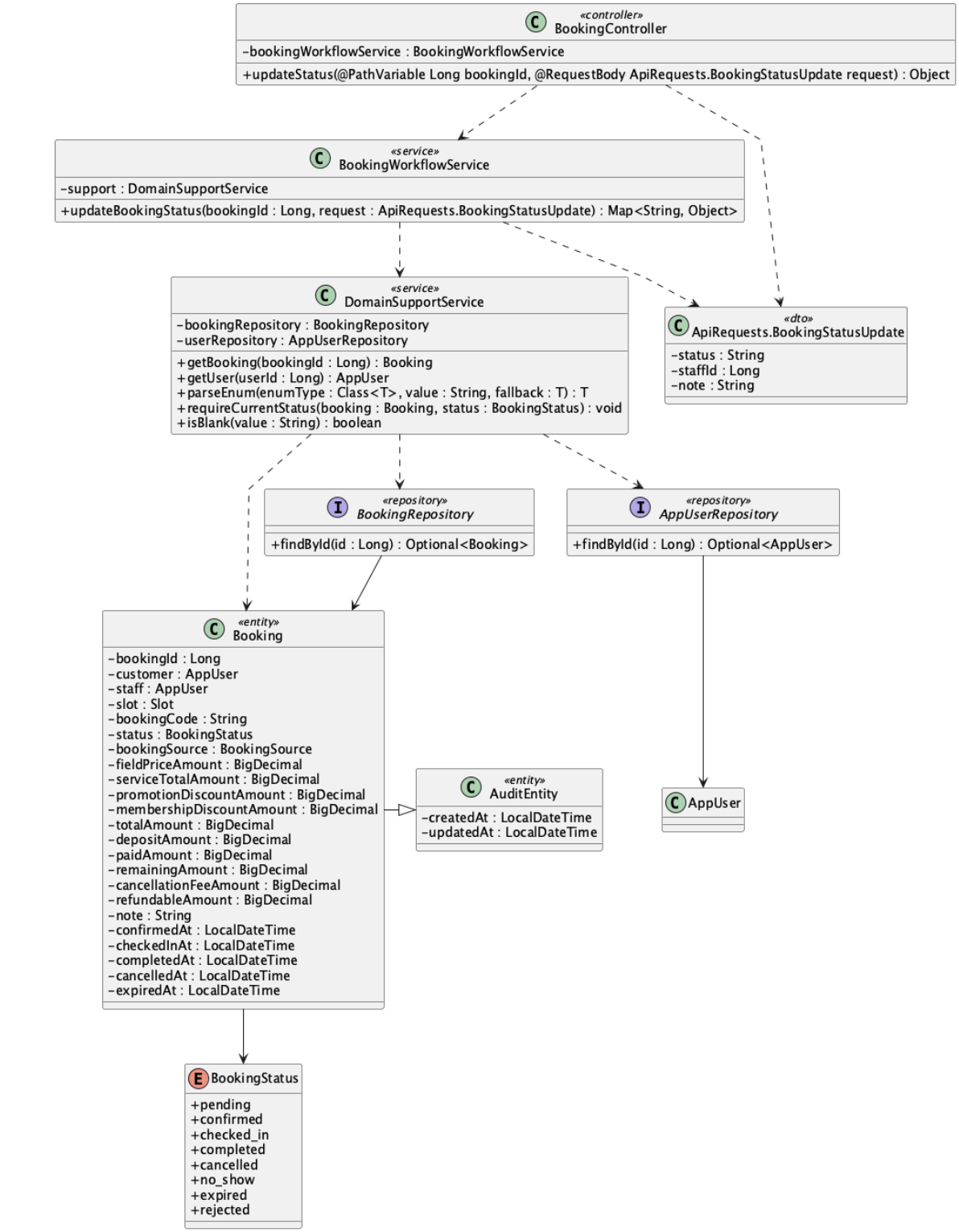
FROM booking

```
WHERE customer_id = :customerId AND status = 'completed';
```

Implementation note: Only completed bookings count; eligibility supports lifetime, current-month, or consecutive qualifying-week rules.

35. Mark no-show

a. Class Diagram



b. Class Specifications

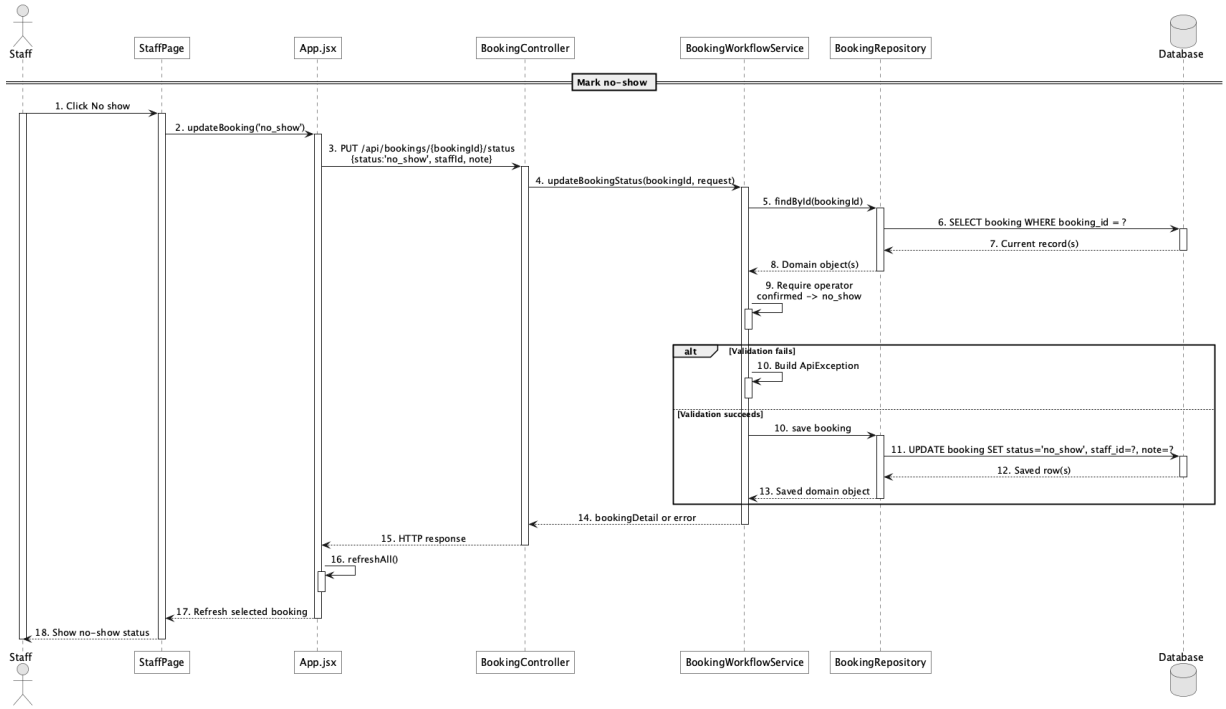
BookingController

No	Method	Description
01	public Object updateStatus(Long bookingId, ApiRequests.BookingStatusUpdate request)	Routes no_show status update.

BookingWorkflowService

No	Method	Description
01	public Map<String, Object> updateBookingStatus(Long bookingId, ApiRequests.BookingStatusUpdate request)	Allows confirmed-to-no_show through the explicit transition map.

c. Sequence Diagram



d. Database Queries

== Mark no-show ==

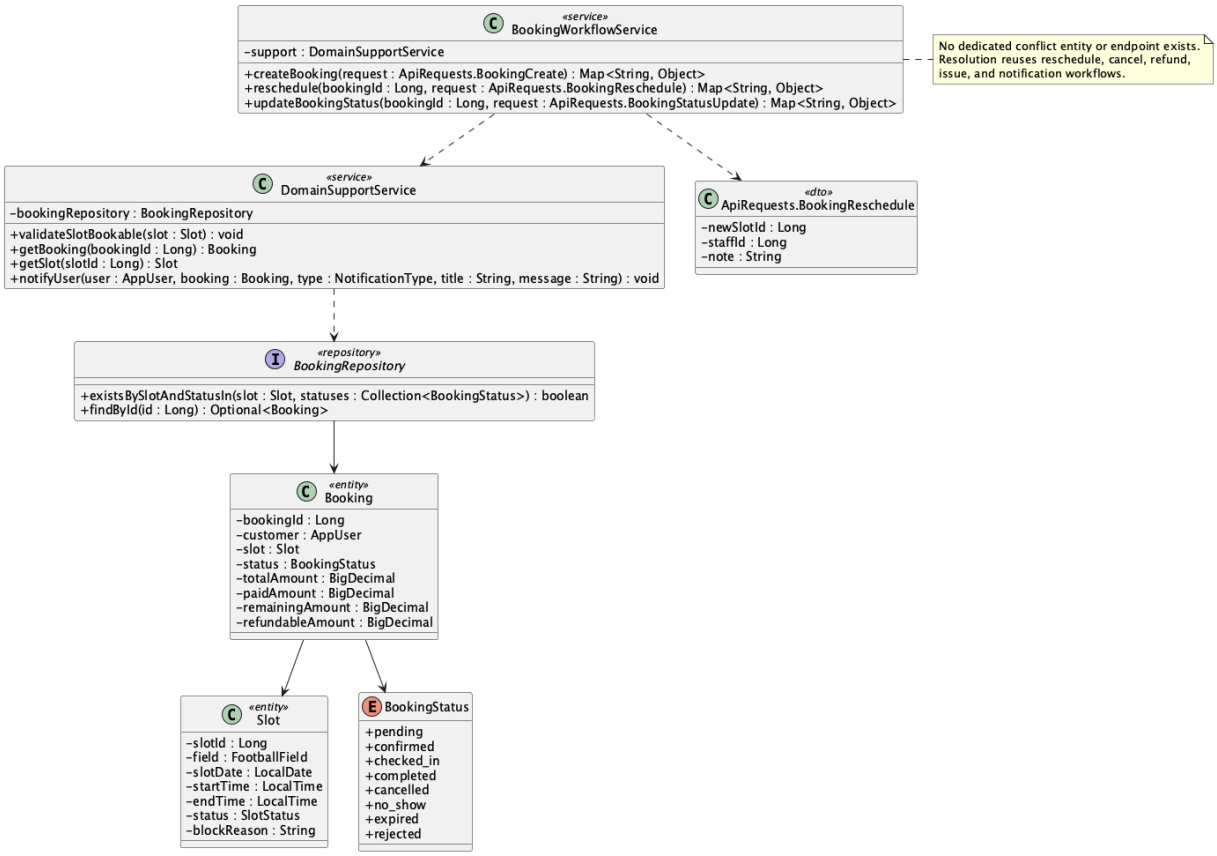
1. Mark confirmed booking no-show

```
UPDATE booking
SET status = 'no_show', staff_id = :staffId, note = COALESCE(:note, note),
   updated_at = CURRENT_TIMESTAMP
WHERE booking_id = :bookingId AND status = 'confirmed';
```

Implementation note: There is no no_show_at column; only status, note, staff_id, and audit timestamp persist this event.

36. Handle booking conflict

a. Class Diagram



b. Class Specifications

DomainSupportService

No	Method	Description
01	void validateSlotBookable(Slot slot)	Raises a conflict when the slot has an active booking.

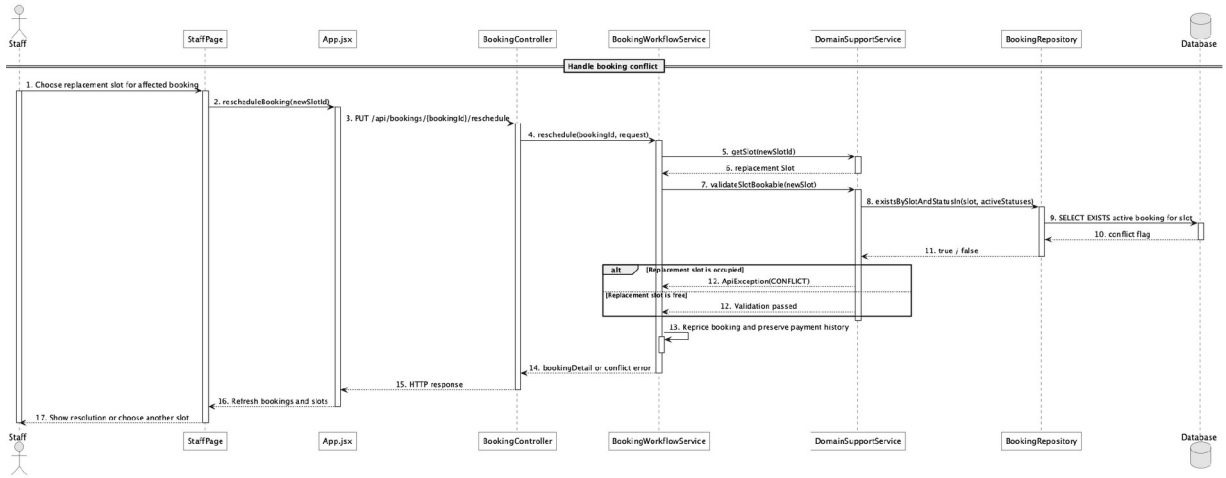
BookingRepository

No	Method	Description
01	boolean existsBySlotAndStatusIn(Slot slot, Collection<BookingStatus> statuses)	Detects active-slot occupancy.

BookingWorkflowService

No	Method	Description
01	public Map<String, Object> reschedule(Long bookingId, ApiRequests.BookingReschedule request)	Provides the implemented resolution route by moving an eligible booking to another free slot.

c. Sequence Diagram



d. Database Queries

== Handle booking conflict ==

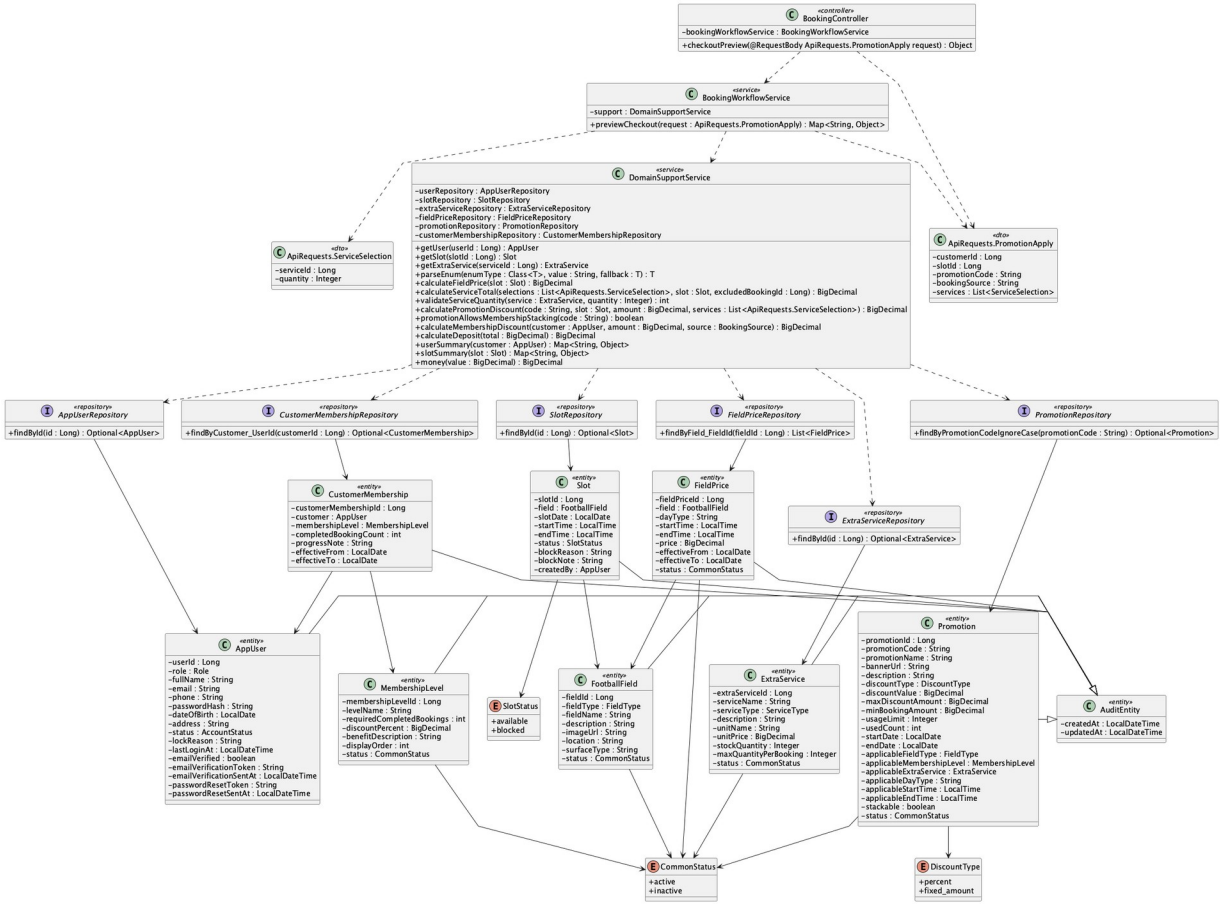
1. Detect active slot conflict

```
SELECT EXISTS (
  SELECT 1
  FROM booking
  WHERE slot_id = :slotId
    AND status IN ('pending','confirmed','checked_in')
) AS has_active_conflict;
```

Implementation note: There is no conflict table, Conflict entity, or dedicated conflict-management endpoint. The implementation prevents double booking and uses reschedule/cancel/refund/notification paths as resolution tools.

37. View checkout summary

a. Class Diagram



b. Class Specifications

BookingController

No	Method	Description
01	public Object checkoutPreview(ApiRequests.PromotionApply request)	Exposes the public checkout preview endpoint.

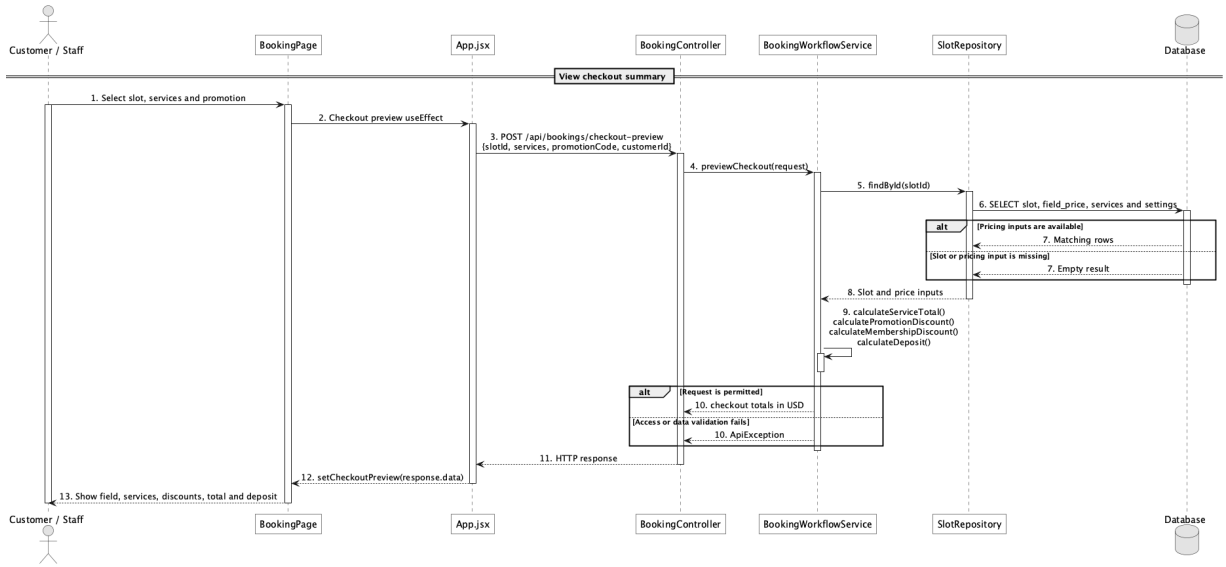
BookingWorkflowService

No	Method	Description
01	previewCheckout(request : ApiRequests.PromotionApply) : Map<String, Object>	Calculates the USD subtotal, caps one promotion at that subtotal, applies membership only when stacking is allowed, and derives deposit from the non-negative final total.

DomainSupportService

No	Method	Description
01	calculatePromotionDiscount(code, slot, baseAmount, services, customer) : BigDecimal	Validates eligibility and caps the promotion between zero and the eligible field-plus-service subtotal.
02	calculateMembershipDiscount(customer, baseAmount, source) : BigDecimal	Applies the eligible online membership rate to a non-negative post-promotion balance.
03	calculateDeposit(total : BigDecimal) : BigDecimal	Applies deposit.default_percent to the final non-negative USD total.

c. Sequence Diagram



d. Database Queries

== View checkout summary ==

1. Checkout pricing inputs

```
SELECT s.slot_id, s.slot_date, s.start_time, s.end_time, f.field_id,
    fp.price AS field_price, ss.setting_value AS deposit_percent
FROM slot s
JOIN field f ON f.field_id = s.field_id
LEFT JOIN field_price fp ON fp.field_id = f.field_id
  AND fp.status = 'active'
  AND (fp.day_type = :dayType OR fp.day_type = 'all')
```

AND s.start_time >= fp.start_time AND s.end_time <= fp.end_time

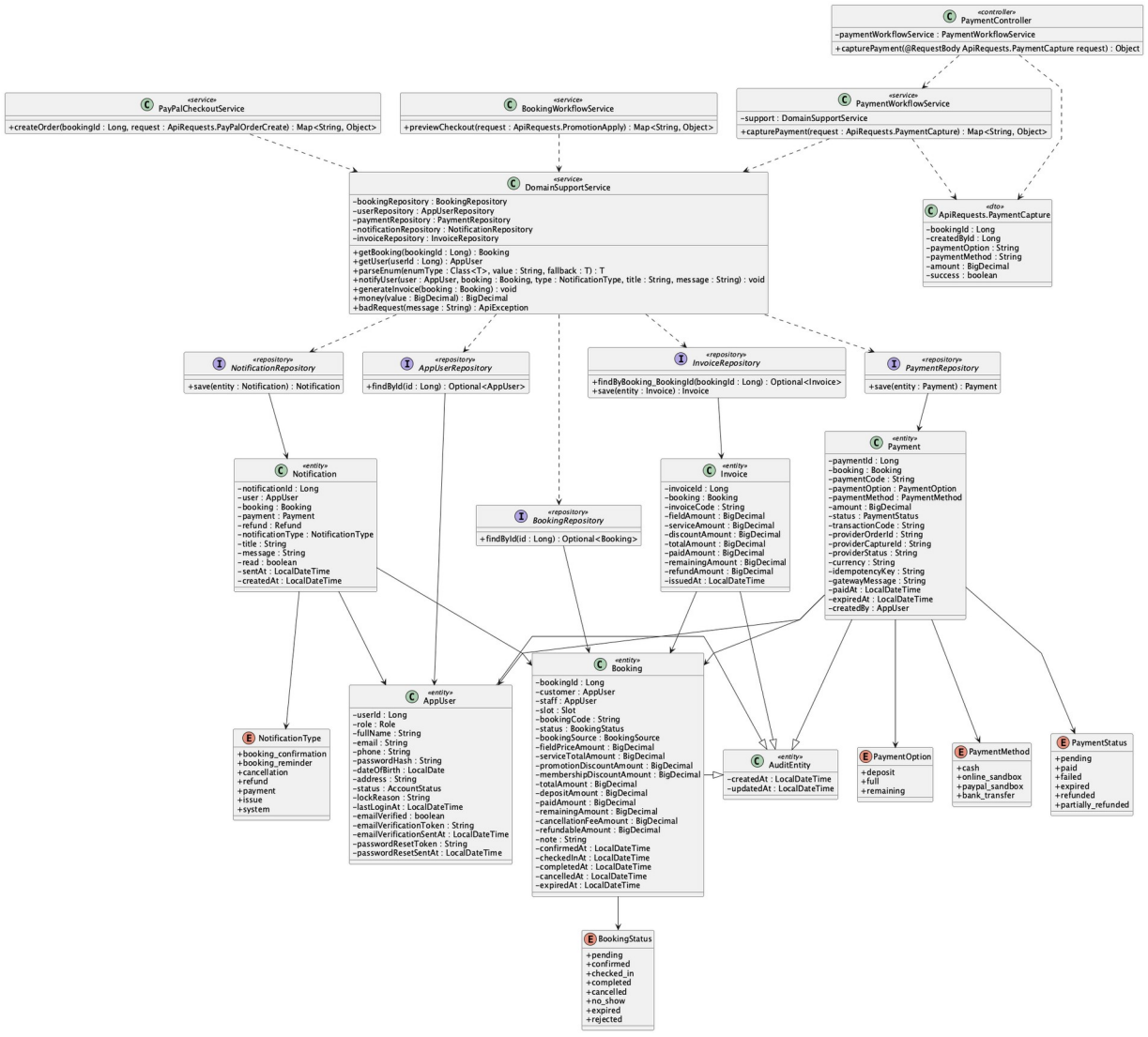
LEFT JOIN system_setting ss ON ss.setting_key = 'deposit.default_percent'

WHERE s.slot_id = :slotId;

Implementation note: This endpoint is intentionally public for anonymous basket estimation and does not return a supplied customer's profile.

38. Choose payment option

a. Class Diagram



b. Class Specifications

PayPalCheckoutService

No	Method	Description
01	public Map<String, Object> createOrder(Long bookingId,	Uses deposit or full to calculate the customer PayPal payable amount.

No	Method	Description
	ApiRequests.PayPalOrderCreate request)	

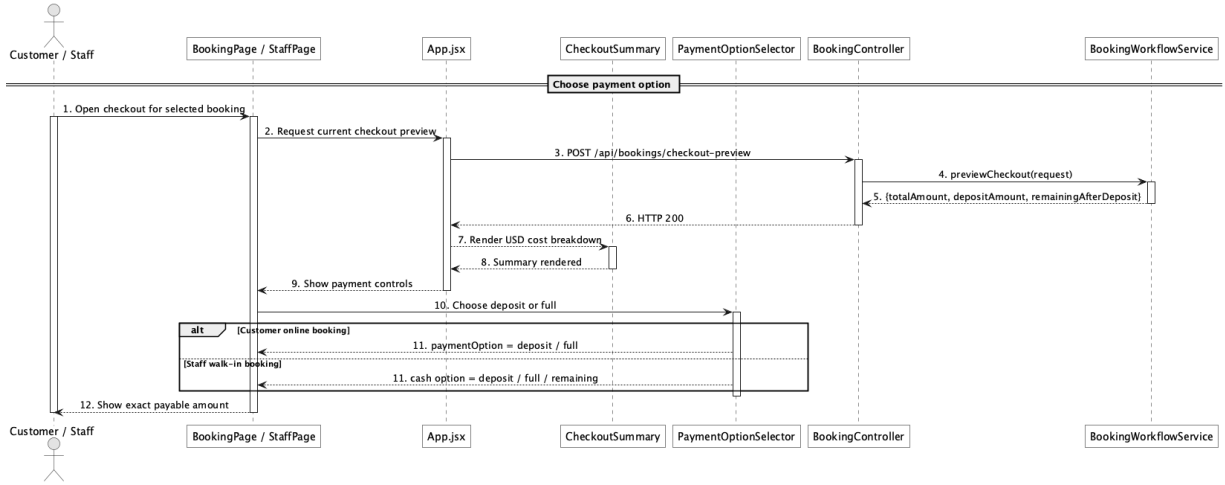
PaymentWorkflowService

No	Method	Description
01	public Map<String, Object> capturePayment(ApiRequests.PaymentCapture request)	Accepts deposit, full, or remaining for Staff/Admin counter payment but requires cash.

BookingWorkflowService

No	Method	Description
01	public Map<String, Object> previewCheckout(ApiRequests.PromotionApply request)	Supplies total, deposit, and remaining values shown before option choice.

c. Sequence Diagram



d. Database Queries

== Choose payment option ==

1. Payment option amounts

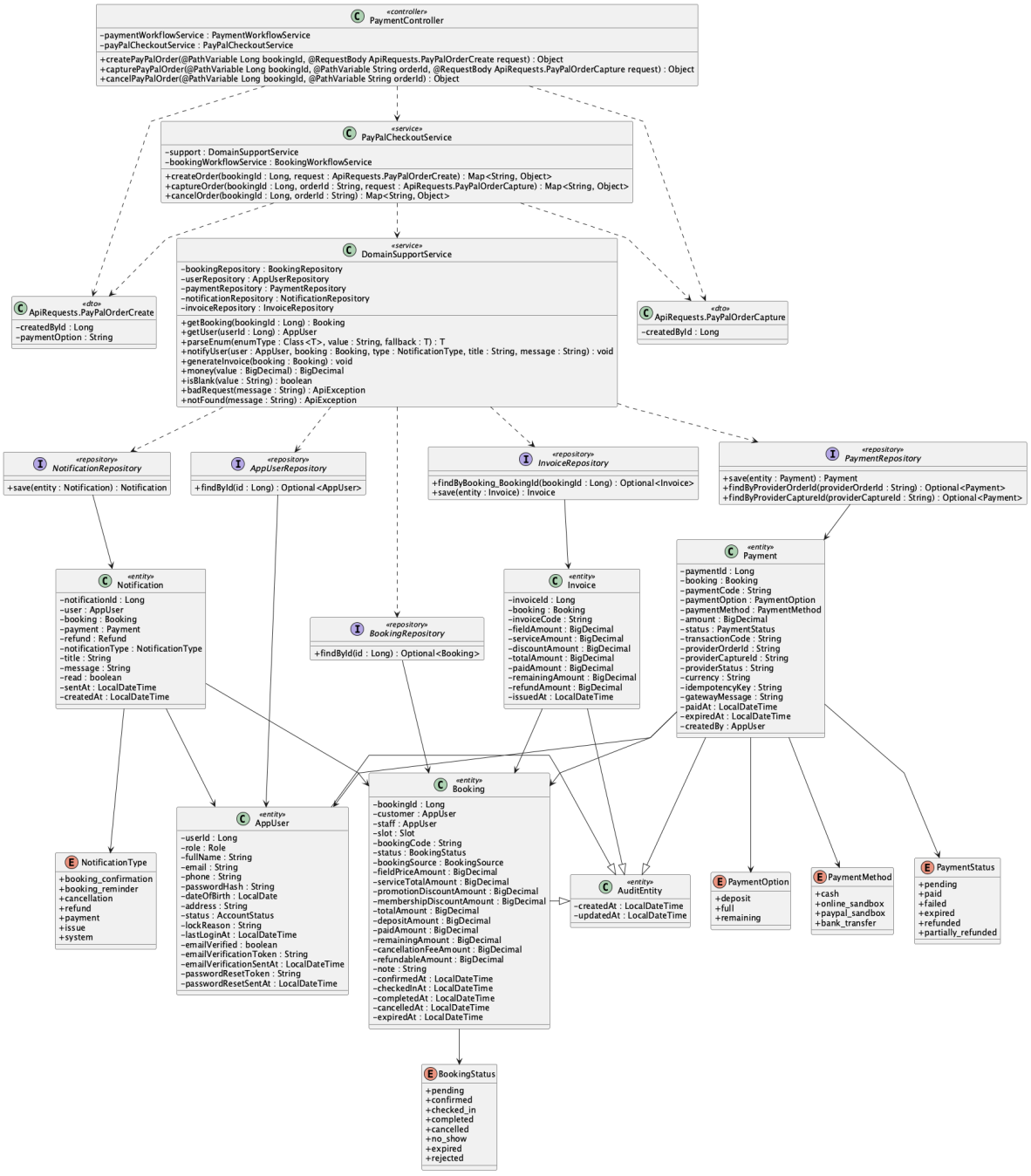
```
SELECT booking_id, total_amount, deposit_amount, paid_amount, remaining_amount,
    CASE :paymentOption
      WHEN 'deposit' THEN GREATEST(deposit_amount - paid_amount, 0)
      WHEN 'full' THEN remaining_amount
      WHEN 'remaining' THEN remaining_amount
```

```
    END AS payable_amount
FROM booking
WHERE booking_id = :bookingId;
```

Implementation note: Customer online checkout supports deposit/full through PayPal; Staff/Admin counter flow records cash and may use remaining.

39. Pay deposit/full amount via online payment sandbox

a. Class Diagram



b. Class Specifications

PaymentController

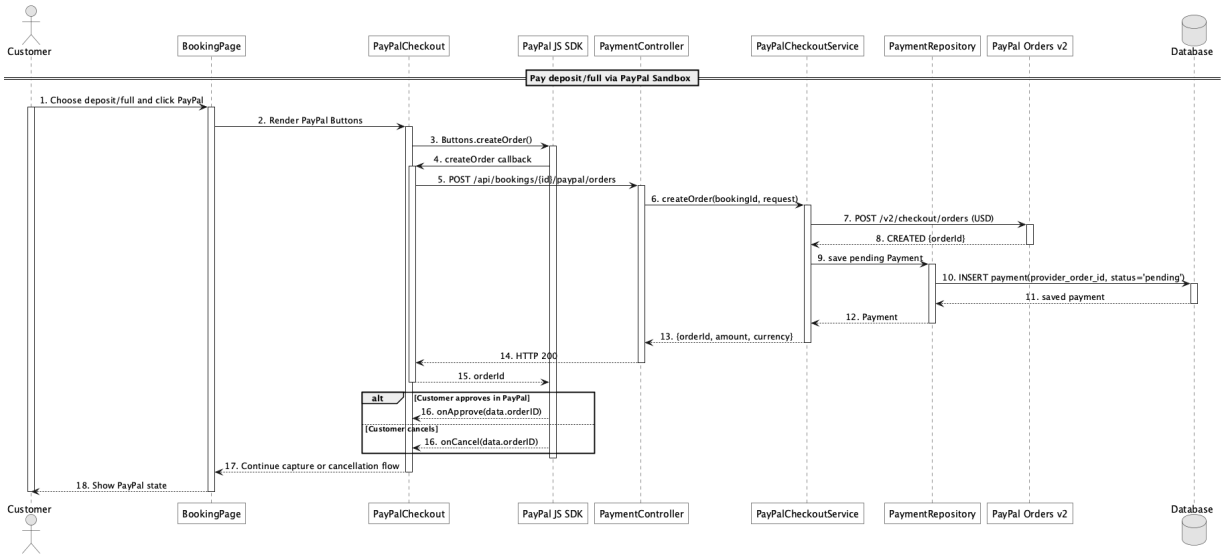
No	Method	Description
01	public Object payPalConfig()	Returns client-side sandbox configuration without exposing the secret.
02	public Object createPayPalOrder(Long bookingId,	Starts a PayPal order for the customer-owned online booking.

No	Method	Description
	ApiRequests.PayPalOrderCreate request)	
03	public Object capturePayPalOrder(Long bookingId, String orderId, ApiRequests.PayPalOrderCapture request)	Captures the approved order.

PayPalCheckoutService

No	Method	Description
01	public Map<String, Object> config()	Reports whether configured sandbox checkout is enabled.
02	public Map<String, Object> createOrder(Long bookingId, ApiRequests.PayPalOrderCreate request)	Creates provider order and pending local payment with idempotency metadata.
03	public Map<String, Object> captureOrder(Long bookingId, String orderId, ApiRequests.PayPalOrderCapture request)	Calls PayPal Orders v2 capture and validates captured currency/amount.

c. Sequence Diagram



d. Database Queries

== Pay deposit/full amount via online payment sandbox ==

1. Persist pending PayPal order

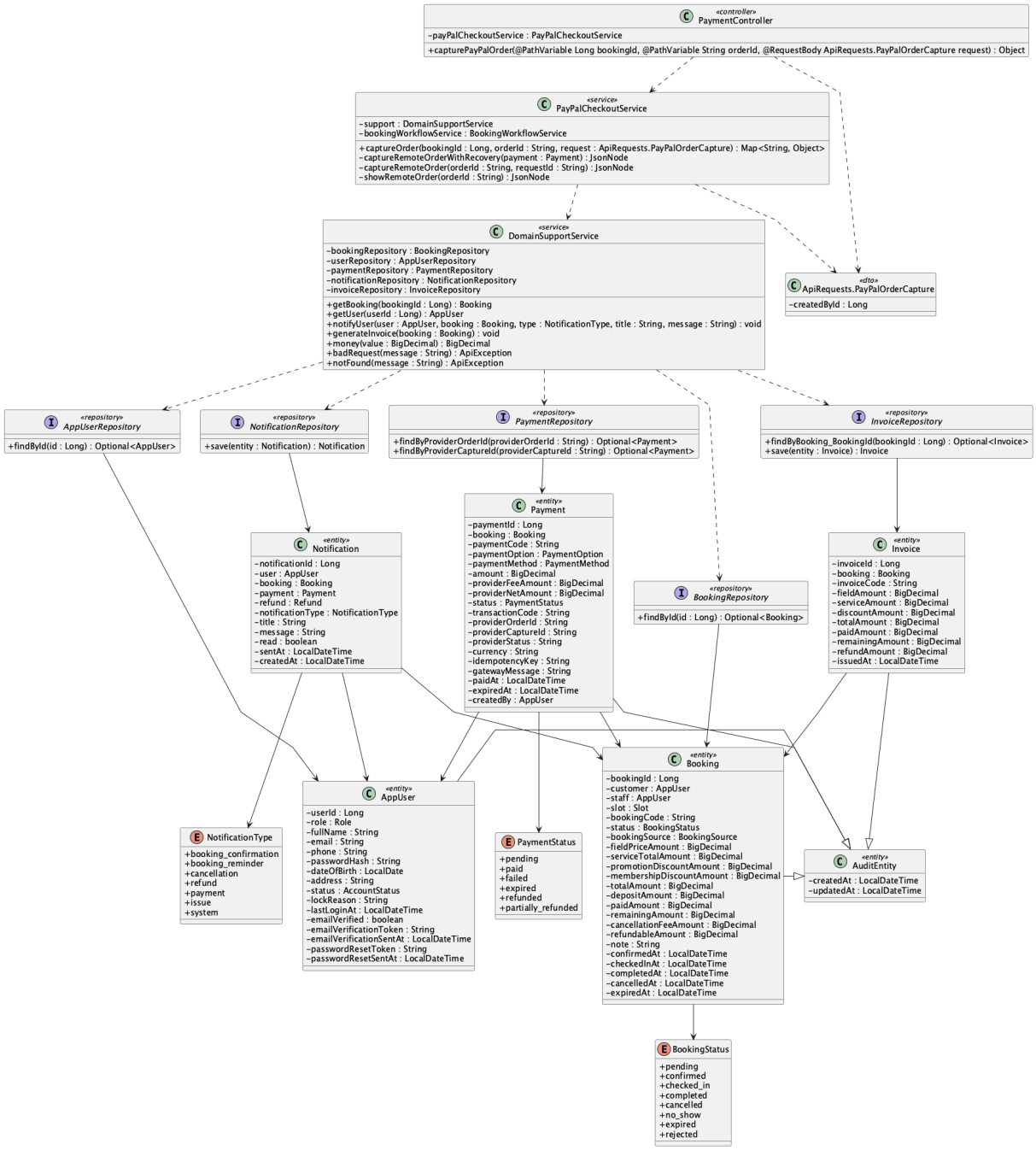
INSERT INTO payment

```
  (booking_id, payment_code, payment_option, payment_method, amount, status,
   provider_order_id, provider_status, currency, idempotency_key, gateway_message,
   created_by, created_at, updated_at)
VALUES
  (:bookingId, :paymentCode, :paymentOption, 'paypal_sandbox', :amount, 'pending',
   :orderId, 'CREATED', 'USD', :idempotencyKey, :gatewayMessage,
   :customerId, CURRENT_TIMESTAMP, CURRENT_TIMESTAMP);
```

Implementation note: The product target is PayPal Sandbox, not live-money certification. PayPal receives the exact USD booking balance; processor fee/net are provider-returned merchant values, not customer tax. Mock mode exists only for automated tests.

40. Capture/confirm online payment

a. Class Diagram



b. Class Specifications

PayPalCheckoutService

No	Method	Description
01	captureOrder(bookingId, orderId, request) : Map<String, Object>	Uses a stable PayPal request identity, reconciles an ambiguous timeout by reading the order, validates exact USD capture, stores provider fee/net, updates gross paid balance, and regenerates the invoice once.

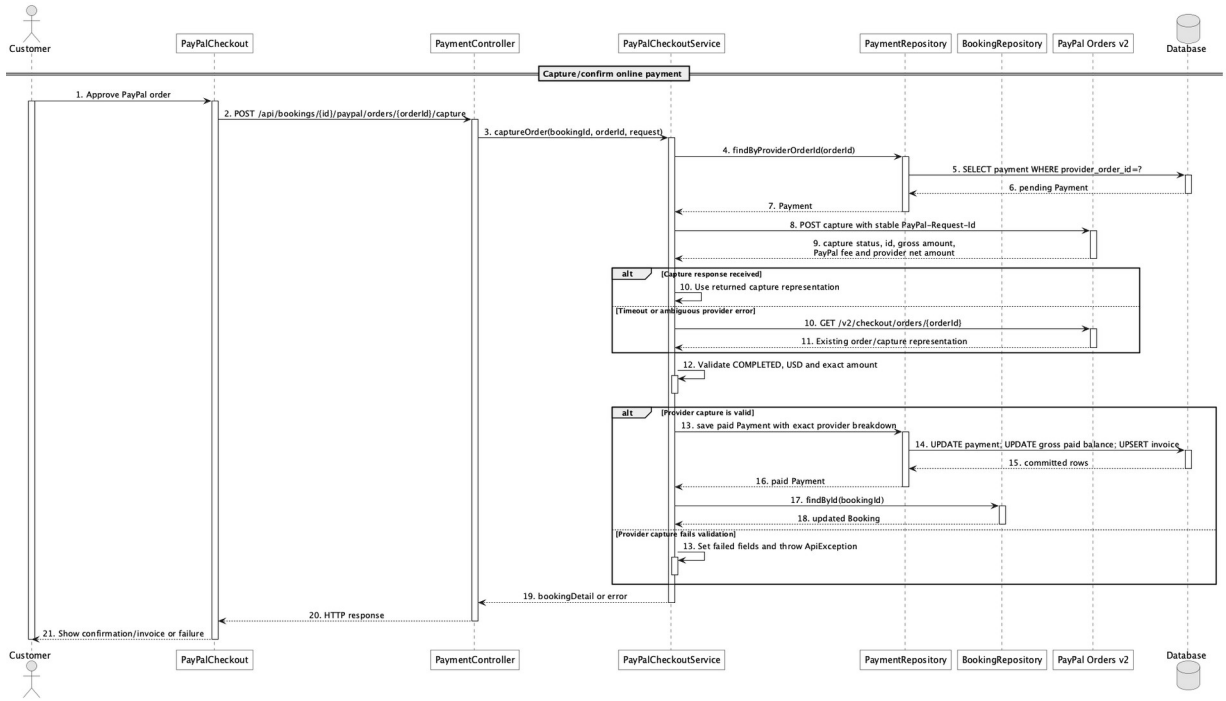
PaymentRepository

No	Method	Description
01	Optional<Payment> findByProviderOrderId(String providerOrderId)	Resolves the local pending payment for capture.
02	Optional<Payment> findByProviderCaptureId(String providerCaptureId)	Prevents a provider capture from being processed twice.

DomainSupportService

No	Method	Description
01	void generateInvoice(Booking booking)	Synchronizes invoice totals after capture.

c. Sequence Diagram



d. Database Queries

== Capture/confirm online payment ==

1. Store successful capture and update balance

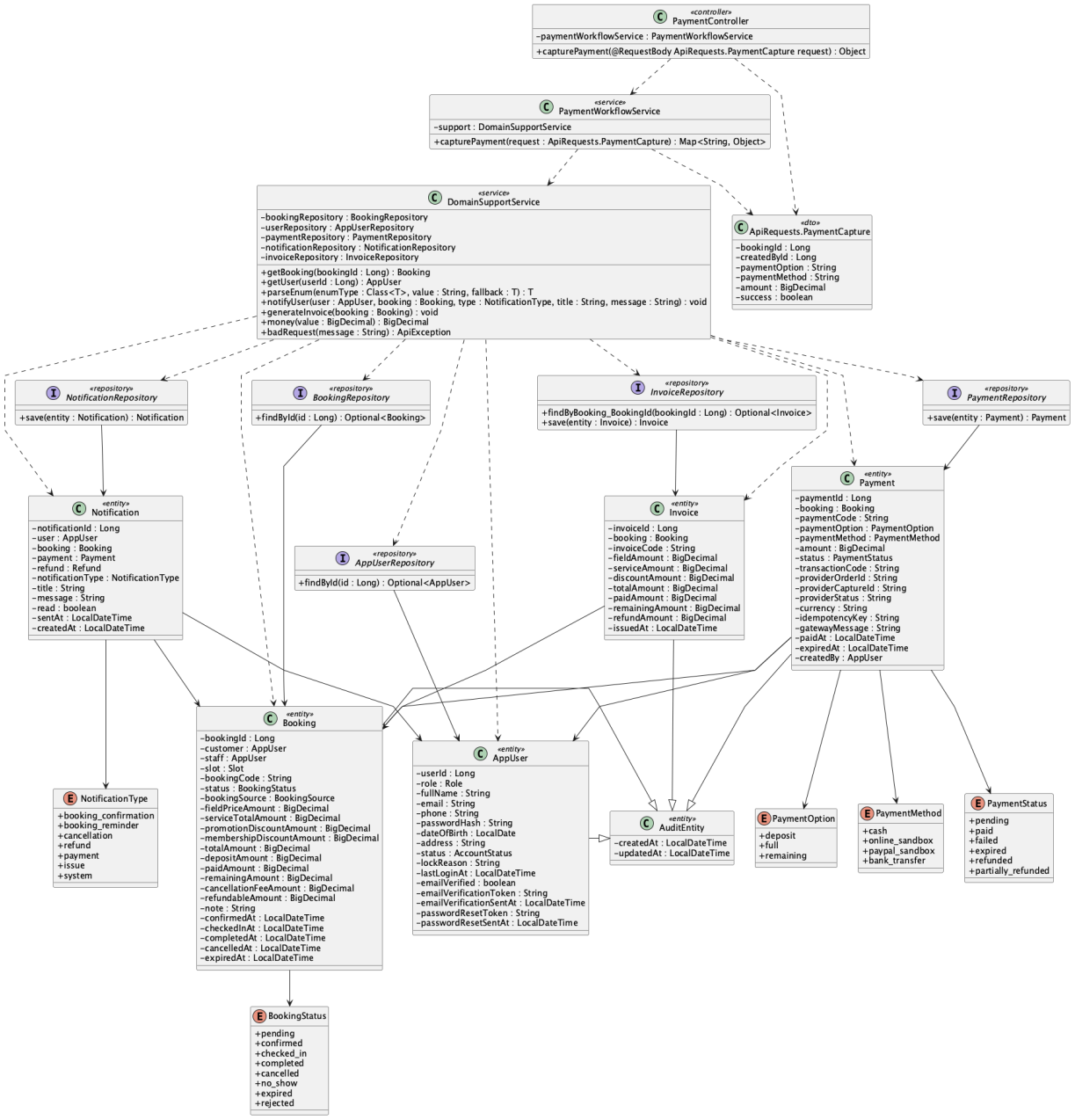
```
UPDATE payment
SET status = 'paid', provider_capture_id = :captureId, provider_status = :providerStatus,
   provider_fee_amount = :paypalFee, provider_net_amount = :providerNet,
   transaction_code = :captureId, gateway_message = :message, paid_at = CURRENT_TIMESTAMP,
   updated_at = CURRENT_TIMESTAMP
WHERE provider_order_id = :orderId AND status = 'pending';


UPDATE booking
SET paid_amount = paid_amount + :capturedAmount,
   remaining_amount = GREATEST(total_amount - (paid_amount + :capturedAmount), 0),
   status = CASE WHEN status = 'pending' AND paid_amount + :capturedAmount >= deposit_amount
            THEN 'confirmed' ELSE status END,
   confirmed_at = CASE WHEN status = 'pending' AND paid_amount + :capturedAmount >=
deposit_amount
                  THEN CURRENT_TIMESTAMP ELSE confirmed_at END
WHERE booking_id = :bookingId;
```

Implementation note: Only provider COMPLETED capture is accepted as paid. Capture uses a stable PayPal-Request-Id and reads the order after an ambiguous timeout before changing the local gross-paid ledger.

41. Confirm remaining payment

a. Class Diagram



b. Class Specifications

PaymentController

No	Method	Description
01	public Object capturePayment(ApiRequests.PaymentCapture request)	Exposes Staff/Admin counter capture.

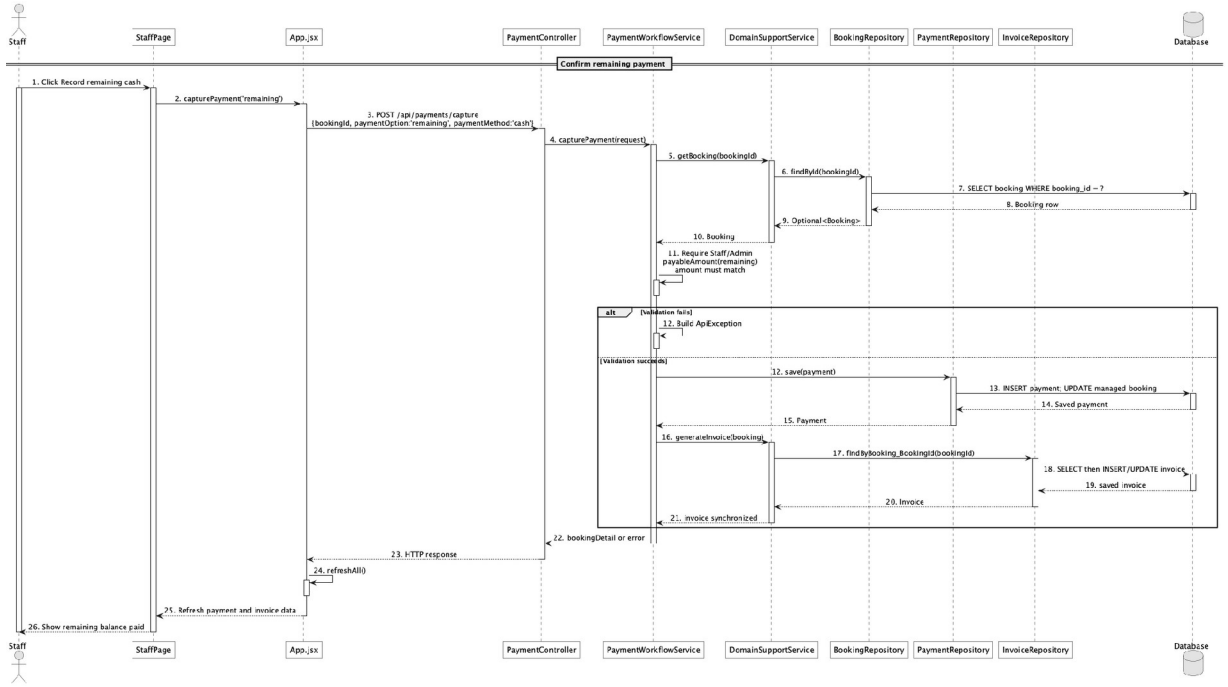
PaymentWorkflowService

No	Method	Description
01	public Map<String, Object> capturePayment(ApiRequests.PaymentCapture request)	Requires operator and cash, validates exact payable amount, records payment, updates booking balance, and refreshes invoice.

DomainSupportService

No	Method	Description
01	void generateInvoice(Booking booking)	Updates the invoice after remaining cash capture.

c. Sequence Diagram



d. Database Queries

== Confirm remaining payment ==

1. Record remaining cash payment

```
INSERT INTO payment
  (booking_id, payment_code, payment_option, payment_method, amount, status,
   transaction_code, paid_at, created_by, created_at, updated_at)
VALUES
  (:bookingId, :paymentCode, 'remaining', 'cash', :remainingAmount, 'paid',
   :transactionCode, CURRENT_TIMESTAMP, :operatorId, CURRENT_TIMESTAMP,
CURRENT_TIMESTAMP);
```

UPDATE booking

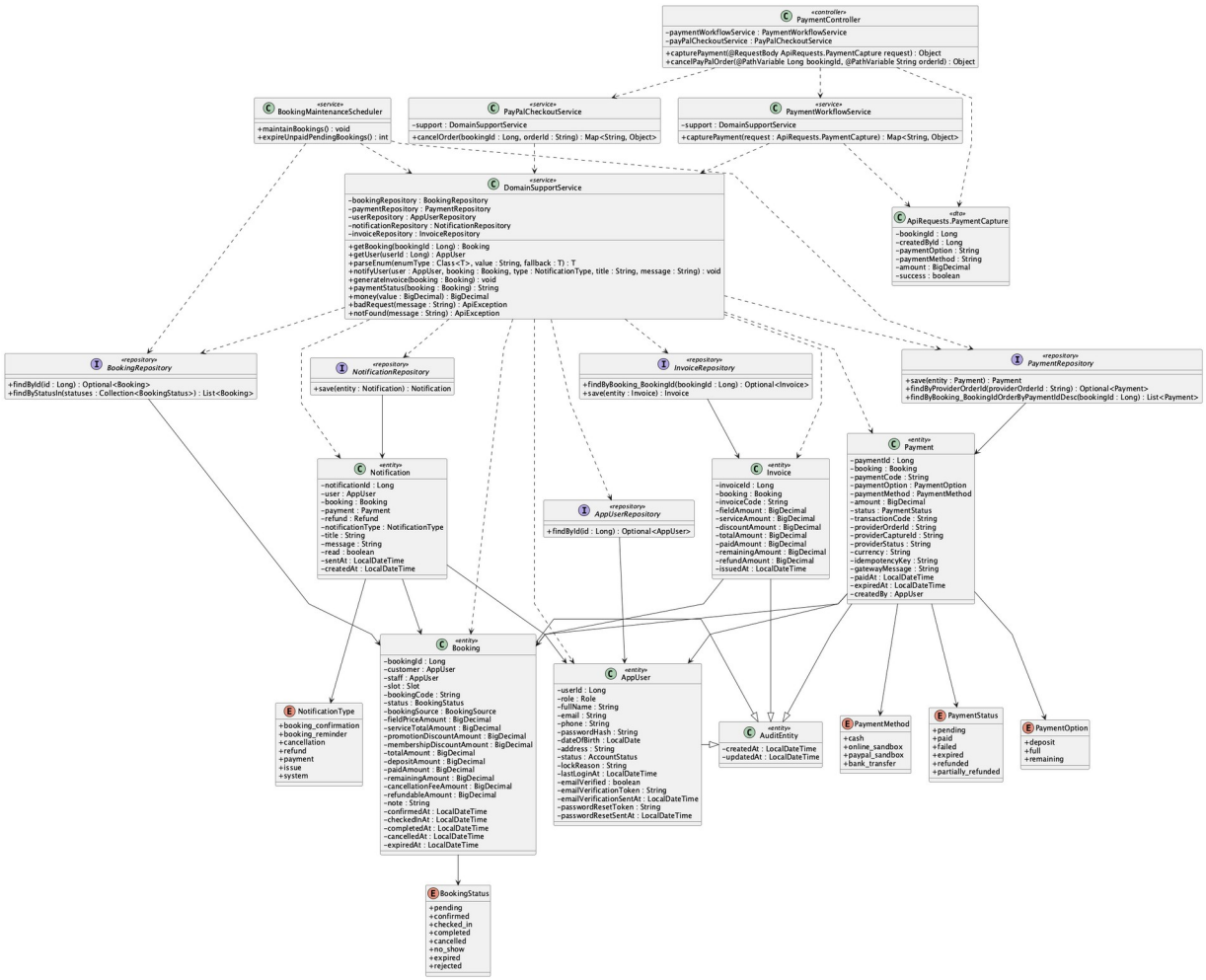
SET paid_amount = paid_amount + :remainingAmount, remaining_amount = 0, updated_at = CURRENT_TIMESTAMP

WHERE booking_id = :bookingId;

Implementation note: Counter capture rejects non-cash methods; customer PayPal is handled by the separate provider flow.

42. Handle failed/expired payment

a. Class Diagram



b. Class Specifications

PayPalCheckoutService

No	Method	Description
01	public Map<String, Object> cancelOrder(Long bookingId, String	Expires an uncaptured payment and an unpaid pending booking when the customer cancels

No	Method	Description
	orderId)	checkout.

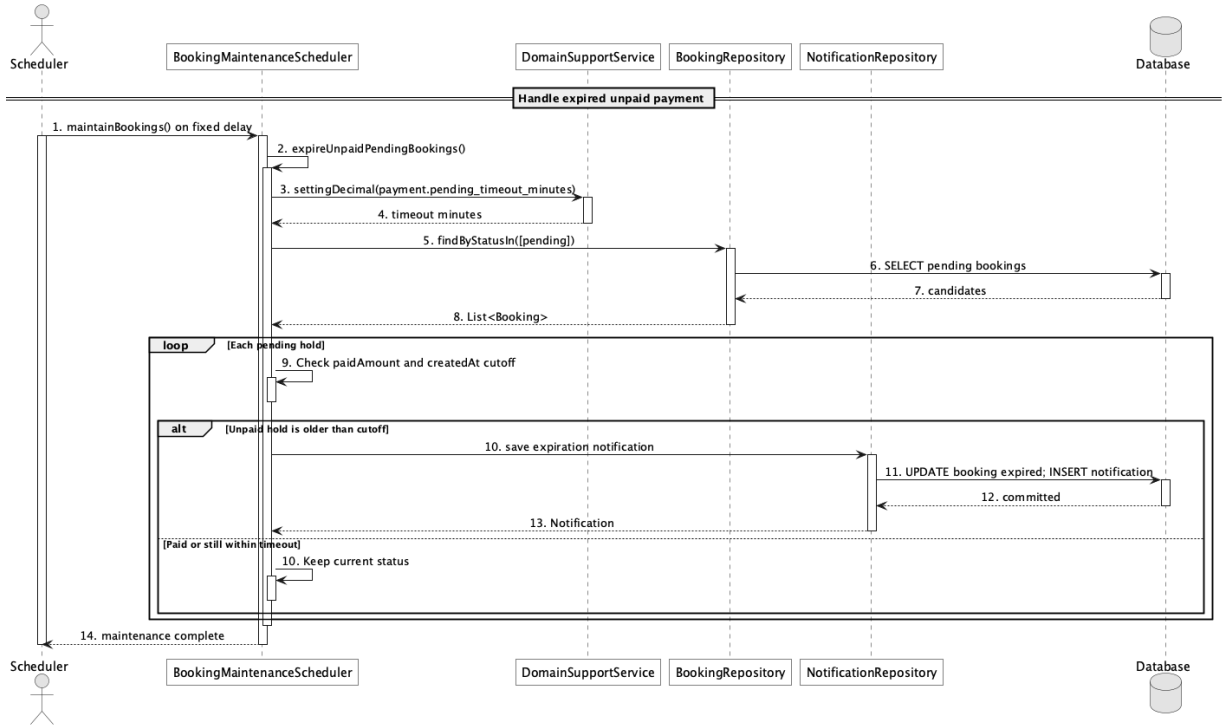
BookingMaintenanceScheduler

No	Method	Description
01	public void maintainBookings()	Scheduled entry point for payment expiry and reminder maintenance.
02	public int expireUnpaidPendingBookings()	Expires aged unpaid pending holds using payment.pending_timeout_minutes.

BookingRepository

No	Method	Description
01	List<Booking> findByStatusIn(Collection<BookingStatus> statuses)	Loads pending expiry candidates.

c. Sequence Diagram



d. Database Queries

== Handle failed/expired payment ==

1. Find expired unpaid holds

SELECT b.booking_id, b.booking_code, b.customer_id, b.created_at

```
FROM booking b
WHERE b.status = 'pending'
  AND b.paid_amount = 0
  AND b.created_at < CURRENT_TIMESTAMP - (:timeoutMinutes * INTERVAL '1 minute');
```

Implementation note: The scheduler now exists; older documentation saying scheduled timeout was missing is stale.

43. View payment history

a. Class Diagram

b. Class Specifications

PaymentController

No	Method	Description
01	public Object payments()	Exposes authenticated GET /api/payments.

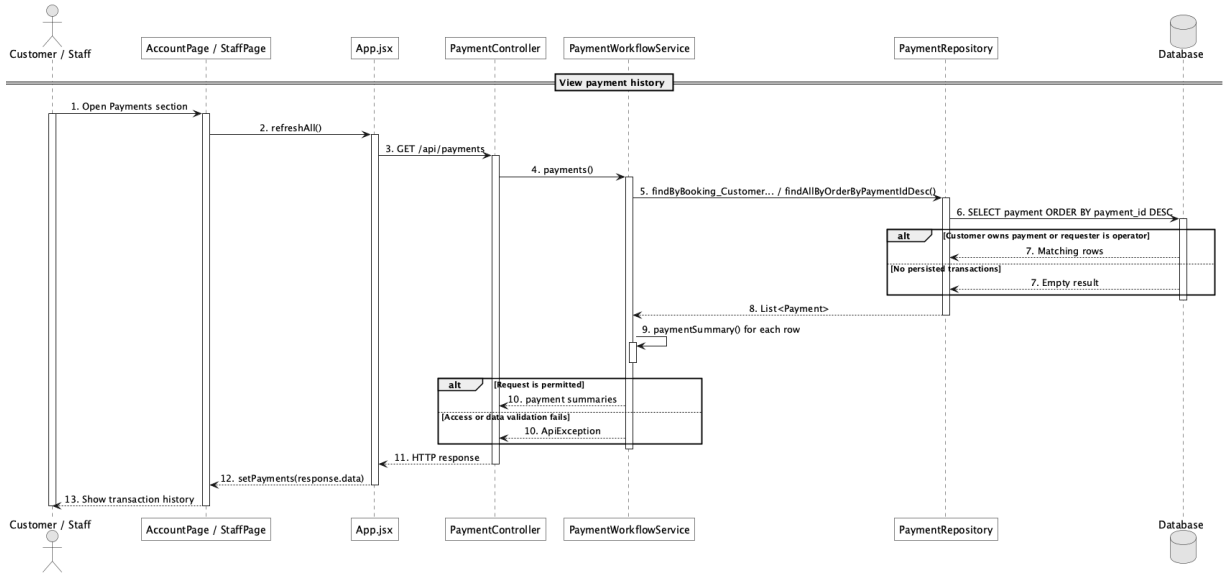
PaymentWorkflowService

No	Method	Description
01	public List<Map<String, Object>> payments()	Returns own payment history for Customer or all payments for operators.

PaymentRepository

No	Method	Description
01	List<Payment> findByBooking_Customer_UserIdOrderByPaymentIdDesc(Long customerId)	Loads the signed-in customer's transactions.
02	List<Payment> findAllByOrderByPaymentIdDesc()	Loads the operator-wide history.

c. Sequence Diagram



d. Database Queries

== View payment history ==

1. Customer payment history

```
SELECT p.payment_id, p.payment_code, p.payment_option, p.payment_method,
    p.amount, p.provider_fee_amount, p.provider_net_amount, p.status,
    p.transaction_code, p.gateway_message, p.paid_at, b.booking_code
```

FROM payment p

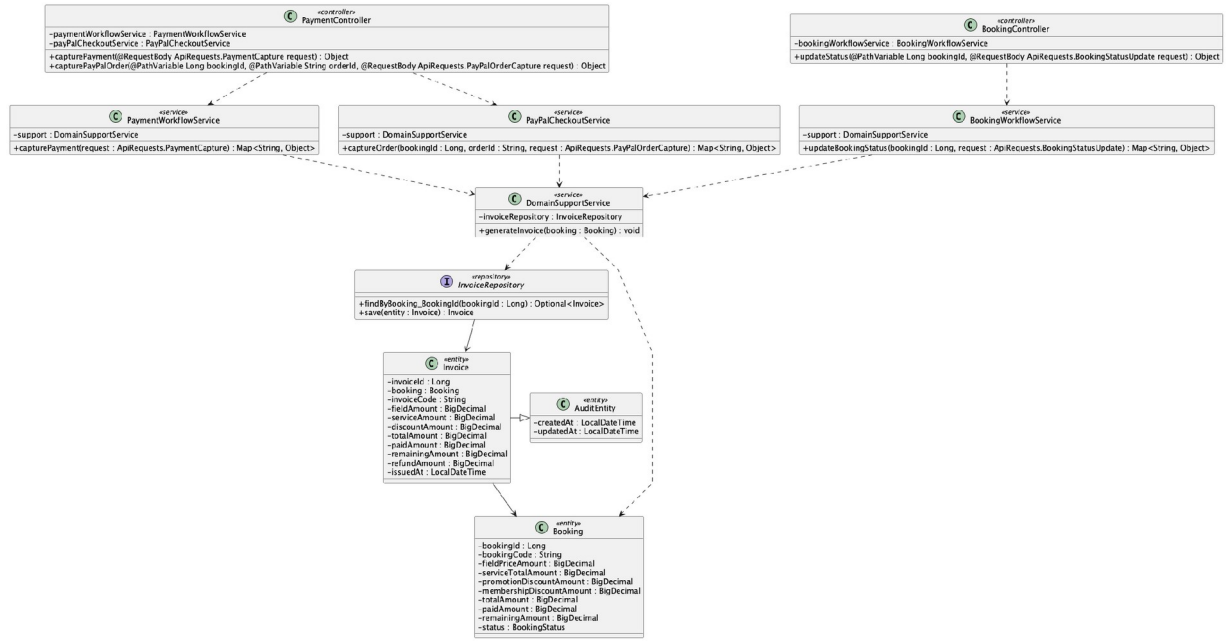
JOIN booking b ON b.booking_id = p.booking_id

WHERE b.customer_id = :customerId

ORDER BY p.payment_id DESC;

44. Generate booking invoice

a. Class Diagram



b. Class Specifications

DomainSupportService

No	Method	Description
01	void generateInvoice(Booking booking)	Upserts one invoice per booking from booking financial snapshots.

InvoiceRepository

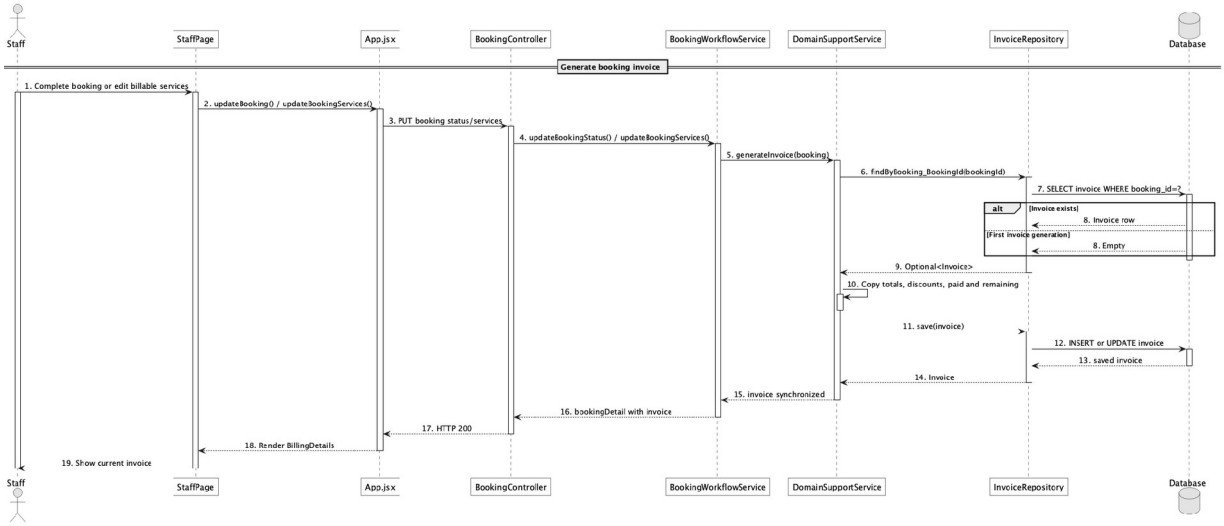
No	Method	Description
01	Optional<Invoice> findByBooking_BookingId(Long bookingId)	Finds the existing invoice before creating or updating it.

PaymentWorkflowService

No	Method	Description
01	public Map<String, Object> capturePayment(ApiRequests.Pay	Triggers invoice generation after a successful counter payment.

No	Method	Description
	mentCapture request)	

c. Sequence Diagram



d. Database Queries

== Generate booking invoice ==

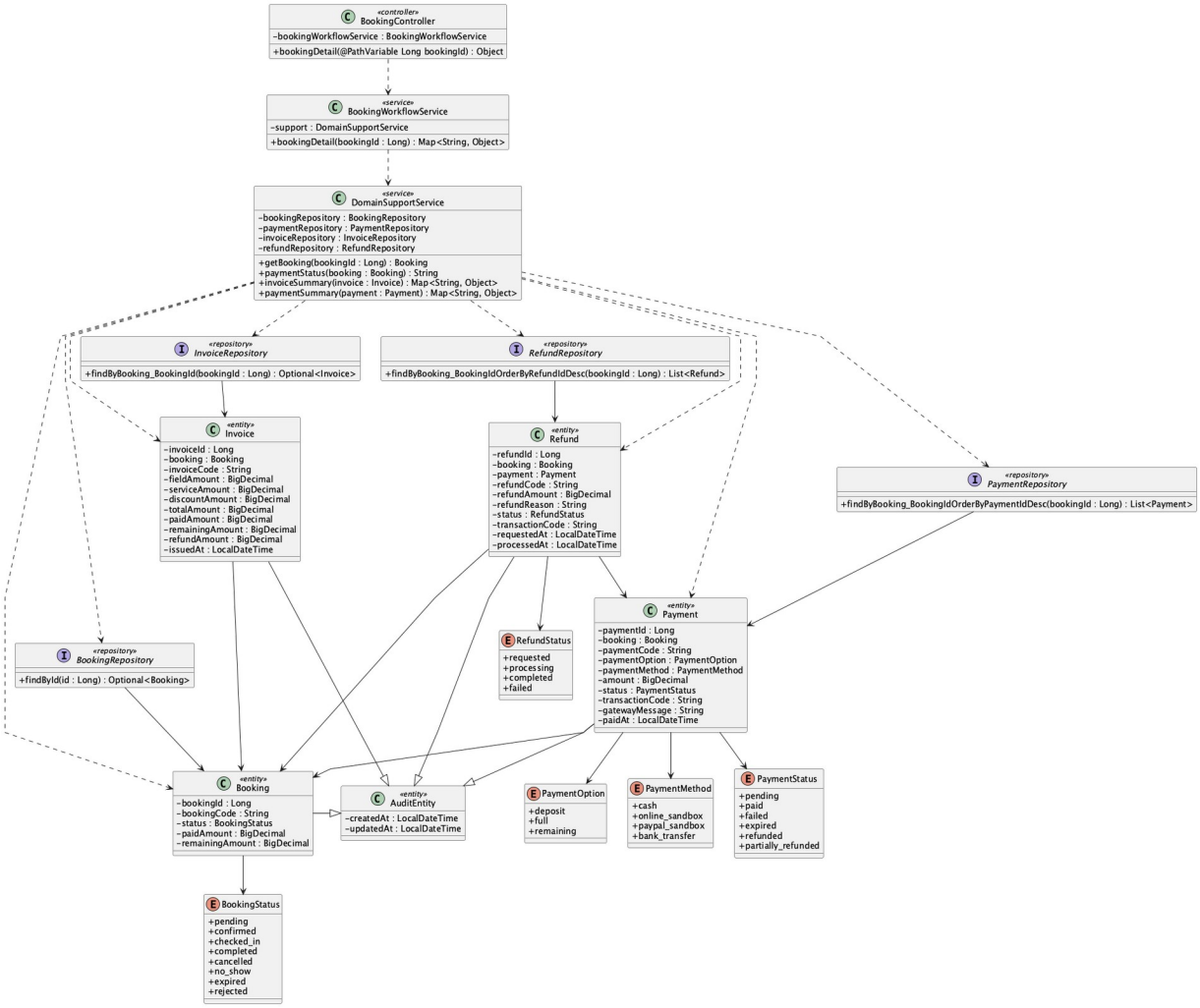
1. Invoice upsert

```
INSERT INTO invoice
  (booking_id, invoice_code, field_amount, service_amount, discount_amount,
   total_amount, paid_amount, remaining_amount, refund_amount, issued_at, created_at, updated_at)
VALUES
  (:bookingId, :invoiceCode, :fieldAmount, :serviceAmount, :discountAmount,
   :totalAmount, :paidAmount, :remainingAmount, :refundAmount, CURRENT_TIMESTAMP,
   CURRENT_TIMESTAMP, CURRENT_TIMESTAMP)
ON CONFLICT (booking_id) DO UPDATE
SET field_amount = EXCLUDED.field_amount, service_amount = EXCLUDED.service_amount,
    discount_amount = EXCLUDED.discount_amount, total_amount = EXCLUDED.total_amount,
    paid_amount = EXCLUDED.paid_amount, remaining_amount = EXCLUDED.remaining_amount,
    issued_at = EXCLUDED.issued_at, updated_at = CURRENT_TIMESTAMP;
```

Implementation note: The Java implementation uses find-or-create plus repository save; the SQL expresses equivalent PostgreSQL behavior.

45. View invoice/payment status

a. Class Diagram



b. Class Specifications

BookingWorkflowService

No	Method	Description
01	public Map<String, Object> bookingDetail(Long bookingId)	Returns invoice, transaction list, refunds, and derived paymentStatus together.

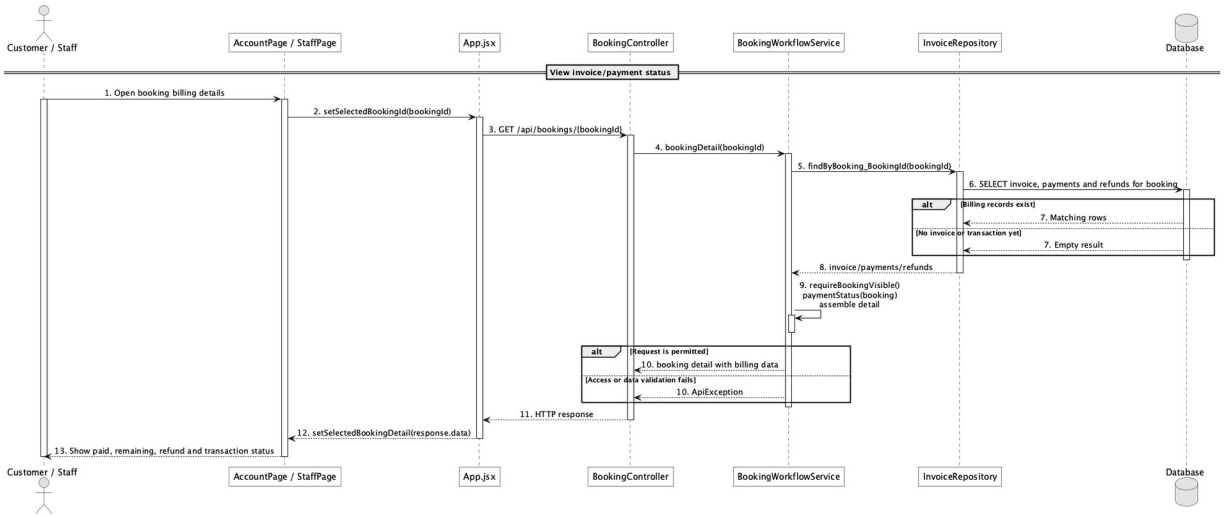
DomainSupportService

No	Method	Description
01	String paymentStatus(Booking booking)	Derives unpaid, failed, expired, partially_paid, paid, partially_refunded, or refunded.

RefundRepository

No	Method	Description
01	List<Refund> findByBooking_BookingIdOrderByRefundIdDesc(Long bookingId)	Provides completed-refund amounts for status derivation.

c. Sequence Diagram



d. Database Queries

== View invoice/payment status ==

1. Invoice and payment status inputs

SELECT b.booking_id, b.booking_code, b.status AS booking_status, b.paid_amount, b.remaining_amount,

i.invoice_code, i.total_amount, i.paid_amount AS invoice_paid,

i.remaining_amount AS invoice_remaining, i.refund_amount,

COALESCE(SUM(CASE WHEN r.status = 'completed' THEN r.refund_amount ELSE 0 END), 0) AS completed_refunds

FROM booking b

LEFT JOIN invoice i ON i.booking_id = b.booking_id

LEFT JOIN refund r ON r.booking_id = b.booking_id

WHERE b.booking_id = :bookingId

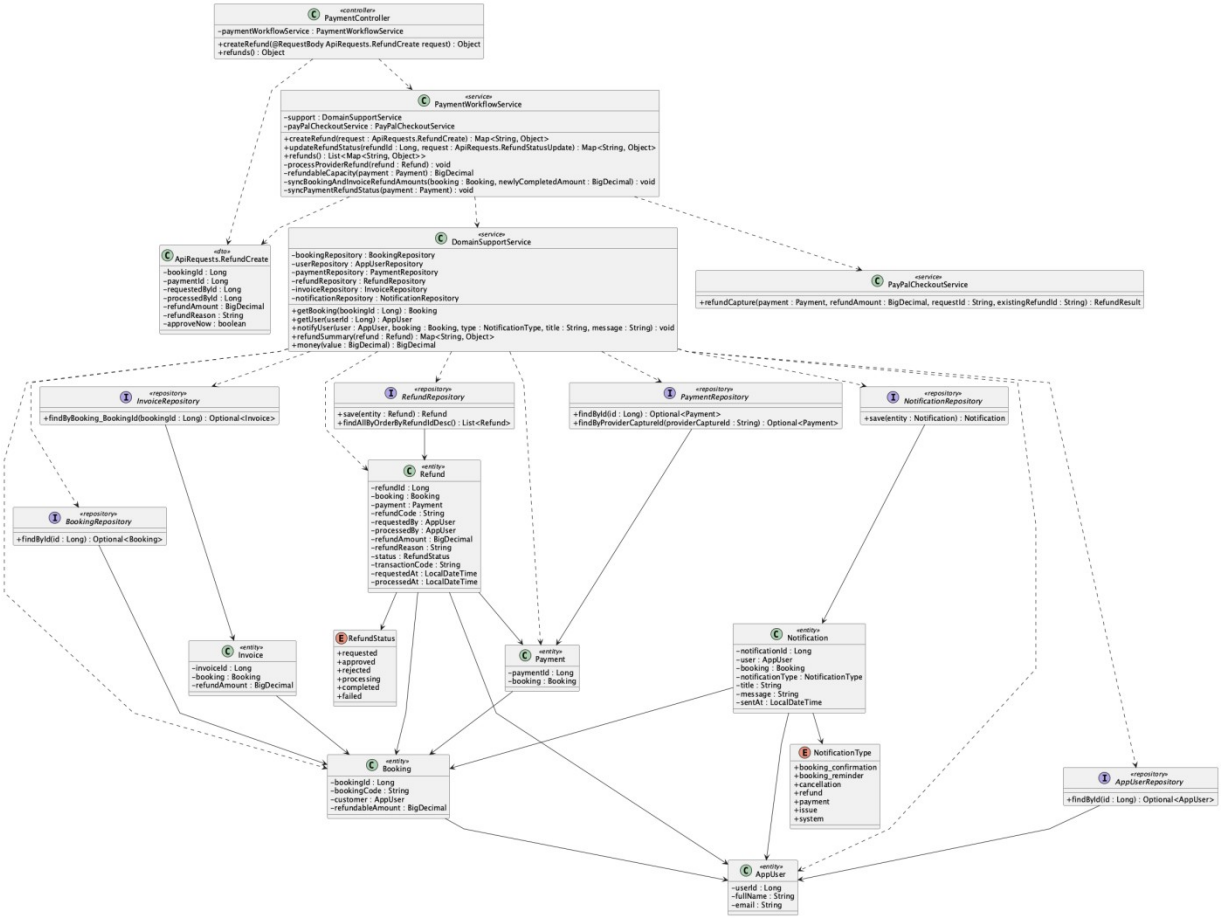
GROUP BY b.booking_id, b.booking_code, b.status, b.paid_amount, b.remaining_amount,

i.invoice_code, i.total_amount, i.paid_amount, i.remaining_amount, i.refund_amount;

Implementation note: paymentStatus is derived in Java; it is not a column on invoice or booking.

46. Process online refund

a. Class Diagram



b. Class Specifications

PayPalCheckoutService

No	Method	Description
01	refundCapture(payment, refundAmount, requestId, existingRefundId) : RefundResult	Uses PayPal Payments v2 with a stable request id, polls an existing provider refund when present, and returns provider truth without inventing completion.

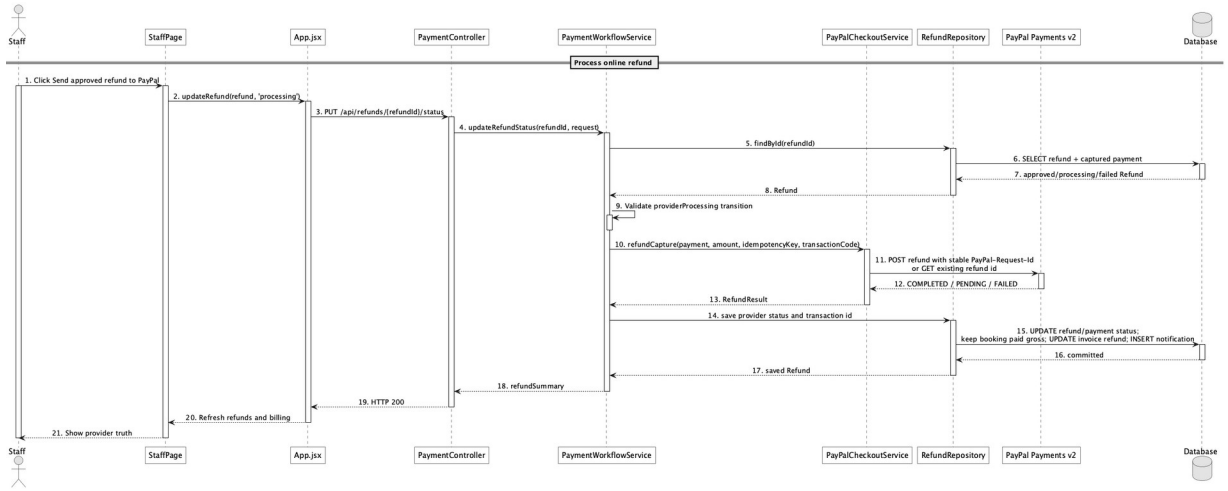
PaymentWorkflowService

No	Method	Description
01	updateRefundStatus(refundId, request) : Map<String, Object>	Maps only provider COMPLETED to completion, preserves booking paidAmount as gross collected, records refund separately, and synchronizes payment and invoice refund state.

PaymentRepository

No	Method	Description
01	Optional<Payment> findByProviderCaptureId(String providerCaptureId)	Represents provider capture reconciliation data held on payment.

c. Sequence Diagram



d. Database Queries

== Process online refund ==

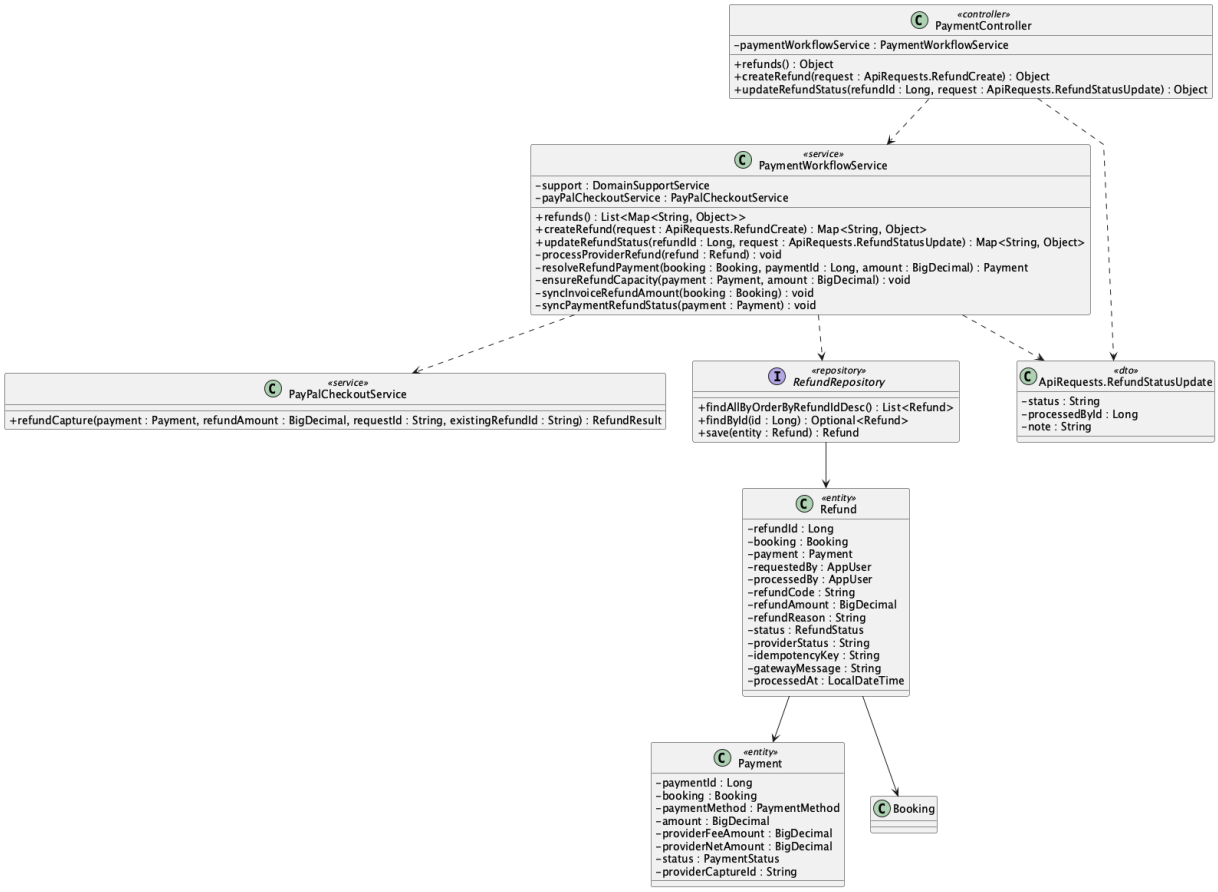
1. Provider refund work item

```
SELECT r.refund_id, r.refund_code, r.refund_amount, r.status,
    r.transaction_code AS provider_refund_id, r.provider_status, r.idempotency_key,
    p.payment_id, p.provider_capture_id, p.payment_method, p.status AS payment_status
FROM refund r
JOIN payment p ON p.payment_id = r.payment_id
WHERE r.refund_id = :refundId;
```

Implementation note: Only PayPal provider COMPLETED becomes completed. paidAmount remains gross collected, refunds are recorded separately, and the stable provider request id makes retries safe.

47. Manage refund requests

a. Class Diagram



b. Class Specifications

PaymentController

No	Method	Description
01	public Object refunds()	Lists all requests for Staff/Admin or only owned requests for a Customer.
02	public Object createRefund(ApiRequests.RefundCreate request)	Creates an owner-authorized Customer refund request.
03	public Object updateRefundStatus(Long refundId, ApiRequests.RefundStatusUpdate request)	Applies an operator status transition.

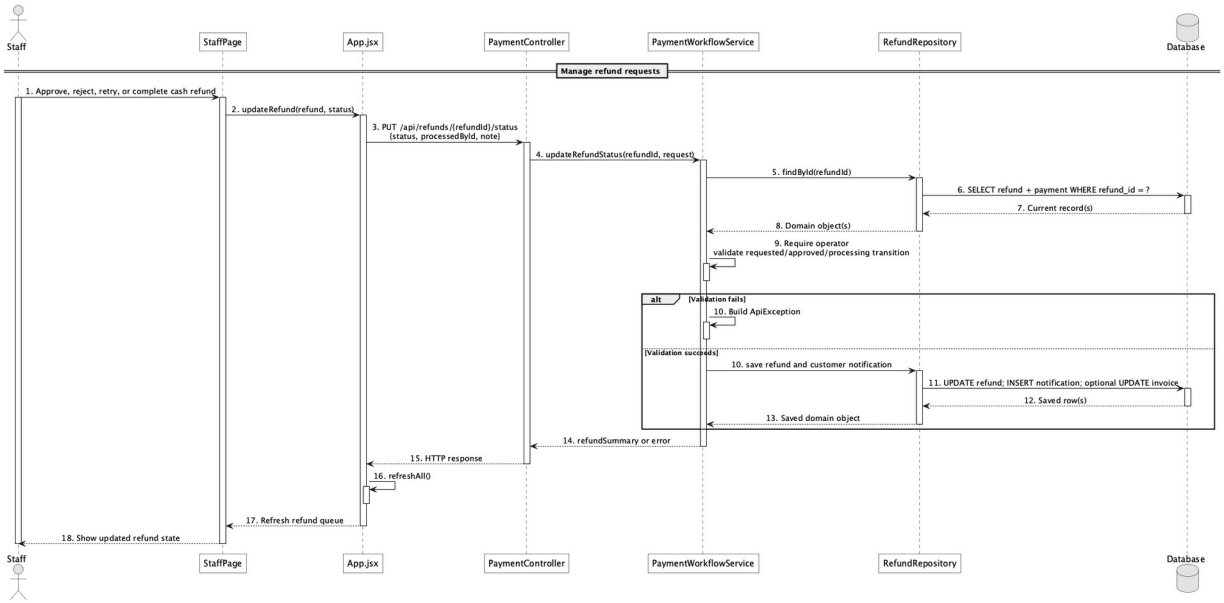
PaymentWorkflowService

No	Method	Description
01	createRefund(request : ApiRequests.RefundCreate) : Map<String, Object>	Validates Customer ownership and reserves only requested, approved, or processing capacity so completed partial refunds are not subtracted twice.
02	updateRefundStatus(refundId, request) : Map<String, Object>	Validates operator transitions and completes either a PayPal provider refund or an explicit manual cash refund.
03	refunds() : List<Map<String, Object>>	Returns newest-first requests with Staff/Admin scope or Customer ownership filtering.

RefundRepository

No	Method	Description
01	List<Refund> findAllByOrderByRefundIdDesc()	Loads the refund queue.

c. Sequence Diagram



d. Database Queries

== Manage refund requests ==

1. Refund management queue

SELECT r.refund_id, r.refund_code, r.refund_amount, r.refund_reason, r.status,

r.provider_status, r.gateway_message, r.requested_at, r.processed_at,

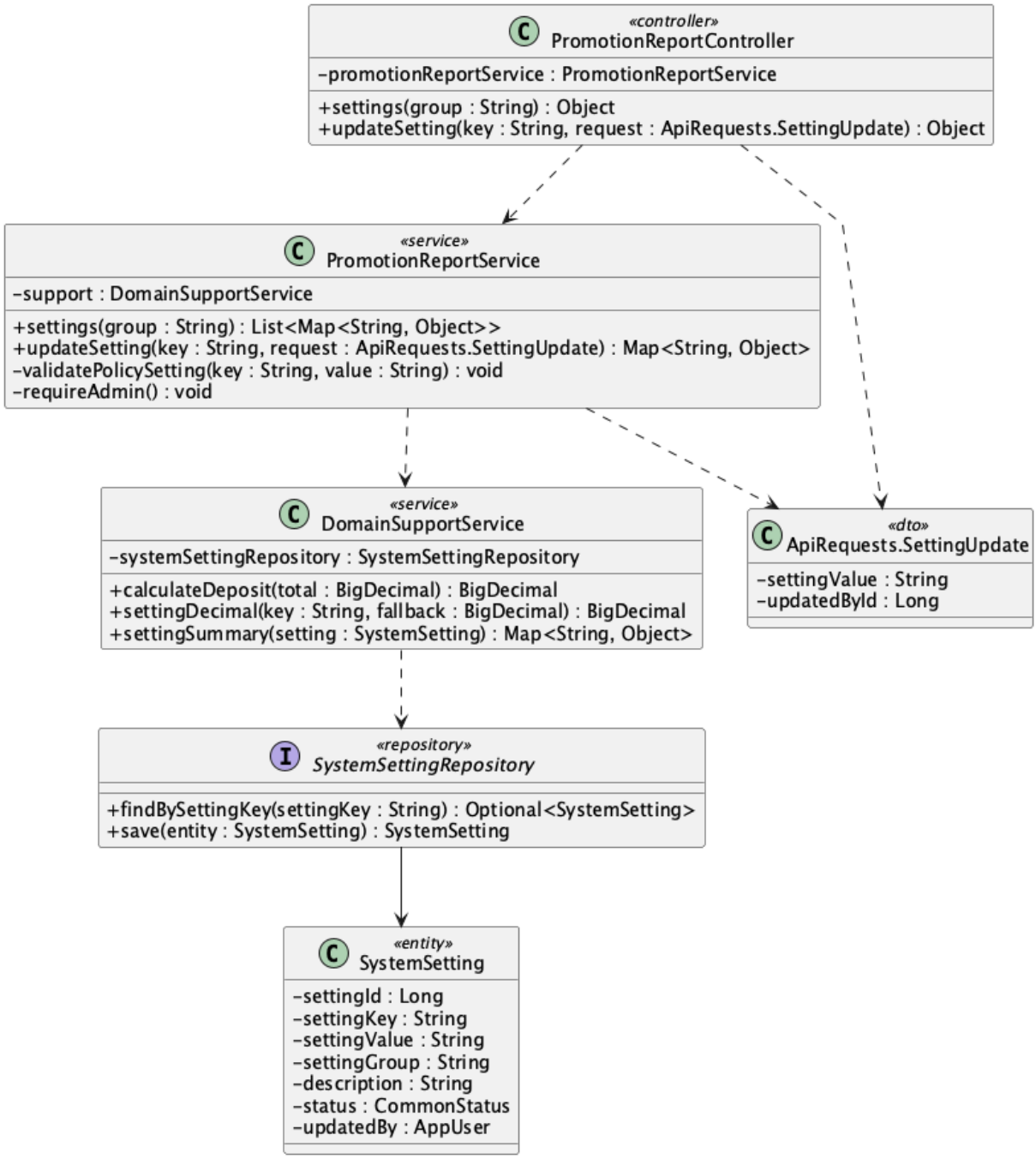
b.booking_code, p.payment_method, p.provider_capture_id

```
FROM refund r
JOIN booking b ON b.booking_id = r.booking_id
LEFT JOIN payment p ON p.payment_id = r.payment_id
ORDER BY r.refund_id DESC;
```

Implementation note: Only the booking owner can submit a request. Staff/Admin review and process it; cash completion is an explicit venue operation marked MANUAL_CASH_REFUND.

48. Configure deposit rules

a. Class Diagram



b. Class Specifications

PromotionReportController

No	Method	Description
01	public Object settings(String group)	Lists policy settings.
02	public Object updateSetting(String key, ApiRequests.SettingUpdate	Exposes Admin policy update.

No	Method	Description
	request)	

PromotionReportService

No	Method	Description
01	public Map<String, Object> updateSetting(String key, ApiRequests.SettingUpdate request)	Validates percentage range and updates deposit.default_percent.

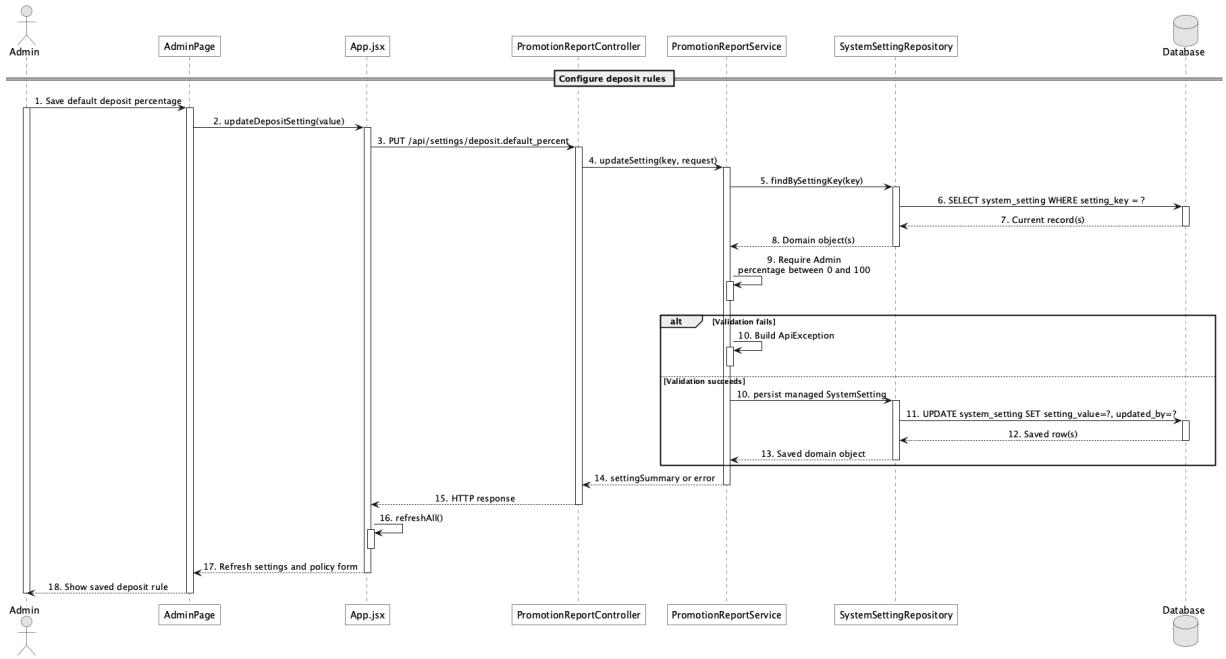
DomainSupportService

No	Method	Description
01	BigDecimal calculateDeposit(BigDecimal total)	Reads deposit.default_percent and computes the booking deposit.

SystemSettingRepository

No	Method	Description
01	Optional<SystemSetting> findBySettingKey(String settingKey)	Loads the deposit setting by stable key.

c. Sequence Diagram



d. Database Queries

== Configure deposit rules ==

1. Update deposit percentage

UPDATE system_setting

SET setting_value = :percent, updated_by = :adminId, updated_at = CURRENT_TIMESTAMP

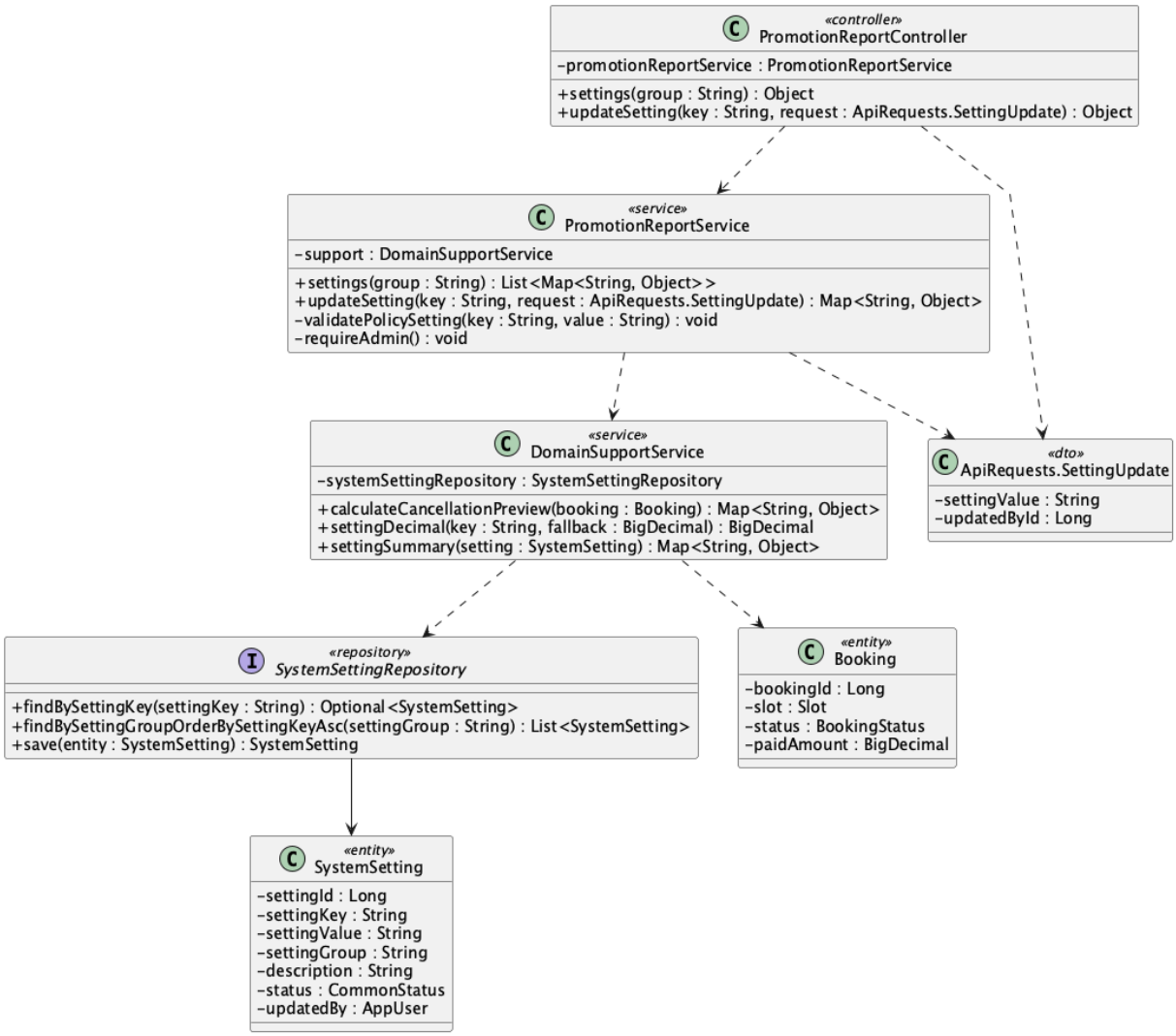
WHERE setting_key = 'deposit.default_percent'

AND CAST(:percent AS numeric) BETWEEN 0 AND 100;

Implementation note: The implemented rule is a single default percentage, not a tiered deposit-rule DSL.

49. Configure cancellation/refund policy

a. Class Diagram



b. Class Specifications

PromotionReportService

No	Method	Description
01	public List<Map<String, Object>> settings(String group)	Reads all or group-filtered system settings.
02	public Map<String, Object> updateSetting(String key, ApiRequests.SettingUpdate request)	Validates and updates refund percentage settings.

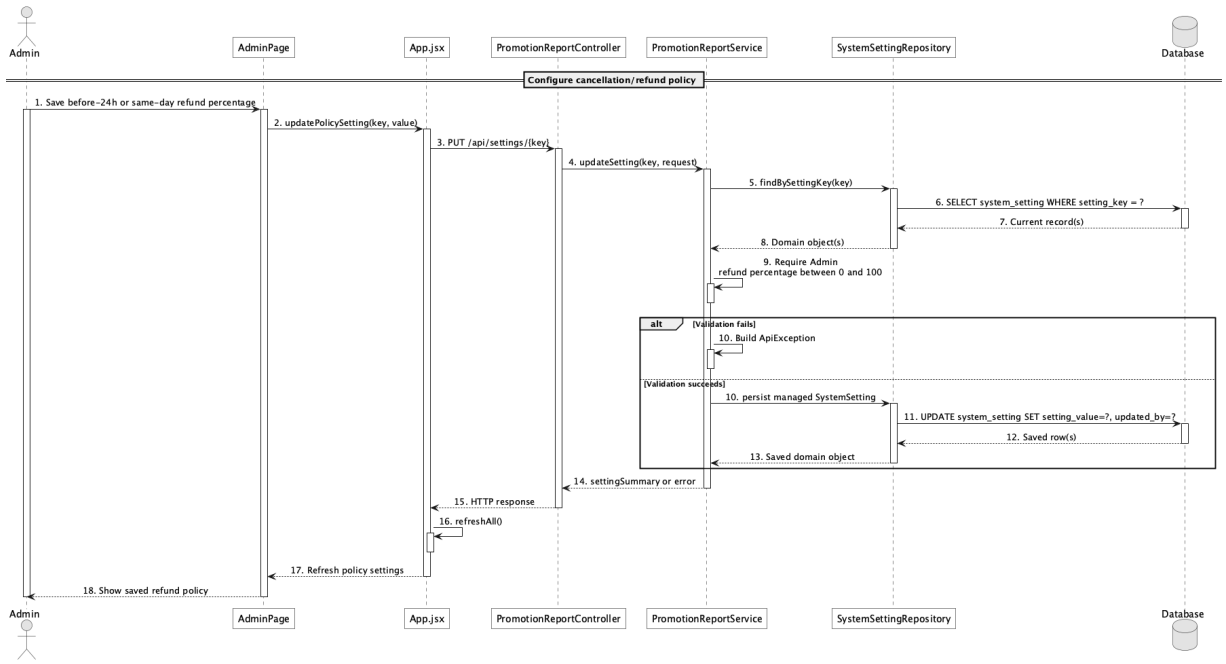
DomainSupportService

No	Method	Description
01	Map<String, Object> calculateCancellationPreview(Booking booking)	Applies refund.before_24h_percent and refund.same_day_percent.
02	BigDecimal settingDecimal(String key, BigDecimal fallback)	Reads numeric policy values with code defaults.

SystemSettingRepository

No	Method	Description
01	List<SystemSetting> findBySettingGroupOrderBySettingKeyAsc(String settingGroup)	Loads the refund policy group for Admin.

c. Sequence Diagram



d. Database Queries

== Configure cancellation/refund policy ==

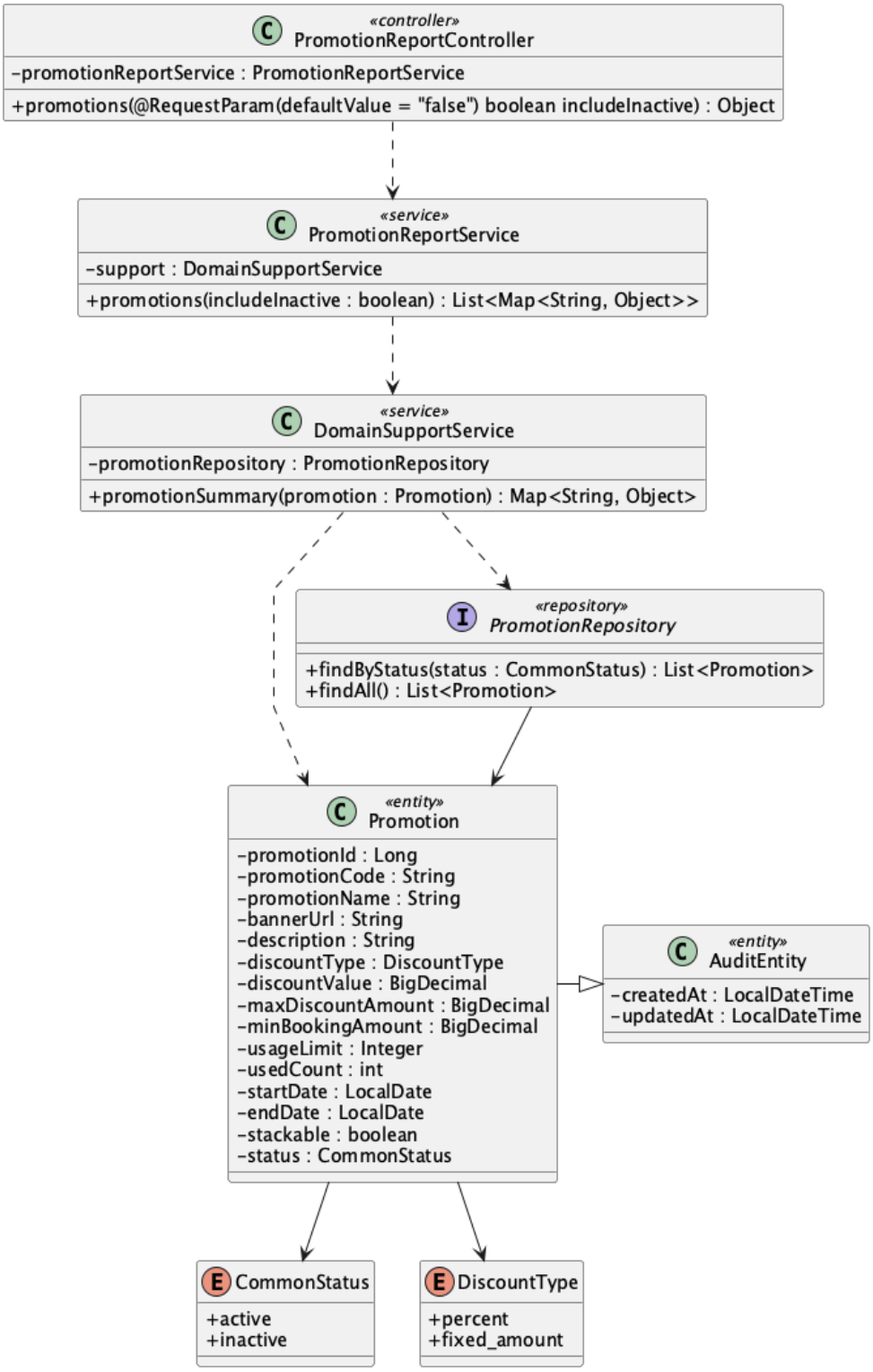
1. Refund policy settings

```
SELECT setting_key, setting_value, description, updated_by, updated_at
FROM system_setting
WHERE setting_group = 'refund'
ORDER BY setting_key;
```

Implementation note: Current cancellation logic has two configured percentage bands: at least 24 hours and same-day before start; after start is zero refund.

50. View active promotions

a. Class Diagram



b. Class Specifications

PromotionReportController

No	Method	Description
01	public Object promotions(boolean includeInactive)	Exposes GET /api/promotions; public callers normally use includeInactive=false.

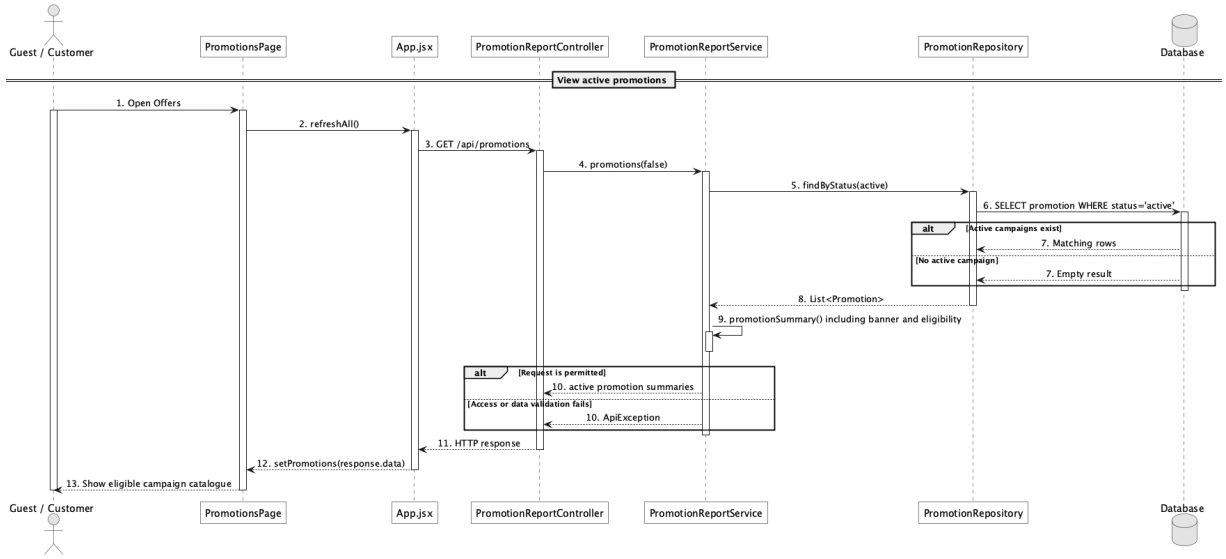
PromotionReportService

No	Method	Description
01	public List<Map<String, Object>> promotions(boolean includeInactive)	Returns active promotions or the full admin catalogue.

PromotionRepository

No	Method	Description
01	List<Promotion> findByStatus(CommonStatus status)	Loads active promotion campaigns.

c. Sequence Diagram



d. Database Queries

== View active promotions ==

1. Active promotion catalogue

```
SELECT promotion_id, promotion_code, promotion_name, banner_url, description,
    discount_type, discount_value, max_discount_amount, min_booking_amount,
    start_date, end_date, usage_limit, used_count
FROM promotion
```

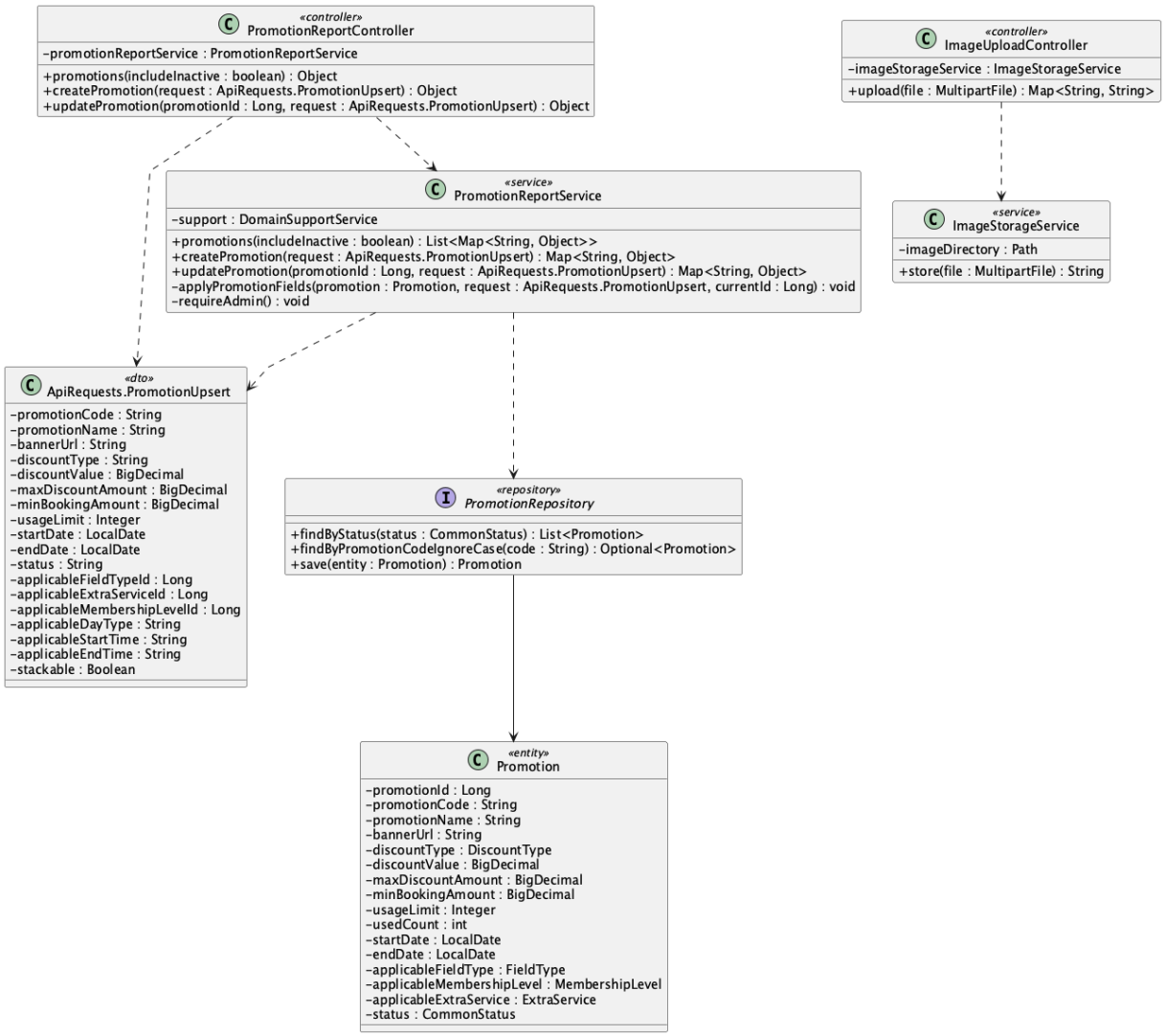
WHERE status = 'active'

ORDER BY promotion_id;

Implementation note: The repository status filter does not itself filter dates; eligibility date validation occurs when applying a code.

51. Manage promotion campaigns

a. Class Diagram



b. Class Specifications

PromotionReportController

No	Method	Description
01	public Object createPromotion(ApiRequests.PromotionUpsert request)	Creates an Admin campaign.
02	public Object	Updates a campaign and active/inactive state.

No	Method	Description
	updatePromotion(Long promotionId, ApiRequests.PromotionUpsert request)	

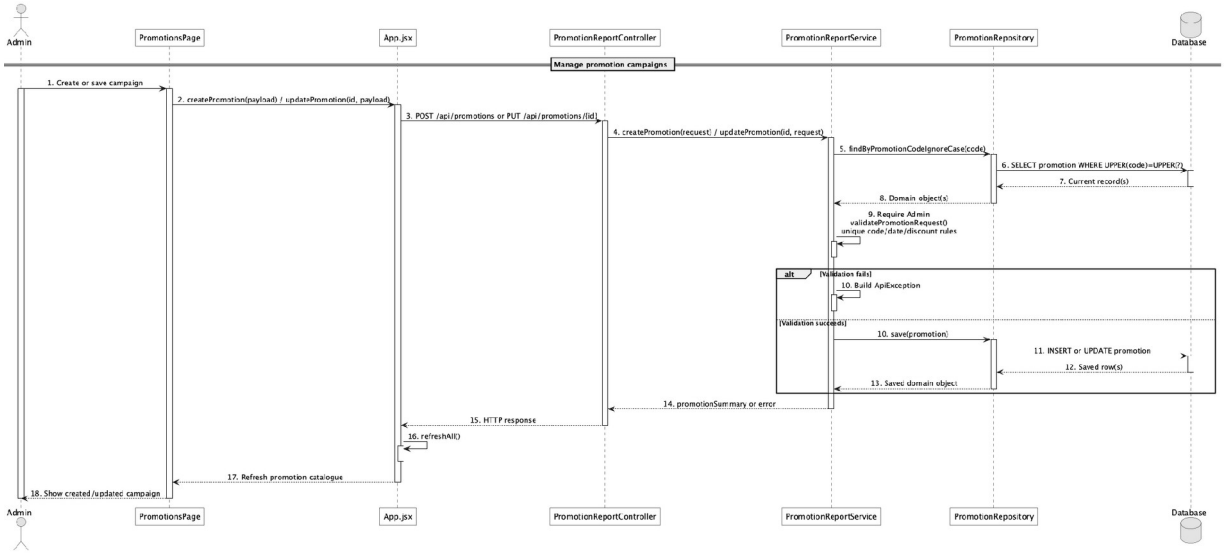
PromotionReportService

No	Method	Description
01	createPromotion(request : ApiRequests.PromotionUpsert) : Map<String, Object>	Validates dates, positive discount, non-negative caps/minimum/usage limit, applicability, and stackable policy before creating a campaign.
02	updatePromotion(id, request) : Map<String, Object>	Applies the same validation while updating campaign eligibility, dates, limits, status, and membership-stacking policy.

PromotionRepository

No	Method	Description
01	Optional<Promotion> findByPromotionCodeIgnoreCase(String promotionCode)	Enforces case-insensitive code uniqueness in service validation.

c. Sequence Diagram



d. Database Queries

== Manage promotion campaigns ==

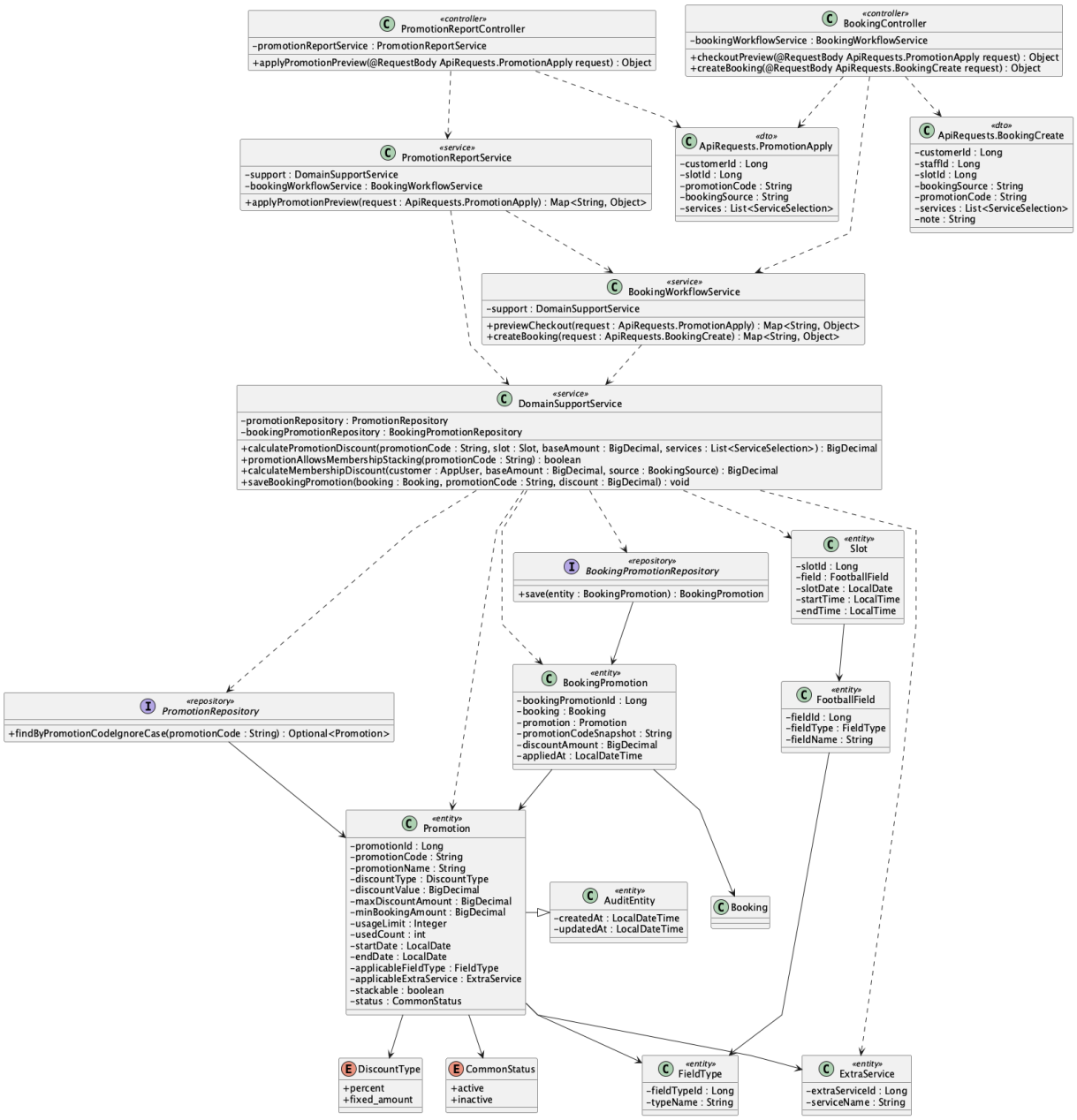
1. Create promotion campaign

```
INSERT INTO promotion
  (promotion_code, promotion_name, banner_url, description, discount_type, discount_value,
   max_discount_amount, min_booking_amount, usage_limit, used_count, start_date, end_date,
   applicable_field_type_id, applicable_membership_level_id, applicable_extra_service_id,
   applicable_day_type, applicable_start_time, applicable_end_time, stackable, status, created_at,
updated_at)
VALUES
  (:code, :name, :bannerUrl, :description, :discountType, :discountValue,
   :maxDiscount, :minBookingAmount, :usageLimit, 0, :startDate, :endDate,
   :fieldTypeId, :membershipLevelId, :extraServiceId, :dayType, :startTime, :endTime,
   false, :status, CURRENT_TIMESTAMP, CURRENT_TIMESTAMP);
```

Implementation note: SecurityConfig allows promotion mutations only for Admin. One code is supported per booking; stackable explicitly controls whether the membership discount may also apply after the capped promotion.

52. Apply promotion to booking

a. Class Diagram



b. Class Specifications

PromotionReportService

No	Method	Description
01	public Map<String, Object> applyPromotionPreview(ApiRequests.PromotionApply request)	Delegates promotion preview to checkout pricing.

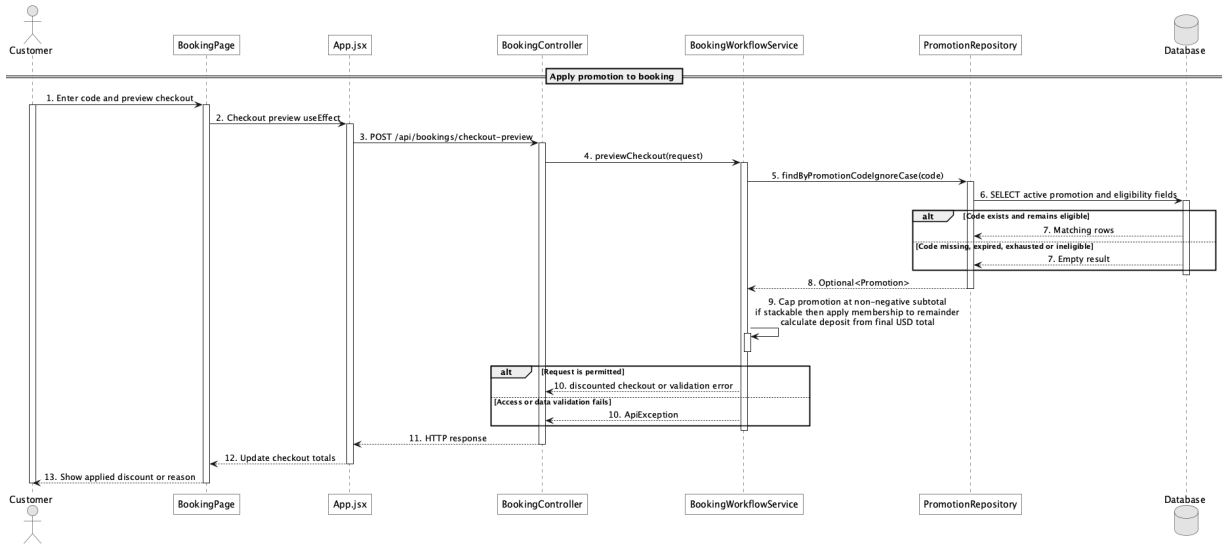
DomainSupportService

No	Method	Description
01	calculatePromotionDiscount(code, slot, baseAmount, services, customer) : BigDecimal	Validates all campaign conditions and caps either percent or fixed discount at the non-negative subtotal.
02	promotionAllowsMembershipStacking(code : String) : boolean	Treats stackable as permission to apply the Customer membership discount after the promotion.
03	saveBookingPromotion(booking, code, discount) : void	Snapshots the applied code/discount and consumes campaign usage once when the booking is created.

PromotionRepository

No	Method	Description
01	Optional<Promotion> findByPromotionCodeIgnoreCase(String promotionCode)	Resolves the entered code.

c. Sequence Diagram



d. Database Queries

== Apply promotion to booking ==

1. Promotion eligibility record

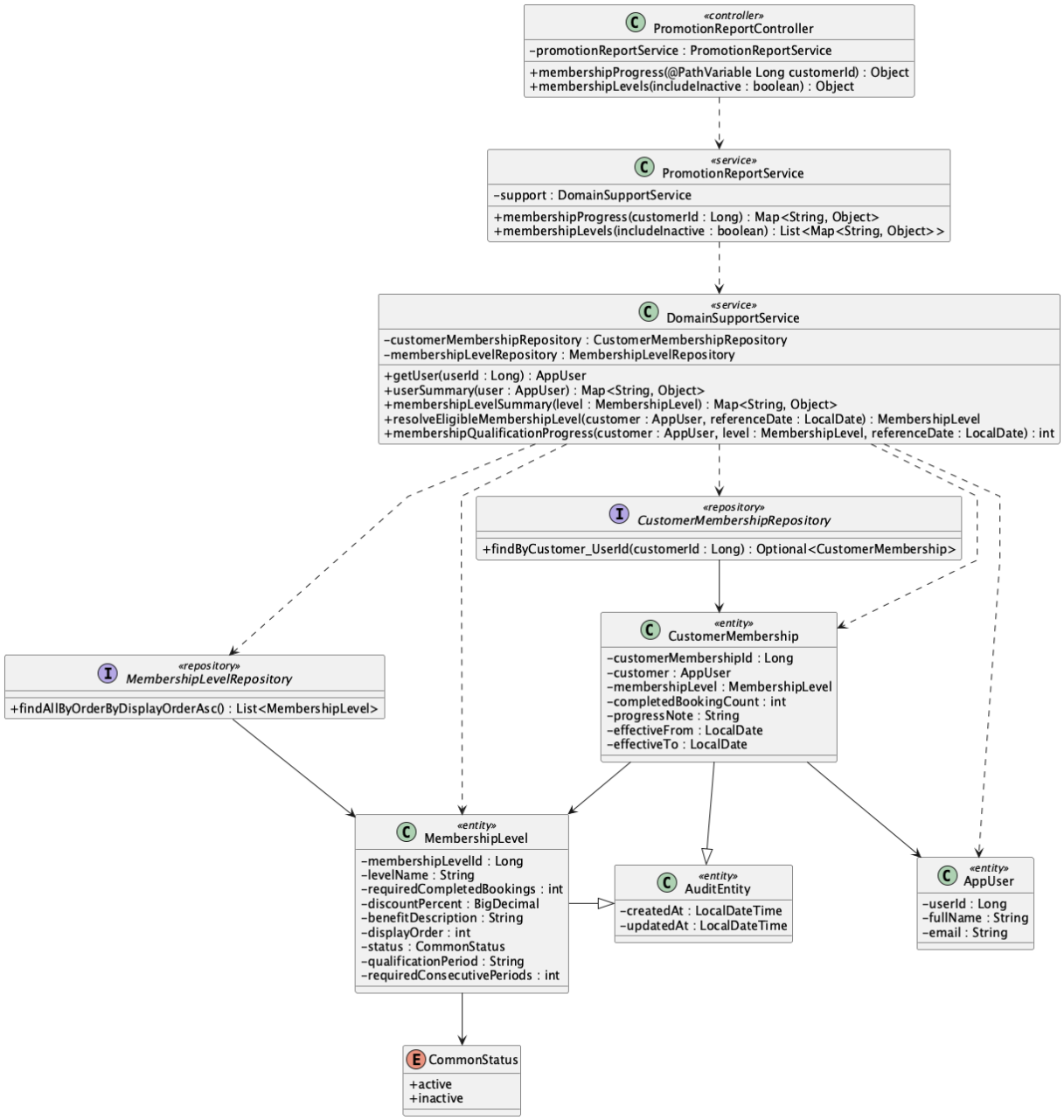
```
SELECT p.promotion_id, p.promotion_code, p.discount_type, p.discount_value,
    p.max_discount_amount, p.min_booking_amount, p.usage_limit, p.used_count,
    p.start_date, p.end_date, p.applicable_field_type_id,
    p.applicable_membership_level_id, p.applicable_extra_service_id,
```

```
    p.applicable_day_type, p.applicable_start_time, p.applicable_end_time
FROM promotion p
WHERE UPPER(p.promotion_code) = UPPER(:promotionCode)
  AND p.status = 'active';
```

Implementation note: booking_promotion is the applied snapshot table; multi-code stacking is intentionally out of scope.

53. View membership progress

a. Class Diagram



b. Class Specifications

PromotionReportController

No	Method	Description
01	public Object membershipProgress(Long customerId)	Exposes GET /api/membership/{customerId}/progress.

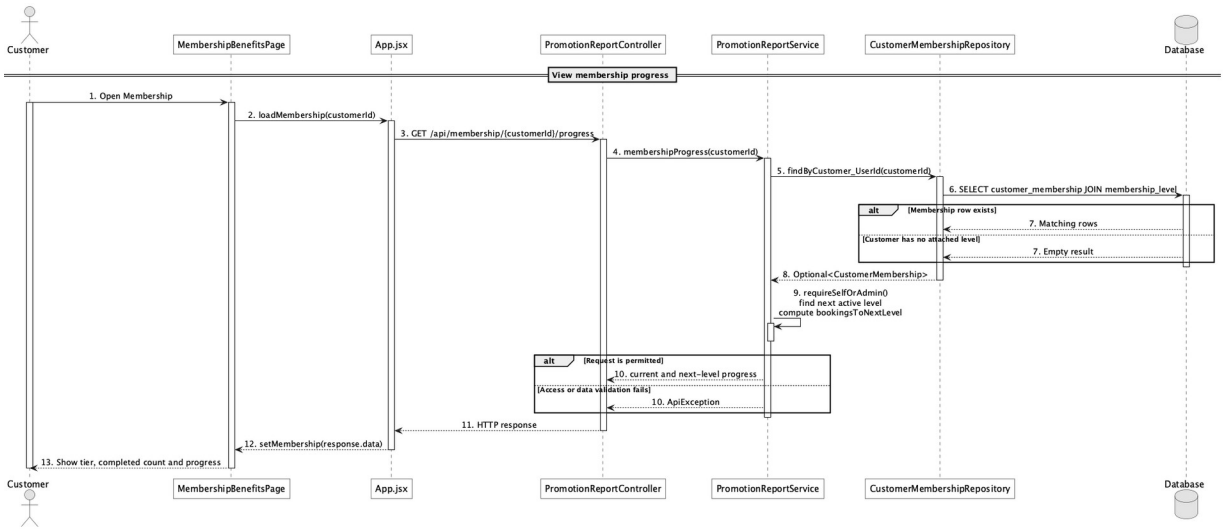
PromotionReportService

No	Method	Description
01	public Map<String, Object> membershipProgress(Long customerId)	Returns current level, lifetime audit count, period-aware next-level progress, target, and remaining value.

CustomerMembershipRepository

No	Method	Description
01	Optional<CustomerMembership> findByCustomer_UserId(Long customerId)	Loads the customer's single membership progress row.

c. Sequence Diagram



d. Database Queries

== View membership progress ==

1. Membership progress and next thresholds

```
SELECT cm.customer_id, cm.completed_booking_count, ml.membership_level_id,
    ml.level_name, ml.discount_percent, ml.required_completed_bookings,
```

ml.qualification_period, ml.required_consecutive_periods, ml.benefit_description

FROM customer_membership cm

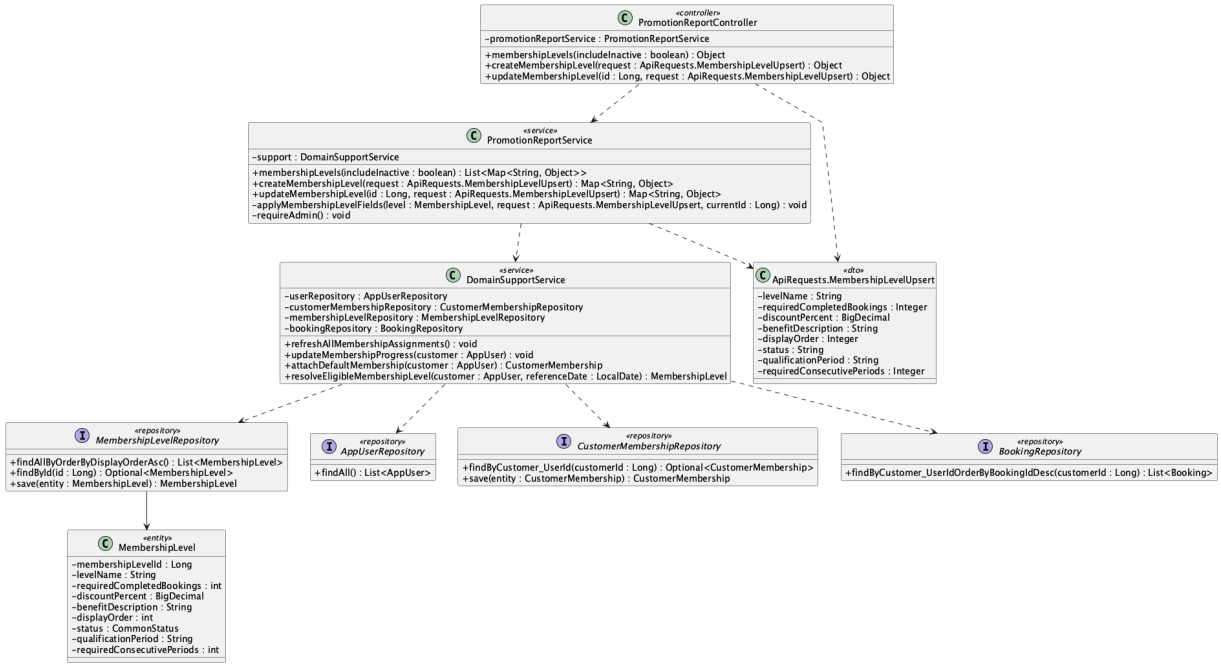
JOIN membership_level ml ON ml.membership_level_id = cm.membership_level_id

WHERE cm.customer_id = :customerId;

Implementation note: Eligibility/progress is computed from completed bookings using each next level's lifetime, current-month, or consecutive-week rule.

54. Manage membership rules

a. Class Diagram



b. Class Specifications

PromotionReportController

No	Method	Description
01	public Object createMembershipLevel(ApiRequests.MembershipLevelUpsert request)	Creates a membership level.
02	public Object updateMembershipLevel(Long id, ApiRequests.MembershipLevelUpsert request)	Updates a level's threshold, discount, benefit, order, and status.

PromotionReportService

No	Method	Description
01	createMembershipLevel(request : ApiRequests.MembershipLevelUpsert) : Map<String, Object>	Validates and persists a new level, then recalculates every customer's existing membership assignment.
02	updateMembershipLevel(id : Long, request : ApiRequests.MembershipLevelUpsert) : Map<String, Object>	Updates an existing rule and recalculates assignments without creating duplicate customer-membership rows.

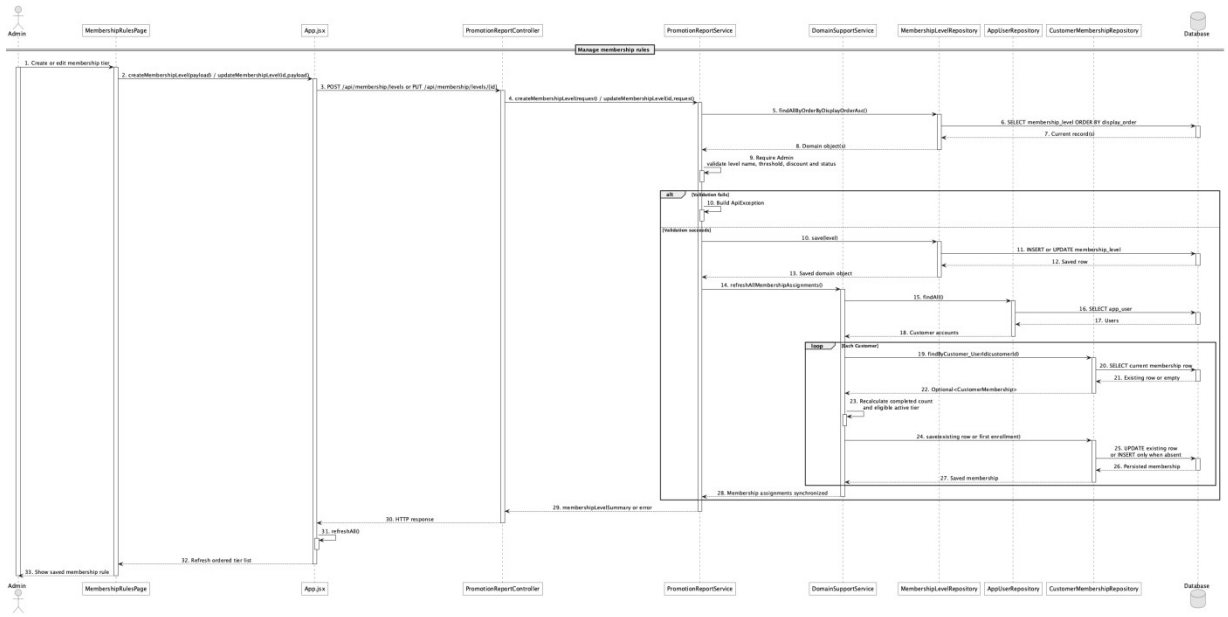
MembershipLevelRepository

No	Method	Description
01	List<MembershipLevel> findAllByOrderByDisplayOrderAsc()	Provides deterministic progression order.

DomainSupportService

No	Method	Description
01	refreshAllMembershipAssignments() : void	Re-evaluates all Customer accounts after an Admin changes a membership rule.
02	updateMembershipProgress(customerId : Long, increment : int) : CustomerMembership	Updates the customer's single membership row in place and falls back to the first active tier when no threshold currently qualifies.
03	attachDefaultMembership(customer : AppUser) : CustomerMembership	Returns the existing customer membership when present; otherwise inserts exactly one default assignment.

c. Sequence Diagram



d. Database Queries

== Manage membership rules ==

1. Membership rule catalogue

SELECT membership_level_id, level_name, required_completed_bookings,

qualification_period, required_consecutive_periods,

discount_percent, benefit_description, display_order, status

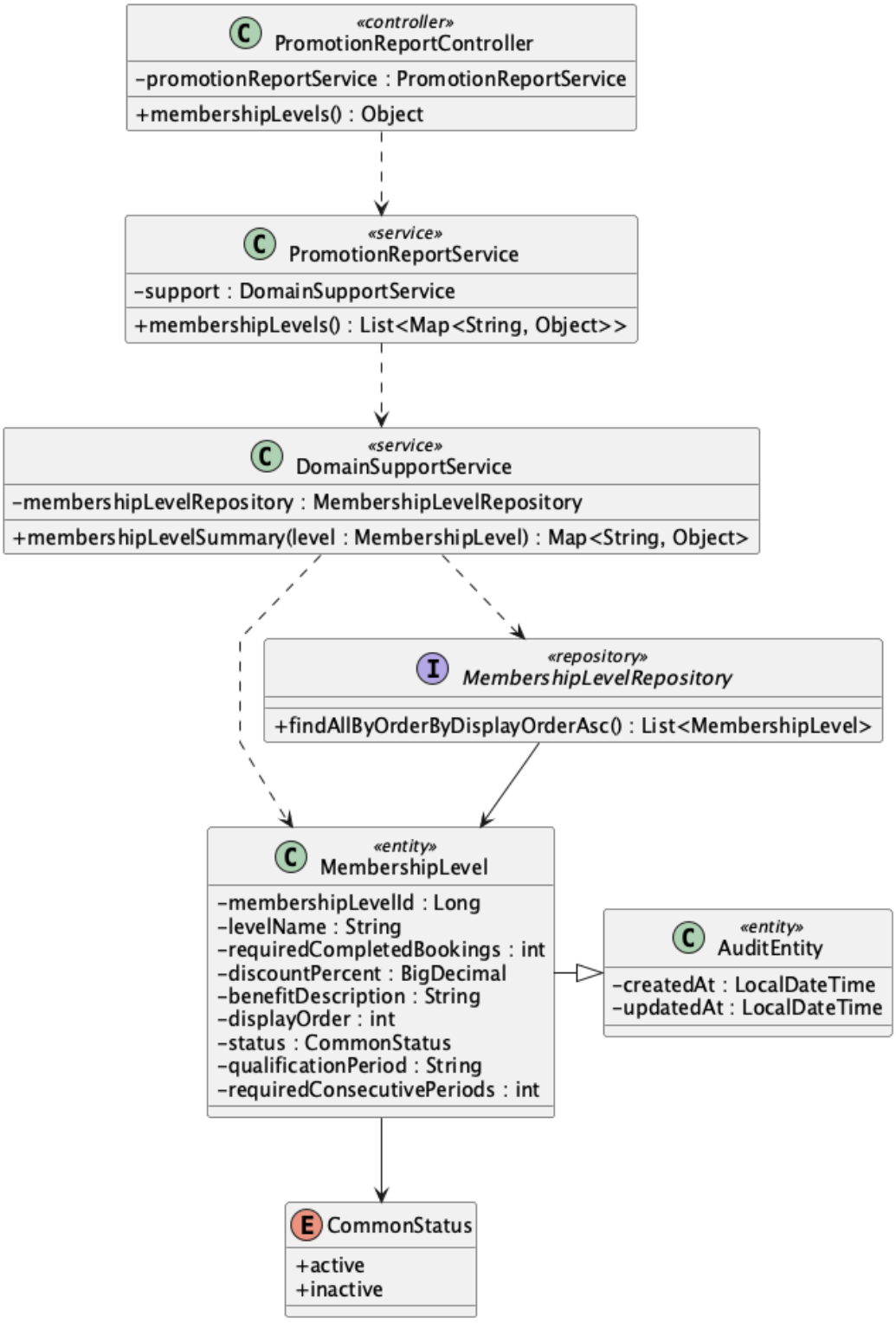
FROM membership_level

ORDER BY display_order ASC;

Implementation note: Qualification period is lifetime, monthly, or weekly; weekly rules require a configurable consecutive qualifying-week streak.

55. View membership benefits

a. Class Diagram



b. Class Specifications

PromotionReportController

No	Method	Description
01	public Object membershipLevels(boolean includeInactive)	Exposes public GET /api/membership/levels.

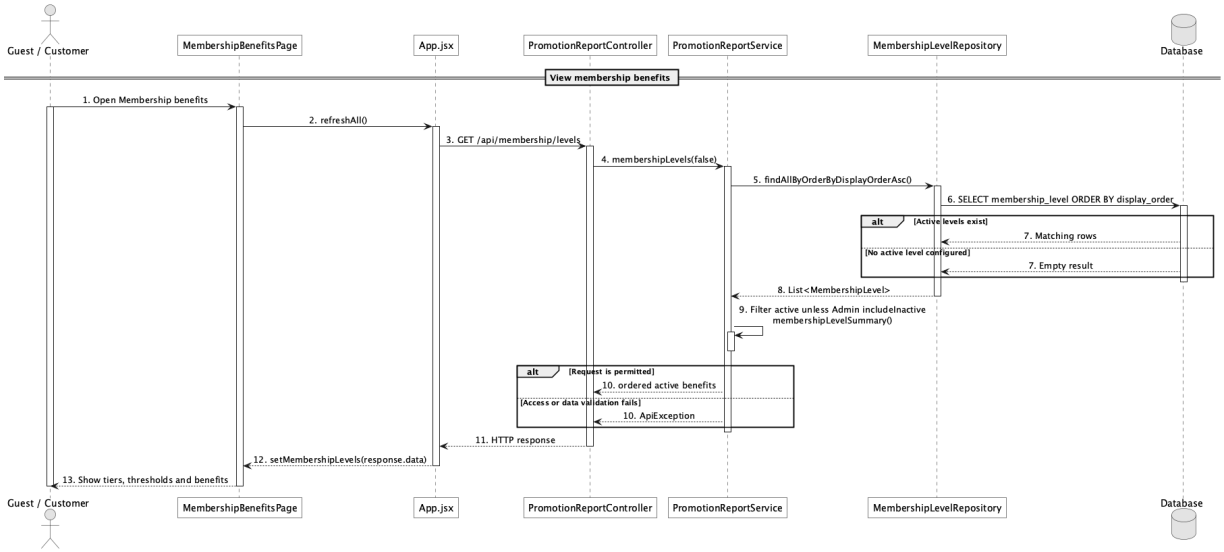
PromotionReportService

No	Method	Description
01	public List<Map<String, Object>> membershipLevels(boolean includeInactive)	Returns ordered level names, thresholds, discounts, and benefit descriptions.

MembershipLevelRepository

No	Method	Description
01	List<MembershipLevel> findAllByOrderByDisplayOrderAsc()	Loads the benefit catalogue in display order.

c. Sequence Diagram



d. Database Queries

== View membership benefits ==

1. Membership benefits

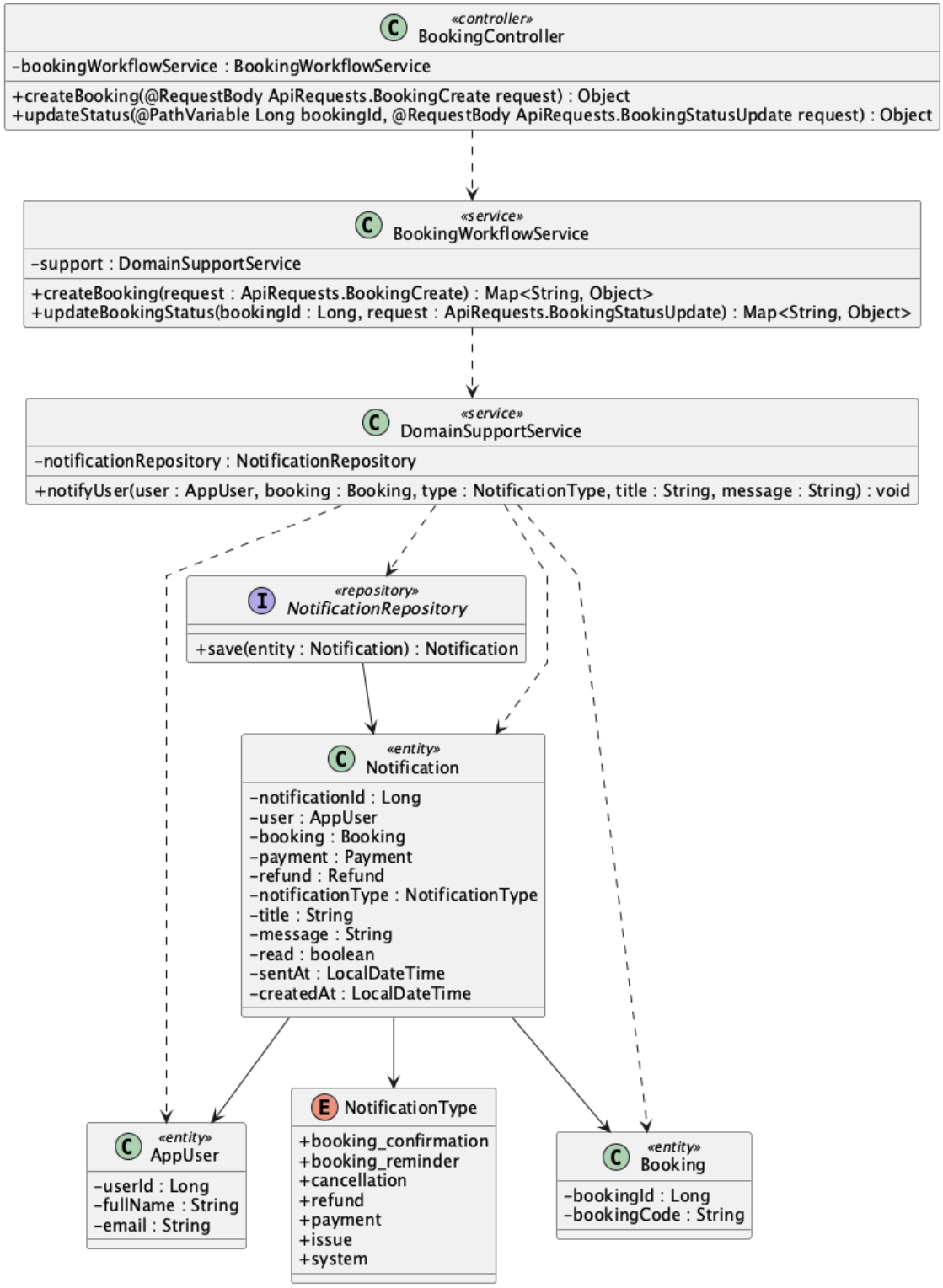
```
SELECT membership_level_id, level_name, required_completed_bookings,
    qualification_period, required_consecutive_periods,
    discount_percent, benefit_description, display_order, status
FROM membership_level
```

```
WHERE status = 'active'
ORDER BY display_order;
```

Implementation note: Public calls exclude inactive levels; only Admin may request includeInactive=true.

56. Send booking confirmation notification

a. Class Diagram



b. Class Specifications

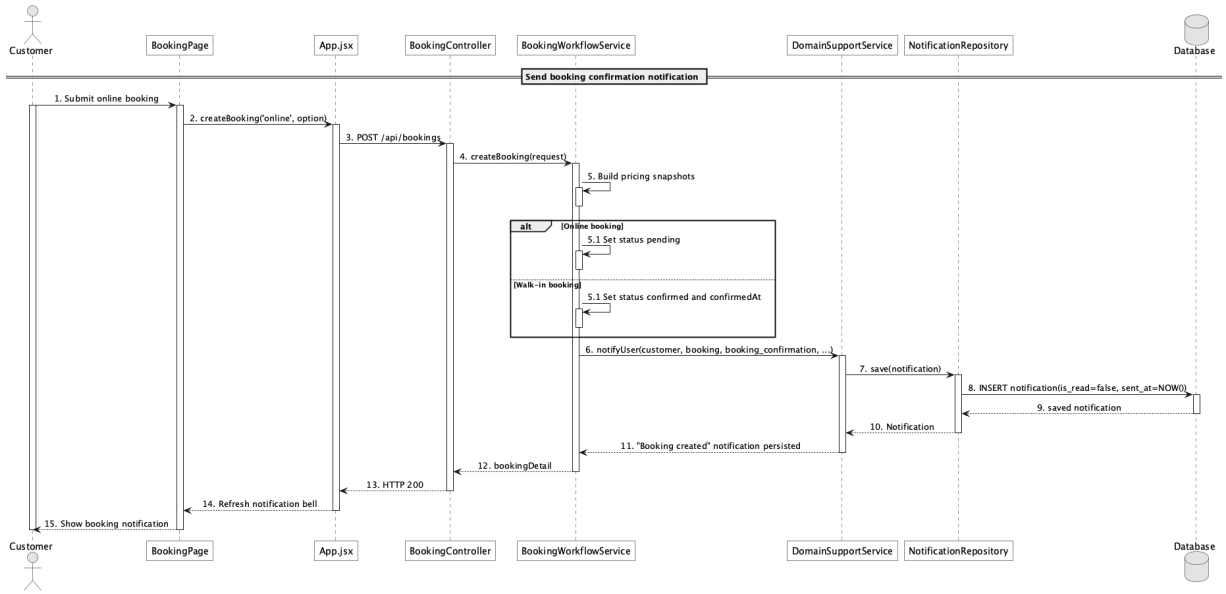
BookingWorkflowService

No	Method	Description
01	public Map<String, Object> createBooking(ApiRequests.BookingCreate request)	Creates a booking-created notification.
02	public Map<String, Object> updateBookingStatus(Long bookingId, ApiRequests.BookingStatusUpdate request)	Creates confirmed/rejected lifecycle notifications.

DomainSupportService

No	Method	Description
01	void notifyUser(AppUser user, Booking booking, NotificationType type, String title, String message)	Persists the in-app notification.

c. Sequence Diagram



d. Database Queries

== Send booking confirmation notification ==

1. Booking confirmation notification

INSERT INTO notification

(user_id, booking_id, notification_type, title, message, is_read, sent_at, created_at)

VALUES

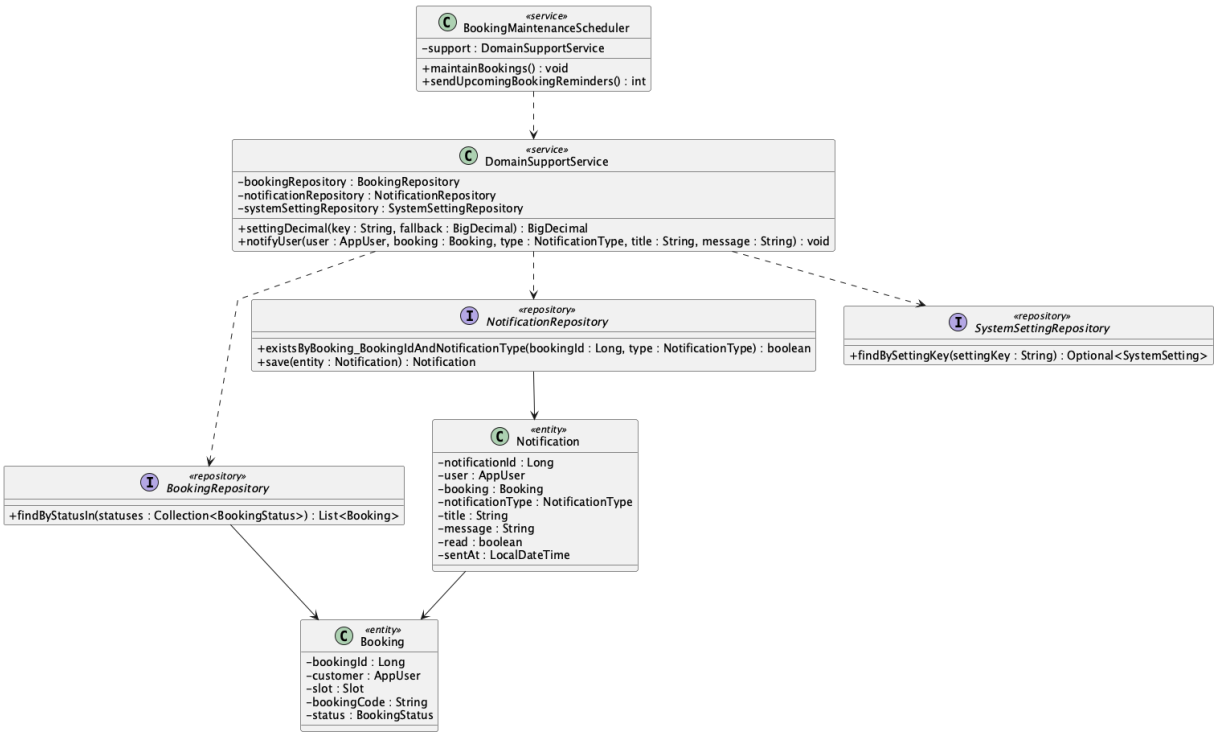
(:customerId, :bookingId, 'booking_confirmation', :title, :message, false,

CURRENT_TIMESTAMP, CURRENT_TIMESTAMP);

Implementation note: This use case is implemented as in-app persistence; it is not an SMTP booking-confirmation email workflow.

57. Send booking reminder notification

a. Class Diagram



b. Class Specifications

BookingMaintenanceScheduler

No	Method	Description
01	public void maintainBookings()	Runs reminder maintenance on the configured fixed delay.
02	public int sendUpcomingBookingReminders()	Creates one reminder for pending/confirmed bookings within the configured window.

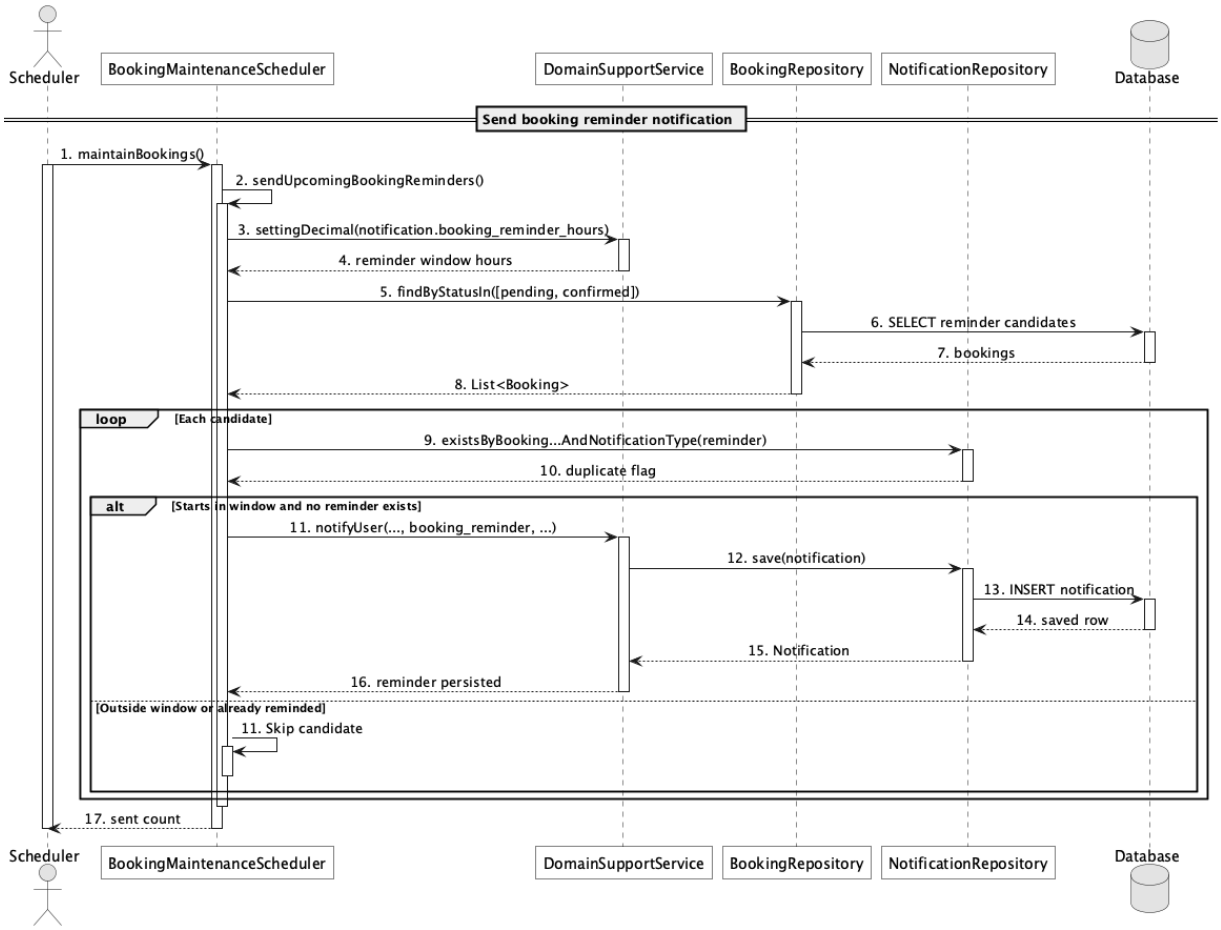
NotificationRepository

No	Method	Description
01	boolean existsByBooking_BookingIdAndNotificationType(Long bookingId, NotificationType notificationType)	Prevents duplicate booking_reminder rows.

BookingRepository

No	Method	Description
01	List<Booking> findByStatusIn(Collection<BookingStatus> statuses)	Loads pending and confirmed reminder candidates.

c. Sequence Diagram



d. Database Queries

== Send booking reminder notification ==

1. Upcoming bookings without a reminder

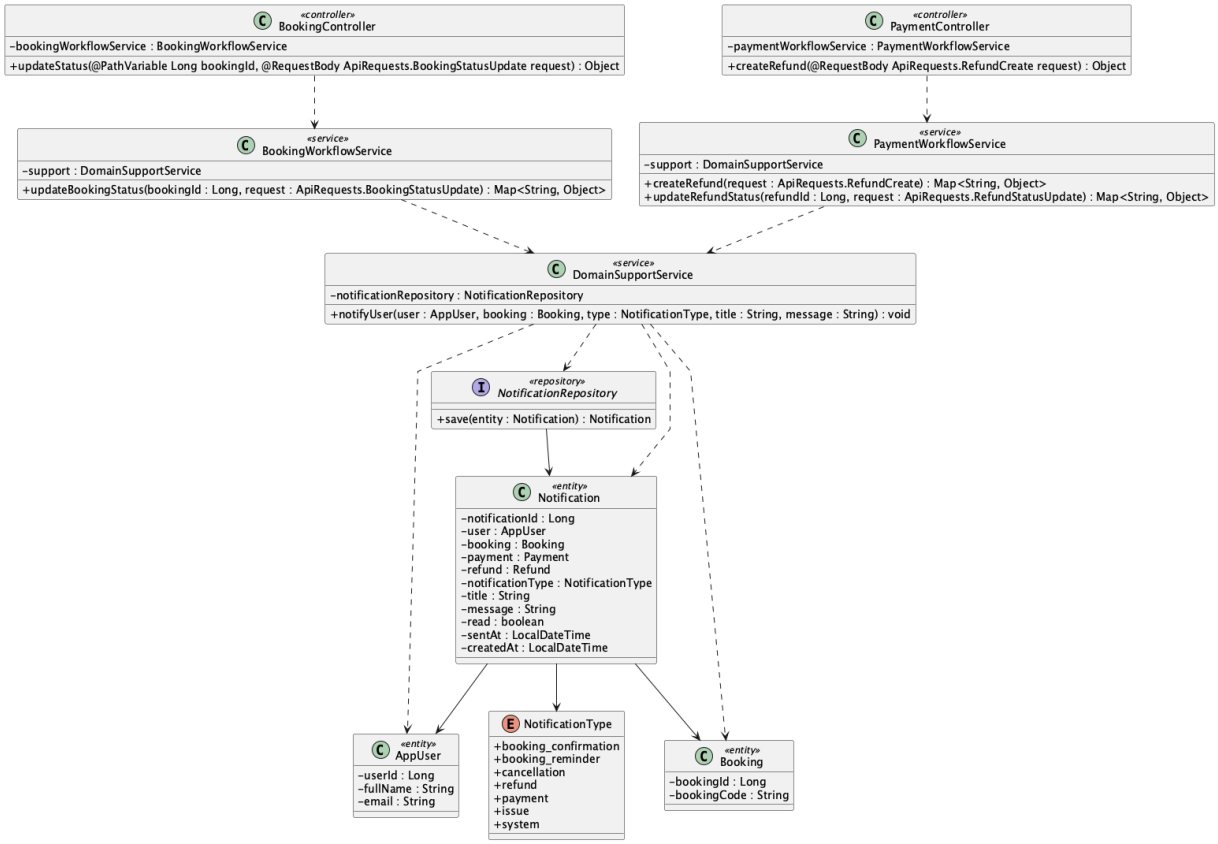
```
SELECT b.booking_id, b.booking_code, b.customer_id, s.slot_date, s.start_time
FROM booking b
JOIN slot s ON s.slot_id = b.slot_id
WHERE b.status IN ('pending','confirmed')
  AND (s.slot_date + s.start_time) BETWEEN CURRENT_TIMESTAMP
      AND CURRENT_TIMESTAMP + (:reminderHours * INTERVAL '1 hour')
```

```
AND NOT EXISTS (
  SELECT 1 FROM notification n
  WHERE n.booking_id = b.booking_id
    AND n.notification_type = 'booking_reminder'
);
```

Implementation note: The reminder is exactly-once at the application-query level for each booking/type; there is no database unique constraint on that pair.

58. Send cancellation/refund notification

a. Class Diagram



b. Class Specifications

BookingWorkflowService

No	Method	Description
01	public Map<String, Object> updateBookingStatus(Long bookingId, ApiRequests.BookingStatusUpdate request)	Creates cancellation notification when status becomes cancelled.

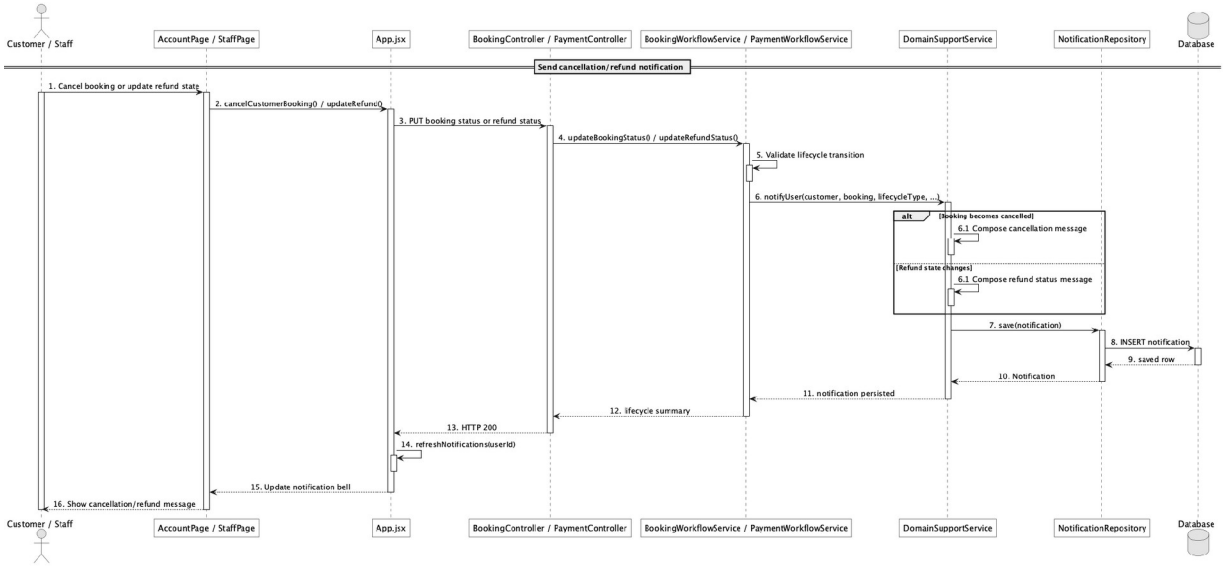
PaymentWorkflowService

No	Method	Description
01	public Map<String, Object> updateRefundStatus(Long refundId, ApiRequests.RefundStatusUpdate request)	Notifies the customer for approved, rejected, processing, completed, and failed refund states.

DomainSupportService

No	Method	Description
01	void notifyUser(AppUser user, Booking booking, NotificationType type, String title, String message)	Writes cancellation/refund messages to notification.

c. Sequence Diagram



d. Database Queries

== Send cancellation/refund notification ==

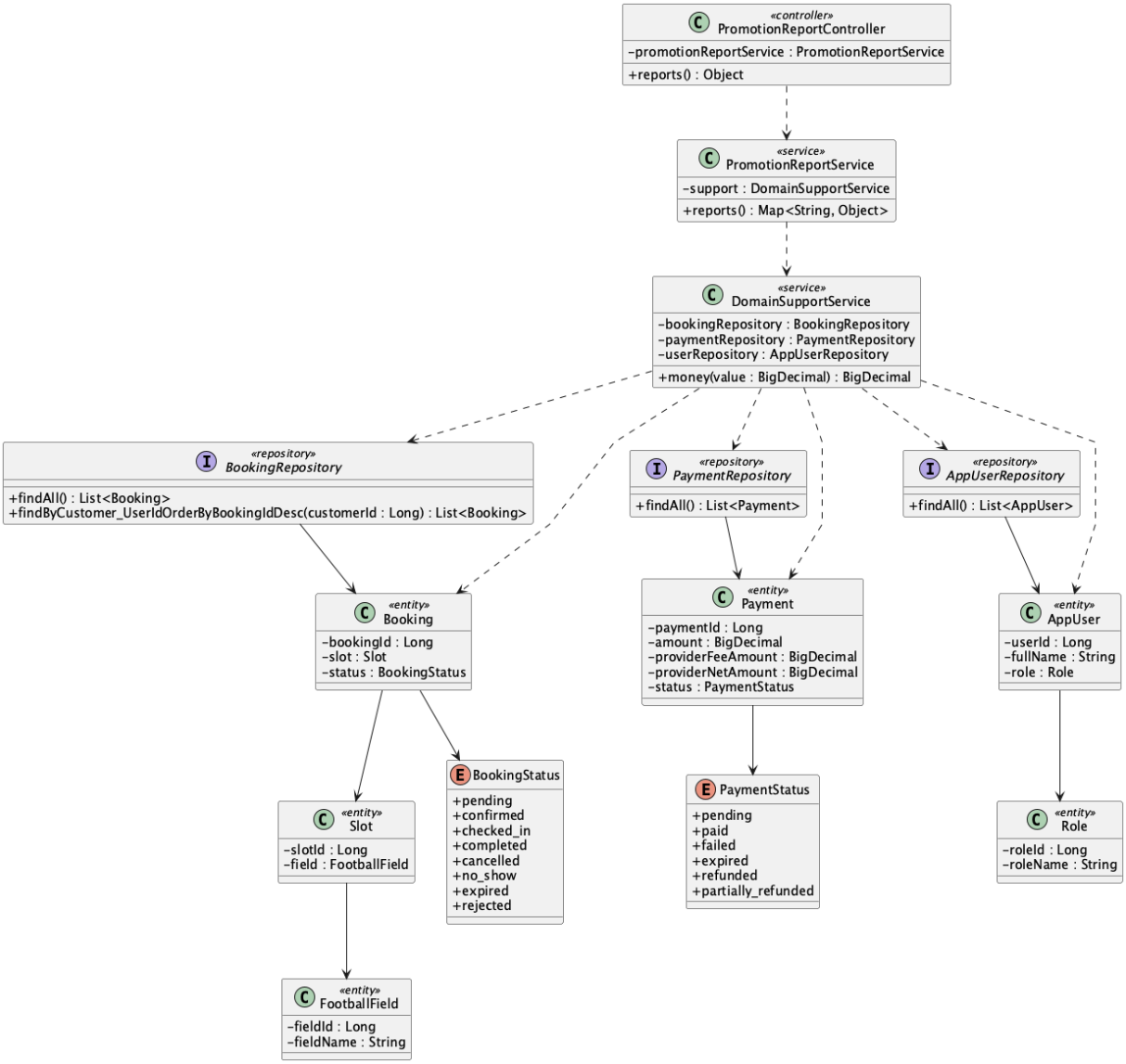
1. Cancellation and refund notifications

```
SELECT notification_id, user_id, booking_id, refund_id, notification_type,
    title, message, is_read, sent_at
FROM notification
WHERE user_id = :customerId
  AND notification_type IN ('cancellation','refund')
ORDER BY notification_id DESC;
```

Implementation note: Notification.refund_id exists in the schema/entity, but notifyUser currently accepts only a booking and does not populate payment_id or refund_id.

59. View revenue report

a. Class Diagram



b. Class Specifications

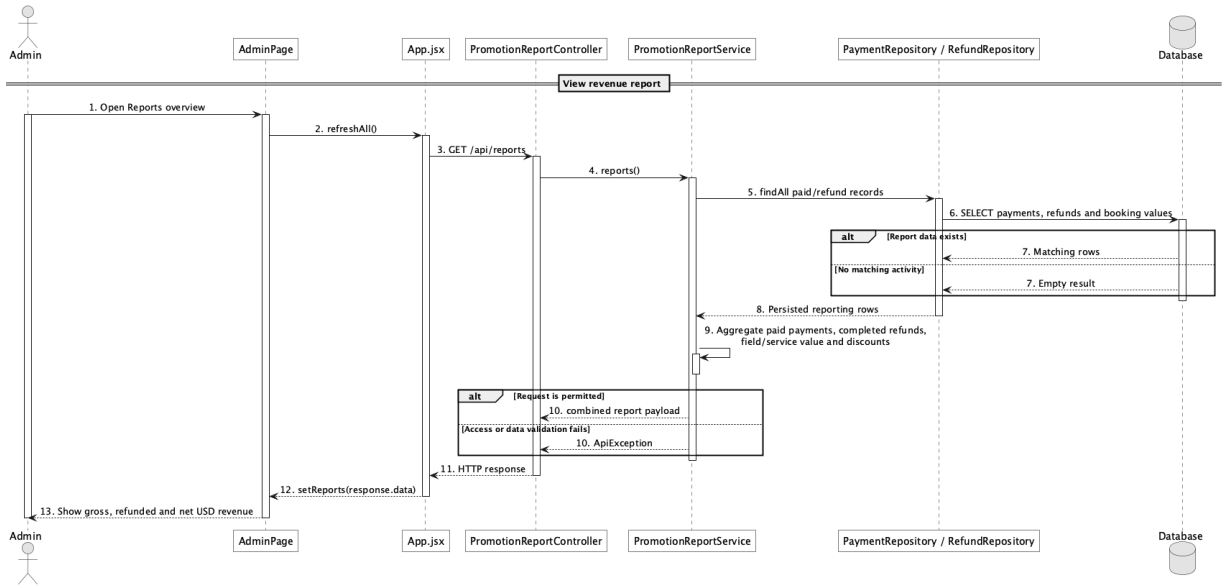
PromotionReportController

No	Method	Description
01	public Object reports()	Exposes Admin GET /api/reports.

PromotionReportService

No	Method	Description
01	public Map<String, Object> reports()	Computes gross collected, completed refunds, tracked PayPal fees, net revenue, field/service value, and discount totals.

c. Sequence Diagram



d. Database Queries

== View revenue report ==

1. Revenue totals

SELECT

COALESCE((SELECT SUM(amount) FROM payment WHERE status IN ('paid','partially_refunded','refunded')), 0) AS gross_collected,

COALESCE((SELECT SUM(refund_amount) FROM refund WHERE status = 'completed'), 0) AS refund_total,

COALESCE((SELECT SUM(provider_fee_amount) FROM payment WHERE status IN ('paid','partially_refunded','refunded')), 0) AS provider_fee_total,

COALESCE((SELECT SUM(amount) FROM payment WHERE status IN ('paid','partially_refunded','refunded')), 0)

- COALESCE((SELECT SUM(refund_amount) FROM refund WHERE status = 'completed'), 0)

- COALESCE((SELECT SUM(provider_fee_amount) FROM payment WHERE status IN ('paid','partially_refunded','refunded')), 0) AS net_revenue,

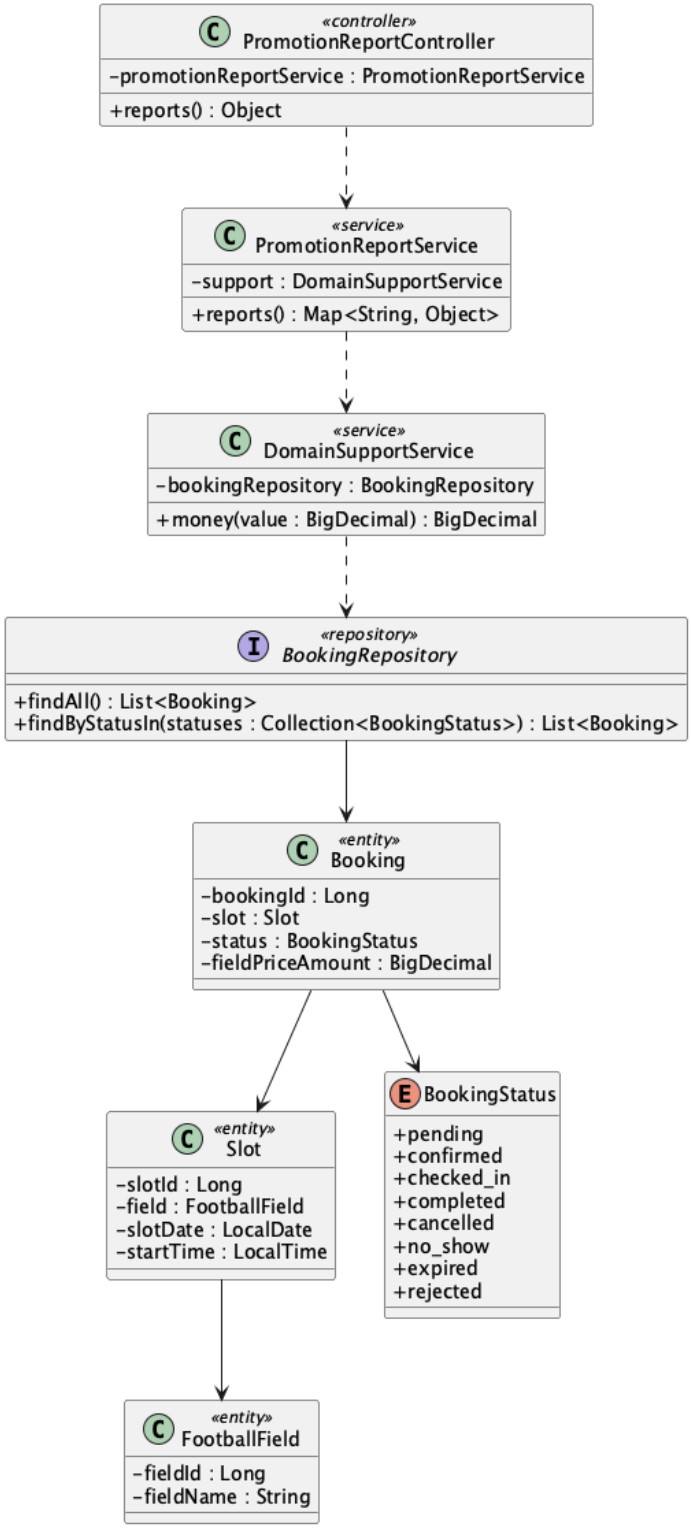
COALESCE((SELECT SUM(field_price_amount) FROM booking WHERE status NOT IN ('rejected','expired')), 0) AS field_value,

```
  COALESCE((SELECT SUM(service_total_amount) FROM booking WHERE status NOT IN
('rejected','expired')), 0) AS service_value;
```

Implementation note: The live service aggregates in Java. Provider fees come from PayPal seller_receivable_breakdown; legacy rows without a returned breakdown are counted as untracked rather than estimated.

60. View booking report

a. Class Diagram (similar to 60)



b. Class Specifications

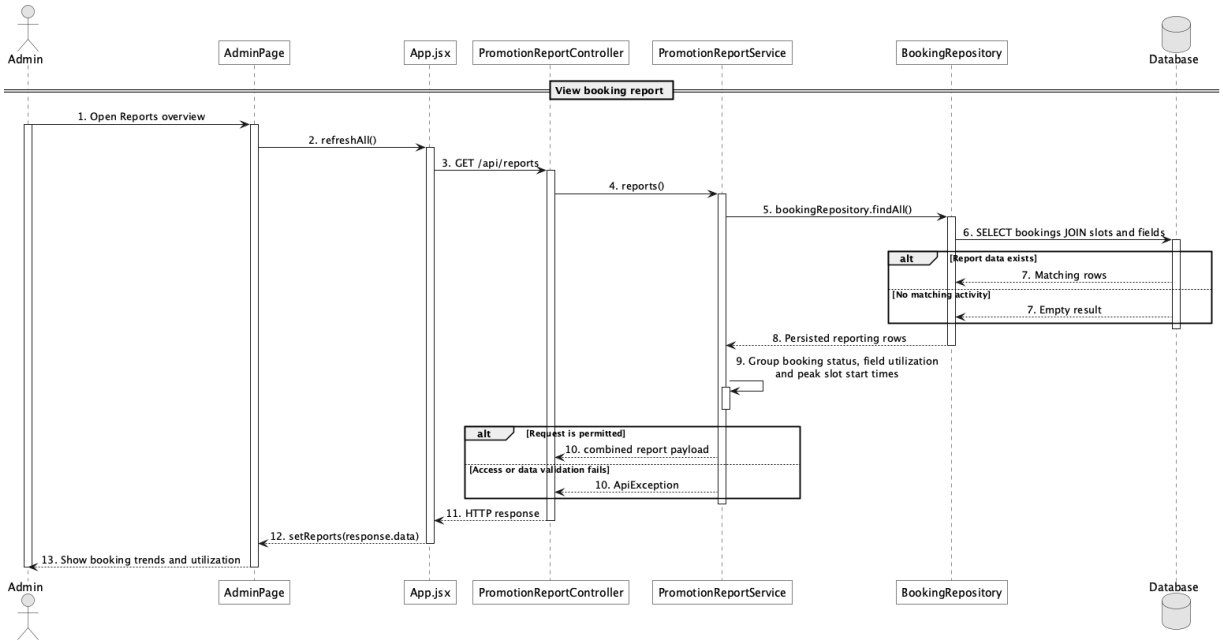
PromotionReportService

No	Method	Description
01	public Map<String, Object> reports()	Builds booking counts, status counts, field utilization, booked field value, and peak start times.

BookingRepository

No	Method	Description
01	List<Booking> findByStatusIn(Collection<BookingStatus> statuses)	Custom status lookup used elsewhere; reports currently use inherited findAll().

c. Sequence Diagram



d. Database Queries

== View booking report ==

1. Booking status, utilization, and peak time

SELECT b.status, f.field_name, s.start_time, COUNT(*) AS booking_count,

SUM(CASE WHEN b.status NOT IN ('rejected','expired') THEN b.field_price_amount ELSE 0 END) AS booked_field_value

FROM booking b

JOIN slot s ON s.slot_id = b.slot_id

JOIN field f ON f.field_id = s.field_id

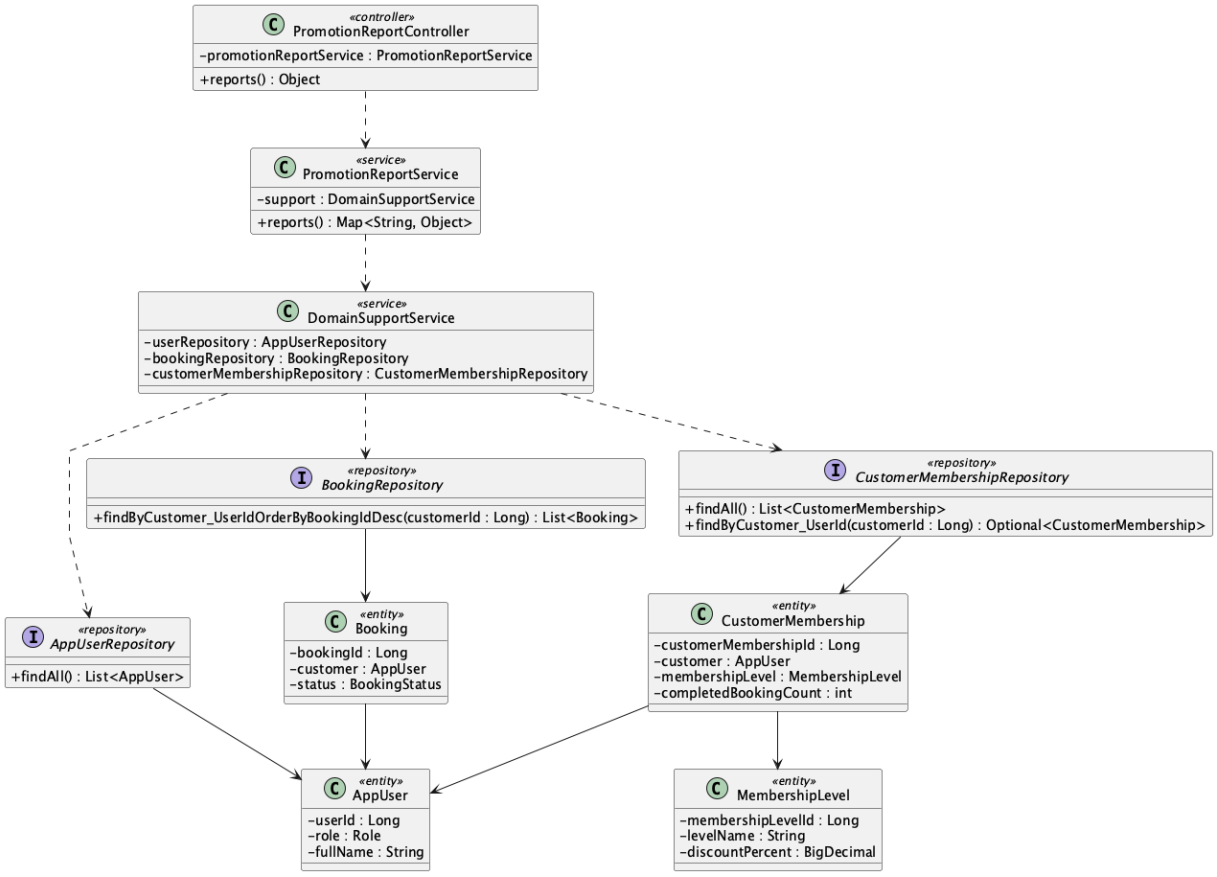
GROUP BY b.status, f.field_name, s.start_time

ORDER BY booking_count DESC;

Implementation note: There is one combined report endpoint rather than separate revenue, booking, and customer report controllers.

61. View customer activity report

a. Class Diagram (similar to 60)



b. Class Specifications

PromotionReportService

No	Method	Description
01	public Map<String, Object> reports()	Computes top customers, returning-customer count, and membership distribution.

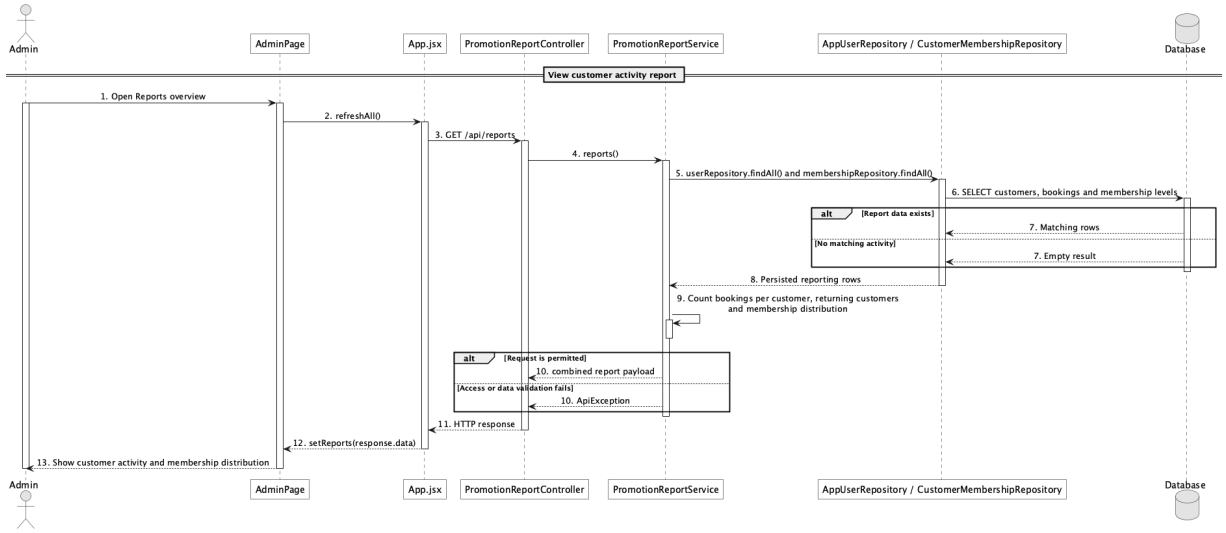
BookingRepository

No	Method	Description
01	List<Booking> findByCustomer_UserIdOrderByBookingIdDesc(Long customerId)	The current Java report calls this per customer to derive booking count.

CustomerMembershipRepository

No	Method	Description
01	Optional<CustomerMembership> findByCustomer_UserId(Long customerId)	Represents the customer-to-level relationship; reports currently aggregate inherited findAll() results.

c. Sequence Diagram



d. Database Queries

== View customer activity report ==

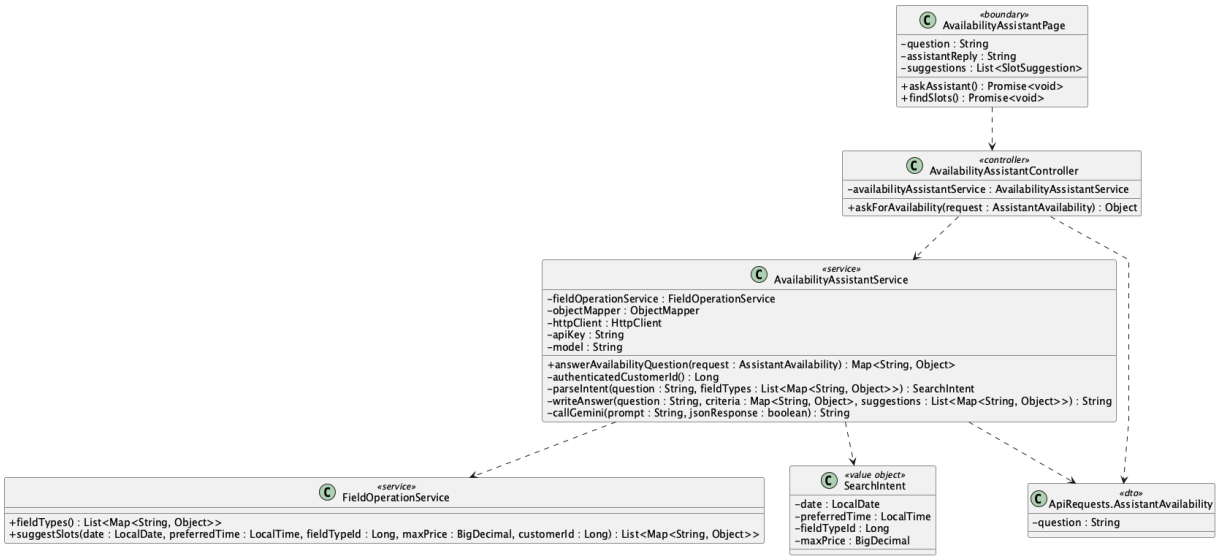
1. Customer activity summary

```
SELECT u.user_id, u.full_name, COUNT(b.booking_id) AS booking_count,
    ml.level_name, cm.completed_booking_count
FROM app_user u
JOIN role r ON r.role_id = u.role_id AND r.role_name = 'Customer'
LEFT JOIN booking b ON b.customer_id = u.user_id
LEFT JOIN customer_membership cm ON cm.customer_id = u.user_id
LEFT JOIN membership_level ml ON ml.membership_level_id = cm.membership_level_id
GROUP BY u.user_id, u.full_name, ml.level_name, cm.completed_booking_count
ORDER BY booking_count DESC;
```

Implementation note: The current implementation has an N+1 query shape for top-customer booking counts because it loads all users and then calls the per-customer booking repository method.

62. Ask availability assistant

a. Class Diagram



b. Class Specifications

AvailabilityAssistantController

No	Method	Description
01	askForAvailability(request : ApiRequests.AssistantAvailability) : Object	Exposes POST /api/assistant/availability and delegates the authenticated Customer question.

AvailabilityAssistantService

No	Method	Description
01	answerAvailabilityQuestion(request : AssistantAvailability) : Map<String, Object>	Validates the question/key, obtains verified ranked slots, and returns answer, criteria, and suggestions.
02	authenticatedCustomerId() : Long	Requires the current authenticated role to be Customer.
03	parseIntent(question : String, fieldTypes : List<Map>) : SearchIntent	Uses constrained Gemini JSON to extract date, preferred time, known field type, and maximum price.
04	callGemini(prompt : String, jsonResponse : boolean) : String	Calls the configured Gemini model and converts provider/configuration failures into safe API errors.

No	Method	Description
05	writeAnswer(question : String, criteria : Map, suggestions : List) : String	Asks Gemini to phrase only the verified GoalZone availability in at most 75 words.

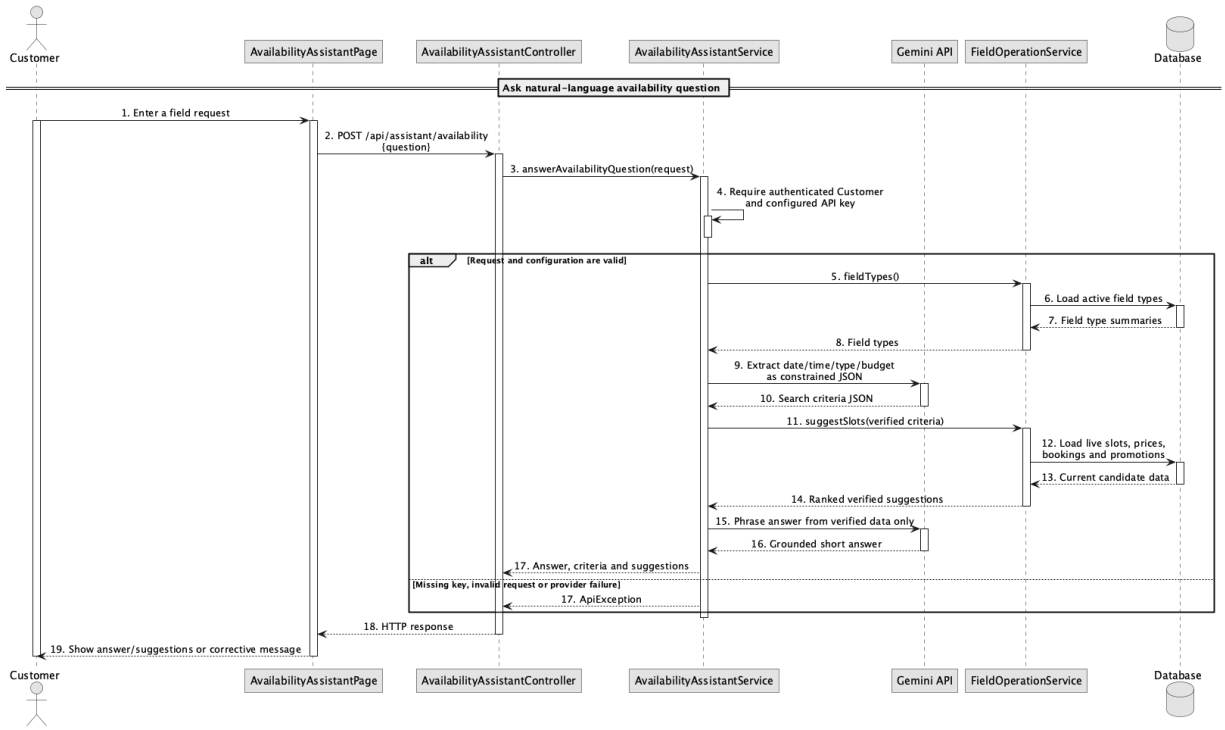
FieldOperationService

No	Method	Description
01	fieldTypes() : List<Map<String, Object>>	Returns the allowed field-type identifiers supplied to Gemini as a closed set.
02	suggestSlots(date, preferredTime, fieldTypeId, maxPrice, customerId) : List<Map>	Authoritatively filters and ranks live availability; Gemini cannot create or alter a result.

AvailabilityAssistantPage

No	Method	Description
01	askAssistant() : Promise<void>	Submits a Customer question and renders the grounded answer, criteria, and ranked suggestions.
02	findSlots() : Promise<void>	Runs the deterministic UC-63 filter path without requiring Gemini.

c. Sequence Diagram



d. Database Queries

== Ask availability assistant ==

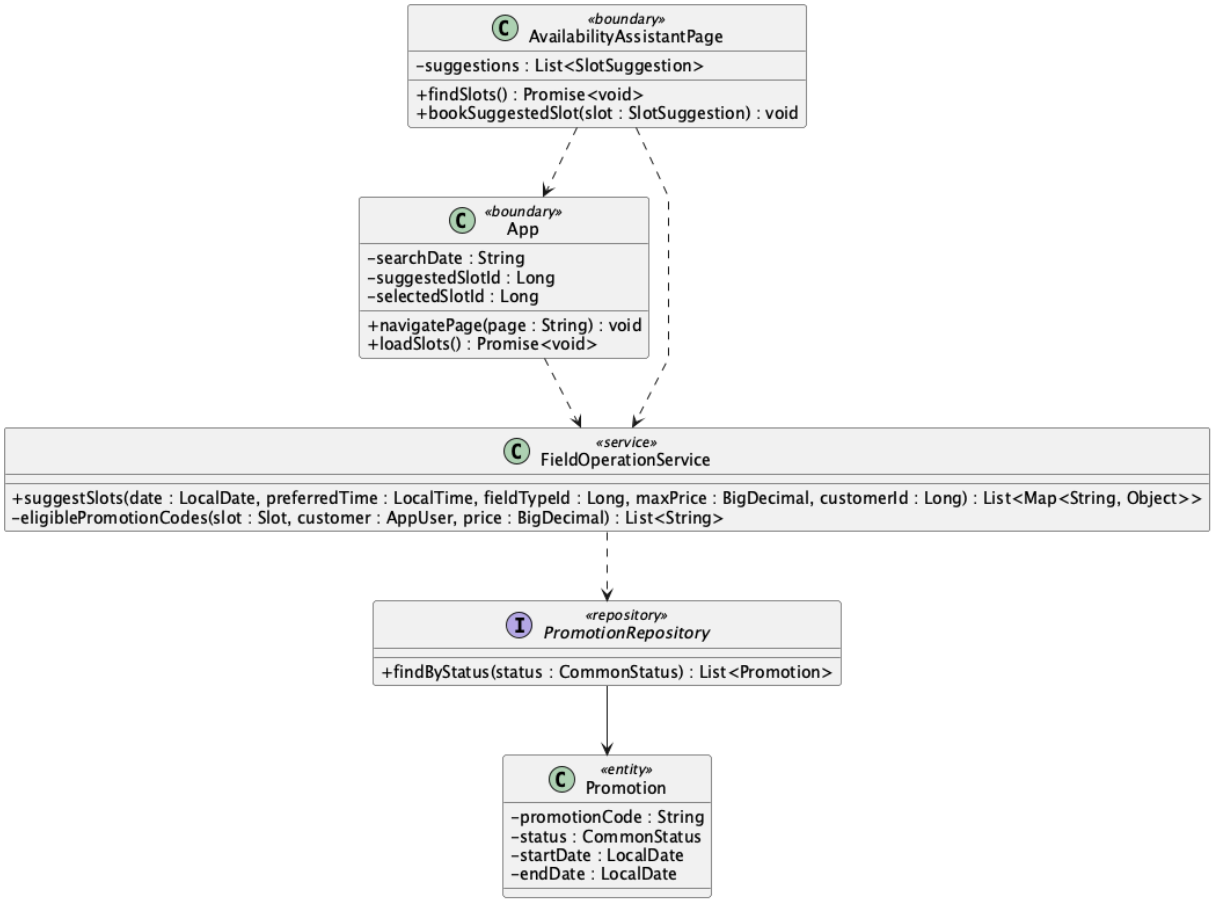
1. Live suggestion candidates

```
SELECT s.slot_id, s.slot_date, s.start_time, s.end_time, f.field_id, f.field_name,
    ft.type_name
FROM slot s
JOIN field f ON f.field_id = s.field_id AND f.status = 'active'
JOIN field_type ft ON ft.field_type_id = f.field_type_id
WHERE s.slot_date = :date AND s.status = 'available'
  AND NOT EXISTS (SELECT 1 FROM booking b WHERE b.slot_id = s.slot_id
          AND b.status IN ('pending','confirmed','checked_in'));
```

Implementation note: Gemini interprets and phrases UC-62 only when GEMINI_API_KEY is configured. UC-63 ranking and promotion eligibility remain authoritative Java logic over live data; no chatbot/RAG table is required.

63. Get suggested available slots

a. Class Diagram



b. Class Specifications

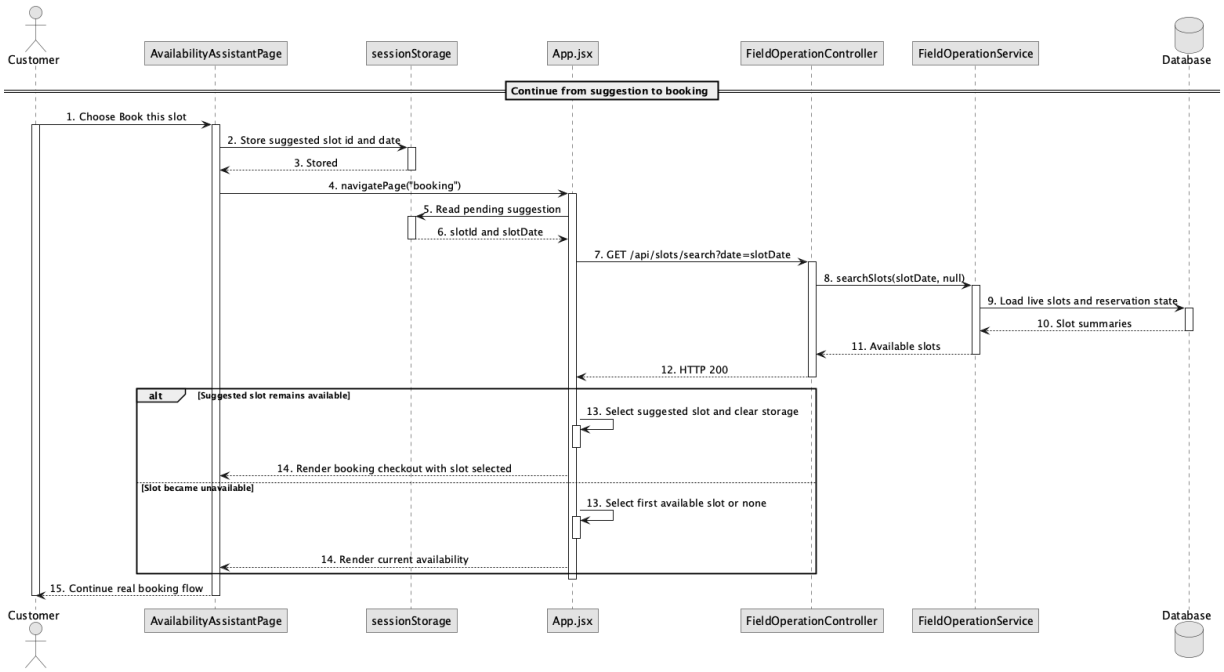
AvailabilityAssistantPage

No	Method	Description
01	findSlots()	Requests and displays ranked live suggestions.
02	bookSuggestedSlot(slot)	Stores slot id/date and navigates to booking.

App

No	Method	Description
01	loadSlots()	Reloads the suggested date and selects the exact slot only while available.

c. Sequence Diagram



d. Database Queries

== Get suggested available slots ==

1. Revalidate suggested slot

```
SELECT s.slot_id, s.status
FROM slot s
WHERE s.slot_id = :suggestedSlotId AND s.slot_date = :suggestedDate
  AND s.status = 'available'
  AND NOT EXISTS (SELECT 1 FROM booking b WHERE b.slot_id = s.slot_id
            AND b.status IN ('pending','confirmed','checked_in'));
```

Implementation note: sessionStorage is only a UI handoff; the booking API remains authoritative and revalidates availability.